

Important Code Reference: template-preview.js

A focused code/control-file explanation covering purpose, owner layer, APIs, variables, implementation signals, source excerpts, and debugging rules.

Item	Explanation
File	template-preview.js
Category	JavaScript source
Folder role	Repository root. Usually an entry file, app config, public website file, package manifest, or top-level documentation.
Size	454,437 bytes
Extension	.js
MIME type	application/javascript
Text lines	10,826

Why this file exists

- Builder frontend brain: state handling, rich editors, project builder, previews, parser triggers, publishing UI, compile workspace, and guide windows.

How it is used

Repository root. Usually an entry file, app config, public website file, package manifest, or top-level documentation.

How this file participates in the system

This generated section reads the actual file and points out line numbers, declarations, endpoints, events, source excerpts, and debugging cues.

API endpoints mentioned or called

Item	Explanation
/api/save-draft	Line 1637. Project-specific local API endpoint. Inspect server.mjs and the fetch caller for exact behavior.
/api/upload	Line 5430. Project-specific local API endpoint. Inspect server.mjs and the fetch caller for exact behavior.
/api/upload	Line 8413. Project-specific local API endpoint. Inspect server.mjs and the fetch caller for exact behavior.
/api/upload	Line 8473. Project-specific local API endpoint. Inspect server.mjs and the fetch caller for exact behavior.
/api/apply-catalog	Line 8613. Loads project/catalog data for the builder.
/api/apply-catalog	Line 8637. Loads project/catalog data for the builder.
/api/apply-catalog	Line 8647. Loads project/catalog data for the builder.
/api/apply-catalog	Line 8662. Loads project/catalog data for the builder.
/api/apply-catalog	Line 8674. Loads project/catalog data for the builder.
/api/save-draft	Line 8685. Project-specific local API endpoint. Inspect server.mjs and the fetch caller for exact behavior.
/api/save-draft	Line 10770. Project-specific local API endpoint. Inspect server.mjs and the fetch caller for exact behavior.
/api/apply-catalog	Line 10772. Loads project/catalog data for the builder.
/api/save-draft	Line 10796. Project-specific local API endpoint. Inspect server.mjs and the fetch caller for exact behavior.

Important implementation signals

Item	Explanation
Browser DOM at line 1	Builder or website interface behavior in the browser/Electron renderer. Evidence: const builderGuideOpen = document.querySelector("#builder-guide-open");
Browser DOM at line 2	Builder or website interface behavior in the browser/Electron renderer. Evidence: const builderGuideDialog = document.querySelector("#builder-guide-dialog");
Browser DOM at line 3	Builder or website interface behavior in the browser/Electron renderer. Evidence: const builderGuideClose = document.querySelector("#builder-guide-close");
Browser DOM at line 4	Builder or website interface behavior in the browser/Electron renderer. Evidence: const builderGuideSearch = document.querySelector("#builder-guide-search");
Browser DOM at line 5	Builder or website interface behavior in the browser/Electron renderer. Evidence: const builderGuideResults = document.querySelector("#builder-guide-results");
Browser DOM at line 6	Builder or website interface behavior in the browser/Electron renderer. Evidence: const builderGuideSections = document.querySelector("#builder-guide-sections");
Browser DOM at line 7	Builder or website interface behavior in the browser/Electron renderer. Evidence: const builderGuideTopicDialog = document.querySelector("#builder-guide-topic-dialog");
Browser DOM at line 8	Builder or website interface behavior in the browser/Electron renderer. Evidence: const builderGuideTopicTitle = document.querySelector("#builder-guide-topic-title");
Browser DOM at line 9	Builder or website interface behavior in the browser/Electron renderer. Evidence: const builderGuideTopicBody = document.querySelector("#builder-guide-topic-body");
Browser DOM at line 10	Builder or website interface behavior in the browser/Electron renderer. Evidence: const builderGuideTopicClose = document.querySelector("#builder-guide-topic-close");
Browser DOM at line 11	Builder or website interface behavior in the browser/Electron renderer. Evidence: const templateSelect = document.querySelector("#template-select");
Browser DOM at line 12	Builder or website interface behavior in the browser/Electron renderer. Evidence: const newCategorySelect = document.querySelector("#new-category-select");
Browser DOM at line 13	Builder or website interface behavior in the browser/Electron renderer. Evidence: const placementSelect = document.querySelector("#placement-select");
Browser DOM at line 14	Builder or website interface behavior in the browser/Electron renderer. Evidence: const newProjectTitle = document.querySelector("#new-project-title");
Browser DOM at line 15	Builder or website interface behavior in the browser/Electron renderer. Evidence: const addProjectButton = document.querySelector("#add-project");
Browser DOM at line 16	Builder or website interface behavior in the browser/Electron renderer. Evidence: const addCategoryButton = document.querySelector("#add-category");
Browser DOM at line 17	Builder or website interface behavior in the browser/Electron renderer. Evidence: const addSiteSectionButton = document.querySelector("#add-site-section");
Browser DOM at line 18	Builder or website interface behavior in the browser/Electron renderer. Evidence: const funFactsInput = document.querySelector("#fun-facts-input");
Browser DOM at line 19	Builder or website interface behavior in the browser/Electron renderer. Evidence: const funFactsCount = document.querySelector("#fun-facts-count");
Browser DOM at line 20	Builder or website interface behavior in the browser/Electron renderer. Evidence: const saveFunFactsButton = document.querySelector("#save-fun-facts");
Browser DOM at line 21	Builder or website interface behavior in the browser/Electron renderer. Evidence: const clearFunFactsButton = document.querySelector("#clear-fun-facts");
Browser DOM at line 22	Builder or website interface behavior in the browser/Electron renderer. Evidence: const siteHeroEyebrowInput = document.querySelector("#site-hero-eyebrow");
Browser DOM at line 23	Builder or website interface behavior in the browser/Electron renderer. Evidence: const siteHeroTitleInput = document.querySelector("#site-hero-title");
Browser DOM at line 24	Builder or website interface behavior in the browser/Electron renderer. Evidence: const siteHeroCopyInput = document.querySelector("#site-hero-copy");
Browser DOM at line 25	Builder or website interface behavior in the browser/Electron renderer. Evidence: const saveSiteContentButton = document.querySelector("#save-site-content");
Browser DOM at line 26	Builder or website interface behavior in the browser/Electron renderer. Evidence: const profileDisplayNameInput = document.querySelector("#profile-display-name");
Browser DOM at line 27	Builder or website interface behavior in the browser/Electron renderer. Evidence: const profilePortfolioLabelInput = document.querySelector("#profile-portfolio-label");

Item	Explanation
Browser DOM at line 28	Builder or website interface behavior in the browser/Electron renderer. Evidence: const profileContactIntroInput = document.querySelector("#profile-contact-intro");
Browser DOM at line 29	Builder or website interface behavior in the browser/Electron renderer. Evidence: const profileEmailInput = document.querySelector("#profile-email");
Browser DOM at line 30	Builder or website interface behavior in the browser/Electron renderer. Evidence: const profilePhoneInput = document.querySelector("#profile-phone");
Browser DOM at line 31	Builder or website interface behavior in the browser/Electron renderer. Evidence: const profileGithubInput = document.querySelector("#profile-github");
Browser DOM at line 32	Builder or website interface behavior in the browser/Electron renderer. Evidence: const profileLinkedInInput = document.querySelector("#profile-linkedin");
Browser DOM at line 33	Builder or website interface behavior in the browser/Electron renderer. Evidence: const profileWebsiteInput = document.querySelector("#profile-website");
Browser DOM at line 34	Builder or website interface behavior in the browser/Electron renderer. Evidence: const saveProfileContentButton = document.querySelector("#save-profile-content");
Browser DOM at line 35	Builder or website interface behavior in the browser/Electron renderer. Evidence: const profileImageUploadButton = document.querySelector("#profile-image-upload");
Browser DOM at line 36	Builder or website interface behavior in the browser/Electron renderer. Evidence: const profileHeroUploadButton = document.querySelector("#profile-hero-upload");
Browser DOM at line 37	Builder or website interface behavior in the browser/Electron renderer. Evidence: const profileResumeUploadButton = document.querySelector("#profile-resume-upload");
Browser DOM at line 38	Builder or website interface behavior in the browser/Electron renderer. Evidence: const profileBrandUploadButton = document.querySelector("#profile-brand-upload");
Browser DOM at line 39	Builder or website interface behavior in the browser/Electron renderer. Evidence: const profileAssetStatus = document.querySelector("#profile-asset-status");
Browser DOM at line 40	Builder or website interface behavior in the browser/Electron renderer. Evidence: const categoryDropdown = document.querySelector("#category-dropdown");
Browser DOM at line 41	Builder or website interface behavior in the browser/Electron renderer. Evidence: const siteSectionList = document.querySelector("#site-section-list");
Browser DOM at line 42	Builder or website interface behavior in the browser/Electron renderer. Evidence: const projectCreateDialog = document.querySelector("#project-create-dialog");
Browser DOM at line 43	Builder or website interface behavior in the browser/Electron renderer. Evidence: const projectCreateForm = document.querySelector("#project-create-form");
Browser DOM at line 44	Builder or website interface behavior in the browser/Electron renderer. Evidence: const projectCreateCategory = document.querySelector("#project-create-category");
Browser DOM at line 45	Builder or website interface behavior in the browser/Electron renderer. Evidence: const projectCreateCancel = document.querySelector("#project-create-cancel");
Browser DOM at line 46	Builder or website interface behavior in the browser/Electron renderer. Evidence: const categoryDialog = document.querySelector("#category-dialog");
Browser DOM at line 47	Builder or website interface behavior in the browser/Electron renderer. Evidence: const categoryForm = document.querySelector("#category-form");
Browser DOM at line 48	Builder or website interface behavior in the browser/Electron renderer. Evidence: const categoryTitle = document.querySelector("#category-title");
Browser DOM at line 49	Builder or website interface behavior in the browser/Electron renderer. Evidence: const categoryDescription = document.querySelector("#category-description");
Browser DOM at line 50	Builder or website interface behavior in the browser/Electron renderer. Evidence: const categoryAccent = document.querySelector("#category-accent");
Browser DOM at line 51	Builder or website interface behavior in the browser/Electron renderer. Evidence: const categoryCancel = document.querySelector("#category-cancel");
Browser DOM at line 52	Builder or website interface behavior in the browser/Electron renderer. Evidence: const projectTree = document.querySelector("#project-tree");
Browser DOM at line 53	Builder or website interface behavior in the browser/Electron renderer. Evidence: const projectSearchInput = document.querySelector("#project-search");
Browser DOM at line 54	Builder or website interface behavior in the browser/Electron renderer. Evidence: const projectSearchClear = document.querySelector("#project-search-clear");

Item	Explanation
Browser DOM at line 55	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const projectSearchStatus = document.querySelector("#project-search-status");</code>
Browser DOM at line 56	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const projectDialog = document.querySelector("#project-dialog");</code>
Browser DOM at line 57	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const projectWindowTitle = document.querySelector("#project-window-title");</code>
Browser DOM at line 58	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const projectWindowClose = document.querySelector("#project-window-close");</code>
Browser DOM at line 59	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const projectWindowDelete = document.querySelector("#project-window-delete");</code>
Browser DOM at line 60	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const viewProjectPreviewButton = document.querySelector("#view-project-preview");</code>
Browser DOM at line 61	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const saveProjectButton = document.querySelector("#save-project");</code>
Browser DOM at line 62	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const saveProjectCloseButton = document.querySelector("#save-project-close");</code>
Browser DOM at line 63	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const projectTitleMenu = document.querySelector("#project-title-menu");</code>
Browser DOM at line 64	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const titleRenameActions = document.querySelector("#title-rename-actions");</code>
Browser DOM at line 65	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const titleRenameSave = document.querySelector("#title-rename-save");</code>
Browser DOM at line 66	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const titleRenameCancel = document.querySelector("#title-rename-cancel");</code>
Browser DOM at line 67	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const projectMetaDialog = document.querySelector("#project-meta-dialog");</code>
Browser DOM at line 68	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const projectMetaTitle = document.querySelector("#project-meta-title");</code>
Browser DOM at line 69	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const projectMetaGrid = document.querySelector("#project-meta-grid");</code>
Browser DOM at line 70	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const projectMetaClose = document.querySelector("#project-meta-close");</code>
Browser DOM at line 71	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const projectFields = document.querySelector("#project-fields");</code>
Browser DOM at line 72	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const sectionTabs = document.querySelector("#section-tabs");</code>
Browser DOM at line 73	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const sectionContent = document.querySelector("#section-content");</code>
Browser DOM at line 74	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const projectWindowContent = document.querySelector(".project-window-content");</code>
Browser DOM at line 75	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const projectPreviewDialog = document.querySelector("#project-preview-dialog");</code>
Browser DOM at line 76	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const projectPreviewTitle = document.querySelector("#project-preview-title");</code>
Browser DOM at line 77	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const projectPreviewContent = document.querySelector("#project-preview-content");</code>
Browser DOM at line 78	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const projectPreviewClose = document.querySelector("#project-preview-close");</code>
Browser DOM at line 79	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const projectPreviewBack = document.querySelector("#project-preview-back");</code>
Browser DOM at line 80	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const projectPreviewForward = document.querySelector("#project-preview-forward");</code>

Important constants and variables

Item	Explanation
builderGuideOpen	Line 1: const builderGuideOpen = document.querySelector("#builder-guide-open");
builderGuideDialog	Line 2: const builderGuideDialog = document.querySelector("#builder-guide-dialog");
builderGuideClose	Line 3: const builderGuideClose = document.querySelector("#builder-guide-close");
builderGuideSearch	Line 4: const builderGuideSearch = document.querySelector("#builder-guide-search");
builderGuideResults	Line 5: const builderGuideResults = document.querySelector("#builder-guide-results");
builderGuideSections	Line 6: const builderGuideSections = document.querySelector("#builder-guide-sections");
builderGuideTopicDialog	Line 7: const builderGuideTopicDialog = document.querySelector("#builder-guide-topic-dialog");
builderGuideTopicTitle	Line 8: const builderGuideTopicTitle = document.querySelector("#builder-guide-topic-title");
builderGuideTopicBody	Line 9: const builderGuideTopicBody = document.querySelector("#builder-guide-topic-body");
builderGuideTopicClose	Line 10: const builderGuideTopicClose = document.querySelector("#builder-guide-topic-close");
templateSelect	Line 11: const templateSelect = document.querySelector("#template-select");
newCategorySelect	Line 12: const newCategorySelect = document.querySelector("#new-category-select");
placementSelect	Line 13: const placementSelect = document.querySelector("#placement-select");
newProjectTitle	Line 14: const newProjectTitle = document.querySelector("#new-project-title");
addProjectButton	Line 15: const addProjectButton = document.querySelector("#add-project");
addCategoryButton	Line 16: const addCategoryButton = document.querySelector("#add-category");
addSiteSectionButton	Line 17: const addSiteSectionButton = document.querySelector("#add-site-section");
funFactsInput	Line 18: const funFactsInput = document.querySelector("#fun-facts-input");
funFactsCount	Line 19: const funFactsCount = document.querySelector("#fun-facts-count");
saveFunFactsButton	Line 20: const saveFunFactsButton = document.querySelector("#save-fun-facts");
clearFunFactsButton	Line 21: const clearFunFactsButton = document.querySelector("#clear-fun-facts");
siteHeroEyebrowInput	Line 22: const siteHeroEyebrowInput = document.querySelector("#site-hero-eyebrow");
siteHeroTitleInput	Line 23: const siteHeroTitleInput = document.querySelector("#site-hero-title");
siteHeroCopyInput	Line 24: const siteHeroCopyInput = document.querySelector("#site-hero-copy");
saveSiteContentButton	Line 25: const saveSiteContentButton = document.querySelector("#save-site-content");
profileDisplayNameInput	Line 26: const profileDisplayNameInput = document.querySelector("#profile-display-name");
profilePortfolioLabelInput	Line 27: const profilePortfolioLabelInput = document.querySelector("#profile-portfolio-label");
profileContactIntroInput	Line 28: const profileContactIntroInput = document.querySelector("#profile-contact-intro");
profileEmailInput	Line 29: const profileEmailInput = document.querySelector("#profile-email");
profilePhoneInput	Line 30: const profilePhoneInput = document.querySelector("#profile-phone");
profileGithubInput	Line 31: const profileGithubInput = document.querySelector("#profile-github");
profileLinkedInInput	Line 32: const profileLinkedInInput = document.querySelector("#profile-linkedin");
profileWebsiteInput	Line 33: const profileWebsiteInput = document.querySelector("#profile-website");
saveProfileContentButton	Line 34: const saveProfileContentButton = document.querySelector("#save-profile-content");
profileImageUploadButton	Line 35: const profileImageUploadButton = document.querySelector("#profile-image-upload");
profileHeroUploadButton	Line 36: const profileHeroUploadButton = document.querySelector("#profile-hero-upload");
profileResumeUploadButton	Line 37: const profileResumeUploadButton = document.querySelector("#profile-resume-upload");
profileBrandUploadButton	Line 38: const profileBrandUploadButton = document.querySelector("#profile-brand-upload");
profileAssetStatus	Line 39: const profileAssetStatus = document.querySelector("#profile-asset-status");
categoryDropdown	Line 40: const categoryDropdown = document.querySelector("#category-dropdown");
siteSectionList	Line 41: const siteSectionList = document.querySelector("#site-section-list");

Item	Explanation
projectCreateDialog	Line 42: const projectCreateDialog = document.querySelector("#project-create-dialog");
projectCreateForm	Line 43: const projectCreateForm = document.querySelector("#project-create-form");
projectCreateCategory	Line 44: const projectCreateCategory = document.querySelector("#project-create-category");
projectCreateCancel	Line 45: const projectCreateCancel = document.querySelector("#project-create-cancel");
categoryDialog	Line 46: const categoryDialog = document.querySelector("#category-dialog");
categoryForm	Line 47: const categoryForm = document.querySelector("#category-form");
categoryTitle	Line 48: const categoryTitle = document.querySelector("#category-title");
categoryDescription	Line 49: const categoryDescription = document.querySelector("#category-description");
categoryAccent	Line 50: const categoryAccent = document.querySelector("#category-accent");
categoryCancel	Line 51: const categoryCancel = document.querySelector("#category-cancel");
projectTree	Line 52: const projectTree = document.querySelector("#project-tree");
projectSearchInput	Line 53: const projectSearchInput = document.querySelector("#project-search");
projectSearchClear	Line 54: const projectSearchClear = document.querySelector("#project-search-clear");
projectSearchStatus	Line 55: const projectSearchStatus = document.querySelector("#project-search-status");
projectDialog	Line 56: const projectDialog = document.querySelector("#project-dialog");
projectWindowTitle	Line 57: const projectWindowTitle = document.querySelector("#project-window-title");
projectWindowClose	Line 58: const projectWindowClose = document.querySelector("#project-window-close");
projectWindowDelete	Line 59: const projectWindowDelete = document.querySelector("#project-window-delete");
viewProjectPreviewButton	Line 60: const viewProjectPreviewButton = document.querySelector("#view-project-preview");
saveProjectButton	Line 61: const saveProjectButton = document.querySelector("#save-project");
saveProjectCloseButton	Line 62: const saveProjectCloseButton = document.querySelector("#save-project-close");
projectTitleMenu	Line 63: const projectTitleMenu = document.querySelector("#project-title-menu");
titleRenameActions	Line 64: const titleRenameActions = document.querySelector("#title-rename-actions");
titleRenameSave	Line 65: const titleRenameSave = document.querySelector("#title-rename-save");
titleRenameCancel	Line 66: const titleRenameCancel = document.querySelector("#title-rename-cancel");
projectMetaDialog	Line 67: const projectMetaDialog = document.querySelector("#project-meta-dialog");
projectMetaTitle	Line 68: const projectMetaTitle = document.querySelector("#project-meta-title");
projectMetaGrid	Line 69: const projectMetaGrid = document.querySelector("#project-meta-grid");
projectMetaClose	Line 70: const projectMetaClose = document.querySelector("#project-meta-close");
projectFields	Line 71: const projectFields = document.querySelector("#project-fields");
sectionTabs	Line 72: const sectionTabs = document.querySelector("#section-tabs");
sectionContent	Line 73: const sectionContent = document.querySelector("#section-content");
projectWindowContent	Line 74: const projectWindowContent = document.querySelector(".project-window-content");
projectPreviewDialog	Line 75: const projectPreviewDialog = document.querySelector("#project-preview-dialog");
projectPreviewTitle	Line 76: const projectPreviewTitle = document.querySelector("#project-preview-title");
projectPreviewContent	Line 77: const projectPreviewContent = document.querySelector("#project-preview-content");
projectPreviewClose	Line 78: const projectPreviewClose = document.querySelector("#project-preview-close");
projectPreviewBack	Line 79: const projectPreviewBack = document.querySelector("#project-preview-back");
projectPreviewForward	Line 80: const projectPreviewForward = document.querySelector("#project-preview-forward");

JSON/YAML-style keys detected

Item	Explanation
skin-light-blue	Line 433: "skin-light-blue": "appearance-light-blue-red-click",
skin-clean-white	Line 434: "skin-clean-white": "appearance-white-blue-click",
skin-deep-navy	Line 435: "skin-deep-navy": "appearance-deep-navy-cyan-click",
skin-red-warm	Line 436: "skin-red-warm": "appearance-warm-red-pale-click",
skin-emerald-instrument	Line 437: "skin-emerald-instrument": "appearance-emerald-mint-click",
skin-amber-power	Line 438: "skin-amber-power": "appearance-amber-cream-click",
skin-violet-mixed	Line 439: "skin-violet-mixed": "appearance-violet-lilac-click",
skin-graphite-asic	Line 440: "skin-graphite-asic": "appearance-graphite-white-click",
analog-opamp-topology	Line 441: "analog-opamp-topology": "appearance-light-blue-red-click",
analog-power-charger	Line 442: "analog-power-charger": "appearance-amber-cream-click",
analog-filter-front-end	Line 443: "analog-filter-front-end": "appearance-light-blue-red-click",
analog-sensor-interface	Line 444: "analog-sensor-interface": "appearance-emerald-mint-click",
analog-mixed-signal-timing	Line 445: "analog-mixed-signal-timing": "appearance-violet-lilac-click",
digital-fpga-pipeline	Line 446: "digital-fpga-pipeline": "appearance-deep-navy-cyan-click",
digital-asic-block	Line 447: "digital-asic-block": "appearance-graphite-white-click",
digital-verification-suite	Line 448: "digital-verification-suite": "appearance-graphite-white-click",
digital-interface-controller	Line 449: "digital-interface-controller": "appearance-deep-navy-cyan-click",
digital-signal-processing	Line 450: "digital-signal-processing": "appearance-violet-lilac-click",
embedded-stm32-sensor	Line 451: "embedded-stm32-sensor": "appearance-emerald-mint-click",
embedded-rtos-control	Line 452: "embedded-rtos-control": "appearance-graphite-white-click",
embedded-power-monitor	Line 453: "embedded-power-monitor": "appearance-amber-cream-click",
embedded-wireless-node	Line 454: "embedded-wireless-node": "appearance-deep-navy-cyan-click",
embedded-motor-control	Line 455: "embedded-motor-control": "appearance-violet-lilac-click"
engineering-file	Line 2512: "engineering-file": "Engineering file link",
google-drive	Line 2515: "google-drive": "Google Drive link",
Brief	Line 2656: "Brief": "Overview",
Project Brief	Line 2657: "Project Brief": "Overview",
Design Brief	Line 2658: "Design Brief": "Design Overview",
Simulation Brief	Line 2659: "Simulation Brief": "Simulation Overview"
--responsive-section-card-min	Line 2901: "--responsive-section-card-min": profile.sectionCardMin,
--responsive-content-max	Line 2902: "--responsive-content-max": profile.contentMax,
--responsive-media-max	Line 2903: "--responsive-media-max": profile.mediaMax,
--responsive-touch-target	Line 2904: "--responsive-touch-target": profile.touchTarget
--accent	Line 2919: "--accent": accent,
--template-bg	Line 2920: "--template-bg": visual.background,
--template-panel	Line 2921: "--template-panel": visual.panel,
--template-accent	Line 2922: "--template-accent": visual.accent,
--template-hover	Line 2923: "--template-hover": visual.hover,
--template-click	Line 2924: "--template-click": visual.click,
--template-click-text	Line 2925: "--template-click-text": visual.clickText,
--template-line	Line 2926: "--template-line": visual.line,

Item	Explanation
--template-text	Line 2927: "--template-text": visual.text,
--accent	Line 2952: "--accent": accent,
--template-bg	Line 2953: "--template-bg": visual.background,
--template-panel	Line 2954: "--template-panel": visual.panel,
--template-accent	Line 2955: "--template-accent": visual.accent,
--template-hover	Line 2956: "--template-hover": visual.hover,
--template-click	Line 2957: "--template-click": visual.click,
--template-click-text	Line 2958: "--template-click-text": visual.clickText,
--template-line	Line 2959: "--template-line": visual.line,
--template-text	Line 2960: "--template-text": visual.text,
compile-code	Line 7449: "compile-code": () => renderCompileCodeSection(project),

Event handlers and user interactions

Item	Explanation
Line 1048	document.addEventListener("pointerdown", beginDialogResize);
Line 1049	document.addEventListener("pointerdown", beginDialogDrag);
Line 1050	document.addEventListener("pointermove", moveDialogResize);
Line 1051	document.addEventListener("pointermove", moveDialogDrag);
Line 1052	document.addEventListener("pointerup", endDialogResize);
Line 1053	document.addEventListener("pointerup", endDialogDrag);
Line 1054	document.addEventListener("pointercancel", endDialogResize);
Line 1055	document.addEventListener("pointercancel", endDialogDrag);
Line 1056	document.addEventListener("click", (event) => {
Line 1061	document.addEventListener("click", (event) => {
Line 1067	dialog.addEventListener("close", () => {
Line 1076	window.addEventListener("resize", () => {
Line 1319	document.addEventListener("visibilitychange", () => {
Line 6166	siteLinkDialog.addEventListener("close", onClose);
Line 8325	dialog.addEventListener("close", onClose);
Line 8747	templateSelect.addEventListener("change", () => {
Line 8752	templatePreviewClose.addEventListener("click", () => {
Line 8880	builderGuideOpen.addEventListener("click", () => {
Line 8887	builderGuideClose.addEventListener("click", () => {
Line 8891	builderGuideTopicClose?.addEventListener("click", () => {
Line 8895	builderGuideTopicDialog?.addEventListener("close", () => {
Line 8899	builderGuideSearch?.addEventListener("input", filterBuilderGuide);
Line 8900	builderGuideDialog?.addEventListener("click", (event) => {
Line 8915	builderGuideDialog?.addEventListener("keydown", (event) => {
Line 8923	projectWindowTitle.addEventListener("dblclick", (event) => {
Line 8929	projectWindowTitle.addEventListener("keydown", (event) => {
Line 8946	projectWindowTitle.addEventListener("contextmenu", (event) => {

Item	Explanation
Line 8954	projectTitleMenu.addEventListener("click", async (event) => {
Line 8991	titleRenameSave.addEventListener("click", () => endTitleRename(true));
Line 8992	titleRenameCancel.addEventListener("click", () => endTitleRename(false));
Line 8993	projectMetaClose.addEventListener("click", () => closeDialogElement(projectMetaDialog));
Line 8995	viewProjectPreviewButton.addEventListener("click", openProjectPortfolioPreview);
Line 8996	projectPreviewClose.addEventListener("click", closeProjectPreviewWindow);
Line 8997	projectPreviewDialog.addEventListener("close", () => {
Line 9001	projectPreviewBack.addEventListener("click", projectPreviewBackStep);
Line 9002	projectPreviewForward.addEventListener("click", projectPreviewForwardStep);
Line 9004	projectPreviewContent.addEventListener("click", (event) => {
Line 9012	saveProjectButton.addEventListener("click", saveSelectedProjectToPortfolio);
Line 9013	saveProjectCloseButton.addEventListener("click", saveSelectedProjectAndClose);
Line 9014	openPortfolioPreviewButton.addEventListener("click", openPortfolioPreview);
Line 9015	saveAllSectionsButton.addEventListener("click", saveAllSections);
Line 9016	portfolioPreviewClose.addEventListener("click", () => {
Line 9019	portfolioPreviewDialog.addEventListener("close", () => {
Line 9022	publishTargetOpen?.addEventListener("click", openPublishTargetDialog);
Line 9023	publishTargetClose?.addEventListener("click", () => {
Line 9026	publishTargetCancel?.addEventListener("click", () => {
Line 9029	publishTargetForm?.addEventListener("submit", savePublishTarget);
Line 9030	publishTargetSync?.addEventListener("click", syncFromPublishTarget);
Line 9031	publishTargetCheck?.addEventListener("click", loadSystemReadiness);
Line 9032	publishTargetInstallGit?.addEventListener("click", installGitForPublishing);
Line 9033	publishTargetRepository?.addEventListener("input", () => {
Line 9036	publishTargetDomain?.addEventListener("input", () => {
Line 9040	publishTargetUsername?.addEventListener("input", markPublishTargetNeedsAuthentication);
Line 9041	publishTargetPassword?.addEventListener("input", markPublishTargetNeedsAuthentication);
Line 9042	publishResultClose.addEventListener("click", () => {
Line 9046	appUpdateClose?.addEventListener("click", () => {
Line 9050	appUpdateLater?.addEventListener("click", () => {
Line 9058	appUpdateSkip?.addEventListener("click", () => {
Line 9065	appUpdateApply?.addEventListener("click", () => {
Line 9069	appUpdateCheckButton?.addEventListener("click", () => {
Line 9073	securityReportOpen?.addEventListener("click", openSecurityReport);
Line 9074	securityReportRefresh?.addEventListener("click", openSecurityReport);
Line 9075	securityReportClose?.addEventListener("click", () => {
Line 9078	securityReportOk?.addEventListener("click", () => {
Line 9082	addProjectButton.addEventListener("click", () => {
Line 9086	projectSearchInput?.addEventListener("input", () => {
Line 9091	projectSearchClear?.addEventListener("click", () => {
Line 9100	quickOpenSelected?.addEventListener("click", () => {

Item	Explanation
Line 9106	quickPreviewSelected?.addEventListener("click", () => {
Line 9112	addCategoryButton?.addEventListener("click", openCategoryDialog);
Line 9113	categoryCancel?.addEventListener("click", () => closeDialogElement(categoryDialog, "cancel"));
Line 9114	categoryForm?.addEventListener("submit", (event) => {
Line 9120	addSiteSectionButton.addEventListener("click", addSiteSection);
Line 9122	funFactsInput?.addEventListener("input", () => {
Line 9128	saveFunFactsButton?.addEventListener("click", () => {
Line 9135	clearFunFactsButton?.addEventListener("click", () => {
Line 9146	input.addEventListener("input", () => {
Line 9154	saveSiteContentButton?.addEventListener("click", () => {
Line 9171	input.addEventListener("input", () => {
Line 9179	saveProfileContentButton?.addEventListener("click", () => {

Function-by-function detail

isHdlLanguage - lines 240-242

Item	Explanation
Source location	Lines 240-242 (3 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function isHdlLanguage(language = "") {
Touches	filesystem
Calls detected	includes, normalizeCodeLanguage
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

240: function isHdlLanguage(language = "") {
241:   return ["verilog", "systemverilog"].includes(normalizeCodeLanguage(language));
242: }

```

inferCompileFileRole - lines 244-251

Item	Explanation
Source location	Lines 244-251 (8 source lines).
Purpose	Part of the Compile Code language/tool/build/run flow.
Declaration	function inferCompileFileRole(fileName = "", code = "", language = "") {
Touches	Mostly local calculations or local UI state.
Calls detected	isHdlLanguage, toLowerCase, b, tb
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 3 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
244: function inferCompileFileRole(fileName = "", code = "", language = "") {
245:   if (!isHdlLanguage(language)) return "source";
246:   const haystack = `${fileName}\n${code}`.toLowerCase();
247:   if (/\\b(tb|testbench)\\b|^|[_-]|tb|[_-]|\\.|)test[_-]?bench|\\$dumpfile|\\$dumpvars|module\\s+tb[_a-z0-9]*\\/i.test(haystack)) {
248:     return "testbench";
249:   }
250:   return "design";
251: }
```

normalizeCompileFileRole - lines 253-257

Item	Explanation
Source location	Lines 253-257 (5 source lines).
Purpose	Part of the Compile Code language/tool/build/run flow.
Declaration	function normalizeCompileFileRole(value = "", language = "") {
Touches	filesystem
Calls detected	isHdlLanguage, trim, toLowerCase, replace, includes
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
253: function normalizeCompileFileRole(value = "", language = "") {
254:   if (!isHdlLanguage(language)) return "source";
255:   const clean = String(value || "").trim().toLowerCase().replace(/[_\s-]+/g, "");
256:   return ["tb", "testbench", "bench"].includes(clean) ? "testbench" : "design";
257: }
```

compileFileRoleLabel - lines 259-263

Item	Explanation
Source location	Lines 259-263 (5 source lines).
Purpose	Part of the Compile Code language/tool/build/run flow.
Declaration	function compileFileRoleLabel(role = "source") {
Touches	Mostly local calculations or local UI state.
Calls detected	No obvious named calls detected by the generator.
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 3 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
259: function compileFileRoleLabel(role = "source") {
260:   if (role === "testbench") return "Testbench";
261:   if (role === "design") return "Design";
262:   return "Program source";
263: }
```

setStatus - lines 509-511

Item	Explanation
Source location	Lines 509-511 (3 source lines).
Purpose	Part of the private builder frontend and editing experience.

Item	Explanation
Declaration	function setStatus(message) {
Touches	Mostly local calculations or local UI state.
Calls detected	No obvious named calls detected by the generator.
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
509: function setStatus(message) {
510:   builderStatus.textContent = message;
511: }
```

setSaveState - lines 513-517

Item	Explanation
Source location	Lines 513-517 (5 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function setSaveState(state, message) {
Touches	browser/Electron DOM
Calls detected	No obvious named calls detected by the generator.
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
513: function setSaveState(state, message) {
514:   if (!builderSaveState) return;
515:   builderSaveState.dataset.state = state || "loaded";
516:   builderSaveState.textContent = message || "Loaded";
517: }
```

projectSearchText - lines 519-521

Item	Explanation
Source location	Lines 519-521 (3 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function projectSearchText(value = "") {
Touches	Mostly local calculations or local UI state.
Calls detected	toLowerCase, replace, trim
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
519: function projectSearchText(value = "") {
520:   return String(value || "").toLowerCase().replace(/\s+/g, " ").trim();
```

521: }

regexEscape - lines 523-525

Item	Explanation
Source location	Lines 523-525 (3 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function regexEscape(value = "") {
Touches	Mostly local calculations or local UI state.
Calls detected	replace
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
523: function regexEscape(value = "") {
524:   return String(value).replace(/[\.*+?^${}()|[\]\$\\"/g, "\\$&");
525: }
```

highlightSearchText - lines 527-533

Item	Explanation
Source location	Lines 527-533 (7 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function highlightSearchText(value = "", query = "") {
Touches	Mostly local calculations or local UI state.
Calls detected	projectSearchText, escapeHtml, regexEscape, replace
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
527: function highlightSearchText(value = "", query = "") {
528:   const text = String(value || "");
529:   const cleanQuery = projectSearchText(query);
530:   if (!cleanQuery) return escapeHtml(text);
531:   const matcher = new RegExp(`${regexEscape(cleanQuery)}`, "ig");
532:   return escapeHtml(text).replace(matcher, "<mark>$1</mark>");
533: }
```

projectSearchHaystack - lines 535-555

Item	Explanation
Source location	Lines 535-555 (21 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function projectSearchHaystack(project = {}, category = {}) {
Touches	Mostly local calculations or local UI state.
Calls detected	projectSearchText, stringify, filter, join
API endpoints	None detected inside this function body.

Item	Explanation
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

535: function projectSearchHaystack(project = {}, category = {}) {
536:   return projectSearchText([
537:     category.label,
538:     category.description,
539:     project.title,
540:     project.summary,
541:     project.status,
542:     project.category,
543:     project.templateLabel,
544:     ...(project.focus || []),
545:     ...(project.highlights || []),
546:     JSON.stringify(project.sections || []),
547:     JSON.stringify(project.design || {}),
548:     JSON.stringify(project.documents || []),
549:     JSON.stringify(project.tests || []),
550:     JSON.stringify(project.tools || []),
551:     JSON.stringify(project.links || []),
552:     JSON.stringify(project.media || []),
553:     JSON.stringify(project.compileCode || {}),
554:   ]).filter(Boolean).join(" ");
555: }

```

projectMatchesSearch - lines 557-561

Item	Explanation
Source location	Lines 557-561 (5 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function projectMatchesSearch(project, category, query) {
Touches	Mostly local calculations or local UI state.
Calls detected	projectSearchText, projectSearchHaystack, includes
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

557: function projectMatchesSearch(project, category, query) {
558:   const cleanQuery = projectSearchText(query);
559:   if (!cleanQuery) return true;
560:   return projectSearchHaystack(project, category).includes(cleanQuery);
561: }

```

updateProjectSearchStatus - lines 563-575

Item	Explanation
Source location	Lines 563-575 (13 source lines).
Purpose	Updates UI, state, cache, or stored data.
Declaration	function updateProjectSearchStatus(matchCount = catalog.projects.length) {
Touches	browser/Electron DOM
Calls detected	projectSearchText
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.

Item	Explanation
Async/return clues	0 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

563: function updateProjectSearchStatus(matchCount = catalog.projects.length) {
564:   const cleanQuery = projectSearchText(projectSearchQuery);
565:   if (projectSearchInput && document.activeElement !== projectSearchInput) {
566:     projectSearchInput.value = projectSearchQuery;
567:   }
568:   if (projectSearchClear) projectSearchClear.hidden = !cleanQuery;
569:   if (!projectSearchStatus) return;
570:   if (!cleanQuery) {
571:     projectSearchStatus.textContent = "Showing all projects.";
572:     return;
573:   }
574:   projectSearchStatus.textContent = `${matchCount} matching project${matchCount === 1 ? "" : "s"} for
"${projectSearchQuery}"`;
575: }

```

updateBuilderWorkflow - lines 577-597

Item	Explanation
Source location	Lines 577-597 (21 source lines).
Purpose	Updates UI, state, cache, or stored data.
Declaration	function updateBuilderWorkflow() {
Touches	Mostly local calculations or local UI state.
Calls detected	selectedProject, Set, map, filter, has, forEach
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

577: function updateBuilderWorkflow() {
578:   const project = selectedProject();
579:   const savedIds = new Set((savedPortfolioCatalog.projects || []).map((item) => item.id));
580:   const savedCount = (catalog.projects || []).filter((projectItem) =>
savedIds.has(projectItem.id)).length;
581:   const visibleSections = (catalog.siteSections || []).filter(siteSectionRenderable).length;
582:   if (workflowSelectedProject) {
583:     workflowSelectedProject.textContent = project ? project.title || "Untitled project" : "No project
selected";
584:   }
585:   if (workflowTotalProjects) {
586:     workflowTotalProjects.textContent = `${catalog.projects.length} project${catalog.projects.length ===
1 ? "" : "s"}`;
587:   }
588:   if (workflowSavedProjects) {
589:     workflowSavedProjects.textContent = `${savedCount} parsed`;
590:   }
591:   if (workflowVisibleSections) {
592:     workflowVisibleSections.textContent = `${visibleSections} live section${visibleSections === 1 ? "" :
"s"}`;
593:   }
594:   [quickOpenSelected, quickPreviewSelected].forEach((button) => {
595:     if (button) button.disabled = !project;
596:   });
597: }

```

savedCount - lines 580-584

Item	Explanation
Source location	Lines 580-584 (5 source lines).
Purpose	Persists data to local state, disk, credentials, or browser storage.

Item	Explanation
Declaration	<code>const savedCount = (catalog.projects []).filter((projectItem) => savedIds.has(projectItem.id)).length;</code>
Touches	Mostly local calculations or local UI state.
Calls detected	filter, has
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

580:   const savedCount = (catalog.projects || []).filter((projectItem) =>
savedIds.has(projectItem.id)).length;
581:   const visibleSections = (catalog.siteSections || []).filter(siteSectionRenderable).length;
582:   if (workflowSelectedProject) {
583:     workflowSelectedProject.textContent = project ? project.title || "Untitled project" : "No project
selected";
584:   }

```

dialogDragHandles - lines 599-604

Item	Explanation
Source location	Lines 599-604 (6 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	<code>function dialogDragHandles(dialog) {</code>
Touches	browser/Electron DOM, filesystem
Calls detected	querySelectorAll
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

599: function dialogDragHandles(dialog) {
600:   return [
601:     ...dialog.querySelectorAll(".project-dialog-heading, .project-preview-heading, .template-dialog-
heading"),
602:     ...dialog.querySelectorAll(".asset-dialog form > div:first-child")
603:   ];
604: }

```

dialogWindowActionTarget - lines 606-608

Item	Explanation
Source location	Lines 606-608 (3 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	<code>function dialogWindowActionTarget(handle) {</code>
Touches	browser/Electron DOM
Calls detected	querySelector
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).

Item	Explanation
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
606: function dialogWindowActionTarget(handle) {
607:   return handle.querySelector(".project-window-actions, .section-view-actions") || handle;
608: }
```

updateDialogWindowButtons - lines 610-635

Item	Explanation
Source location	Lines 610-635 (26 source lines).
Purpose	Updates UI, state, cache, or stored data.
Declaration	function updateDialogWindowButtons(dialog) {
Touches	browser/Electron DOM
Calls detected	contains, querySelectorAll, forEach, setAttribute, dialogCanStepBack, dialogCanStepForward
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
610: function updateDialogWindowButtons(dialog) {
611:   const isMinimized = dialog.classList.contains("is-minimized-dialog");
612:   const isHidden = dialog.classList.contains("is-hidden-dialog");
613:   const isMaximized = dialog.classList.contains("is-maximized-dialog");
614:   dialog.querySelectorAll("[data-dialog-minimize='true']").forEach((button) => {
615:     button.textContent = isMinimized ? "+" : "\u2212";
616:     button.title = isMinimized ? "Restore window" : "Minimize window";
617:     button.setAttribute("aria-label", isMinimized ? "Restore window" : "Minimize window");
618:   });
619:   dialog.querySelectorAll("[data-dialog-hide='true']").forEach((button) => {
620:     button.textContent = isHidden ? "\u25b4" : "\u25be";
621:     button.title = isHidden ? "Show window" : "Hide window";
622:     button.setAttribute("aria-label", isHidden ? "Show window" : "Hide window");
623:   });
624:   dialog.querySelectorAll("[data-dialog-maximize='true']").forEach((button) => {
625:     button.textContent = isMaximized ? "\u2750" : "\u25a1";
626:     button.title = isMaximized ? "Restore size" : "Maximize window";
627:     button.setAttribute("aria-label", isMaximized ? "Restore size" : "Maximize window");
628:   });
629:   dialog.querySelectorAll("[data-dialog-back='true']").forEach((button) => {
630:     button.disabled = !dialogCanStepBack(dialog);
631:   });
632:   dialog.querySelectorAll("[data-dialog-forward='true']").forEach((button) => {
633:     button.disabled = !dialogCanStepForward(dialog);
634:   });
635: }
```

dialogTitleForDock - lines 637-639

Item	Explanation
Source location	Lines 637-639 (3 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function dialogTitleForDock(dialog) {
Touches	browser/Electron DOM
Calls detected	querySelector, trim
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).

Item	Explanation
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

637: function dialogTitleForDock(dialog) {
638:   return dialog.querySelector("h2")?.textContent?.trim() || "Window";
639: }

```

makeDialogControl - lines 641-650

Item	Explanation
Source location	Lines 641-650 (10 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function makeDialogControl(action, label, title) {
Touches	browser/Electron DOM
Calls detected	createElement, setAttribute
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

641: function makeDialogControl(action, label, title) {
642:   const button = document.createElement("button");
643:   button.type = "button";
644:   button.className = `dialog-window-control dialog-window-${action}-control`;
645:   button.dataset.dialogAction = action;
646:   button.textContent = label;
647:   button.title = title;
648:   button.setAttribute("aria-label", title);
649:   return button;
650: }

```

ensureDialogLeftControls - lines 652-674

Item	Explanation
Source location	Lines 652-674 (23 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function ensureDialogLeftControls(dialog, handle) {
Touches	browser/Electron DOM
Calls detected	querySelector, createElement, makeDialogControl, append, prepend, forEach, toUpperCase, slice
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

652: function ensureDialogLeftControls(dialog, handle) {
653:   if (handle.querySelector(".dialog-window-left-controls")) return;
654:   const titlebar = handle.querySelector(".section-view-titlebar");
655:   const controls = document.createElement("div");
656:   controls.className = "dialog-window-left-controls";
657:
658:   if (titlebar?.querySelector(".section-view-back, .section-view-forward")) {
659:     const refresh = makeDialogControl("refresh", "\u21bb", "Refresh window");
660:     refresh.dataset.dialogRefresh = "true";
661:     controls.append(refresh);

```

```

662:     titlebar.prepend(controls);
663:   } else {
664:     [
665:       makeDialogControl("back", "\u2190", "Back"),
666:       makeDialogControl("forward", "\u2192", "Forward"),
667:       makeDialogControl("refresh", "\u21bb", "Refresh window")
668:     ].forEach((button) => {
669: button.dataset[`dialog${button.dataset.dialogAction[0].toUpperCase()}${button.dataset.dialogAction.slice(1)}`] =
"true";
670:       controls.append(button);
671:     });
672:     handle.prepend(controls);
673:   }
674: }

```

ensureDialogWindowControls - lines 676-720

Item	Explanation
Source location	Lines 676-720 (45 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function ensureDialogWindowControls(dialog, handle) {
Touches	browser/Electron DOM, filesystem
Calls detected	querySelectorAll, forEach, remove, ensureDialogLeftControls, dialogWindowActionTarget, querySelector, createElement, append, makeDialogControl, find, closest, add
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

676: function ensureDialogWindowControls(dialog, handle) {
677:   dialog.querySelectorAll(".dialog-minimize-button").forEach((button) => button.remove());
678:   ensureDialogLeftControls(dialog, handle);
679:   const target = dialogWindowActionTarget(handle);
680:   let cluster = target.querySelector(".dialog-window-control-cluster");
681:   if (!cluster) {
682:     cluster = document.createElement("div");
683:     cluster.className = "dialog-window-control-cluster";
684:     target.append(cluster);
685:   }
686:
687:   if (!cluster.querySelector("[data-dialog-hide='true']")) {
688:     const hide = makeDialogControl("hide", "\u25be", "Hide window");
689:     hide.dataset.dialogHide = "true";
690:     cluster.append(hide);
691:   }
692:   if (!cluster.querySelector("[data-dialog-minimize='true']")) {
693:     const minimize = makeDialogControl("minimize", "\u2212", "Minimize window");
694:     minimize.dataset.dialogMinimize = "true";
695:     cluster.append(minimize);
696:   }
697:   if (!cluster.querySelector("[data-dialog-maximize='true']")) {
698:     const maximize = makeDialogControl("maximize", "\u25a1", "Maximize window");
699:     maximize.dataset.dialogMaximize = "true";
700:     cluster.append(maximize);
701:   }
702:
703:   let closeButton = cluster.querySelector(".project-window-close, .close-control, [data-dialog-
close='true']");
704:   if (!closeButton) {
705:     closeButton = [...dialog.querySelectorAll(".project-window-close, .close-control")]
706:       .find((button) => button.closest(".project-dialog-heading, .project-preview-heading, .template-
dialog-heading, .asset-dialog form > div:first-child") === handle);
707:   }
708:   if (closeButton) {
709:     closeButton.classList.add("dialog-window-control", "dialog-close-button");
710:     closeButton.dataset.dialogAction = "close";
711:     closeButton.dataset.dialogClose = "true";
712:     cluster.append(closeButton);
713:   } else {
714:     const cloneClose = makeDialogControl("close", "\u00d7", "Close window");
715:     cloneClose.classList.add("project-window-close", "dialog-close-button");
716:     cloneClose.dataset.dialogClose = "true";
717:     cluster.append(cloneClose);
718:   }
719:   updateDialogWindowButtons(dialog);

```

720: }

ensureDialogResizeHandles - lines 722-731

Item	Explanation
Source location	Lines 722-731 (10 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function ensureDialogResizeHandles(dialog) {
Touches	browser/Electron DOM
Calls detected	querySelector, forEach, createElement, setAttribute, append
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
722: function ensureDialogResizeHandles(dialog) {
723:   if (dialog.querySelector("[data-dialog-resize-handle]")) return;
724:   ["n", "e", "s", "w", "ne", "nw", "se", "sw"].forEach((direction) => {
725:     const handle = document.createElement("div");
726:     handle.className = `dialog-resize-handle resize-${direction}`;
727:     handle.dataset.dialogResizeHandle = direction;
728:     handle.setAttribute("aria-hidden", "true");
729:     dialog.append(handle);
730:   });
731: }
```

markDialogDragHandles - lines 733-740

Item	Explanation
Source location	Lines 733-740 (8 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function markDialogDragHandles(dialog) {
Touches	browser/Electron DOM
Calls detected	dialogDragHandles, forEach, ensureDialogWindowControls, ensureDialogResizeHandles
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
733: function markDialogDragHandles(dialog) {
734:   dialogDragHandles(dialog).forEach((handle) => {
735:     handle.dataset.dialogDragHandle = "true";
736:     handle.title = handle.title || "Drag to move this window";
737:     ensureDialogWindowControls(dialog, handle);
738:   });
739:   ensureDialogResizeHandles(dialog);
740: }
```

draggableDialogFromEvent - lines 742-751

Item	Explanation
Source location	Lines 742-751 (10 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function draggableDialogFromEvent(event) {

Item	Explanation
Touches	browser/Electron DOM
Calls detected	closest, contains
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 6 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

742: function draggableDialogFromEvent(event) {
743:   if (event.target.closest("[data-dialog-resize-handle]")) return null;
744:   const handle = event.target.closest("[data-dialog-drag-handle='true']");
745:   if (!handle) return null;
746:   const dialog = handle.closest("dialog");
747:   if (!dialog?.open) return null;
748:   if (dialog.classList.contains("is-hidden-dialog")) return null;
749:   if (event.target.closest("button, a, input, textarea, select, label, summary,
[contenteditable='true']")) return null;
750:   return dialog;
751: }

```

clampDialogPosition - lines 753-762

Item	Explanation
Source location	Lines 753-762 (10 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function clampDialogPosition(left, top, dialog) {
Touches	Mostly local calculations or local UI state.
Calls detected	getBoundingClientRect, max, min
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

753: function clampDialogPosition(left, top, dialog) {
754:   const rect = dialog.getBoundingClientRect();
755:   const margin = 12;
756:   const maxLeft = Math.max(margin, window.innerWidth - rect.width - margin);
757:   const maxTop = Math.max(margin, window.innerHeight - rect.height - margin);
758:   return {
759:     left: Math.min(Math.max(left, margin), maxLeft),
760:     top: Math.min(Math.max(top, margin), maxTop)
761:   };
762: }

```

anchorDialogForDrag - lines 764-773

Item	Explanation
Source location	Lines 764-773 (10 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function anchorDialogForDrag(dialog) {
Touches	browser/Electron DOM
Calls detected	getBoundingClientRect, clampDialogPosition, add
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.

Item	Explanation
Async/return clues	0 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

764: function anchorDialogForDrag(dialog) {
765:   const rect = dialog.getBoundingClientRect();
766:   const position = clampDialogPosition(rect.left, rect.top, dialog);
767:   dialog.classList.add("is-draggable-dialog");
768:   dialog.style.left = `${position.left}px`;
769:   dialog.style.top = `${position.top}px`;
770:   dialog.style.right = "auto";
771:   dialog.style.bottom = "auto";
772:   dialog.style.margin = "0";
773: }

```

saveDialogRestoreState - lines 775-788

Item	Explanation
Source location	Lines 775-788 (14 source lines).
Purpose	Persists data to local state, disk, credentials, or browser storage.
Declaration	function saveDialogRestoreState(dialog) {
Touches	browser/Electron DOM
Calls detected	getBoundingClientRect, stringify
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

775: function saveDialogRestoreState(dialog) {
776:   if (dialog.dataset.restoreWindowState) return;
777:   const rect = dialog.getBoundingClientRect();
778:   dialog.dataset.restoreWindowState = JSON.stringify({
779:     bottom: dialog.style.bottom || "",
780:     height: dialog.style.height || `${rect.height}px`,
781:     left: dialog.style.left || `${rect.left}px`,
782:     margin: dialog.style.margin || "",
783:     maxHeight: dialog.style.maxHeight || "",
784:     right: dialog.style.right || "",
785:     top: dialog.style.top || `${rect.top}px`,
786:     width: dialog.style.width || `${rect.width}px`
787:   });
788: }

```

restoreDialogWindowState - lines 790-805

Item	Explanation
Source location	Lines 790-805 (16 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function restoreDialogWindowState(dialog) {
Touches	browser/Electron DOM
Calls detected	parse, removeAttribute, entries, forEach
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
790: function restoreDialogWindowState(dialog) {
791:   let state = null;
792:   try {
793:     state = JSON.parse(dialog.dataset.restoreWindowState || "null");
794:   } catch {
795:     state = null;
796:   }
797:   delete dialog.dataset.restoreWindowState;
798:   if (!state) {
799:     dialog.removeAttribute("style");
800:     return;
801:   }
802:   Object.entries(state).forEach(([key, value]) => {
803:     dialog.style[key] = value;
804:   });
805: }
```

anchorDialogForResize - lines 807-814

Item	Explanation
Source location	Lines 807-814 (8 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function anchorDialogForResize(dialog) {
Touches	browser/Electron DOM
Calls detected	anchorDialogForDrag, getBoundingClientRect, add
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
807: function anchorDialogForResize(dialog) {
808:   anchorDialogForDrag(dialog);
809:   const rect = dialog.getBoundingClientRect();
810:   dialog.classList.add("is-resized-dialog");
811:   dialog.style.width = `${rect.width}px`;
812:   dialog.style.height = `${rect.height}px`;
813:   dialog.style.maxHeight = "none";
814: }
```

resizeDialogBounds - lines 816-843

Item	Explanation
Source location	Lines 816-843 (28 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function resizeDialogBounds(state, event) {
Touches	Mostly local calculations or local UI state.
Calls detected	min, max, includes
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
816: function resizeDialogBounds(state, event) {
817:   const margin = 12;
818:   const direction = state.direction;
819:   const minWidth = Math.min(320, Math.max(180, window.innerWidth - margin * 2));
820:   const minHeight = Math.min(170, Math.max(120, window.innerHeight - margin * 2));
821:   let { left, top, width, height } = state.rect;
```

```

822:   const dx = event.clientX - state.startX;
823:   const dy = event.clientY - state.startY;
824:
825:   if (direction.includes("e")) {
826:     width = Math.min(Math.max(state.rect.width + dx, minWidth), window.innerWidth - left - margin);
827:   }
828:   if (direction.includes("s")) {
829:     height = Math.min(Math.max(state.rect.height + dy, minHeight), window.innerHeight - top - margin);
830:   }
831:   if (direction.includes("w")) {
832:     const right = state.rect.left + state.rect.width;
833:     left = Math.min(Math.max(state.rect.left + dx, margin), right - minWidth);
834:     width = right - left;
835:   }
836:   if (direction.includes("n")) {
837:     const bottom = state.rect.top + state.rect.height;
838:     top = Math.min(Math.max(state.rect.top + dy, margin), bottom - minHeight);
839:     height = bottom - top;
840:   }
841:
842:   return { height, left, top, width };
843: }

```

beginDialogResize - lines 845-863

Item	Explanation
Source location	Lines 845-863 (19 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function beginDialogResize(event) {
Touches	browser/Electron DOM
Calls detected	closest, contains, anchorDialogForResize, getBoundingClientRect, add, preventDefault
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

845: function beginDialogResize(event) {
846:   if (event.button !== 0) return;
847:   const handle = event.target.closest("[data-dialog-resize-handle]");
848:   const dialog = handle?.closest("dialog");
849:   if (!handle || !dialog?.open || dialog.classList.contains("is-minimized-dialog") ||
dialog.classList.contains("is-hidden-dialog")) return;
850:   anchorDialogForResize(dialog);
851:   activeDialogResize = {
852:     dialog,
853:     direction: handle.dataset.dialogResizeHandle,
854:     pointerId: event.pointerId,
855:     pointerTarget: event.target,
856:     rect: dialog.getBoundingClientRect(),
857:     startX: event.clientX,
858:     startY: event.clientY
859:   };
860:   event.target.setPointerCapture?.(event.pointerId);
861:   dialog.classList.add("is-resizing-dialog");
862:   event.preventDefault();
863: }

```

moveDialogResize - lines 865-872

Item	Explanation
Source location	Lines 865-872 (8 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function moveDialogResize(event) {
Touches	Mostly local calculations or local UI state.
Calls detected	resizeDialogBounds
API endpoints	None detected inside this function body.

Item	Explanation
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

865: function moveDialogResize(event) {
866:   if (!activeDialogResize) return;
867:   const next = resizeDialogBounds(activeDialogResize, event);
868:   activeDialogResize.dialog.style.left = `${next.left}px`;
869:   activeDialogResize.dialog.style.top = `${next.top}px`;
870:   activeDialogResize.dialog.style.width = `${next.width}px`;
871:   activeDialogResize.dialog.style.height = `${next.height}px`;
872: }

```

endDialogResize - lines 874-879

Item	Explanation
Source location	Lines 874-879 (6 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function endDialogResize() {
Touches	browser/Electron DOM
Calls detected	remove
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

874: function endDialogResize() {
875:   if (!activeDialogResize) return;
876:   activeDialogResize.pointerTarget?.releasePointerCapture?.(activeDialogResize.pointerId);
877:   activeDialogResize.dialog.classList.remove("is-resizing-dialog");
878:   activeDialogResize = null;
879: }

```

beginDialogDrag - lines 881-897

Item	Explanation
Source location	Lines 881-897 (17 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function beginDialogDrag(event) {
Touches	browser/Electron DOM
Calls detected	draggableDialogFromEvent, anchorDialogForDrag, getBoundingClientRect, add, preventDefault
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

881: function beginDialogDrag(event) {
882:   if (event.button !== 0) return;
883:   const dialog = draggableDialogFromEvent(event);
884:   if (!dialog) return;
885:   anchorDialogForDrag(dialog);

```

```

886:   const rect = dialog.getBoundingClientRect();
887:   activeDialogDrag = {
888:     dialog,
889:     offsetX: event.clientX - rect.left,
890:     offsetY: event.clientY - rect.top,
891:     pointerId: event.pointerId,
892:     pointerTarget: event.target
893:   };
894:   event.target.setPointerCapture?.(event.pointerId);
895:   dialog.classList.add("is-dragging-dialog");
896:   event.preventDefault();
897: }

```

moveDialogDrag - lines 899-908

Item	Explanation
Source location	Lines 899-908 (10 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function moveDialogDrag(event) {
Touches	Mostly local calculations or local UI state.
Calls detected	clampDialogPosition
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

899: function moveDialogDrag(event) {
900:   if (!activeDialogDrag) return;
901:   const next = clampDialogPosition(
902:     event.clientX - activeDialogDrag.offsetX,
903:     event.clientY - activeDialogDrag.offsetY,
904:     activeDialogDrag.dialog
905:   );
906:   activeDialogDrag.dialog.style.left = `${next.left}px`;
907:   activeDialogDrag.dialog.style.top = `${next.top}px`;
908: }

```

endDialogDrag - lines 910-915

Item	Explanation
Source location	Lines 910-915 (6 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function endDialogDrag() {
Touches	browser/Electron DOM
Calls detected	remove
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

910: function endDialogDrag() {
911:   if (!activeDialogDrag) return;
912:   activeDialogDrag.pointerTarget?.releasePointerCapture?.(activeDialogDrag.pointerId);
913:   activeDialogDrag.dialog.classList.remove("is-dragging-dialog");
914:   activeDialogDrag = null;
915: }

```

toggleDialogMinimized - lines 917-927

Item	Explanation
Source location	Lines 917-927 (11 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function toggleDialogMinimized(dialog) {
Touches	browser/Electron DOM
Calls detected	contains, toggleDialogHidden, anchorDialogForDrag, remove, toggle, updateDialogWindowButtons
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

917: function toggleDialogMinimized(dialog) {
918:   if (!dialog) return;
919:   if (dialog.classList.contains("is-hidden-dialog")) {
920:     toggleDialogHidden(dialog);
921:     return;
922:   }
923:   anchorDialogForDrag(dialog);
924:   dialog.classList.remove("is-maximized-dialog");
925:   dialog.classList.toggle("is-minimized-dialog");
926:   updateDialogWindowButtons(dialog);
927: }

```

toggleDialogMaximized - lines 929-949

Item	Explanation
Source location	Lines 929-949 (21 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function toggleDialogMaximized(dialog) {
Touches	browser/Electron DOM
Calls detected	contains, remove, restoreDialogWindowState, saveDialogRestoreState, add, updateDialogWindowButtons
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

929: function toggleDialogMaximized(dialog) {
930:   if (!dialog) return;
931:   if (dialog.classList.contains("is-hidden-dialog")) dialog.classList.remove("is-hidden-dialog");
932:   if (dialog.classList.contains("is-maximized-dialog")) {
933:     dialog.classList.remove("is-maximized-dialog");
934:     restoreDialogWindowState(dialog);
935:   } else {
936:     saveDialogRestoreState(dialog);
937:     dialog.classList.remove("is-minimized-dialog");
938:     dialog.classList.add("is-draggable-dialog", "is-maximized-dialog");
939:     dialog.style.left = "0";
940:     dialog.style.top = "0";
941:     dialog.style.right = "auto";
942:     dialog.style.bottom = "auto";
943:     dialog.style.margin = "0";
944:     dialog.style.width = "100vw";
945:     dialog.style.height = "100dvh";
946:     dialog.style.maxHeight = "none";
947:   }
948:   updateDialogWindowButtons(dialog);
949: }

```

toggleDialogHidden - lines 951-970

Item	Explanation
Source location	Lines 951-970 (20 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function toggleDialogHidden(dialog) {
Touches	browser/Electron DOM
Calls detected	contains, remove, restoreDialogWindowState, saveDialogRestoreState, add, min, calc, updateDialogWindowButtons
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

951: function toggleDialogHidden(dialog) {
952:   if (!dialog) return;
953:   if (dialog.classList.contains("is-hidden-dialog")) {
954:     dialog.classList.remove("is-hidden-dialog");
955:     restoreDialogWindowState(dialog);
956:   } else {
957:     saveDialogRestoreState(dialog);
958:     dialog.classList.remove("is-minimized-dialog", "is-maximized-dialog");
959:     dialog.classList.add("is-draggable-dialog", "is-hidden-dialog");
960:     dialog.style.left = "auto";
961:     dialog.style.top = "auto";
962:     dialog.style.right = "12px";
963:     dialog.style.bottom = "10px";
964:     dialog.style.margin = "0";
965:     dialog.style.width = "min(300px, calc(100vw - 24px))";
966:     dialog.style.height = "32px";
967:     dialog.style.maxHeight = "32px";
968:   }
969:   updateDialogWindowButtons(dialog);
970: }

```

dialogCanStepBack - lines 972-976

Item	Explanation
Source location	Lines 972-976 (5 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function dialogCanStepBack(dialog) {
Touches	Mostly local calculations or local UI state.
Calls detected	No obvious named calls detected by the generator.
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 3 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

972: function dialogCanStepBack(dialog) {
973:   if (dialog === projectDialog) return projectWindowBackStack.length > 0;
974:   if (dialog === projectPreviewDialog) return activeProjectPreviewSectionIndex >= 0;
975:   return false;
976: }

```

dialogCanStepForward - lines 978-982

Item	Explanation
Source location	Lines 978-982 (5 source lines).
Purpose	Part of the private builder frontend and editing experience.

Item	Explanation
Declaration	function dialogCanStepForward(dialog) {
Touches	Mostly local calculations or local UI state.
Calls detected	No obvious named calls detected by the generator.
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 3 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

978: function dialogCanStepForward(dialog) {
979:   if (dialog === projectDialog) return projectWindowForwardStack.length > 0;
980:   if (dialog === projectPreviewDialog) return activeProjectPreviewForwardStack.length > 0;
981:   return false;
982: }

```

renderProjectWindowSectionFromHistory - lines 984-990

Item	Explanation
Source location	Lines 984-990 (7 source lines).
Purpose	Renders UI, markup, or display output.
Declaration	function renderProjectWindowSectionFromHistory(sectionId) {
Touches	Mostly local calculations or local UI state.
Calls detected	renderAll, updateDialogWindowButtons
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

984: function renderProjectWindowSectionFromHistory(sectionId) {
985:   suppressProjectWindowHistory = true;
986:   activeSectionId = sectionId || "brief";
987:   renderAll();
988:   suppressProjectWindowHistory = false;
989:   updateDialogWindowButtons(projectDialog);
990: }

```

stepProjectWindowBack - lines 992-996

Item	Explanation
Source location	Lines 992-996 (5 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function stepProjectWindowBack() {
Touches	Mostly local calculations or local UI state.
Calls detected	push, renderProjectWindowSectionFromHistory, pop
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
992: function stepProjectWindowBack() {
993:   if (!projectWindowBackStack.length) return;
994:   projectWindowForwardStack.push(activeSectionId);
995:   renderProjectWindowSectionFromHistory(projectWindowBackStack.pop());
996: }
```

stepProjectWindowForward - lines 998-1002

Item	Explanation
Source location	Lines 998-1002 (5 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function stepProjectWindowForward() {
Touches	Mostly local calculations or local UI state.
Calls detected	push, renderProjectWindowSectionFromHistory, pop
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
998: function stepProjectWindowForward() {
999:   if (!projectWindowForwardStack.length) return;
1000:   projectWindowBackStack.push(activeSectionId);
1001:   renderProjectWindowSectionFromHistory(projectWindowForwardStack.pop());
1002: }
```

refreshDialogWindow - lines 1004-1015

Item	Explanation
Source location	Lines 1004-1015 (12 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function refreshDialogWindow(dialog) {
Touches	Mostly local calculations or local UI state.
Calls detected	saveVisibleRichEditors, refreshOpenPreviews, renderPreview, renderAll, setStatus, updateDialogWindowButtons
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
1004: function refreshDialogWindow(dialog) {
1005:   saveVisibleRichEditors();
1006:   if (dialog === projectPreviewDialog) {
1007:     refreshOpenPreviews();
1008:   } else if (dialog === portfolioPreviewDialog) {
1009:     renderPreview();
1010:   } else if (dialog === projectDialog) {
1011:     renderAll();
1012:   }
1013:   setStatus("Window refreshed.");
1014:   updateDialogWindowButtons(dialog);
1015: }
```

closeDialogFromControl - lines 1017-1025

Item	Explanation
Source location	Lines 1017-1025 (9 source lines).
Purpose	Controls a window, dialog, panel, menu, or visible state.
Declaration	function closeDialogFromControl(dialog, button) {
Touches	browser/Electron DOM
Calls detected	querySelectorAll, find, contains, click, closeDialogElement
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

1017: function closeDialogFromControl(dialog, button) {
1018:   const originalClose = [...dialog.querySelectorAll(".project-window-close, .close-control")]
1019:     .find((item) => item !== button && !item.dataset.dialogClose);
1020:   if (originalClose && !button.classList.contains("close-control") && !button.id) {
1021:     originalClose.click();
1022:     return;
1023:   }
1024:   closeDialogElement(dialog, "close");
1025: }

```

handleDialogWindowAction - lines 1027-1044

Item	Explanation
Source location	Lines 1027-1044 (18 source lines).
Purpose	Handles an event, request, command, or user action.
Declaration	function handleDialogWindowAction(button) {
Touches	browser/Electron DOM
Calls detected	closest, toggleDialogHidden, toggleDialogMinimized, toggleDialogMaximized, refreshDialogWindow, stepProjectWindowBack, projectPreviewBackStep, stepProjectWindowForward, projectPreviewForwardStep, closeDialogFromControl
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

1027: function handleDialogWindowAction(button) {
1028:   const dialog = button.closest("dialog");
1029:   if (!dialog?.open) return;
1030:   const action = button.dataset.dialogAction;
1031:   if (action === "hide") toggleDialogHidden(dialog);
1032:   if (action === "minimize") toggleDialogMinimized(dialog);
1033:   if (action === "maximize") toggleDialogMaximized(dialog);
1034:   if (action === "refresh") refreshDialogWindow(dialog);
1035:   if (action === "back") {
1036:     if (dialog === projectDialog) stepProjectWindowBack();
1037:     else if (dialog === projectPreviewDialog) projectPreviewBackStep();
1038:   }
1039:   if (action === "forward") {
1040:     if (dialog === projectDialog) stepProjectWindowForward();
1041:     else if (dialog === projectPreviewDialog) projectPreviewForwardStep();
1042:   }
1043:   if (action === "close") closeDialogFromControl(dialog, button);
1044: }

```

enableDraggableDialogs - lines 1046-1084

Item	Explanation
Source location	Lines 1046-1084 (39 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function enableDraggableDialogs() {
Touches	browser/Electron DOM, filesystem
Calls detected	querySelectorAll, forEach, addEventListener, closest, handleDialogWindowAction, toggleDialogHidden, remove, updateDialogWindowButtons, getBoundingClientRect, clampDialogPosition
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

1046: function enableDraggableDialogs() {
1047:   document.querySelectorAll("dialog").forEach(markDialogDragHandles);
1048:   document.addEventListener("pointerdown", beginDialogResize);
1049:   document.addEventListener("pointerdown", beginDialogDrag);
1050:   document.addEventListener("pointermove", moveDialogResize);
1051:   document.addEventListener("pointermove", moveDialogDrag);
1052:   document.addEventListener("pointerup", endDialogResize);
1053:   document.addEventListener("pointerup", endDialogDrag);
1054:   document.addEventListener("pointercancel", endDialogResize);
1055:   document.addEventListener("pointercancel", endDialogDrag);
1056:   document.addEventListener("click", (event) => {
1057:     const button = event.target.closest("[data-dialog-action]");
1058:     if (!button) return;
1059:     handleDialogWindowAction(button);
1060:   });
1061:   document.addEventListener("click", (event) => {
1062:     const hiddenDialog = event.target.closest("dialog.is-hidden-dialog");
1063:     if (!hiddenDialog || event.target.closest("button")) return;
1064:     toggleDialogHidden(hiddenDialog);
1065:   });
1066:   document.querySelectorAll("dialog").forEach((dialog) => {
1067:     dialog.addEventListener("close", () => {
1068:       dialog.classList.remove("is-minimized-dialog");
1069:       dialog.classList.remove("is-hidden-dialog");
1070:       dialog.classList.remove("is-maximized-dialog");
1071:       dialog.classList.remove("is-resizing-dialog");
1072:       delete dialog.dataset.restoreWindowState;
1073:       updateDialogWindowButtons(dialog);
1074:     });
1075:   });
1076:   window.addEventListener("resize", () => {
1077:     document.querySelectorAll("dialog.is-draggable-dialog[open]").forEach((dialog) => {
1078:       const rect = dialog.getBoundingClientRect();
1079:       const next = clampDialogPosition(rect.left, rect.top, dialog);
1080:       dialog.style.left = `${next.left}px`;
1081:       dialog.style.top = `${next.top}px`;
1082:     });
1083:   });
1084: }

```

markDraftNeedsSave - lines 1086-1089

Item	Explanation
Source location	Lines 1086-1089 (4 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function markDraftNeedsSave() {
Touches	Mostly local calculations or local UI state.
Calls detected	setSaveState
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 0 return statement(s).

Item	Explanation
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

1086: function markDraftNeedsSave() {
1087:   draftSavedSinceChanges = false;
1088:   setSaveState("dirty", "Save draft needed");
1089: }

```

showBuilderError - lines 1091-1097

Item	Explanation
Source location	Lines 1091-1097 (7 source lines).
Purpose	Controls a window, dialog, panel, menu, or visible state.
Declaration	function showBuilderError(title, message, details = "") {
Touches	Mostly local calculations or local UI state.
Calls detected	showModal
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

1091: function showBuilderError(title, message, details = "") {
1092:   publishResultEyebrow.textContent = "Builder check";
1093:   publishResultTitle.textContent = title;
1094:   publishResultMessage.textContent = message;
1095:   publishResultOutput.textContent = details;
1096:   publishResultDialog.showModal();
1097: }

```

appUpdateWasSnoozed - lines 1105-1110

Item	Explanation
Source location	Lines 1105-1110 (6 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function appUpdateWasSnoozed(version = "") {
Touches	Mostly local calculations or local UI state.
Calls detected	getItem, now
API endpoints	None detected inside this function body.
Storage keys	omb-snoozed-update-at, omb-snoozed-update-version
Async/return clues	0 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

1105: function appUpdateWasSnoozed(version = "") {
1106:   if (!version) return false;
1107:   const snoozedVersion = localStorage.getItem("omb-snoozed-update-version") || "";
1108:   const snoozedAt = Number(localStorage.getItem("omb-snoozed-update-at") || 0);
1109:   return snoozedVersion === version && Date.now() - snoozedAt < APP_UPDATE_SNOOZE_MS;
1110: }

```

shouldShowUpdateDialog - lines 1112-1117

Item	Explanation
Source location	Lines 1112-1117 (6 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function shouldShowUpdateDialog(update = {}) {
Touches	Mostly local calculations or local UI state.
Calls detected	getItem, appUpdateWasSnoozed
API endpoints	None detected inside this function body.
Storage keys	omb-dismissed-update-version, omb-skipped-update-version
Async/return clues	0 await expression(s), 3 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

1112: function shouldShowUpdateDialog(update = {}) {
1113:   if (!update.updateAvailable || !update.latestVersion) return false;
1114:   const skippedVersion = localStorage.getItem("omb-skipped-update-version") || localStorage.getItem("omb-dismissed-update-version") || "";
1115:   if (skippedVersion === update.latestVersion) return false;
1116:   return !appUpdateWasSnoozed(update.latestVersion);
1117: }

```

showUpdateDialog - lines 1119-1158

Item	Explanation
Source location	Lines 1119-1158 (40 source lines).
Purpose	Controls a window, dialog, panel, menu, or visible state.
Declaration	function showUpdateDialog(update = {}, options = {}) {
Touches	Mostly local calculations or local UI state.
Calls detected	shouldShowUpdateDialog, escapeHtml, filter, join, showModal, show
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

1119: function showUpdateDialog(update = {}, options = {}) {
1120:   const force = Boolean(options.force);
1121:   if (!appUpdateDialog || (!force && !shouldShowUpdateDialog(update))) return;
1122:   pendingAppUpdate = update;
1123:   const hasInstaller = Boolean(update.installerUrl);
1124:   const updateBlocked = Boolean(update.updateBlocked);
1125:   const hasUpdate = Boolean(update.updateAvailable && update.latestVersion);
1126:   appUpdateTitle.textContent = updateBlocked ? "Update paused" : hasUpdate ? "Update available" :
"Builder is up to date";
1127:   appUpdateMessage.textContent = updateBlocked
1128:     ? `OMB Portfolio Builder ${update.latestVersion || "this release"} is available, but this version is
paused for automatic updates.`
1129:     : hasUpdate
1130:     ? `OMB Portfolio Builder ${update.latestVersion} is available. You are running
${update.currentVersion || "an older version"}.`
1131:     : `You are running OMB Portfolio Builder ${update.currentVersion || "the current installed
version"}.`;
1132:   appUpdateDetails.textContent = updateBlocked
1133:     ? (update.blockedReason || "This release is temporarily blocked from automatic updates. Check again
after the next release is available.")
1134:     : hasUpdate
1135:     ? "Choose Update to download the latest installer, close this app, and install the new version
automatically. Remind me later postpones the prompt; Skip this version ignores this release."
1136:     : "No newer released builder was found on GitHub Releases.";
1137:   appUpdateReleaseMeta.innerHTML = (hasUpdate || updateBlocked)
1138:     ? [
1139:       `<span>Current version: ${escapeHtml(update.currentVersion || "unknown")}</span>`,
1140:       `<span>Available version: ${escapeHtml(update.latestVersion || "unknown")}</span>`,
1141:       `update.releaseUrl ? `<span>Release source: GitHub Releases</span>` : ""

```

```

1142:     ].filter(Boolean).join("")
1143:     : [
1144:       `<span>Current version: ${escapeHtml(update.currentVersion || "unknown")}</span>`,
1145:       update.checkedAt ? `<span>Checked: ${escapeHtml(update.checkedAt)}</span>` : ""
1146:     ].filter(Boolean).join("");
1147: appUpdateLater.textContent = hasUpdate ? "Remind me later" : "Close";
1148: appUpdateSkip.hidden = !hasUpdate;
1149: appUpdateApply.hidden = !hasUpdate;
1150: appUpdateApply.disabled = hasUpdate && !hasInstaller;
1151: appUpdateApply.textContent = hasInstaller ? "Update" : "Open release page";
1152: if (appUpdateDialog.open) return;
1153: try {
1154:   appUpdateDialog.showModal();
1155: } catch {
1156:   appUpdateDialog.show();
1157: }
1158: }

```

startAppUpdateInstall - lines 1160-1193

Item	Explanation
Source location	Lines 1160-1193 (34 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	async function startAppUpdateInstall() {
Touches	network/API requests
Calls detected	fetch, now, stringify, json, Error, showBuilderError
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	2 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

1160: async function startAppUpdateInstall() {
1161:   const update = pendingAppUpdate || {};
1162:   appUpdateApply.disabled = true;
1163:   appUpdateLater.disabled = true;
1164:   appUpdateSkip.disabled = true;
1165:   appUpdateApply.textContent = "Updating...";
1166:   appUpdateDetails.textContent = "Downloading the installer. The builder will close automatically, then
the installer will update the app.";
1167:   try {
1168:     const response = await fetch(`/api/app-update/install?t=${Date.now()}`, {
1169:       method: "POST",
1170:       cache: "no-store",
1171:       headers: { "Content-Type": "application/json" },
1172:       body: JSON.stringify({ latestVersion: update.latestVersion || "" })
1173:     });
1174:     const result = await response.json();
1175:     if (!response.ok || !result.ok) throw new Error(result.error || "The update could not be started.");
1176:     appUpdateMessage.textContent = "Update started";
1177:     appUpdateDetails.textContent = "The installer is ready. This window will close so the new version can
be installed.";
1178:   } catch (error) {
1179:     appUpdateApply.disabled = false;
1180:     appUpdateLater.disabled = false;
1181:     appUpdateSkip.disabled = false;
1182:     appUpdateApply.textContent = "Update";
1183:     appUpdateDetails.textContent = error.message || "The update could not be started.";
1184:     const fallback = update.releaseUrl || update.installerUrl || update.portableUrl || "";
1185:     if (fallback) {
1186:       showBuilderError(
1187:         "Automatic update could not start",
1188:         "The app could not run the installer automatically. The release page can still be opened
manually.",
1189:         fallback
1190:       );
1191:     }
1192:   }
1193: }

```

checkForAppUpdates - lines 1195-1235

Item	Explanation
Source location	Lines 1195-1235 (41 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	async function checkForAppUpdates(options = {}) {
Touches	network/API requests
Calls detected	now, fetch, json, showUpdateDialog, toLocaleString
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	2 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

1195: async function checkForAppUpdates(options = {}) {
1196:   const force = Boolean(options.force);
1197:   const manual = Boolean(options.manual);
1198:   if (!force && Date.now() - lastAppUpdateCheckAt < 5 * 60 * 1000) return;
1199:   lastAppUpdateCheckAt = Date.now();
1200:   if (manual) {
1201:     if (appUpdateCheckButton) {
1202:       appUpdateCheckButton.disabled = true;
1203:       appUpdateCheckButton.textContent = "Checking...";
1204:     }
1205:   }
1206:   try {
1207:     const response = await fetch(`/api/app-update?t=${Date.now()}`, { cache: "no-store" });
1208:     const update = await response.json();
1209:     if (response.ok && update.ok !== false) {
1210:       showUpdateDialog({ ...update, checkedAt: new Date().toLocaleString() }, { force: manual });
1211:     } else if (manual) {
1212:       showUpdateDialog({
1213:         currentVersion: update.currentVersion || "unknown",
1214:         updateAvailable: false,
1215:         checkedAt: new Date().toLocaleString()
1216:       }, { force: true });
1217:     }
1218:   } catch {
1219:     if (manual) {
1220:       showUpdateDialog({
1221:         currentVersion: "unknown",
1222:         updateAvailable: false,
1223:         checkedAt: new Date().toLocaleString()
1224:       }, { force: true });
1225:     }
1226:     // Update checks should never interrupt builder work.
1227:   } finally {
1228:     if (manual) {
1229:       if (appUpdateCheckButton) {
1230:         appUpdateCheckButton.disabled = false;
1231:         appUpdateCheckButton.textContent = "Check updates";
1232:       }
1233:     }
1234:   }
1235: }

```

formatReportDate - lines 1237-1240

Item	Explanation
Source location	Lines 1237-1240 (4 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function formatReportDate(value = "") {
Touches	filesystem
Calls detected	isNaN, getTime, toLocaleString
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).

Item	Explanation
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

1237: function formatReportDate(value = "") {
1238:   const date = new Date(value);
1239:   return Number.isNaN(date.getTime()) ? "" : date.toLocaleString();
1240: }

```

renderSecurityReport - lines 1242-1294

Item	Explanation
Source location	Lines 1242-1294 (53 source lines).
Purpose	Renders UI, markup, or display output.
Declaration	function renderSecurityReport(report = {}) {
Touches	filesystem, authentication/security data
Calls detected	isArray, map, escapeHtml, toLocaleString, join, formatReportDate
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

1242: function renderSecurityReport(report = {}) {
1243:   if (!securityReportBody) return;
1244:   const downloads = report.appDownloads || {};
1245:   const assets = Array.isArray(downloads.assets) ? downloads.assets : [];
1246:   const auth = report.publishingAuth || {};
1247:   const access = report.websiteAccess || {};
1248:   const controls = Array.isArray(report.securityControls) ? report.securityControls : [];
1249:   const assetRows = assets.length
1250:     ? assets.map((asset) => `
1251:       <li>
1252:         <span>${escapeHtml(asset.name || "Release asset")}</span>
1253:         <span>${Number(asset.downloadCount || 0).toLocaleString()} downloads</span>
1254:       </li>
1255:     `).join("")
1256:   : `<li><span>No release assets found</span><span>0 downloads</span></li>`;
1257:   securityReportBody.innerHTML = `
1258:     <div class="security-report-grid">
1259:       <section class="security-report-card">
1260:         <h3>Builder downloads</h3>
1261:         <strong>${downloads.ok ? Number(downloads.totalDownloads || 0).toLocaleString() :
"Unavailable"}</strong>
1262:         <p>${downloads.ok
1263:           ? `Latest release: ${escapeHtml(downloads.releaseTag || downloads.releaseName || "latest")}`
1264:           : escapeHtml(downloads.error || "GitHub release counts are not available.")}</p>
1265:         <ul class="security-report-assets">${assetRows}</ul>
1266:       </section>
1267:       <section class="security-report-card">
1268:         <h3>Publishing authorization</h3>
1269:         <strong>${auth.authenticated ? "Active" : "Not active"}</strong>
1270:         <p>${auth.authenticated
1271:           ? `This builder has a remembered GitHub publishing session until
${escapeHtml(formatReportDate(auth.expiresAt) || auth.expiresAt || "the cache expires")}.`
1272:           : "No fresh GitHub publishing session is cached for the current target."}</p>
1273:         <ul>
1274:           <li>Target: ${escapeHtml(auth.targetRepository || "No publishing target saved")}</li>
1275:           <li>Branch: ${escapeHtml(auth.branch || "Not set")}</li>
1276:           <li>Successful authorizations this week: ${Number(auth.successCountLastWeek ||
0).toLocaleString()}</li>
1277:         </ul>
1278:       </section>
1279:       <section class="security-report-card">
1280:         <h3>Website access reporting</h3>
1281:         <strong>${access.available ? "Configured" : "Needs Cloudflare"}</strong>
1282:         <p>${escapeHtml(access.summary || "Visitor and IP reporting needs a server-side analytics
source.")}</p>
1283:         <ul>${(access.recommendedSetup || []).map((item) =>
1284:           <li>${escapeHtml(item)}</li>`)}</ul>
1285:       </section>
1286:       <section class="security-report-card">

```

```

1286:         <h3>Security controls</h3>
1287:         <strong>${controls.length.toLocaleString()}</strong>
1288:         <p>Controls currently enabled or prepared for publishing.</p>
1289:         <ul>${controls.map((item) => `<li>${escapeHtml(item)}</li>`).join("")}</ul>
1290:     </section>
1291: </div>
1292: <p>Generated ${escapeHtml(formatReportDate(report.generatedAt) || "just now")}</p>
1293: `;
1294: }

```

openSecurityReport - lines 1296-1314

Item	Explanation
Source location	Lines 1296-1314 (19 source lines).
Purpose	Controls a window, dialog, panel, menu, or visible state.
Declaration	async function openSecurityReport() {
Touches	network/API requests
Calls detected	showModal, updateDialogWindowButtons, fetch, now, json, Error, renderSecurityReport, escapeHtml
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	2 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

1296: async function openSecurityReport() {
1297:   if (!securityReportDialog || !securityReportBody) return;
1298:   securityReportBody.innerHTML = "<p>Loading security report...</p>";
1299:   if (!securityReportDialog.open) securityReportDialog.showModal();
1300:   updateDialogWindowButtons(securityReportDialog);
1301:   try {
1302:     const response = await fetch(`/api/security-report?t=${Date.now()}`, { cache: "no-store" });
1303:     const report = await response.json();
1304:     if (!response.ok || report.ok === false) throw new Error(report.error || "Security report could not
be loaded.");
1305:     renderSecurityReport(report);
1306:   } catch (error) {
1307:     securityReportBody.innerHTML = `
1308:       <div class="security-report-card">
1309:         <h3>Security report unavailable</h3>
1310:         <p>${escapeHtml(error.message || "The report could not be loaded.")}</p>
1311:       </div>
1312:     `;
1313:   }
1314: }

```

scheduleAppUpdateChecks - lines 1316-1325

Item	Explanation
Source location	Lines 1316-1325 (10 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function scheduleAppUpdateChecks() {
Touches	browser/Electron DOM
Calls detected	checkForAppUpdates, addEventListener, now
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

1316: function scheduleAppUpdateChecks() {
1317:   checkForAppUpdates({ force: true });
1318:   window.setInterval(() => checkForAppUpdates({ force: true }), APP_UPDATE_CHECK_INTERVAL_MS);

```

```

1319: document.addEventListener("visibilitychange", () => {
1320:   if (document.visibilityState !== "visible") return;
1321:   if (Date.now() - lastAppUpdateCheckAt >= APP_UPDATE_FOCUS_RECHECK_MS) {
1322:     checkForAppUpdates({ force: true });
1323:   }
1324: });
1325: }

```

renderPublishTargetInfo - lines 1327-1353

Item	Explanation
Source location	Lines 1327-1353 (27 source lines).
Purpose	Renders UI, markup, or display output.
Declaration	function renderPublishTargetInfo(target = {}) {
Touches	browser/Electron DOM, authentication/security data
Calls detected	toLocaleString, map, escapeHtml, join
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

1327: function renderPublishTargetInfo(target = {}) {
1328:   if (!publishTargetCurrent) return;
1329:   currentPublishTarget = target;
1330:   const checkedText = target.checkedAt ? ` at ${new Date(target.checkedAt).toLocaleString()}` : "";
1331:   const expiresText = target.expiresAt ? `; remembered until ${new
Date(target.expiresAt).toLocaleString()}` : "";
1332:   const trustText = target.extendedTrust ? " (30-day trusted target)" : target.authorizationCached ? "
(daily cache)" : "";
1333:   const authorizationStatus = target.authorizationChecked
1334:     ? `Verified${checkedText}${trustText}${expiresText}`
1335:     : "Not verified for this repository and branch";
1336:   const rows = [
1337:     ["Builder workspace", target.workspace || "Not available"],
1338:     ["Portfolio workspace", target.portfolioWorkspace || target.workspace || "Not available"],
1339:     ["Repository", target.repository || target.remote || "No repository connected"],
1340:     ["Branch", target.branch || "No branch selected"],
1341:     ["GitHub authorization", authorizationStatus],
1342:     ["Compatible site", target.compatible ? "Yes" : "No"],
1343:     ["Git-backed", target.gitBacked ? "Yes" : "No"]
1344:   ];
1345:   publishTargetCurrent.innerHTML = rows
1346:     .map(([label, value]) =>
`<div><strong>${escapeHtml(label)}</strong><span>${escapeHtml(value)}</span></div>`)
1347:     .join("");
1348:   if (publishTargetRepository) {
1349:     publishTargetRepository.value = target.remote || "";
1350:     publishTargetRepository.dataset.loadedValue = target.remote || "";
1351:   }
1352:   if (publishTargetSync) publishTargetSync.disabled = !target.authorizationChecked;
1353: }

```

renderSystemReadiness - lines 1355-1394

Item	Explanation
Source location	Lines 1355-1394 (40 source lines).
Purpose	Renders UI, markup, or display output.
Declaration	function renderSystemReadiness(system = {}, target = {}) {
Touches	authentication/security data
Calls detected	toLocaleString, map, escapeHtml, join
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).

Item	Explanation
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

1355: function renderSystemReadiness(system = {}, target = {}) {
1356:   if (!publishTargetReadinessList) return;
1357:   const rows = [
1358:     {
1359:       ok: true,
1360:       text: `Electron/Node runtime: bundled with the app${system.nodeRuntime?.version ? `
1361: (${system.nodeRuntime.version})` : ""}.`
1362:     },
1363:     {
1364:       ok: system.pnpm?.required === false,
1365:       text: "pnpm: not required for installed users."
1366:     },
1367:     {
1368:       ok: Boolean(system.git?.ok),
1369:       text: system.git?.ok
1370:       ? `Git for Windows: ${system.git.version || "installed"}.`
1371:       : "Git for Windows: missing. Install it before loading or publishing a target."
1372:     },
1373:     {
1374:       ok: Boolean(system.credentialManager?.ok),
1375:       text: system.credentialManager?.ok
1376:       ? `Git Credential Manager: ${system.credentialManager.version || "installed"}.`
1377:       : "Git Credential Manager: missing. Install Git for Windows with Git Credential Manager enabled."
1378:     },
1379:     {
1380:       ok: Boolean(target?.remote),
1381:       text: target?.remote
1382:       ? `Publishing target: ${target.repository || target.remote}.`
1383:       : "Publishing target: enter a repository URL and click Authenticate target."
1384:     },
1385:     {
1386:       ok: Boolean(target?.authorizationChecked),
1387:       text: target?.authorizationChecked
1388:       ? `GitHub authorization: verified for ${target.branch || "the selected branch"}${target.expiresAt
1389: ? ` until ${new Date(target.expiresAt).toLocaleString()}` : ""}.`
1390:       : "GitHub authorization: required before Load from target or Apply to site."
1391:     }
1392:   ];
1393:   publishTargetReadinessList.innerHTML = rows
1394:     .map((row) => `<li class="${row.ok ? "ready" : "blocked"}">${escapeHtml(row.ok ? `OK - ${row.text}` :
1395: `Needs action - ${row.text}`)}</li>`)
1396:     .join("");
1397: }
1398: }
1399: }

```

markPublishTargetNeedsAuthentication - lines 1396-1405

Item	Explanation
Source location	Lines 1396-1405 (10 source lines).
Purpose	Part of publishing, Git, target repository, or authorization behavior.
Declaration	function markPublishTargetNeedsAuthentication() {
Touches	authentication/security data
Calls detected	No obvious named calls detected by the generator.
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

1396: function markPublishTargetNeedsAuthentication() {
1397:   if (publishTargetSync) publishTargetSync.disabled = true;
1398:   if (currentPublishTarget) {
1399:     currentPublishTarget = {
1400:       ...currentPublishTarget,
1401:       authorizationChecked: false,
1402:       authorizationCached: false
1403:     };
1404:   }
1405: }

```


loadSystemReadiness - lines 1407-1418

Item	Explanation
Source location	Lines 1407-1418 (12 source lines).
Purpose	Loads data from disk, network, browser storage, or a service.
Declaration	async function loadSystemReadiness() {
Touches	network/API requests
Calls detected	fetch, now, json, Error, renderSystemReadiness, escapeHtml
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	2 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
1407: async function loadSystemReadiness() {
1408:   if (!publishTargetReadinessList) return;
1409:   publishTargetReadinessList.innerHTML = "<li>Checking local publishing tools...</li>";
1410:   try {
1411:     const response = await fetch(`/api/system-check?t=${Date.now()}`, { cache: "no-store" });
1412:     const result = await response.json();
1413:     if (!response.ok || !result.ok) throw new Error(result.error || "System check failed.");
1414:     renderSystemReadiness(result.system || {}, result.target || currentPublishTarget || {});
1415:   } catch (error) {
1416:     publishTargetReadinessList.innerHTML = `<li class="blocked">${escapeHtml(error.message || "System
check failed.")}</li>`;
1417:   }
1418: }
```

setPublishTargetProgress - lines 1420-1426

Item	Explanation
Source location	Lines 1420-1426 (7 source lines).
Purpose	Part of publishing, Git, target repository, or authorization behavior.
Declaration	function setPublishTargetProgress(title, steps = []) {
Touches	Mostly local calculations or local UI state.
Calls detected	escapeHtml, toLocaleTimeString, map, join
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
1420: function setPublishTargetProgress(title, steps = []) {
1421:   if (!publishTargetCurrent) return;
1422:   publishTargetCurrent.innerHTML =
1423:     <div><strong>${escapeHtml(title)}</strong><span>${escapeHtml(new
Date().toLocaleTimeString())}</span></div>
1424:     ${steps.map((step) =>
`<div><strong>${escapeHtml(step.label)}</strong><span>${escapeHtml(step.value)}</span></div>`).join("")}
1425:   `;
1426: }
```

loadPublishTargetInfo - lines 1428-1439

Item	Explanation
Source location	Lines 1428-1439 (12 source lines).
Purpose	Loads data from disk, network, browser storage, or a service.

Item	Explanation
Declaration	async function loadPublishTargetInfo() {
Touches	network/API requests
Calls detected	fetch, now, json, Error, renderPublishTargetInfo
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	2 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

1428: async function loadPublishTargetInfo() {
1429:   if (!publishTargetDialog) return;
1430:   publishTargetCurrent.textContent = "Current target is loading.";
1431:   try {
1432:     const response = await fetch(`/api/publish-target?t=${Date.now()}`, { cache: "no-store" });
1433:     const result = await response.json();
1434:     if (!response.ok || !result.ok) throw new Error(result.error || "Publishing target could not be
loaded.");
1435:     renderPublishTargetInfo(result.target || {});
1436:   } catch (error) {
1437:     publishTargetCurrent.textContent = error.message || "Publishing target could not be loaded.";
1438:   }
1439: }

```

openPublishTargetDialog - lines 1441-1445

Item	Explanation
Source location	Lines 1441-1445 (5 source lines).
Purpose	Part of publishing, Git, target repository, or authorization behavior.
Declaration	async function openPublishTargetDialog() {
Touches	Mostly local calculations or local UI state.
Calls detected	loadPublishTargetInfo, loadSystemReadiness, showModal
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	2 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

1441: async function openPublishTargetDialog() {
1442:   await loadPublishTargetInfo();
1443:   await loadSystemReadiness();
1444:   publishTargetDialog.showModal();
1445: }

```

savePublishTarget - lines 1447-1450

Item	Explanation
Source location	Lines 1447-1450 (4 source lines).
Purpose	Persists data to local state, disk, credentials, or browser storage.
Declaration	async function savePublishTarget(event) {
Touches	authentication/security data
Calls detected	preventDefault, authenticatePublishTarget
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.

Item	Explanation
Async/return clues	1 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

1447: async function savePublishTarget(event) {
1448:   event.preventDefault();
1449:   await authenticatePublishTarget({ fromSave: true });
1450: }

```

currentPublishTargetPayload - lines 1452-1456

Item	Explanation
Source location	Lines 1452-1456 (5 source lines).
Purpose	Part of publishing, Git, target repository, or authorization behavior.
Declaration	function currentPublishTargetPayload() {
Touches	Mostly local calculations or local UI state.
Calls detected	No obvious named calls detected by the generator.
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

1452: function currentPublishTargetPayload() {
1453:   return {
1454:     repositoryUrl: publishTargetRepository?.value || ""
1455:   };
1456: }

```

installGitForPublishing - lines 1458-1481

Item	Explanation
Source location	Lines 1458-1481 (24 source lines).
Purpose	Part of publishing, Git, target repository, or authorization behavior.
Declaration	async function installGitForPublishing() {
Touches	network/API requests, authentication/security data
Calls detected	fetch, now, json, Error, open, loadSystemReadiness, showBuilderError
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	3 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

1458: async function installGitForPublishing() {
1459:   publishTargetCurrent.textContent = "Starting Git for Windows setup...";
1460:   try {
1461:     const response = await fetch(`/api/install-git?t=${Date.now()}`, {
1462:       method: "POST",
1463:       cache: "no-store",
1464:       headers: { "Content-Type": "application/json" },
1465:       body: "{}"
1466:     });
1467:     const result = await response.json();
1468:     if (!response.ok || !result.ok) throw new Error(result.error || "Git installer could not be
started.");

```

```

1469:     if (!result.install?.launched && result.install?.downloadUrl) {
1470:         window.open(result.install.downloadUrl, "_blank", "noreferrer");
1471:     }
1472:     await loadSystemReadiness();
1473:     showBuilderError(
1474:         result.install?.launched ? "Git setup started" : "Open Git download",
1475:         result.install?.message || "Install Git for Windows with Git Credential Manager enabled, then
reopen the builder.",
1476:         result.install?.downloadUrl || ""
1477:     );
1478: } catch (error) {
1479:     publishTargetCurrent.textContent = error.message || "Git installer could not be started.";
1480: }
1481: }

```

authenticatePublishTarget - lines 1483-1536

Item	Explanation
Source location	Lines 1483-1536 (54 source lines).
Purpose	Part of publishing, Git, target repository, or authorization behavior.
Declaration	async function authenticatePublishTarget(options = {}) {
Touches	network/API requests, authentication/security data
Calls detected	setPublishTargetProgress, fetch, now, stringify, currentPublishTargetPayload, json, Error, filter, join, renderPublishTargetInfo, renderSystemReadiness, showBuilderError
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	2 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

1483: async function authenticatePublishTarget(options = {}) {
1484:     const fromSave = Boolean(options.fromSave);
1485:     if (publishTargetSync) publishTargetSync.disabled = true;
1486:     setPublishTargetProgress(
1487:         fromSave ? "Authenticating publishing target" : "Checking GitHub authorization",
1488:         [
1489:             { label: "Step 1", value: "Validating the repository URL and local Git tools." },
1490:             { label: "Step 2", value: "Using the saved daily authorization if it is still valid." },
1491:             { label: "Step 3", value: "Finding the target repository default branch." },
1492:             { label: "Step 4", value: "Opening GitHub sign-in only when a fresh authorization is required." }
1493:         ]
1494:     );
1495:     try {
1496:         const response = await fetch(`/api/github-authenticate?t=${Date.now()}`, {
1497:             method: "POST",
1498:             cache: "no-store",
1499:             headers: { "Content-Type": "application/json" },
1500:             body: JSON.stringify(currentPublishTargetPayload())
1501:         });
1502:         const result = await response.json();
1503:         if (!response.ok || !result.ok) {
1504:             const message = result.error || "GitHub authentication did not complete.";
1505:             throw new Error([message, result.details].filter(Boolean).join("\n\n"));
1506:         }
1507:         renderPublishTargetInfo(result.target || {});
1508:         renderSystemReadiness(result.auth?.system || {}, result.target || {});
1509:         if (publishTargetPassword) publishTargetPassword.value = "";
1510:         const usedCachedAuth = Boolean(result.auth?.authorizationCached ||
result.target?.authorizationCached);
1511:         setPublishTargetProgress(
1512:             usedCachedAuth ? "Target authenticated from saved authorization" : "Target authenticated",
1513:             [
1514:                 { label: "Repository", value: result.target?.repository || result.auth?.repository || "Selected
target" },
1515:                 { label: "Branch", value: result.target?.branch || result.auth?.branch || "Detected target
branch" },
1516:                 { label: "Authorization", value: usedCachedAuth ? "A valid saved authorization was reused; no new
GitHub sign-in was needed." : "GitHub write access was verified now." },
1517:                 { label: "Next step", value: "Click Load from target when you want to import the latest
compatible website files into this builder." }
1518:             ]
1519:         );
1520:         if (publishTargetSync) publishTargetSync.disabled = false;
1521:     } catch (error) {
1522:         if (publishTargetSync) publishTargetSync.disabled = true;
1523:         setPublishTargetProgress(
1524:             "Target was not saved",

```

```

1525:     [
1526:       { label: "Reason", value: error.message || "GitHub authentication did not complete." },
1527:       { label: "Status", value: "The previous publishing target was kept. Fix the repository or sign-
in, then try Authenticate target again." }
1528:     ]
1529:   );
1530:   showBuilderError(
1531:     "Target authentication failed",
1532:     "The builder did not save this publishing target because GitHub write access was not verified.",
1533:     error.message || "GitHub authentication did not complete."
1534:   );
1535: }
1536: }

```

syncFromPublishTarget - lines 1538-1584

Item	Explanation
Source location	Lines 1538-1584 (47 source lines).
Purpose	Part of publishing, Git, target repository, or authorization behavior.
Declaration	async function syncFromPublishTarget(options = {}) {
Touches	network/API requests, authentication/security data
Calls detected	fetch, now, json, Error, filter, join, renderPublishTargetInfo, closeDialogElement, loadData, showBuilderError
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	3 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

1538: async function syncFromPublishTarget(options = {}) {
1539:   if (!publishTargetDialog) return;
1540:   const afterAuth = Boolean(options.afterAuth);
1541:   publishTargetCurrent.textContent = afterAuth
1542:     ? "GitHub authentication was accepted. Importing the target portfolio now..."
1543:     : "Checking GitHub authorization and loading compatible portfolio files from the target
repository...";
1544:   try {
1545:     const response = await fetch(`/api/sync-from-publish-target?t=${Date.now()}`, {
1546:       method: "POST",
1547:       cache: "no-store",
1548:       headers: { "Content-Type": "application/json" },
1549:       body: "{}"
1550:     });
1551:     const result = await response.json();
1552:     if (!response.ok || !result.ok) {
1553:       throw new Error([
1554:         result.error || "Publishing target could not be imported.",
1555:         result.details || "Click Authenticate target, complete the browser sign-in if prompted, then try
Load from target again."
1556:       ].filter(Boolean).join("\n\n"));
1557:     }
1558:     renderPublishTargetInfo(result.target || {});
1559:     closeDialogElement(publishTargetDialog);
1560:     await loadData();
1561:     draftSavedSinceChanges = true;
1562:     showBuilderError(
1563:       afterAuth ? "GitHub connected and target loaded" : "Target content loaded",
1564:       afterAuth
1565:         ? "Authentication succeeded, and compatible portfolio files were loaded into this local builder
workspace. The local draft now matches the imported target catalog."
1566:         : "Compatible portfolio files were loaded from the authenticated publishing target into this
local builder workspace. The local draft now matches the imported target catalog.",
1567:       [
1568:         `Imported: ${result.sync?.imported || []}.join(", ")`,
1569:         result.sync?.backup ? `Backup: ${result.sync.backup}` : ""
1570:       ].filter(Boolean).join("\n")
1571:     );
1572:   } catch (error) {
1573:     publishTargetCurrent.textContent = afterAuth
1574:       ? `Authentication completed, but target loading failed. ${error.message || "Publishing target could
not be imported."}`
1575:       : error.message || "Publishing target could not be imported.";
1576:     if (afterAuth) {
1577:       showBuilderError(
1578:         "Target saved, loading failed",
1579:         "GitHub authorization succeeded, but the builder could not import compatible portfolio files from
that target.",
1580:         error.message || "Publishing target could not be imported."

```

```

1581:         });
1582:     }
1583: }
1584: }

```

showPublishResult - lines 1586-1619

Item	Explanation
Source location	Lines 1586-1619 (34 source lines).
Purpose	Part of publishing, Git, target repository, or authorization behavior.
Declaration	function showPublishResult(result) {
Touches	authentication/security data
Calls detected	filter, join, showModal
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

1586: function showPublishResult(result) {
1587:   const publish = result.publish || {};
1588:   const pushed = Boolean(publish.pushed);
1589:   const authorizationRequired = Boolean(publish.authorizationRequired);
1590:   publishResultEyebrow.textContent = "GitHub publish";
1591:   publishResultTitle.textContent = pushed
1592:     ? "Successfully pushed to GitHub"
1593:     : authorizationRequired
1594:       ? "Publishing access required"
1595:       : "Site applied, push failed";
1596:   publishResultMessage.textContent = pushed
1597:     ? `Your portfolio changes were committed and pushed to ${publish.branch || "GitHub"}.`
1598:     : authorizationRequired
1599:       ? "No live website files were applied. Open Publishing target, click Authenticate with GitHub,
complete the browser sign-in, then try Apply to site again."
1600:       : "The site was applied locally, but Git push did not complete.";
1601:
1602:   const output = [
1603:     pushed ? "PUSH STATUS: SUCCESS" : "PUSH STATUS: FAILED",
1604:     `FILE: ${result.file || "projects.json"}`,
1605:     authorizationRequired ? "AUTHORIZATION: REQUIRED BEFORE WEBSITE CHANGES" : "",
1606:     publish.repository ? `REPOSITORY: ${publish.repository}` : "",
1607:     publish.remote ? `REMOTE: ${publish.remote}` : "",
1608:     publish.branch ? `BRANCH: ${publish.branch}` : "",
1609:     publish.committed === false ? "COMMIT: No file changes to commit." : "",
1610:     publish.commitOutput ? `COMMIT OUTPUT:\n${publish.commitOutput}` : "",
1611:     publish.pushOutput ? `PUSH OUTPUT:\n${publish.pushOutput}` : "",
1612:     publish.error ? `ERROR:\n${publish.error}` : "",
1613:     publish.details ? `DETAILS:\n${publish.details}` : "",
1614:     publish.help ? `NEXT STEP:\n${publish.help}` : "",
1615:   ].filter(Boolean).join("\n\n");
1616:
1617:   publishResultOutput.textContent = output || "No Git output was returned.";
1618:   publishResultDialog.showModal();
1619: }

```

scheduleAutosave - lines 1621-1627

Item	Explanation
Source location	Lines 1621-1627 (7 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function scheduleAutosave(delay = 900) {
Touches	Mostly local calculations or local UI state.
Calls detected	markDraftNeedsSave, autosaveDraft
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.

Item	Explanation
Async/return clues	0 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

1621: function scheduleAutosave(delay = 900) {
1622:   markDraftNeedsSave();
1623:   clearTimeout(autosaveTimer);
1624:   autosaveTimer = setTimeout(() => {
1625:     autosaveDraft();
1626:   }, delay);
1627: }

```

autosaveDraft - lines 1629-1659

Item	Explanation
Source location	Lines 1629-1659 (31 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	async function autosaveDraft() {
Touches	network/API requests
Calls detected	setSaveState, fetch, stringify, json, setStatus, scheduleAutosave
API endpoints	/api/save-draft
Storage keys	No local/session storage keys detected.
Async/return clues	2 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

1629: async function autosaveDraft() {
1630:   if (autosaveInFlight) {
1631:     autosaveQueued = true;
1632:     return;
1633:   }
1634:   autosaveInFlight = true;
1635:   setSaveState("saving", "Autosaving...");
1636:   try {
1637:     const response = await fetch("/api/save-draft", {
1638:       method: "POST",
1639:       headers: { "Content-Type": "application/json" },
1640:       body: JSON.stringify({ catalog })
1641:     });
1642:     const result = await response.json();
1643:     if (!response.ok) {
1644:       setStatus(result.error || "Autosave failed.");
1645:       setSaveState("error", "Autosave failed");
1646:     } else if (!draftSavedSinceChanges) {
1647:       setSaveState("dirty", "Autosaved locally");
1648:     }
1649:   } catch {
1650:     setStatus("Autosave failed. Make sure the local server is running.");
1651:     setSaveState("error", "Autosave failed");
1652:   } finally {
1653:     autosaveInFlight = false;
1654:     if (autosaveQueued) {
1655:       autosaveQueued = false;
1656:       scheduleAutosave(300);
1657:     }
1658:   }
1659: }

```

schedulePreviewRender - lines 1661-1667

Item	Explanation
Source location	Lines 1661-1667 (7 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function schedulePreviewRender() {

Item	Explanation
Touches	Mostly local calculations or local UI state.
Calls detected	renderPreview
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

1661: function schedulePreviewRender() {
1662:   if (!portfolioPreviewDialog?.open) return;
1663:   clearTimeout(previewRenderTimer);
1664:   previewRenderTimer = setTimeout(() => {
1665:     renderPreview();
1666:   }, 220);
1667: }

```

scheduleChromeRender - lines 1669-1676

Item	Explanation
Source location	Lines 1669-1676 (8 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function scheduleChromeRender() {
Touches	Mostly local calculations or local UI state.
Calls detected	renderTree, renderSectionTabs, selectedProject, renderPreview
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

1669: function scheduleChromeRender() {
1670:   clearTimeout(chromeRenderTimer);
1671:   chromeRenderTimer = setTimeout(() => {
1672:     renderTree();
1673:     renderSectionTabs(selectedProject());
1674:     if (portfolioPreviewDialog?.open) renderPreview();
1675:   }, 260);
1676: }

```

updateAssetDialogVisibility - lines 1678-1691

Item	Explanation
Source location	Lines 1678-1691 (14 source lines).
Purpose	Updates UI, state, cache, or stored data.
Declaration	function updateAssetDialogVisibility() {
Touches	browser/Electron DOM
Calls detected	querySelector, trim, labelForUrlAssetKind, inferUrlAssetKind
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
1678: function updateAssetDialogVisibility() {
1679:   const isLocal = assetSource.value === "local";
1680:   const isCaptionFile = captionSource.value === "file";
1681:   document.querySelector(".asset-local-field").hidden = !isLocal;
1682:   document.querySelector(".asset-url-field").hidden = isLocal;
1683:   document.querySelector(".caption-text-field").hidden = isCaptionFile;
1684:   document.querySelector(".caption-file-field").hidden = !isCaptionFile;
1685:   const url = assetUrl?.value?.trim() || "";
1686:   if (assetDetectedNote) {
1687:     assetDetectedNote.textContent = isLocal || !url
1688:       ? ""
1689:       : `Detected: ${labelForUrlAssetKind(inferUrlAssetKind(url, assetSource.value))}`;
1690:   }
1691: }
```

slugify - lines 1693-1699

Item	Explanation
Source location	Lines 1693-1699 (7 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function slugify(value) {
Touches	Mostly local calculations or local UI state.
Calls detected	toLowerCase, trim, replace
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
1693: function slugify(value) {
1694:   return String(value || "")
1695:     .toLowerCase()
1696:     .trim()
1697:     .replace(/[^\a-z0-9]+/g, "-")
1698:     .replace(/^-+|-+$/g, "");
1699: }
```

wordCount - lines 1701-1703

Item	Explanation
Source location	Lines 1701-1703 (3 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function wordCount(value) {
Touches	Mostly local calculations or local UI state.
Calls detected	trim, split, filter
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
1701: function wordCount(value) {
1702:   return String(value || "").trim().split(/\s+/).filter(Boolean).length;
1703: }
```

limitWords - lines 1705-1708

Item	Explanation
Source location	Lines 1705-1708 (4 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function limitWords(value, maxWords = 1000) {
Touches	Mostly local calculations or local UI state.
Calls detected	trim, split, filter, slice, join
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
1705: function limitWords(value, maxWords = 1000) {
1706:   const words = String(value || "").trim().split(/\s+/).filter(Boolean);
1707:   return words.length > maxWords ? words.slice(0, maxWords).join(" ") : value;
1708: }
```

normalizeFunFacts - lines 1710-1716

Item	Explanation
Source location	Lines 1710-1716 (7 source lines).
Purpose	Validates or normalizes input so later code receives safe predictable values.
Declaration	function normalizeFunFacts(value) {
Touches	filesystem
Calls detected	isArray, split, map, replace, trim, filter, slice
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
1710: function normalizeFunFacts(value) {
1711:   const items = Array.isArray(value) ? value : String(value || "").split(/\r?\n/);
1712:   return items
1713:     .map((item) => String(item || "").replace(/\s+/g, " ").trim())
1714:     .filter(Boolean)
1715:     .slice(0, 3);
1716: }
```

syncFunFactsFromInput - lines 1718-1726

Item	Explanation
Source location	Lines 1718-1726 (9 source lines).
Purpose	Keeps two places aligned, such as local data and target files.
Declaration	function syncFunFactsFromInput() {
Touches	filesystem
Calls detected	extractRichSummary, normalizeFunFacts, plainTextFromRich
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 2 return statement(s).

Item	Explanation
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

1718: function syncFunFactsFromInput() {
1719:   if (!funFactsInput) return [];
1720:   const rich = extractRichSummary(funFactsInput);
1721:   const facts = normalizeFunFacts(plainTextFromRich(rich));
1722:   catalog.funFacts = facts;
1723:   catalog.funFactsRich = rich;
1724:   if (funFactsCount) funFactsCount.textContent = `${facts.length}/3`;
1725:   return facts;
1726: }

```

renderFunFactsEditor - lines 1728-1734

Item	Explanation
Source location	Lines 1728-1734 (7 source lines).
Purpose	Renders UI, markup, or display output.
Declaration	function renderFunFactsEditor() {
Touches	browser/Electron DOM, filesystem
Calls detected	normalizeFunFacts, populateStandaloneRichEditor, join
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

1728: function renderFunFactsEditor() {
1729:   if (!funFactsInput) return;
1730:   const facts = normalizeFunFacts(catalog.funFacts || []);
1731:   if (funFactsCount) funFactsCount.textContent = `${facts.length}/3`;
1732:   if (document.activeElement === funFactsInput) return;
1733:   populateStandaloneRichEditor(funFactsInput, catalog.funFactsRich, facts.join("\n"));
1734: }

```

populateTextOnlyRichField - lines 1736-1740

Item	Explanation
Source location	Lines 1736-1740 (5 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function populateTextOnlyRichField(field, rich, fallbackText = "") {
Touches	browser/Electron DOM
Calls detected	populateStandaloneRichEditor
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

1736: function populateTextOnlyRichField(field, rich, fallbackText = "") {
1737:   if (!field) return;
1738:   populateStandaloneRichEditor(field, rich, fallbackText);
1739:   field.dataset.richTextOnly = "true";
1740: }

```

richFieldValue - lines 1742-1754

Item	Explanation
Source location	Lines 1742-1754 (13 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function richFieldValue(field, fallbackText = "") {
Touches	Mostly local calculations or local UI state.
Calls detected	trim, textBlocksFromPlainText, extractRichSummary, plainTextFromRich
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
1742: function richFieldValue(field, fallbackText = "") {
1743:   if (!field) {
1744:     return {
1745:       plain: String(fallbackText || "").trim(),
1746:       rich: { blocks: textBlocksFromPlainText(fallbackText) }
1747:     };
1748:   }
1749:   const rich = extractRichSummary(field);
1750:   return {
1751:     plain: plainTextFromRich(rich, "\n").trim(),
1752:     rich
1753:   };
1754: }
```

mergeRichFieldMaps - lines 1756-1761

Item	Explanation
Source location	Lines 1756-1761 (6 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function mergeRichFieldMaps(current = {}, fallback = {}) {
Touches	Mostly local calculations or local UI state.
Calls detected	clone
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
1756: function mergeRichFieldMaps(current = {}, fallback = {}) {
1757:   return {
1758:     ...clone(fallback || {}),
1759:     ...clone(current || {})
1760:   };
1761: }
```

parsedSiteContentRich - lines 1763-1765

Item	Explanation
Source location	Lines 1763-1765 (3 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function parsedSiteContentRich() {

Item	Explanation
Touches	Mostly local calculations or local UI state.
Calls detected	mergeRichFieldMaps
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
1763: function parsedSiteContentRich() {
1764:   return mergeRichFieldMaps(catalog.siteContentRich, savedPortfolioCatalog.siteContentRich);
1765: }
```

parsedProfileRich - lines 1767-1769

Item	Explanation
Source location	Lines 1767-1769 (3 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function parsedProfileRich() {
Touches	Mostly local calculations or local UI state.
Calls detected	mergeRichFieldMaps
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
1767: function parsedProfileRich() {
1768:   return mergeRichFieldMaps(catalog.profileRich, savedPortfolioCatalog.profileRich);
1769: }
```

parsedFunFactsRich - lines 1771-1775

Item	Explanation
Source location	Lines 1771-1775 (5 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function parsedFunFactsRich() {
Touches	Mostly local calculations or local UI state.
Calls detected	call, clone
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
1771: function parsedFunFactsRich() {
1772:   return Object.prototype.hasOwnProperty.call(catalog, "funFactsRich")
1773:     ? clone(catalog.funFactsRich || null)
1774:     : clone(savedPortfolioCatalog.funFactsRich || null);
1775: }
```

syncPortfolioTextInputsForParsing - lines 1777-1781

Item	Explanation
Source location	Lines 1777-1781 (5 source lines).
Purpose	Keeps two places aligned, such as local data and target files.
Declaration	function syncPortfolioTextInputsForParsing() {
Touches	Mostly local calculations or local UI state.
Calls detected	syncFunFactsFromInput, syncSiteContentFromInputs, syncProfileFromInputs
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
1777: function syncPortfolioTextInputsForParsing() {
1778:   if (funFactsInput) syncFunFactsFromInput();
1779:   if (siteHeroTitleInput || siteHeroCopyInput || siteHeroEyebrowInput) syncSiteContentFromInputs();
1780:   if (profileDisplayNameInput || profileEmailInput || profilePhoneInput) syncProfileFromInputs();
1781: }
```

parsedPortfolioGlobals - lines 1783-1796

Item	Explanation
Source location	Lines 1783-1796 (14 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function parsedPortfolioGlobals() {
Touches	filesystem
Calls detected	syncPortfolioTextInputsForParsing, clone, normalizeFieldStyles, normalizeFunFacts, parsedFunFactsRich, normalizeProfile, parsedProfileRich, normalizeSiteContent, parsedSiteContentRich, filter, map
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
1783: function parsedPortfolioGlobals() {
1784:   syncPortfolioTextInputsForParsing();
1785:   return {
1786:     categories: clone(catalog.categories || savedPortfolioCatalog.categories || []),
1787:     fieldStyles: clone(normalizeFieldStyles(catalog.fieldStyles || savedPortfolioCatalog.fieldStyles || {})),
1788:     funFacts: normalizeFunFacts(catalog.funFacts || savedPortfolioCatalog.funFacts || []),
1789:     funFactsRich: parsedFunFactsRich(),
1790:     profile: clone(normalizeProfile(catalog.profile || savedPortfolioCatalog.profile || {})),
1791:     profileRich: parsedProfileRich(),
1792:     siteContent: clone(normalizeSiteContent(catalog.siteContent || savedPortfolioCatalog.siteContent || {})),
1793:     siteContentRich: parsedSiteContentRich(),
1794:     siteSections: (catalog.siteSections || savedPortfolioCatalog.siteSections || []).filter(siteSectionRenderable).map(clone)
1795:   };
1796: }
```

syncParsedPortfolioGlobalsIntoSaved - lines 1798-1800

Item	Explanation
Source location	Lines 1798-1800 (3 source lines).
Purpose	Keeps two places aligned, such as local data and target files.

Item	Explanation
Declaration	function syncParsedPortfolioGlobalsIntoSaved() {
Touches	Mostly local calculations or local UI state.
Calls detected	assign, parsedPortfolioGlobals
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
1798: function syncParsedPortfolioGlobalsIntoSaved() {
1799:   Object.assign(savedPortfolioCatalog, parsedPortfolioGlobals());
1800: }
```

siteContentRichValue - lines 1802-1804

Item	Explanation
Source location	Lines 1802-1804 (3 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function siteContentRichValue(key) {
Touches	Mostly local calculations or local UI state.
Calls detected	No obvious named calls detected by the generator.
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
1802: function siteContentRichValue(key) {
1803:   return catalog.siteContentRich?.[key] || savedPortfolioCatalog.siteContentRich?.[key] || null;
1804: }
```

profileRichValue - lines 1806-1808

Item	Explanation
Source location	Lines 1806-1808 (3 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function profileRichValue(key) {
Touches	Mostly local calculations or local UI state.
Calls detected	No obvious named calls detected by the generator.
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
1806: function profileRichValue(key) {
1807:   return catalog.profileRich?.[key] || savedPortfolioCatalog.profileRich?.[key] || null;
1808: }
```

syncSiteRichField - lines 1810-1823

Item	Explanation
Source location	Lines 1810-1823 (14 source lines).
Purpose	Keeps two places aligned, such as local data and target files.
Declaration	function syncSiteRichField(editor) {
Touches	browser/Electron DOM, filesystem
Calls detected	richFieldValue, normalizeSiteContent, markDraftNeedsSave, scheduleAutosave, schedulePreviewRender
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
1810: function syncSiteRichField(editor) {
1811:   const key = editor?.dataset.siteField;
1812:   if (!key) return;
1813:   catalog.siteContentRich = catalog.siteContentRich || {};
1814:   const { plain, rich } = richFieldValue(editor, catalog.siteContent?.[key] || "");
1815:   catalog.siteContent = normalizeSiteContent({
1816:     ...(catalog.siteContent || {}),
1817:     [key]: plain
1818:   });
1819:   catalog.siteContentRich[key] = rich;
1820:   markDraftNeedsSave();
1821:   scheduleAutosave();
1822:   schedulePreviewRender();
1823: }
```

syncProfileRichField - lines 1825-1838

Item	Explanation
Source location	Lines 1825-1838 (14 source lines).
Purpose	Keeps two places aligned, such as local data and target files.
Declaration	function syncProfileRichField(editor) {
Touches	browser/Electron DOM, filesystem
Calls detected	richFieldValue, normalizeProfile, markDraftNeedsSave, scheduleAutosave, schedulePreviewRender
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
1825: function syncProfileRichField(editor) {
1826:   const key = editor?.dataset.profileField;
1827:   if (!key) return;
1828:   catalog.profileRich = catalog.profileRich || {};
1829:   const { plain, rich } = richFieldValue(editor, catalog.profile?.[key] || "");
1830:   catalog.profile = normalizeProfile({
1831:     ...(catalog.profile || {}),
1832:     [key]: plain
1833:   });
1834:   catalog.profileRich[key] = rich;
1835:   markDraftNeedsSave();
1836:   scheduleAutosave();
1837:   schedulePreviewRender();
1838: }
```

renderSiteContentEditor - lines 1840-1851

Item	Explanation
Source location	Lines 1840-1851 (12 source lines).
Purpose	Renders UI, markup, or display output.
Declaration	function renderSiteContentEditor() {
Touches	browser/Electron DOM, filesystem
Calls detected	normalizeSiteContent, populateTextOnlyRichField, siteContentRichValue
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

1840: function renderSiteContentEditor() {
1841:   const content = normalizeSiteContent(catalog.siteContent || {});
1842:   if (siteHeroEyebrowInput && document.activeElement !== siteHeroEyebrowInput) {
1843:     populateTextOnlyRichField(siteHeroEyebrowInput, siteContentRichValue("heroEyebrow"),
content.heroEyebrow);
1844:   }
1845:   if (siteHeroTitleInput && document.activeElement !== siteHeroTitleInput) {
1846:     populateTextOnlyRichField(siteHeroTitleInput, siteContentRichValue("heroTitle"), content.heroTitle);
1847:   }
1848:   if (siteHeroCopyInput && document.activeElement !== siteHeroCopyInput) {
1849:     populateTextOnlyRichField(siteHeroCopyInput, siteContentRichValue("heroCopy"), content.heroCopy);
1850:   }
1851: }

```

renderProfileEditor - lines 1853-1881

Item	Explanation
Source location	Lines 1853-1881 (29 source lines).
Purpose	Renders UI, markup, or display output.
Declaration	function renderProfileEditor() {
Touches	browser/Electron DOM, filesystem
Calls detected	normalizeProfile, forEach, populateTextOnlyRichField, profileRichValue, filter, join
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

1853: function renderProfileEditor() {
1854:   const profile = normalizeProfile(catalog.profile || {});
1855:   const fields = [
1856:     [profileDisplayNameInput, profile.displayName],
1857:     [profilePortfolioLabelInput, profile.portfolioLabel],
1858:     [profileContactIntroInput, profile.contactIntro],
1859:     [profileEmailInput, profile.email],
1860:     [profilePhoneInput, profile.phone],
1861:     [profileGithubInput, profile.githubUrl],
1862:     [profileLinkedInInput, profile.linkedinUrl],
1863:     [profileWebsiteInput, profile.websiteUrl]
1864:   ];
1865:   fields.forEach(([field, value]) => {
1866:     if (field && document.activeElement !== field) {
1867:       populateTextOnlyRichField(field, profileRichValue(field.dataset.profileField), value);
1868:     }
1869:   });
1870:   if (profileAssetStatus) {
1871:     const assetLabels = [
1872:       profile.profileImage ? "profile photo" : "",
1873:       profile.heroImage ? "main background" : "",
1874:       profile.resumeUrl ? "resume" : "",
1875:       profile.brandImage ? "brand icon" : ""

```

```

1876:     ].filter(Boolean);
1877:     profileAssetStatus.textContent = assetLabels.length
1878:     ? `Added: ${assetLabels.join(", ")}`.
1879:     : "No profile photo, resume, brand icon, or main background added yet.";
1880:   }
1881: }

```

syncSiteContentFromInputs - lines 1883-1897

Item	Explanation
Source location	Lines 1883-1897 (15 source lines).
Purpose	Keeps two places aligned, such as local data and target files.
Declaration	function syncSiteContentFromInputs() {
Touches	filesystem
Calls detected	richFieldValue, normalizeSiteContent
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

1883: function syncSiteContentFromInputs() {
1884:   catalog.siteContentRich = catalog.siteContentRich || {};
1885:   const eyebrow = richFieldValue(siteHeroEyebrowInput, catalog.siteContent?.heroEyebrow || "");
1886:   const title = richFieldValue(siteHeroTitleInput, catalog.siteContent?.heroTitle || "");
1887:   const copy = richFieldValue(siteHeroCopyInput, catalog.siteContent?.heroCopy || "");
1888:   catalog.siteContent = normalizeSiteContent({
1889:     heroCopy: copy.plain,
1890:     heroEyebrow: eyebrow.plain,
1891:     heroTitle: title.plain
1892:   });
1893:   catalog.siteContentRich.heroCopy = copy.rich;
1894:   catalog.siteContentRich.heroEyebrow = eyebrow.rich;
1895:   catalog.siteContentRich.heroTitle = title.rich;
1896:   return catalog.siteContent;
1897: }

```

syncProfileFromInputs - lines 1899-1926

Item	Explanation
Source location	Lines 1899-1926 (28 source lines).
Purpose	Keeps two places aligned, such as local data and target files.
Declaration	function syncProfileFromInputs() {
Touches	filesystem
Calls detected	richFieldValue, normalizeProfile, entries, forEach
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

1899: function syncProfileFromInputs() {
1900:   catalog.profileRich = catalog.profileRich || {};
1901:   const fields = {
1902:     contactIntro: richFieldValue(profileContactIntroInput, catalog.profile?.contactIntro || ""),
1903:     displayName: richFieldValue(profileDisplayNameInput, catalog.profile?.displayName || ""),
1904:     email: richFieldValue(profileEmailInput, catalog.profile?.email || ""),
1905:     githubUrl: richFieldValue(profileGithubInput, catalog.profile?.githubUrl || ""),
1906:     linkedinUrl: richFieldValue(profileLinkedinInput, catalog.profile?.linkedinUrl || ""),
1907:     phone: richFieldValue(profilePhoneInput, catalog.profile?.phone || ""),
1908:     portfolioLabel: richFieldValue(profilePortfolioLabelInput, catalog.profile?.portfolioLabel || ""),
1909:     websiteUrl: richFieldValue(profileWebsiteInput, catalog.profile?.websiteUrl || "")

```

```

1910:   };
1911:   catalog.profile = normalizeProfile({
1912:     ...(catalog.profile || {}),
1913:     contactIntro: fields.contactIntro.plain,
1914:     displayName: fields.displayName.plain,
1915:     email: fields.email.plain,
1916:     githubUrl: fields.githubUrl.plain,
1917:     linkedinUrl: fields.linkedinUrl.plain,
1918:     phone: fields.phone.plain,
1919:     portfolioLabel: fields.portfolioLabel.plain,
1920:     websiteUrl: fields.websiteUrl.plain
1921:   });
1922:   Object.entries(fields).forEach(([key, value]) => {
1923:     catalog.profileRich[key] = value.rich;
1924:   });
1925:   return catalog.profile;
1926: }

```

clone - lines 1928-1930

Item	Explanation
Source location	Lines 1928-1930 (3 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function clone(value) {
Touches	Mostly local calculations or local UI state.
Calls detected	parse, stringify
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

1928: function clone(value) {
1929:   return JSON.parse(JSON.stringify(value));
1930: }

```

closeDialogElement - lines 1932-1948

Item	Explanation
Source location	Lines 1932-1948 (17 source lines).
Purpose	Controls a window, dialog, panel, menu, or visible state.
Declaration	function closeDialogElement(dialog, returnValue = "close") {
Touches	Mostly local calculations or local UI state.
Calls detected	hasAttribute, close, removeAttribute, dispatchEvent, Event
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

1932: function closeDialogElement(dialog, returnValue = "close") {
1933:   if (!dialog) return;
1934:   const closeValue = String(returnValue || "close");
1935:   const wasOpen = Boolean(dialog.open || dialog.hasAttribute("open"));
1936:   if (typeof dialog.close === "function") {
1937:     try {
1938:       if (dialog.open) dialog.close(closeValue);
1939:     } catch {
1940:       // Fall through to manual cleanup if the browser rejects the native close call.
1941:     }
1942:     if (!dialog.open && !dialog.hasAttribute("open")) return;
1943:   }

```

```

1944:   dialog.returnValue = closeValue;
1945:   dialog.open = false;
1946:   dialog.removeAttribute("open");
1947:   if (wasOpen) dialog.dispatchEvent(new Event("close"));
1948: }

```

getByPath - lines 1950-1952

Item	Explanation
Source location	Lines 1950-1952 (3 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function getByPath(target, pathValue) {
Touches	Mostly local calculations or local UI state.
Calls detected	split, filter, reduce
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

1950: function getByPath(target, pathValue) {
1951:   return String(pathValue || "").split(".").filter(Boolean).reduce((current, key) => current?.[key],
target);
1952: }

```

setByPath - lines 1954-1963

Item	Explanation
Source location	Lines 1954-1963 (10 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function setByPath(target, pathValue, value) {
Touches	Mostly local calculations or local UI state.
Calls detected	split, filter, pop, forEach
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

1954: function setByPath(target, pathValue, value) {
1955:   const keys = String(pathValue || "").split(".").filter(Boolean);
1956:   const last = keys.pop();
1957:   let current = target;
1958:   keys.forEach((key) => {
1959:     current[key] = current[key] || {};
1960:     current = current[key];
1961:   });
1962:   if (last) current[last] = value;
1963: }

```

defaultDesignModel - lines 1965-1971

Item	Explanation
Source location	Lines 1965-1971 (7 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function defaultDesignModel() {

Item	Explanation
Touches	Mostly local calculations or local UI state.
Calls detected	No obvious named calls detected by the generator.
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

1965: function defaultDesignModel() {
1966:     return {
1967:         brief: { summary: "", summaryRich: null, files: [] },
1968:         documentation: { summary: "", summaryRich: null, files: [], references: [], mathAnalysis: [] },
1969:         simulation: { summary: "", summaryRich: null, files: [], results: [] }
1970:     };
1971: }

```

isAnalogProject - lines 1973-1975

Item	Explanation
Source location	Lines 1973-1975 (3 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function isAnalogProject(project) {
Touches	Mostly local calculations or local UI state.
Calls detected	No obvious named calls detected by the generator.
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

1973: function isAnalogProject(project) {
1974:     return project?.category === "analog-mixed-signal";
1975: }

```

ensureDesignModel - lines 1977-1992

Item	Explanation
Source location	Lines 1977-1992 (16 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function ensureDesignModel(project) {
Touches	Mostly local calculations or local UI state.
Calls detected	defaultDesignModel
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

1977: function ensureDesignModel(project) {
1978:     if (!project) return null;
1979:     const defaults = defaultDesignModel();

```

```

1980:   project.design = project.design || {};
1981:   project.design.brief = { ...defaults.brief, ...(project.design.brief || {}) };
1982:   project.design.documentation = { ...defaults.documentation, ...(project.design.documentation || {}) };
1983:   project.design.documentation.summaryRich = project.design.documentation.summaryRich || null;
1984:   project.design.simulation = { ...defaults.simulation, ...(project.design.simulation || {}) };
1985:   project.design.brief.files = project.design.brief.files || [];
1986:   project.design.documentation.files = project.design.documentation.files || [];
1987:   project.design.documentation.references = project.design.documentation.references || [];
1988:   project.design.documentation.mathAnalysis = project.design.documentation.mathAnalysis || [];
1989:   project.design.simulation.files = project.design.simulation.files || [];
1990:   project.design.simulation.results = project.design.simulation.results || [];
1991:   return project.design;
1992: }

```

ensureCompileCode - lines 1994-2034

Item	Explanation
Source location	Lines 1994-2034 (41 source lines).
Purpose	Part of the Compile Code language/tool/build/run flow.
Declaration	function ensureCompileCode(project) {
Touches	filesystem
Calls detected	isArray, map, normalizeCodeLanguage, detectCodeLanguage, safeClientCodeFileName, defaultCodeFileName, normalizeCompileFileRole, inferCompileFileRole, slugify, now, some, compileAppendDestinations
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 3 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

1994: function ensureCompileCode(project) {
1995:   if (!project) return null;
1996:   project.compileCode = project.compileCode || {};
1997:   project.compileCode.files = Array.isArray(project.compileCode.files) ? project.compileCode.files : [];
1998:   project.compileCode.files = project.compileCode.files.map((file, index) => {
1999:     const language = normalizeCodeLanguage(file.language || detectCodeLanguage(file.code || "",
file.fileName || ""));
2000:     const fileName = safeClientCodeFileName(file.fileName || defaultCodeFileName(language), language);
2001:     const role = normalizeCompileFileRole(file.role || inferCompileFileRole(fileName, file.code || "",
language), language);
2002:     return {
2003:       id: file.id || slugify(`${fileName}-${index}-${Date.now()}`),
2004:       title: file.title || fileName,
2005:       fileName,
2006:       language,
2007:       role,
2008:       code: String(file.code || ""),
2009:       stdin: String(file.stdin || ""),
2010:       savedPath: file.savedPath || file.workspacePath || "",
2011:       savedAt: file.savedAt || "",
2012:       waveform: file.waveform || file.lastResult?.waveform || null,
2013:       lastResult: file.lastResult || null,
2014:       dirty: Boolean(file.dirty)
2015:     };
2016:   });
2017:   if (!project.compileCode.activeFileId || !project.compileCode.files.some((file) => file.id ===
project.compileCode.activeFileId)) {
2018:     project.compileCode.activeFileId = project.compileCode.files[0]?.id || "";
2019:   }
2020:   project.compileCode.terminal = String(project.compileCode.terminal || "");
2021:   const appendDestinations = compileAppendDestinations(project);
2022:   if (!project.compileCode.appendTargetId || !appendDestinations.some((destination) => destination.id ===
project.compileCode.appendTargetId)) {
2023:     project.compileCode.appendTargetId = appendDestinations[0]?.id || "";
2024:   }
2025:   project.compileCode.messages = Array.isArray(project.compileCode.messages)
2026:     ? project.compileCode.messages.map((message) => ({
2027:       id: message.id || slugify(`message-${Date.now()}-${Math.random().toString(16).slice(2)}`),
2028:       at: message.at || new Date().toISOString(),
2029:       level: ["info", "success", "warning", "error"].includes(message.level) ? message.level : "info",
2030:       text: String(message.text || "")
2031:     })).filter((message) => message.text)
2032:     : [];
2033:   return project.compileCode;
2034: }

```

escapeHtml - lines 2036-2043

Item	Explanation
Source location	Lines 2036-2043 (8 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function escapeHtml(value) {
Touches	Mostly local calculations or local UI state.
Calls detected	replaceAll
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
2036: function escapeHtml(value) {
2037:   return String(value || "")
2038:     .replaceAll("&", "&amp;")
2039:     .replaceAll("<", "&lt;")
2040:     .replaceAll(">", "&gt;")
2041:     .replaceAll("'", "&quot;")
2042:     .replaceAll('"', "&#039;");
2043: }
```

escapeCodeHtml - lines 2045-2051

Item	Explanation
Source location	Lines 2045-2051 (7 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function escapeCodeHtml(value) {
Touches	Mostly local calculations or local UI state.
Calls detected	replaceAll
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
2045: function escapeCodeHtml(value) {
2046:   return String(value || "")
2047:     .replaceAll("&", "&amp;")
2048:     .replaceAll("<", "&lt;")
2049:     .replaceAll(">", "&gt;")
2050:     .replaceAll("'", "&quot;");
2051: }
```

normalizeCodeLanguage - lines 2053-2059

Item	Explanation
Source location	Lines 2053-2059 (7 source lines).
Purpose	Validates or normalizes input so later code receives safe predictable values.
Declaration	function normalizeCodeLanguage(value = "") {
Touches	filesystem
Calls detected	trim, toLowerCase, replace, find, includes

Item	Explanation
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

2053: function normalizeCodeLanguage(value = "") {
2054:   const clean = String(value || "").trim().toLowerCase().replace(/[_-]+/g, " ");
2055:   const match = supportedCodeLanguages.find((language) =>
2056:     language.id === clean || language.aliases.includes(clean)
2057:   );
2058:   return match?.id || "javascript";
2059: }

```

codeLanguageProfile - lines 2061-2063

Item	Explanation
Source location	Lines 2061-2063 (3 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function codeLanguageProfile(value = "") {
Touches	filesystem
Calls detected	find, normalizeCodeLanguage
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

2061: function codeLanguageProfile(value = "") {
2062:   return supportedCodeLanguages.find((language) => language.id === normalizeCodeLanguage(value)) ||
supportedCodeLanguages.find((language) => language.id === "javascript");
2063: }

```

codeLanguageLabel - lines 2065-2067

Item	Explanation
Source location	Lines 2065-2067 (3 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function codeLanguageLabel(value = "") {
Touches	filesystem
Calls detected	find, normalizeCodeLanguage
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

2065: function codeLanguageLabel(value = "") {
2066:   return supportedCodeLanguages.find((language) => language.id === normalizeCodeLanguage(value))?.label
|| "Code";
2067: }

```


sourceLooksCpp - lines 2069-2073

Item	Explanation
Source location	Lines 2069-2073 (5 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function sourceLooksCpp(source = "") {
Touches	Mostly local calculations or local UI state.
Calls detected	b
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
2069: function sourceLooksCpp(source = "") {
2070:   const text = String(source || "");
2071:   return
#include\s*<(?:iostream|string|vector|array|map|unordered_map|memory|algorithm|optional|variant)>/.test(text)
||
2072:   /\b(std::|cout|cin|cerr|namespace|template\s*<|class\s+\w+|new\s+\w+|delete\s+|constexpr|nullptr|using
\s+namespace)\b/.test(text);
2073: }
```

sourceLooksC - lines 2075-2079

Item	Explanation
Source location	Lines 2075-2079 (5 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function sourceLooksC(source = "") {
Touches	Mostly local calculations or local UI state.
Calls detected	b
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
2075: function sourceLooksC(source = "") {
2076:   const text = String(source || "");
2077:   return #include\s*<(?:stdio|stdlib|string|stdint|stdbool|math)\.h>/.test(text) ||
2078:   /\b(printf|scanf|malloc|calloc|realloc|free|struct\s+\w+|typedef\s+struct)\b/.test(text);
2079: }
```

languageFromFileName - lines 2081-2090

Item	Explanation
Source location	Lines 2081-2090 (10 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function languageFromFileName(fileName = "", source = "") {
Touches	Mostly local calculations or local UI state.
Calls detected	toLowerCase, includes, split, pop, sourceLooksCpp, sourceLooksC, find
API endpoints	None detected inside this function body.

Item	Explanation
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 4 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

2081: function languageFromFileName(fileName = "", source = "") {
2082:   const clean = String(fileName || "").toLowerCase();
2083:   const extension = clean.includes(".") ? `${clean.split(".").pop()}` : "";
2084:   if (extension === ".h") {
2085:     if (sourceLooksCpp(source)) return "cpp";
2086:     if (sourceLooksC(source)) return "c";
2087:     return "c";
2088:   }
2089:   return supportedCodeLanguages.find((language) => language.extensions?.includes(extension))?.id || "";
2090: }

```

defaultCodeFileName - lines 2092-2094

Item	Explanation
Source location	Lines 2092-2094 (3 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function defaultCodeFileName(language = "javascript") {
Touches	Mostly local calculations or local UI state.
Calls detected	codeLanguageProfile
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

2092: function defaultCodeFileName(language = "javascript") {
2093:   return codeLanguageProfile(language)?.defaultFile || "main.js";
2094: }

```

safeClientCodeFileName - lines 2096-2104

Item	Explanation
Source location	Lines 2096-2104 (9 source lines).
Purpose	Validates or normalizes input so later code receives safe predictable values.
Declaration	function safeClientCodeFileName(fileName = "", language = "javascript") {
Touches	Mostly local calculations or local UI state.
Calls detected	codeLanguageProfile, trim, replace, match, toLowerCase, includes
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

2096: function safeClientCodeFileName(fileName = "", language = "javascript") {
2097:   const profile = codeLanguageProfile(language);
2098:   const fallback = profile.defaultFile || "main.js";
2099:   const clean = String(fileName || fallback).trim().replace(/[\\/:?*"<>|]+/g, "-");
2100:   const namePart = clean.replace(/\.([^.]+)$/, "").replace(/^[a-zA-Z0-9_-]+/g, "-").replace(/^-+|-+$/g,
  "")) || fallback.replace(/\.([^.]+)$/, "");

```

```

2101:   const extMatch = clean.match(/(\.[a-zA-Z0-9_-]+)$/);
2102:   const ext = (extMatch?.[1] || fallback.match(/(\.[^.]+)$/)?.[1] || ".js").toLowerCase();
2103:   return `${namePart}${profile.extensions?.includes(ext) ? ext : fallback.match(/(\.[^.]+)$/)?.[1] ||
".js"}`;
2104: }

```

normalizeCodePasteMode - lines 2106-2108

Item	Explanation
Source location	Lines 2106-2108 (3 source lines).
Purpose	Validates or normalizes input so later code receives safe predictable values.
Declaration	function normalizeCodePasteMode(value = "") {
Touches	filesystem
Calls detected	No obvious named calls detected by the generator.
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

2106: function normalizeCodePasteMode(value = "") {
2107:   return value === "source" ? "source" : "basic";
2108: }

```

normalizeCodeText - lines 2110-2119

Item	Explanation
Source location	Lines 2110-2119 (10 source lines).
Purpose	Validates or normalizes input so later code receives safe predictable values.
Declaration	function normalizeCodeText(value = "", pasteMode = "source") {
Touches	filesystem
Calls detected	replace, normalizeCodePasteMode, split, map, trimEnd, join, trim
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

2110: function normalizeCodeText(value = "", pasteMode = "source") {
2111:   const normalized = String(value || "").replace(/\r\n?/g, "\n").replace(/\u00a0/g, " ");
2112:   if (normalizeCodePasteMode(pasteMode) === "source") return normalized.replace(/\t/g, "
").replace(/\s+$/g, "");
2113:   return normalized
2114:     .split("\n")
2115:     .map((line) => line.trimEnd())
2116:     .join("\n")
2117:     .replace(/\n{4,}/g, "\n\n\n")
2118:     .trim();
2119: }

```

detectCodeLanguage - lines 2121-2136

Item	Explanation
Source location	Lines 2121-2136 (16 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function detectCodeLanguage(value = "", fileName = "") {

Item	Explanation
Touches	Mostly local calculations or local UI state.
Calls detected	languageFromFileName, b, sourceLooksCpp, sourceLooksC
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 10 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

2121: function detectCodeLanguage(value = "", fileName = "") {
2122:   const byFile = languageFromFileName(fileName, value);
2123:   if (byFile) return byFile;
2124:   const code = String(value || "");
2125:   if (/<\/?[a-z][\s\S]*?>/i.test(code) || /<!doctype\s+html/i.test(code)) return "html";
2126:   if (/\\b(always_ff|always_comb|always_latch|logic|interface|covergroup|assert\s+property|typedef\s+enum|
class\s+w+)\b/.test(code)) return "systemverilog";
2127:   if (/\\b(module|endmodule|always|assign|reg|wire|initial|posedge|negedge)\b/.test(code)) return
"verilog";
2128:   if (/^\\s*\\.?(tran|ac|dc|op|model|subckt|ends|param)\\b/im.test(code) ||
/bv\\w+\\s+\\w+\\s+\\w+\\s+(?:DC|SIN|PULSE)?/i.test(code)) return "ltspice";
2129:   if (/\\b(def|elif|import\\s+\\w+|from\\s+\\w+\\s+import|self|None|True|False)\\b/.test(code)) return "python";
2130:   if (/\\b(public\\s+class|private\\s+|protected\\s+|static\\s+void\\s+main|System\\.out)\\b/.test(code)) return
"java";
2131:   if (sourceLooksCpp(code) || sourceLooksC(code) ||
/b(#include|main\\s*\\(|puts\\s*\\(|fprintf|sizeof\\s*\\(|\\b/.test(code)) {
2132:     return sourceLooksCpp(code) ? "cpp" : "c";
2133:   }
2134:   if (/\\b(const|let|var|function|=>|console\\.log|document\\.|window\\.|)\\b/.test(code)) return "javascript";
2135:   return "javascript";
2136: }

```

tokenizedCodeHtml - lines 2138-2192

Item	Explanation
Source location	Lines 2138-2192 (55 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function tokenizedCodeHtml(code = "", language = "javascript") {
Touches	filesystem, authentication/security data
Calls detected	escapeCodeHtml, fromCharCode, floor, replace, tokenName, push, protect, includes, normalizeCodeLanguage, b, forEach, replaceAll
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	2 await expression(s), 9 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

2138: function tokenizedCodeHtml(code = "", language = "javascript") {
2139:   let html = escapeCodeHtml(code);
2140:   const tokens = [];
2141:   const tokenName = (index) => {
2142:     let value = Number(index) + 1;
2143:     let output = "";
2144:     while (value > 0) {
2145:       value -= 1;
2146:       output = String.fromCharCode(65 + (value % 26)) + output;
2147:       value = Math.floor(value / 26);
2148:     }
2149:     return `uE000${output}uE001`;
2150:   };
2151:   const protect = (pattern, className) => {
2152:     html = html.replace(pattern, (match) => {
2153:       const token = tokenName(tokens.length);
2154:       tokens.push({ token, html: `<span class="${className}">${match}</span>` });
2155:       return token;
2156:     });
2157:   };

```

```

2158:
2159:   if (language === "html") {
2160:     protect(/&lt;!--[\s\S]*?--&gt;/g, "code-token-comment");
2161:     protect(/&lt;\/?([a-zA-Z0-9:-]+)([\s\S]*?)(\/?&gt;)/g, "code-token-tag");
2162:   } else {
2163:     const commentPattern = ["python", "ltspice"].includes(normalizeCodeLanguage(language))
2164:       ? /\^\/\^*[\s\S]*?*\^\/\^\/\^\/\^[\n]*#[^\n]*/g
2165:       : /\^\/\^*[\s\S]*?*\^\/\^\/\^\/\^[\n]*/g;
2166:     protect(commentPattern, "code-token-comment");
2167:     protect(/"(?:\\\.|[\^"\\\])*"|'(?:\\\.|[\^"\\\])*'|`(?:\\\.|[\^"\\\])*`"/g, "code-token-string");
2168:     if (["c", "cpp"].includes(normalizeCodeLanguage(language))) {
2169:       protect(/#\s*w+[\n]*/gm, "code-token-preprocessor");
2170:     }
2171:   }
2172:
2173:   protect(/\\b(?:0x[\\da-f]+|\\d+(?:\\.\\d+)?(?:e[+-]?\\d+)?)/gi, "code-token-number");
2174:
2175:   const keywordMap = {
2176:     c: "_Alignas|_Alignof|_Atomic|_Bool|_Complex|_Generic|_Imaginary|_Noreturn|_Static_assert|_Thread_loc
al|auto|break|case|char|const|continue|default|do|double|else|enum|extern|float|for|goto|if|inline|int|long|regi
ster|restrict|return|short|signed|sizeof|static|struct|switch|typedef|union|unsigned|void|volatile|while",
2177:     cpp: "alignas|alignof|and|and_eq|asm|auto|bitand|bitor|bool|break|case|catch|char|char8_t|char16_t|ch
ar32_t|class|compl|concept|const|constexpr|constinit|const_cast|continue|co_await|co_return|co_yield|d
ecltype|default|delete|do|double|dynamic_cast|else|enum|explicit|export|extern|false|float|for|friend|if|inline|
int|long|mutable|namespace|new|noexcept|not|not_eq|nullptr|operator|or|or_eq|private|protected|public|register|r
einterpret_cast|requires|return|short|signed|sizeof|static|static_assert|static_cast|struct|switch|template|this
|thread_local|throw|true|try|typedef|typeid|typename|union|unsigned|using|virtual|void|volatile|wchar_t|while|xor
r|xor_eq",
2178:     verilog: "always|and|assign|begin|buf|case|casex|casez|deassign|default|defparam|disable|edge|else|en
d|endcase|endfunction|endmodule|endprimitive|endspecify|endtable|endtask|event|for|force|forever|fork|function|g
enerate|genvar|if|initial|inout|input|integer|join|module|nand|negedge|nor|not|or|output|parameter|posedge|primi
tive|reg|release|repeat|signed|specify|supply0|supply1|table|task|tri|wand|while|wire|wor|xnor|xor",
2179:     systemverilog: "always|always_comb|always_ff|always_latch|assign|automatic|begin|bit|case|class|clock
ing|covergroup|default|disable|do|else|end|endcase|endclass|endclocking|endfunction|endmodule|endpackage|endtask
|enum|for|forever|function|generate|genvar|if|initial|input|int|interface|logic|module|negedge|output|package|pa
rameter|posedge|reg|return|signed|task|typedef|wire",
2180:     ltspice: "ac|dc|end|ends|four|func|global|ic|include|lib|meas|model|nodeset|op|options|param|plot|pro
be|save|step|subckt|temp|tran",
2181:     java: "abstract|assert|boolean|break|byte|case|catch|char|class|const|continue|default|do|double|else
|enum|extends|final|finally|float|for|if|implements|import|instanceof|int|interface|long|native|new|null|package
|private|protected|public|return|short|static|strictfp|super|switch|synchronized|this|throw|throws|transient|try
|void|volatile|while",
2182:     javascript: "async|await|break|case|catch|class|const|continue|debugger|default|delete|do|else|export
|extends|false|finally|for|from|function|if|import|in|instanceof|let|new|null|of|return|static|super|switch|this
|throw|true|try|typeof|undefined|var|void|while|yield",
2183:     python: "and|as|assert|async|await|break|class|continue|def|del|elif|else|except|False|finally|for|fr
om|global|if|import|in|is|lambda|None|nonlocal|not|or|pass|raise|return|True|try|while|with|yield",
2184:     html: "html|head|body|main|section|article|div|span|script|style|link|meta|title|header|footer|nav|bu
tton|input|form|label|img|a|p|pre|code"
2185:   };
2186:   const keywords = keywordMap[normalizeCodeLanguage(language)] || keywordMap.javascript;
2187:   html = html.replace(new RegExp(`\\b(${keywords})\\b`, "g"), '<span class="code-token-
keyword">$1</span>');
2188:   tokens.forEach((token) => {
2189:     html = html.replaceAll(token.token, token.html);
2190:   });
2191:   return html;
2192: }

```

tokenName - lines 2141-2150

Item	Explanation
Source location	Lines 2141-2150 (10 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	const tokenName = (index) => {
Touches	authentication/security data
Calls detected	fromCharCode, floor
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

2141:   const tokenName = (index) => {
2142:     let value = Number(index) + 1;
2143:     let output = "";
2144:     while (value > 0) {

```

```

2145:     value -= 1;
2146:     output = String.fromCharCode(65 + (value % 26)) + output;
2147:     value = Math.floor(value / 26);
2148:   }
2149:   return ``\uE000${output}\uE001`;
2150: };

```

protect - lines 2151-2157

Item	Explanation
Source location	Lines 2151-2157 (7 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	const protect = (pattern, className) => {
Touches	authentication/security data
Calls detected	replace, tokenName, push
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

2151:   const protect = (pattern, className) => {
2152:     html = html.replace(pattern, (match) => {
2153:       const token = tokenName(tokens.length);
2154:       tokens.push({ token, html: `<span class="${className}">${match}</span>` });
2155:       return token;
2156:     });
2157:   };

```

renderRichCodeBlock - lines 2194-2204

Item	Explanation
Source location	Lines 2194-2204 (11 source lines).
Purpose	Renders UI, markup, or display output.
Declaration	function renderRichCodeBlock(block = {}) {
Touches	filesystem, authentication/security data
Calls detected	normalizeCodeLanguage, detectCodeLanguage, normalizeCodeText, codeLanguageLabel, escapeHtml, tokenizedCodeHtml
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

2194: function renderRichCodeBlock(block = {}) {
2195:   const language = normalizeCodeLanguage(block.language || detectCodeLanguage(block.code || ""));
2196:   const code = normalizeCodeText(block.code || "", block.pasteMode || "source");
2197:   const label = codeLanguageLabel(language);
2198:   return
2199:     <figure class="rich-code-block rich-code-language-${language}">
2200:       <figcaption><span>&lt;/span> </span> ${escapeHtml(label)}</figcaption>
2201:       <pre><code>${tokenizedCodeHtml(code, language)}</code></pre>
2202:     </figure>
2203:   `;
2204: }

```

renderPlainMultiline - lines 2206-2208

Item	Explanation
Source location	Lines 2206-2208 (3 source lines).
Purpose	Renders UI, markup, or display output.
Declaration	function renderPlainMultiline(value = "") {
Touches	Mostly local calculations or local UI state.
Calls detected	escapeHtml, replace
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

2206: function renderPlainMultiline(value = "") {
2207:   return escapeHtml(value).replace(/\r\n?|\n/g, "<br>");
2208: }

```

normalizeSiteContent - lines 2210-2216

Item	Explanation
Source location	Lines 2210-2216 (7 source lines).
Purpose	Validates or normalizes input so later code receives safe predictable values.
Declaration	function normalizeSiteContent(content = {}) {
Touches	filesystem
Calls detected	trim
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

2210: function normalizeSiteContent(content = {}) {
2211:   return {
2212:     heroCopy: String(content.heroCopy || "").trim() || defaultSiteContent.heroCopy,
2213:     heroEyebrow: String(content.heroEyebrow || "").trim() || defaultSiteContent.heroEyebrow,
2214:     heroTitle: String(content.heroTitle || "").trim() || defaultSiteContent.heroTitle
2215:   };
2216: }

```

normalizeProfile - lines 2218-2234

Item	Explanation
Source location	Lines 2218-2234 (17 source lines).
Purpose	Validates or normalizes input so later code receives safe predictable values.
Declaration	function normalizeProfile(profile = {}) {
Touches	filesystem
Calls detected	trim
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).

Item	Explanation
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

2218: function normalizeProfile(profile = {}) {
2219:   return {
2220:     brandImage: String(profile.brandImage || "").trim(),
2221:     brandText: String(profile.brandText || "").trim() || String(profile.displayName || "").trim() ||
defaultProfile.brandText,
2222:     contactIntro: String(profile.contactIntro || "").trim(),
2223:     displayName: String(profile.displayName || "").trim(),
2224:     email: String(profile.email || "").trim(),
2225:     githubUrl: String(profile.githubUrl || "").trim(),
2226:     heroImage: String(profile.heroImage || "").trim(),
2227:     linkedinUrl: String(profile.linkedinUrl || "").trim(),
2228:     phone: String(profile.phone || "").trim(),
2229:     portfolioLabel: String(profile.portfolioLabel || "").trim() || defaultProfile.portfolioLabel,
2230:     profileImage: String(profile.profileImage || "").trim(),
2231:     resumeUrl: String(profile.resumeUrl || "").trim(),
2232:     websiteUrl: String(profile.websiteUrl || "").trim()
2233:   };
2234: }

```

normalizePlainFieldStyle - lines 2236-2250

Item	Explanation
Source location	Lines 2236-2250 (15 source lines).
Purpose	Validates or normalizes input so later code receives safe predictable values.
Declaration	function normalizePlainFieldStyle(style = {}) {
Touches	filesystem
Calls detected	cleanFontFamily, normalizeFontPx, normalizeTextColor, rgbColorToHex
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

2236: function normalizePlainFieldStyle(style = {}) {
2237:   const fontFamily = cleanFontFamily(style.fontFamily || "");
2238:   const fontPx = normalizeFontPx(style.fontPx || style.fontSize || "");
2239:   const color = normalizeTextColor(style.color || "");
2240:   const normalized = {};
2241:
2242:   if (fontFamily) normalized.fontFamily = fontFamily;
2243:   if (fontPx) normalized.fontPx = fontPx;
2244:   if (color) normalized.color = rgbColorToHex(color);
2245:   if (style.bold === true || style.bold === "true") normalized.bold = true;
2246:   if (style.italic === true || style.italic === "true") normalized.italic = true;
2247:   if (style.underline === true || style.underline === "true") normalized.underline = true;
2248:
2249:   return normalized;
2250: }

```

normalizeFieldStyles - lines 2252-2259

Item	Explanation
Source location	Lines 2252-2259 (8 source lines).
Purpose	Validates or normalizes input so later code receives safe predictable values.
Declaration	function normalizeFieldStyles(styles = {}) {
Touches	filesystem
Calls detected	fromEntries, entries, filter, has, map, normalizePlainFieldStyle, keys
API endpoints	None detected inside this function body.

Item	Explanation
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

2252: function normalizeFieldStyles(styles = {}) {
2253:   return Object.fromEntries(
2254:     Object.entries(styles || {})
2255:       .filter(([fieldId]) => persistentPlainStyleFieldIds.has(fieldId))
2256:       .map(([fieldId, style]) => [fieldId, normalizePlainFieldStyle(style)])
2257:       .filter(([, style]) => Object.keys(style).length)
2258:   );
2259: }

```

loadBuilderPreferences - lines 2261-2273

Item	Explanation
Source location	Lines 2261-2273 (13 source lines).
Purpose	Loads data from disk, network, browser storage, or a service.
Declaration	function loadBuilderPreferences() {
Touches	Mostly local calculations or local UI state.
Calls detected	parse, getItem, includes, applyBuilderPreferences
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

2261: function loadBuilderPreferences() {
2262:   try {
2263:     const stored = JSON.parse(localStorage.getItem(preferenceStorageKey) || "{}");
2264:     builderPreferences = {
2265:       ...defaultBuilderPreferences,
2266:       ...stored,
2267:       theme: ["light", "dark"].includes(stored.theme) ? stored.theme : defaultBuilderPreferences.theme
2268:     };
2269:   } catch {
2270:     builderPreferences = { ...defaultBuilderPreferences };
2271:   }
2272:   applyBuilderPreferences();
2273: }

```

applyBuilderPreferences - lines 2275-2280

Item	Explanation
Source location	Lines 2275-2280 (6 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function applyBuilderPreferences() {
Touches	browser/Electron DOM
Calls detected	includes
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
2275: function applyBuilderPreferences() {
2276:   const theme = ["light", "dark"].includes(builderPreferences.theme) ? builderPreferences.theme :
"light";
2277:   document.documentElement.dataset.builderTheme = theme;
2278:   document.body.dataset.builderTheme = theme;
2279:   if (preferenceTheme) preferenceTheme.value = theme;
2280: }
```

setBuilderTheme - lines 2282-2287

Item	Explanation
Source location	Lines 2282-2287 (6 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function setBuilderTheme(theme = "light") {
Touches	Mostly local calculations or local UI state.
Calls detected	includes, setItem, stringify, applyBuilderPreferences, setStatus
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
2282: function setBuilderTheme(theme = "light") {
2283:   builderPreferences.theme = ["light", "dark"].includes(theme) ? theme : "light";
2284:   localStorage.setItem(preferenceStorageKey, JSON.stringify(builderPreferences));
2285:   applyBuilderPreferences();
2286:   setStatus(`Builder ${builderPreferences.theme} mode enabled.`);
2287: }
```

openPreferencesDialog - lines 2289-2292

Item	Explanation
Source location	Lines 2289-2292 (4 source lines).
Purpose	Controls a window, dialog, panel, menu, or visible state.
Declaration	function openPreferencesDialog() {
Touches	Mostly local calculations or local UI state.
Calls detected	applyBuilderPreferences, showModal
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
2289: function openPreferencesDialog() {
2290:   applyBuilderPreferences();
2291:   if (!preferencesDialog?.open) preferencesDialog.showModal();
2292: }
```

saveBuilderPreferencesFromDialog - lines 2294-2297

Item	Explanation
Source location	Lines 2294-2297 (4 source lines).
Purpose	Persists data to local state, disk, credentials, or browser storage.
Declaration	function saveBuilderPreferencesFromDialog() {

Item	Explanation
Touches	Mostly local calculations or local UI state.
Calls detected	setBuilderTheme, setStatus
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

2294: function saveBuilderPreferencesFromDialog() {
2295:   setBuilderTheme(preferenceTheme?.value || "light");
2296:   setStatus("Preferences saved.");
2297: }

```

plainFieldStyleToCss - lines 2299-2309

Item	Explanation
Source location	Lines 2299-2309 (11 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function plainFieldStyleToCss(style = {}) {
Touches	filesystem
Calls detected	normalizePlainFieldStyle, push, join
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

2299: function plainFieldStyleToCss(style = {}) {
2300:   const normalized = normalizePlainFieldStyle(style);
2301:   const rules = [];
2302:   if (normalized.fontFamily) rules.push(`font-family: ${normalized.fontFamily}`);
2303:   if (normalized.fontPx) rules.push(`font-size: ${normalized.fontPx}px`);
2304:   if (normalized.color) rules.push(`color: ${normalized.color}`);
2305:   if (normalized.bold) rules.push("font-weight: 700");
2306:   if (normalized.italic) rules.push("font-style: italic");
2307:   if (normalized.underline) rules.push("text-decoration: underline");
2308:   return rules.join("; ");
2309: }

```

plainFieldStyleAttribute - lines 2311-2314

Item	Explanation
Source location	Lines 2311-2314 (4 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function plainFieldStyleAttribute(styles = {}, fieldId = "") {
Touches	Mostly local calculations or local UI state.
Calls detected	plainFieldStyleToCss, escapeHtml
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
2311: function plainFieldStyleAttribute(styles = {}, fieldId = "") {
2312:   const css = plainFieldStyleToCss(styles[fieldId]);
2313:   return css ? `style="${escapeHtml(css)}"` : "";
2314: }
```

plainStyleFromControl - lines 2316-2326

Item	Explanation
Source location	Lines 2316-2326 (11 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function plainStyleFromControl(control) {
Touches	browser/Electron DOM, filesystem
Calls detected	normalizePlainFieldStyle
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
2316: function plainStyleFromControl(control) {
2317:   if (!control) return {};
2318:   return normalizePlainFieldStyle({
2319:     bold: control.dataset.builderBold,
2320:     color: control.dataset.builderColor,
2321:     fontFamily: control.dataset.builderFontFamily,
2322:     fontPx: control.dataset.builderFontPx,
2323:     italic: control.dataset.builderItalic,
2324:     underline: control.dataset.builderUnderline
2325:   });
2326: }
```

applyPlainFieldStyleToControl - lines 2328-2343

Item	Explanation
Source location	Lines 2328-2343 (16 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function applyPlainFieldStyleToControl(control, style = {}) {
Touches	browser/Electron DOM, filesystem
Calls detected	normalizePlainFieldStyle
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
2328: function applyPlainFieldStyleToControl(control, style = {}) {
2329:   if (!control) return;
2330:   const normalized = normalizePlainFieldStyle(style);
2331:   control.dataset.builderFontFamily = normalized.fontFamily || "";
2332:   control.dataset.builderFontPx = normalized.fontPx || "";
2333:   control.dataset.builderColor = normalized.color || "";
2334:   control.dataset.builderBold = normalized.bold ? "true" : "false";
2335:   control.dataset.builderItalic = normalized.italic ? "true" : "false";
2336:   control.dataset.builderUnderline = normalized.underline ? "true" : "false";
2337:   control.style.fontFamily = normalized.fontFamily || "";
2338:   control.style.fontSize = normalized.fontPx ? `${normalized.fontPx}px` : "";
2339:   control.style.color = normalized.color || "";
2340:   control.style.fontWeight = normalized.bold ? "700" : "";
2341:   control.style.fontStyle = normalized.italic ? "italic" : "";
2342:   control.style.textDecoration = normalized.underline ? "underline" : "";
```

2343: }

applyStoredPlainFieldStyle - lines 2345-2349

Item	Explanation
Source location	Lines 2345-2349 (5 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function applyStoredPlainFieldStyle(control) {
Touches	filesystem
Calls detected	has, normalizeFieldStyles, applyPlainFieldStyleToControl
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
2345: function applyStoredPlainFieldStyle(control) {
2346:   if (!control?.id || !persistentPlainStyleFieldIds.has(control.id)) return;
2347:   const styles = normalizeFieldStyles(catalog.fieldStyles || {});
2348:   applyPlainFieldStyleToControl(control, styles[control.id] || {});
2349: }
```

storePlainControlStyle - lines 2351-2365

Item	Explanation
Source location	Lines 2351-2365 (15 source lines).
Purpose	Persists data to local state, disk, credentials, or browser storage.
Declaration	function storePlainControlStyle(control) {
Touches	filesystem
Calls detected	has, normalizeFieldStyles, plainStyleFromControl, keys, clone, markDraftNeedsSave, scheduleAutosave, schedulePreviewRender
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
2351: function storePlainControlStyle(control) {
2352:   if (!control?.id || !persistentPlainStyleFieldIds.has(control.id)) return;
2353:   const nextStyles = normalizeFieldStyles(catalog.fieldStyles || {});
2354:   const style = plainStyleFromControl(control);
2355:   if (Object.keys(style).length) {
2356:     nextStyles[control.id] = style;
2357:   } else {
2358:     delete nextStyles[control.id];
2359:   }
2360:   catalog.fieldStyles = nextStyles;
2361:   savedPortfolioCatalog.fieldStyles = clone(nextStyles);
2362:   markDraftNeedsSave();
2363:   scheduleAutosave();
2364:   schedulePreviewRender();
2365: }
```

mailComposeLink - lines 2367-2370

Item	Explanation
Source location	Lines 2367-2370 (4 source lines).

Item	Explanation
Purpose	Part of the private builder frontend and editing experience.
Declaration	function mailComposeLink(email = "") {
Touches	network/API requests
Calls detected	trim
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

2367: function mailComposeLink(email = "") {
2368:   const clean = String(email || "").trim();
2369:   return clean ? `https://mail.google.com/mail/?view=cm&fs=1&to=${encodeURIComponent(clean)}` : "";
2370: }

```

phoneLink - lines 2372-2375

Item	Explanation
Source location	Lines 2372-2375 (4 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function phoneLink(phone = "") {
Touches	Mostly local calculations or local UI state.
Calls detected	trim, replace
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

2372: function phoneLink(phone = "") {
2373:   const clean = String(phone || "").trim();
2374:   return clean ? `tel:${clean.replace(/^[^d+]/g, "")}` : "";
2375: }

```

textBlocksFromPlainText - lines 2377-2385

Item	Explanation
Source location	Lines 2377-2385 (9 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function textBlocksFromPlainText(text) {
Touches	filesystem
Calls detected	No obvious named calls detected by the generator.
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

2377: function textBlocksFromPlainText(text) {
2378:   return [{
2379:     align: "left",
2380:     fontFamily: "Arial",
2381:     fontSize: "normal",
2382:     text: String(text || ""),
2383:     type: "paragraph"
2384:   }];
2385: }

```

richBlocksForProject - lines 2387-2389

Item	Explanation
Source location	Lines 2387-2389 (3 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function richBlocksForProject(project) {
Touches	Mostly local calculations or local UI state.
Calls detected	clone, textBlocksFromPlainText
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

2387: function richBlocksForProject(project) {
2388:   return project?.summaryRich?.blocks?.length ? clone(project.summaryRich.blocks) :
textBlocksFromPlainText(project?.summary || "");
2389: }

```

plainTextFromRich - lines 2391-2402

Item	Explanation
Source location	Lines 2391-2402 (12 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function plainTextFromRich(rich, separator = "\n\n") {
Touches	filesystem
Calls detected	map, filter, join
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 6 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

2391: function plainTextFromRich(rich, separator = "\n\n") {
2392:   return (rich?.blocks || [])
2393:     .map((block) => {
2394:       if (block.type === "paragraph") return block.text || "";
2395:       if (block.type === "formula") return block.formula || "";
2396:       if (block.type === "code") return block.code || "";
2397:       if (block.type === "image") return [block.title, block.caption].filter(Boolean).join(" ");
2398:       return "";
2399:     })
2400:     .filter(Boolean)
2401:     .join(separator);
2402: }

```

unwrapFormula - lines 2404-2417

Item	Explanation
Source location	Lines 2404-2417 (14 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function unwrapFormula(value) {
Touches	filesystem
Calls detected	trim, match
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

2404: function unwrapFormula(value) {
2405:   const text = String(value || "").trim();
2406:   const wrappers = [
2407:     [/^\$\$([\s\S]+\)\$\$$/, 1],
2408:     [/^\$\$([\s\S]+\)\$\$$/, 1],
2409:     [/^\$\$([\s\S]+\)\$\$$/, 1],
2410:     [/^\$([\s\S]+\)\$\$$/, 1]
2411:   ];
2412:   for (const [pattern, group] of wrappers) {
2413:     const match = text.match(pattern);
2414:     if (match) return match[group].trim();
2415:   }
2416:   return text;
2417: }

```

isFormulaOnly - lines 2419-2426

Item	Explanation
Source location	Lines 2419-2426 (8 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function isFormulaOnly(value) {
Touches	filesystem
Calls detected	trim, looksLikeWebOrContactText
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 4 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

2419: function isFormulaOnly(value) {
2420:   const text = String(value || "").trim();
2421:   if (!text) return false;
2422:   if (looksLikeWebOrContactText(text)) return false;
2423:   if (/^\$\$([\s\S]+\)\$\$$/ .test(text) || /^\$\$([\s\S]+\)\$\$$/ .test(text) || /^\$([\s\S]+\)\$\$$/ .test(text))
return true;
2424:   const mathSignals = /\frac|\sqrt|\sum|\int|\Delta|\omega|\pi|[\^_/\Sigma\Omega]/;
2425:   return text.length <= 180 && mathSignals.test(text) && !/[.!?]\s*$/ .test(text);
2426: }

```

renderInlineMath - lines 2428-2437

Item	Explanation
Source location	Lines 2428-2437 (10 source lines).
Purpose	Renders UI, markup, or display output.
Declaration	function renderInlineMath(text) {

Item	Explanation
Touches	filesystem
Calls detected	escapeHtml, replace, b, match, slice, normalizeLinkTarget
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

2428: function renderInlineMath(text) {
2429:   const escaped = escapeHtml(text);
2430:   const withMath = escaped.replace(/\$([\^$]+)\$/g, '<span class="rich-inline-formula">$1</span>');
2431:   return withMath.replace(/\b(https?:\/\/[\^\\s<]+|www\.[\^\\s<]+|(?:(?:github|linkedin|gitlab|bitbucket|drive|d
ocs)\.com\/[\^\\s<]+))/gi, (match) => {
2432:     const trailing = match.match(/[(),.;:!?]+\$/)?.[0] || "";
2433:     const clean = trailing ? match.slice(0, -trailing.length) : match;
2434:     const href = normalizeLinkTarget(clean, { assumeWeb: true });
2435:     return `<a href="${href}" target="_blank" rel="noreferrer">${clean}</a>${trailing}`;
2436:   });
2437: }

```

renderMultilineInlineText - lines 2439-2441

Item	Explanation
Source location	Lines 2439-2441 (3 source lines).
Purpose	Renders UI, markup, or display output.
Declaration	function renderMultilineInlineText(text = "") {
Touches	filesystem
Calls detected	renderInlineMath, replace
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

2439: function renderMultilineInlineText(text = "") {
2440:   return renderInlineMath(text).replace(/\r\n?|\n/g, "<br>");
2441: }

```

looksLikeBareWebAddress - lines 2443-2450

Item	Explanation
Source location	Lines 2443-2450 (8 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function looksLikeBareWebAddress(value = "") {
Touches	Mostly local calculations or local UI state.
Calls detected	trim, split, toLowerCase
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 4 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
2443: function looksLikeBareWebAddress(value = "") {
2444:   const clean = String(value || "").trim();
2445:   if (!clean || /\s/.test(clean) || /^(\.?\.\.?\/|#|mailto:|tel:)/i.test(clean)) return false;
2446:   if (/^[a-z][a-z0-9+.-]*:/i.test(clean)) return false;
2447:   const host = clean.split(/[/?#]/)[0].toLowerCase();
2448:   if (!/^[a-z0-9-]+\.(?![a-z0-9-]+)$/.test(host)) return false;
2449:   return !/\.(pdf|docx?|xlsx?|pptx?|zip|7z|rar|png|jpe?g|gif|svg|webp|txt|md|csv|json|xml|log|c|h|cpp|hpp|py|js|mjs|ts|v|sv|vhd|l?|spice|cir|net|asc|sch|kicad_sch|kicad_pcb)$/i.test(host);
2450: }
```

looksLikeWebOrContactText - lines 2452-2460

Item	Explanation
Source location	Lines 2452-2460 (9 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function looksLikeWebOrContactText(value = "") {
Touches	Mostly local calculations or local UI state.
Calls detected	trim, b, looksLikeBareWebAddress, split, some, replace
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 6 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
2452: function looksLikeWebOrContactText(value = "") {
2453:   const clean = String(value || "").trim();
2454:   if (!clean) return false;
2455:   if (/^b(?:https?:\/\/|www\.)[^s<>"]+$/i.test(clean)) return true;
2456:   if (/^b[a-z0-9._%+~]@[a-z0-9.-]+\.[a-z]{2,}\b/i.test(clean)) return true;
2457:   if (/^b(?:github|linkedin|gitlab|bitbucket|drive|docs)\.com\/[^s<>"]+$/i.test(clean)) return true;
2458:   if (looksLikeBareWebAddress(clean)) return true;
2459:   return clean.split(/\s+/).some((word) => looksLikeBareWebAddress(word.replace(/^[("'+|[]'".,;:!?]+$/g, "")));
2460: }
```

normalizeLinkTarget - lines 2462-2469

Item	Explanation
Source location	Lines 2462-2469 (8 source lines).
Purpose	Validates or normalizes input so later code receives safe predictable values.
Declaration	function normalizeLinkTarget(target = "", options = {}) {
Touches	filesystem, network/API requests
Calls detected	trim, looksLikeBareWebAddress
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 5 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
2462: function normalizeLinkTarget(target = "", options = {}) {
2463:   const clean = String(target || "").trim();
2464:   if (!clean) return "";
2465:   if (/^\/\//.test(clean)) return `https:${clean}`;
2466:   if (/^www\./i.test(clean)) return `https://${clean}`;
2467:   if (options.assumeWeb && looksLikeBareWebAddress(clean)) return `https://${clean}`;
2468:   return clean;
2469: }
```

isWebsiteLinkItem - lines 2471-2479

Item	Explanation
Source location	Lines 2471-2479 (9 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function isWebsiteLinkItem(item = {}, target = "") {
Touches	Mostly local calculations or local UI state.
Calls detected	trim, filter, join, toLowerCase, b
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
2471: function isWebsiteLinkItem(item = {}, target = "") {
2472:   const clean = String(target || "").trim();
2473:   if (/^(https?:)?\:\/\/.i.test(clean) || /^www\..i.test(clean)) return true;
2474:   const typeText = [item.kind, item.type, item.status, item.meta, item.label, item.title]
2475:     .filter(Boolean)
2476:     .join(" ")
2477:     .toLowerCase();
2478:   return /\b(link|website|web link|url|external|github|linkedin|drive|social|profile)\b/.test(typeText);
2479: }
```

extensionFromUrl - lines 2481-2485

Item	Explanation
Source location	Lines 2481-2485 (5 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function extensionFromUrl(value = "") {
Touches	Mostly local calculations or local UI state.
Calls detected	split, match, toLowerCase
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
2481: function extensionFromUrl(value = "") {
2482:   const clean = String(value || "").split(/[?#]/)[0];
2483:   const match = clean.match(/(\.[a-z0-9_]+)$/i);
2484:   return match ? match[1].toLowerCase() : "";
2485: }
```

urlLooksLikeDirectImage - lines 2487-2491

Item	Explanation
Source location	Lines 2487-2491 (5 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function urlLooksLikeDirectImage(value = "") {
Touches	filesystem
Calls detected	normalizeLinkTarget
API endpoints	None detected inside this function body.

Item	Explanation
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

2487: function urlLooksLikeDirectImage(value = "") {
2488:   const clean = normalizeLinkTarget(value, { assumeweb: true });
2489:   return /^(data:image\/|blob:)/i.test(clean) ||
2490:     /\.(png|jpe?g|webp|gif|svg|bmp|avif)([?#].*)?$/i.test(clean);
2491: }

```

inferUrlAssetKind - lines 2493-2506

Item	Explanation
Source location	Lines 2493-2506 (14 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function inferUrlAssetKind(value = "", source = "web") {
Touches	filesystem
Calls detected	normalizeLinkTarget, extensionFromUrl, urlLooksLikeDirectImage, includes, looksLikeBareWebAddress
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 10 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

2493: function inferUrlAssetKind(value = "", source = "web") {
2494:   const clean = normalizeLinkTarget(value, { assumeweb: true });
2495:   const extension = extensionFromUrl(clean);
2496:   if (source === "drive" || /^https?:\/\/?(\drive|docs)\.google\.com\/\//i.test(clean)) return "google-
drive";
2497:   if (/github\.com|raw\.githubusercontent\.com/i.test(clean)) return "github";
2498:   if (/linkedin\.com/i.test(clean)) return "linkedin";
2499:   if (urlLooksLikeDirectImage(clean)) return "image";
2500:   if ([".pdf"].includes(extension)) return "pdf";
2501:   if ([".doc", ".docx", ".ppt", ".pptx", ".xls", ".xlsx", ".csv", ".txt", ".md"].includes(extension))
return "document";
2502:   if ([".zip", ".7z", ".rar"].includes(extension)) return "archive";
2503:   if ([".c", ".h", ".cpp", ".hpp", ".py", ".js", ".mjs", ".ts", ".v", ".sv", ".vhd", ".vhdl", ".spice",
".cir", ".net", ".asc", ".sch", ".kicad_sch", ".kicad_pcb"].includes(extension)) return "engineering-file";
2504:   if (/^https?:\/\//i.test(clean) || /^www\./i.test(value) || looksLikeBareWebAddress(value)) return
"webpage";
2505:   return extension ? "file" : "link";
2506: }

```

labelForUrlAssetKind - lines 2508-2522

Item	Explanation
Source location	Lines 2508-2522 (15 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function labelForUrlAssetKind(kind = "link") {
Touches	Mostly local calculations or local UI state.
Calls detected	No obvious named calls detected by the generator.
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).

Item	Explanation
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

2508: function labelForUrlAssetKind(kind = "link") {
2509:   return {
2510:     archive: "Archive link",
2511:     document: "Document link",
2512:     "engineering-file": "Engineering file link",
2513:     file: "File link",
2514:     github: "GitHub link",
2515:     "google-drive": "Google Drive link",
2516:     image: "Image URL",
2517:     linkedin: "LinkedIn link",
2518:     link: "Link",
2519:     pdf: "PDF link",
2520:     webpage: "Website link"
2521:   }[kind] || "Link";
2522: }

```

displayNameFromUrl - lines 2524-2533

Item	Explanation
Source location	Lines 2524-2533 (10 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function displayNameFromUrl(value = "", fallback = "Linked asset") {
Touches	filesystem
Calls detected	normalizeLinkTarget, URL, split, filter, pop, replace
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

2524: function displayNameFromUrl(value = "", fallback = "Linked asset") {
2525:   const clean = normalizeLinkTarget(value, { assumeWeb: true });
2526:   try {
2527:     const parsed = new URL(clean);
2528:     const lastPath = parsed.pathname.split("/").filter(Boolean).pop();
2529:     return decodeURIComponent(lastPath || parsed.hostname.replace(/^www\./, "")) || fallback;
2530:   } catch {
2531:     return String(value || "").split("/").filter(Boolean).pop() || fallback;
2532:   }
2533: }

```

linkifyRichTextNodes - lines 2535-2570

Item	Explanation
Source location	Lines 2535-2570 (36 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function linkifyRichTextNodes(root) {
Touches	browser/Electron DOM, filesystem
Calls detected	createTreeWalker, acceptNode, closest, nextNode, push, forEach, createDocumentFragment, replace, b, append, createTextNode, slice
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 3 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
2535: function linkifyRichTextNodes(root) {
2536:   const walker = document.createTreeWalker(root, NodeFilter.SHOW_TEXT, {
2537:     acceptNode(node) {
2538:       if (!node.textContent ||
!/(https?:\/\/|www\.(?:github|linkedin|gitlab|bitbucket|drive|docs)\.com\/)/i.test(node.textContent)) return
NodeFilter.FILTER_REJECT;
2539:       return node.parentElement?.closest("a") ? NodeFilter.FILTER_REJECT : NodeFilter.FILTER_ACCEPT;
2540:     }
2541:   });
2542:   const nodes = [];
2543:   let node = walker.nextNode();
2544:   while (node) {
2545:     nodes.push(node);
2546:     node = walker.nextNode();
2547:   }
2548:
2549:   nodes.forEach((textNode) => {
2550:     const text = textNode.textContent || "";
2551:     const fragment = document.createDocumentFragment();
2552:     let cursor = 0;
2553:     text.replace(/\b(https?:\/\/|www\.(?:github|linkedin|gitlab|bitbucket|drive|docs)\.com
\/(?:[^\s<]+)\/gi, (match, _url, offset) => {
2554:       if (offset > cursor) fragment.append(document.createTextNode(text.slice(cursor, offset)));
2555:       const trailing = match.match(/(?:,;:!?)+$/)?.[0] || "";
2556:       const clean = trailing ? match.slice(0, -trailing.length) : match;
2557:       const link = document.createElement("a");
2558:       link.href = normalizeLinkTarget(clean, { assumeWeb: true });
2559:       link.target = "_blank";
2560:       link.rel = "noreferrer";
2561:       link.textContent = clean;
2562:       fragment.append(link);
2563:       if (trailing) fragment.append(document.createTextNode(trailing));
2564:       cursor = offset + match.length;
2565:       return match;
2566:     });
2567:     if (cursor < text.length) fragment.append(document.createTextNode(text.slice(cursor)));
2568:     textNode.replaceWith(fragment);
2569:   });
2570: }
```

inlineStyleSignature - lines 2572-2583

Item	Explanation
Source location	Lines 2572-2583 (12 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function inlineStyleSignature(element) {
Touches	Mostly local calculations or local UI state.
Calls detected	join
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
2572: function inlineStyleSignature(element) {
2573:   const style = element?.style;
2574:   if (!style) return "";
2575:   return [
2576:     style.fontFamily || "",
2577:     style.fontSize || "",
2578:     style.color || "",
2579:     style.fontWeight || "",
2580:     style.fontStyle || "",
2581:     style.textDecoration || ""
2582:   ].join("|");
2583: }
```

normalizeInlineStyleSpans - lines 2585-2605

Item	Explanation
Source location	Lines 2585-2605 (21 source lines).

Item	Explanation
Purpose	Validates or normalizes input so later code receives safe predictable values.
Declaration	function normalizeInlineStyleSpans(root) {
Touches	browser/Electron DOM, filesystem
Calls detected	querySelectorAll, forEach, remove, getAttribute, replaceWith, normalize, matches, inlineStyleSignature, append
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

2585: function normalizeInlineStyleSpans(root) {
2586:   if (!root) return;
2587:   root.querySelectorAll("span.rich-inline-style, span[style]").forEach((span) => {
2588:     span.style.display = "inline";
2589:     if (!span.textContent) {
2590:       span.remove();
2591:       return;
2592:     }
2593:     if (!span.getAttribute("style")) span.replaceWith(...span.childNodes);
2594:   });
2595:   root.normalize();
2596:   root.querySelectorAll("span.rich-inline-style, span[style]").forEach((span) => {
2597:     let next = span.nextSibling;
2598:     while (next?.nodeType === Node.ELEMENT_NODE && next.matches("span.rich-inline-style, span[style]") &&
inlineStyleSignature(next) === inlineStyleSignature(span)) {
2599:       span.append(...next.childNodes);
2600:       const remove = next;
2601:       next = next.nextSibling;
2602:       remove.remove();
2603:     }
2604:   });
2605: }

```

sanitizeRichInlineHtml - lines 2607-2651

Item	Explanation
Source location	Lines 2607-2651 (45 source lines).
Purpose	Validates or normalizes input so later code receives safe predictable values.
Declaration	function sanitizeRichInlineHtml(value = "") {
Touches	browser/Electron DOM, filesystem
Calls detected	createElement, Set, querySelectorAll, forEach, has, replaceWith, getAttribute, trim, normalizeLinkTarget, cleanFontFamily, normalizeFontPx, normalizeTextColor
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

2607: function sanitizeRichInlineHtml(value = "") {
2608:   const template = document.createElement("template");
2609:   template.innerHTML = String(value || "");
2610:   const allowedTags = new Set(["A", "B", "BR", "EM", "I", "SPAN", "STRONG", "U"]);
2611:
2612:   [...template.content.querySelectorAll("*")].forEach((node) => {
2613:     if (!allowedTags.has(node.tagName)) {
2614:       node.replaceWith(...node.childNodes);
2615:       return;
2616:     }
2617:
2618:     const href = node.tagName === "A" ? String(node.getAttribute("href") || "").trim() : "";
2619:     const normalizedHref = normalizeLinkTarget(href, { assumeweab: true });
2620:     const safeHref = /^((https?:|mailto:|tel:|)|\s)/i.test(normalizedHref) ? normalizedHref : "";
2621:     const fontFamily = cleanFontFamily(node.style.fontFamily || "");

```

```

2622:     const fontPx = normalizeFontPx(parseFloat(node.style.fontSize || ""));
2623:     const color = normalizeTextColor(node.style.color || "");
2624:     const rawFontWeight = node.style.fontWeight || "";
2625:     const fontWeight = /^(bold|[6-9]00)$/i.test(rawFontWeight)
2626:       ? "700"
2627:       : /^(normal|[1-5]00)$/i.test(rawFontWeight)
2628:       ? "400"
2629:       : "";
2630:     const fontStyle = node.style.fontStyle === "italic" ? "italic" : "";
2631:     const textDecoration = node.style.textDecoration.includes("underline") ? "underline" : "";
2632:
2633:     [...node.attributes].forEach((attribute) => node.removeAttribute(attribute.name));
2634:
2635:     if (node.tagName === "A" && safeHref) {
2636:       node.setAttribute("href", safeHref);
2637:       node.setAttribute("target", "_blank");
2638:       node.setAttribute("rel", "noreferrer");
2639:     }
2640:     if (fontFamily) node.style.fontFamily = fontFamily;
2641:     if (fontPx) node.style.fontSize = `${fontPx}px`;
2642:     if (color) node.style.color = color;
2643:     if (fontWeight) node.style.fontWeight = fontWeight;
2644:     if (fontStyle) node.style.fontStyle = fontStyle;
2645:     if (textDecoration) node.style.textDecoration = textDecoration;
2646:   });
2647:
2648:   linkifyRichTextNodes(template.content);
2649:   normalizeInlineStyleSpans(template.content);
2650:   return template.innerHTML;
2651: }

```

displayTitle - lines 2653-2662

Item	Explanation
Source location	Lines 2653-2662 (10 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function displayTitle(value, fallback = "Untitled item") {
Touches	Mostly local calculations or local UI state.
Calls detected	trim
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

2653: function displayTitle(value, fallback = "Untitled item") {
2654:   const title = String(value || fallback).trim();
2655:   const replacements = {
2656:     "Brief": "Overview",
2657:     "Project Brief": "Overview",
2658:     "Design Brief": "Design Overview",
2659:     "Simulation Brief": "Simulation Overview"
2660:   };
2661:   return replacements[title] || title;
2662: }

```

cleanFontFamily - lines 2664-2671

Item	Explanation
Source location	Lines 2664-2671 (8 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function cleanFontFamily(value = "") {
Touches	Mostly local calculations or local UI state.
Calls detected	split, map, replace, trim, filter, slice, join
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.

Item	Explanation
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

2664: function cleanFontFamily(value = "") {
2665:   return String(value)
2666:     .split(",")
2667:     .map((item) => item.replace(/^[^w\s-]/g, "").trim())
2668:     .filter(Boolean)
2669:     .slice(0, 3)
2670:     .join(", ");
2671: }

```

normalizeFontPx - lines 2673-2676

Item	Explanation
Source location	Lines 2673-2676 (4 source lines).
Purpose	Validates or normalizes input so later code receives safe predictable values.
Declaration	function normalizeFontPx(value) {
Touches	filesystem
Calls detected	isFinite
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

2673: function normalizeFontPx(value) {
2674:   const size = Number(value);
2675:   return Number.isFinite(size) && size >= 8 && size <= 24 ? String(size) : "";
2676: }

```

normalizeTextColor - lines 2677-2684

Item	Explanation
Source location	Lines 2677-2684 (8 source lines).
Purpose	Validates or normalizes input so later code receives safe predictable values.
Declaration	function normalizeTextColor(value = "") {
Touches	filesystem
Calls detected	trim
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

2677: function normalizeTextColor(value = "") {
2678:   const color = String(value || "").trim();
2679:   return /^(#[0-9a-f]{3}|[0-9a-f]{6})$/i.test(color) ||
2680:     /^rgba?\(/i.test(color) ||
2681:     /^hsla?\(/i.test(color)
2682:     ? color
2683:     : "";
2684: }

```

summaryRichPathFromTextPath - lines 2686-2688

Item	Explanation
Source location	Lines 2686-2688 (3 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function summaryRichPathFromTextPath(pathValue = "") {
Touches	Mostly local calculations or local UI state.
Calls detected	replace
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
2686: function summaryRichPathFromTextPath(pathValue = "") {
2687:   return String(pathValue || "").replace(/\.summary$/, ".summaryRich");
2688: }
```

overviewFolderFromPath - lines 2690-2695

Item	Explanation
Source location	Lines 2690-2695 (6 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function overviewFolderFromPath(pathValue = "") {
Touches	Mostly local calculations or local UI state.
Calls detected	includes
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 4 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
2690: function overviewFolderFromPath(pathValue = "") {
2691:   if (pathValue.includes("design.brief")) return "design-overview";
2692:   if (pathValue.includes("design.documentation")) return "documentation-overview";
2693:   if (pathValue.includes("design.simulation")) return "simulation-overview";
2694:   return "overview";
2695: }
```

richBlocksFromValue - lines 2697-2699

Item	Explanation
Source location	Lines 2697-2699 (3 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function richBlocksFromValue(rich, fallbackText = "") {
Touches	Mostly local calculations or local UI state.
Calls detected	clone, textBlocksFromPlainText
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).

Item	Explanation
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

2697: function richBlocksFromValue(rich, fallbackText = "") {
2698:     return rich?.blocks?.length ? clone(rich.blocks) : textBlocksFromPlainText(fallbackText);
2699: }

```

richTextStyle - lines 2700-2714

Item	Explanation
Source location	Lines 2700-2714 (15 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function richTextStyle(block = {}) {
Touches	filesystem
Calls detected	cleanFontFamily, normalizeFontPx, normalizeTextColor, push, escapeHtml, join
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

2700: function richTextStyle(block = {}) {
2701:     const styles = [];
2702:     const fontFamily = cleanFontFamily(block.fontFamily || "Arial") || "Arial";
2703:     const fontPx = normalizeFontPx(block.fontPx);
2704:     const color = normalizeTextColor(block.color || "");
2705:
2706:     if (fontFamily) styles.push(`font-family: ${fontFamily}`);
2707:     if (fontPx) styles.push(`font-size: ${fontPx}px`);
2708:     if (color) styles.push(`color: ${color}`);
2709:     if (block.bold) styles.push("font-weight: 700");
2710:     if (block.italic) styles.push("font-style: italic");
2711:     if (block.underline) styles.push("text-decoration: underline");
2712:
2713:     return styles.length ? ` style="${escapeHtml(styles.join("; "))}"` : "";
2714: }

```

normalizeCropAspect - lines 2716-2718

Item	Explanation
Source location	Lines 2716-2718 (3 source lines).
Purpose	Validates or normalizes input so later code receives safe predictable values.
Declaration	function normalizeCropAspect(value = "") {
Touches	filesystem
Calls detected	includes
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

2716: function normalizeCropAspect(value = "") {
2717:     return ["1 / 1", "4 / 3", "16 / 9", "3 / 4"].includes(value) ? value : "original";
2718: }

```

normalizeCropZoom - lines 2720-2723

Item	Explanation
Source location	Lines 2720-2723 (4 source lines).
Purpose	Validates or normalizes input so later code receives safe predictable values.
Declaration	function normalizeCropZoom(value = 1) {
Touches	filesystem
Calls detected	isFinite, min, max
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
2720: function normalizeCropZoom(value = 1) {  
2721:   const zoom = Number(value);  
2722:   return Number.isFinite(zoom) ? Math.min(3, Math.max(1, zoom)) : 1;  
2723: }
```

normalizeCropPosition - lines 2725-2728

Item	Explanation
Source location	Lines 2725-2728 (4 source lines).
Purpose	Validates or normalizes input so later code receives safe predictable values.
Declaration	function normalizeCropPosition(value = 50) {
Touches	filesystem
Calls detected	isFinite, min, max
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
2725: function normalizeCropPosition(value = 50) {  
2726:   const position = Number(value);  
2727:   return Number.isFinite(position) ? Math.min(100, Math.max(0, position)) : 50;  
2728: }
```

richImageCropStyle - lines 2730-2738

Item	Explanation
Source location	Lines 2730-2738 (9 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function richImageCropStyle(block = {}) {
Touches	filesystem
Calls detected	normalizeCropAspect, normalizeCropZoom, normalizeCropPosition, push, escapeHtml, join
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).

Item	Explanation
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

2730: function richImageCropStyle(block = {}) {
2731:   const aspect = normalizeCropAspect(block.cropAspect);
2732:   const zoom = normalizeCropZoom(block.cropZoom);
2733:   const x = normalizeCropPosition(block.cropX);
2734:   const y = normalizeCropPosition(block.cropY);
2735:   const styles = [ `--crop-zoom: ${zoom}`, `--crop-x: ${x}%`, `--crop-y: ${y}%` ];
2736:   if (aspect !== "original") styles.push(`--crop-aspect: ${aspect}`);
2737:   return ` style="${escapeHtml(styles.join(" ");)}"`;
2738: }

```

richImageDownloadLink - lines 2740-2745

Item	Explanation
Source location	Lines 2740-2745 (6 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function richImageDownloadLink(block = {}) {
Touches	filesystem
Calls detected	escapeHtml, cleanRichImageTitle, normalizeLinkTarget, isWebsiteLinkItem, linkAttributes, downloadAttribute
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

2740: function richImageDownloadLink(block = {}) {
2741:   const label = escapeHtml(cleanRichImageTitle(block) || block.caption || "Image file");
2742:   const url = block.url || "#";
2743:   const target = normalizeLinkTarget(url, { assumeWeb: isWebsiteLinkItem(block, url) });
2744:   return `<p class="rich-download-only"><a class="resource-link"
href="${escapeHtml(target)}"${linkAttributes(target, block)}${downloadAttribute(target,
block)}>${label}</a></p>`;
2745: }

```

cleanRichImageTitle - lines 2747-2750

Item	Explanation
Source location	Lines 2747-2750 (4 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function cleanRichImageTitle(block = {}) {
Touches	Mostly local calculations or local UI state.
Calls detected	trim
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

2747: function cleanRichImageTitle(block = {}) {
2748:   const title = String(block.title || "").trim();
2749:   return /^pasted image\b/i.test(title) ? "" : title;
2750: }

```

renderRichContent - lines 2752-2787

Item	Explanation
Source location	Lines 2752-2787 (36 source lines).
Purpose	Renders UI, markup, or display output.
Declaration	function renderRichContent(rich, fallbackText = "") {
Touches	filesystem
Calls detected	textBlocksFromPlainText, map, includes, richImageDownloadLink, cleanRichImageTitle, normalizeLinkTarget, normalizeCropAspect, richImageCropStyle, escapeHtml, unwrapFormula, looksLikeWebOrContactText, renderInlineMath
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 7 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
2752: function renderRichContent(rich, fallbackText = "") {
2753:   const blocks = rich?.blocks?.length ? rich.blocks : textBlocksFromPlainText(fallbackText);
2754:   return `
2755:     <div class="rich-content">
2756:       ${blocks.map((block) => {
2757:         const align = ["left", "center", "right"].includes(block.align) ? block.align : "left";
2758:         if (block.type === "image") {
2759:           if (block.display === "download") return richImageDownloadLink(block);
2760:           const title = cleanRichImageTitle(block);
2761:           const imageSrc = normalizeLinkTarget(block.url, { assumeWeb: true });
2762:           return `
2763:             <figure class="rich-image justify-${align}">
2764:               <span class="rich-image-viewport crop-${normalizeCropAspect(block.cropAspect)} ===
"original" ? "original" : "active"${richImageCropStyle(block)}>
2765:                 
2766:               </span>
2767:               ${((title || block.caption) ? `<figcaption>${title ? `<strong>${escapeHtml(title)}</strong>`
: ""}${block.caption ? `<span>${escapeHtml(block.caption)}</span>` : ""}</figcaption>` : ""}
2768:             </figure>
2769:           `;
2770:         }
2771:         if (block.type === "formula") {
2772:           const formula = unwrapFormula(block.formula);
2773:           if (looksLikeWebOrContactText(formula)) {
2774:             return `<p class="rich-paragraph rich-text-normal
text-${align}">${renderInlineMath(formula)}</p>`;
2775:           }
2776:           return `<div class="rich-formula justify-${align}">${escapeHtml(formula)}</div>`;
2777:         }
2778:         if (block.type === "code") return renderRichCodeBlock(block);
2779:         const size = ["small", "normal", "large"].includes(block.fontSize) ? block.fontSize : "normal";
2780:         const content = block.html
          ? sanitizeRichInlineHtml(block.html)
2781:          : renderInlineMath(block.text || "");
2782:         return `<p class="rich-paragraph rich-text-${size}
text-${align}"${richTextStyle(block)}>${content}</p>`;
2783:       }).join("")}
2784:     </div>
2785:   `;
2786: }
2787: }
```

renderRichFieldContent - lines 2789-2799

Item	Explanation
Source location	Lines 2789-2799 (11 source lines).
Purpose	Renders UI, markup, or display output.
Declaration	function renderRichFieldContent(rich, fallbackText = "") {
Touches	Mostly local calculations or local UI state.
Calls detected	textBlocksFromPlainText, map, includes, sanitizeRichInlineHtml, renderInlineMath, richTextStyle, filter, join
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.

Item	Explanation
Async/return clues	0 await expression(s), 3 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

2789: function renderRichFieldContent(rich, fallbackText = "") {
2790:   const blocks = rich?.blocks?.length ? rich.blocks : textBlocksFromPlainText(fallbackText);
2791:   return blocks.map((block) => {
2792:     if (block.type !== "paragraph") return "";
2793:     const align = ["left", "center", "right"].includes(block.align) ? block.align : "left";
2794:     const content = block.html
2795:       ? sanitizeRichInlineHtml(block.html)
2796:       : renderInlineMath(block.text || "");
2797:     return `<span class="rich-field-line text-${align}"${richTextStyle(block)}>${content}</span>`;
2798:   }).filter(Boolean).join("");
2799: }

```

saveSelectedProjectToPortfolio - lines 2801-2817

Item	Explanation
Source location	Lines 2801-2817 (17 source lines).
Purpose	Persists data to local state, disk, credentials, or browser storage.
Declaration	function saveSelectedProjectToPortfolio() {
Touches	Mostly local calculations or local UI state.
Calls detected	selectedProject, parseProjectForPortfolio, syncParsedPortfolioGlobalsIntoSaved, filter, Set, map, has, findIndex, max, splice, markDraftNeedsSave, refreshOpenPreviews
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

2801: function saveSelectedProjectToPortfolio() {
2802:   const project = selectedProject();
2803:   if (!project) return false;
2804:   const parsedProject = parseProjectForPortfolio(project);
2805:   syncParsedPortfolioGlobalsIntoSaved();
2806:   const nextProjects = (savedPortfolioCatalog.projects || []).filter((item) => item.id !== project.id);
2807:   const workingIds = new Set((catalog.projects || []).map((item) => item.id));
2808:   savedPortfolioCatalog.projects = nextProjects.filter((item) => workingIds.has(item.id));
2809:   const workingIndex = catalog.projects.findIndex((item) => item.id === project.id);
2810:   const insertAt = Math.max(0, workingIndex);
2811:   savedPortfolioCatalog.projects.splice(insertAt, 0, parsedProject);
2812:   markDraftNeedsSave();
2813:   refreshOpenPreviews();
2814:   setStatus(`${project.title} saved into the portfolio preview.`);
2815:   updateBuilderWorkflow();
2816:   return true;
2817: }

```

nextProjects - lines 2806-2814

Item	Explanation
Source location	Lines 2806-2814 (9 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	const nextProjects = (savedPortfolioCatalog.projects []).filter((item) => item.id !== project.id);
Touches	Mostly local calculations or local UI state.
Calls detected	filter, Set, map, has, findIndex, max, splice, markDraftNeedsSave, refreshOpenPreviews, setStatus
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.

Item	Explanation
Async/return clues	0 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

2806: const nextProjects = (savedPortfolioCatalog.projects || []).filter((item) => item.id !== project.id);
2807: const workingIds = new Set((catalog.projects || []).map((item) => item.id));
2808: savedPortfolioCatalog.projects = nextProjects.filter((item) => workingIds.has(item.id));
2809: const workingIndex = catalog.projects.findIndex((item) => item.id === project.id);
2810: const insertAt = Math.max(0, workingIndex);
2811: savedPortfolioCatalog.projects.splice(insertAt, 0, parsedProject);
2812: markDraftNeedsSave();
2813: refreshOpenPreviews();
2814: setStatus(`${project.title} saved into the portfolio preview.`);

```

removeProjectFromPortfolio - lines 2819-2821

Item	Explanation
Source location	Lines 2819-2821 (3 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function removeProjectFromPortfolio(projectId) {
Touches	Mostly local calculations or local UI state.
Calls detected	filter
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

2819: function removeProjectFromPortfolio(projectId) {
2820:   savedPortfolioCatalog.projects = (savedPortfolioCatalog.projects || []).filter((item) => item.id !==
projectId);
2821: }

```

deleteProjectById - lines 2823-2836

Item	Explanation
Source location	Lines 2823-2836 (14 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function deleteProjectById(projectId) {
Touches	browser/Electron DOM
Calls detected	find, filter, removeProjectFromPortfolio, closeDialogElement, setStatus, scheduleAutosave, renderAll
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

2823: function deleteProjectById(projectId) {
2824:   const project = catalog.projects.find((item) => item.id === projectId);
2825:   if (!project) return;
2826:   catalog.projects = catalog.projects.filter((item) => item.id !== project.id);
2827:   removeProjectFromPortfolio(project.id);
2828:   selectedProjectId = catalog.projects[0]?.id || "";
2829:   activeSectionId = "brief";
2830:   if (projectDialog?.open && project.id === projectWindowTitle.dataset.projectId) {

```



```

2831:     closeModalElement(projectDialog);
2832:   }
2833:   setStatus("Project deleted locally.");
2834:   scheduleAutosave();
2835:   renderAll();
2836: }

```

openDeleteConfirm - lines 2838-2843

Item	Explanation
Source location	Lines 2838-2843 (6 source lines).
Purpose	Controls a window, dialog, panel, menu, or visible state.
Declaration	function openDeleteConfirm({ title = "Delete project", message, onConfirm }) {
Touches	filesystem
Calls detected	showModal
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

2838: function openDeleteConfirm({ title = "Delete project", message, onConfirm }) {
2839:   pendingDeleteAction = onConfirm;
2840:   deleteConfirmTitle.textContent = title;
2841:   deleteConfirmMessage.textContent = message;
2842:   deleteConfirmDialog.showModal();
2843: }

```

requestProjectDelete - lines 2845-2853

Item	Explanation
Source location	Lines 2845-2853 (9 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function requestProjectDelete(projectId) {
Touches	filesystem
Calls detected	find, openDeleteConfirm, deleteProjectById
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

2845: function requestProjectDelete(projectId) {
2846:   const project = catalog.projects.find((item) => item.id === projectId);
2847:   if (!project) return;
2848:   openDeleteConfirm({
2849:     title: "Delete project",
2850:     message: "Are you sure you want to delete this project?",
2851:     onConfirm: () => deleteProjectById(project.id)
2852:   });
2853: }

```

parserItem - lines 2855-2866

Item	Explanation
Source location	Lines 2855-2866 (12 source lines).
Purpose	Part of the private builder frontend and editing experience.

Item	Explanation
Declaration	function parserItem(title, description = "", url = "", meta = "", kind = "text", rich = null, children = []) {
Touches	Mostly local calculations or local UI state.
Calls detected	filter, clone
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

2855: function parserItem(title, description = "", url = "", meta = "", kind = "text", rich = null, children =
[]) {
2856:   const item = {
2857:     children: children.filter((child) => child && (child.title || child.description || child.url ||
child.children?.length)),
2858:     description: description || "",
2859:     kind,
2860:     meta: meta || "",
2861:     title: title || "Untitled item",
2862:     url: url || ""
2863:   };
2864:   if (rich?.blocks?.length) item.rich = clone(rich);
2865:   return item;
2866: }

```

canonicalTemplateId - lines 2868-2870

Item	Explanation
Source location	Lines 2868-2870 (3 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function canonicalTemplateId(id) {
Touches	Mostly local calculations or local UI state.
Calls detected	No obvious named calls detected by the generator.
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

2868: function canonicalTemplateId(id) {
2869:   return legacyTemplateSkins[id] || id || "";
2870: }

```

projectTemplateId - lines 2872-2874

Item	Explanation
Source location	Lines 2872-2874 (3 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function projectTemplateId(project) {
Touches	Mostly local calculations or local UI state.
Calls detected	canonicalTemplateId
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).

Item	Explanation
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

2872: function projectTemplateId(project) {
2873:   return canonicalTemplateId(project?.portfolioView?.template?.id || project?.templateId || "");
2874: }

```

projectTemplateFor - lines 2876-2879

Item	Explanation
Source location	Lines 2876-2879 (4 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function projectTemplateFor(project) {
Touches	Mostly local calculations or local UI state.
Calls detected	projectTemplateId, find
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

2876: function projectTemplateFor(project) {
2877:   const templateId = projectTemplateId(project);
2878:   return templates.find((template) => template.id === templateId) || {};
2879: }

```

projectTemplateClass - lines 2881-2884

Item	Explanation
Source location	Lines 2881-2884 (4 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function projectTemplateClass(project) {
Touches	Mostly local calculations or local UI state.
Calls detected	projectTemplateId
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

2881: function projectTemplateClass(project) {
2882:   const templateId = projectTemplateId(project);
2883:   return templateId ? `project-template project-template-${templateId}` : "project-template-white";
2884: }

```

responsiveClassName - lines 2886-2891

Item	Explanation
Source location	Lines 2886-2891 (6 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function responsiveClassName(value, fallback) {

Item	Explanation
Touches	Mostly local calculations or local UI state.
Calls detected	toLowerCase, replace
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

2886: function responsiveClassName(value, fallback) {
2887:   return String(value || fallback || "")
2888:     .toLowerCase()
2889:     .replace(/^[^a-z0-9]+/g, "-")
2890:     .replace(/^-|-$/g, "") || fallback;
2891: }

```

projectResponsiveClass - lines 2893-2896

Item	Explanation
Source location	Lines 2893-2896 (4 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function projectResponsiveClass(project) {
Touches	Mostly local calculations or local UI state.
Calls detected	projectResponsiveProfile, responsiveClassName
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

2893: function projectResponsiveClass(project) {
2894:   const profile = projectResponsiveProfile(project);
2895:   return `responsive-project responsive-density-${responsiveClassName(profile.density, "balanced")}
responsive-card-${responsiveClassName(profile.cardLayout, "single")}`;
2896: }

```

responsiveStyleValues - lines 2898-2906

Item	Explanation
Source location	Lines 2898-2906 (9 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function responsiveStyleValues(project) {
Touches	Mostly local calculations or local UI state.
Calls detected	projectResponsiveProfile
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

2898: function responsiveStyleValues(project) {
2899:   const profile = projectResponsiveProfile(project);

```

```

2900:   return {
2901:     "--responsive-section-card-min": profile.sectionCardMin,
2902:     "--responsive-content-max": profile.contentMax,
2903:     "--responsive-media-max": profile.mediaMax,
2904:     "--responsive-touch-target": profile.touchTarget
2905:   };
2906: }

```

hasPublicTemplate - lines 2908-2910

Item	Explanation
Source location	Lines 2908-2910 (3 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function hasPublicTemplate(project) {
Touches	Mostly local calculations or local UI state.
Calls detected	projectTemplateId
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

2908: function hasPublicTemplate(project) {
2909:   return Boolean(projectTemplateId(project));
2910: }

```

projectTemplateVisual - lines 2912-2914

Item	Explanation
Source location	Lines 2912-2914 (3 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function projectTemplateVisual(project) {
Touches	Mostly local calculations or local UI state.
Calls detected	projectTemplateFor
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

2912: function projectTemplateVisual(project) {
2913:   return project?.portfolioView?.template?.visual || project?.templateVisual ||
projectTemplateFor(project).visual || null;
2914: }

```

projectTemplateStyle - lines 2916-2934

Item	Explanation
Source location	Lines 2916-2934 (19 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function projectTemplateStyle(project, accent) {
Touches	Mostly local calculations or local UI state.
Calls detected	projectTemplateVisual, responsiveStyleValues, entries, filter, map, join

Item	Explanation
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

2916: function projectTemplateStyle(project, accent) {
2917:   const visual = projectTemplateVisual(project) || {};
2918:   const values = {
2919:     "--accent": accent,
2920:     "--template-bg": visual.background,
2921:     "--template-panel": visual.panel,
2922:     "--template-accent": visual.accent,
2923:     "--template-hover": visual.hover,
2924:     "--template-click": visual.click,
2925:     "--template-click-text": visual.clickText,
2926:     "--template-line": visual.line,
2927:     "--template-text": visual.text,
2928:     ...responsiveStyleValues(project)
2929:   };
2930:   return Object.entries(values)
2931:     .filter(([, value]) => value)
2932:     .map(([key, value]) => `${key}: ${value}`)
2933:     .join("; ");
2934: }

```

applyProjectTemplateToElement - lines 2936-2970

Item	Explanation
Source location	Lines 2936-2970 (35 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function applyProjectTemplateToElement(element, project, accent) {
Touches	browser/Electron DOM
Calls detected	filter, startsWith, forEach, remove, projectTemplateClass, split, add, projectResponsiveClass, projectTemplateVisual, responsiveStyleValues, entries, setProperty
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

2936: function applyProjectTemplateToElement(element, project, accent) {
2937:   if (!element) return;
2938:   [...element.classList]
2939:     .filter((className) =>
2940:       className === "project-template" ||
2941:       className === "project-template-white" ||
2942:       className === "responsive-project" ||
2943:       className.startsWith("project-template-") ||
2944:       className.startsWith("responsive-density-") ||
2945:       className.startsWith("responsive-card-")
2946:     )
2947:     .forEach((className) => element.classList.remove(className));
2948:   projectTemplateClass(project).split(" ").forEach((className) => element.classList.add(className));
2949:   projectResponsiveClass(project).split(" ").forEach((className) => element.classList.add(className));
2950:   const visual = projectTemplateVisual(project) || {};
2951:   const values = {
2952:     "--accent": accent,
2953:     "--template-bg": visual.background,
2954:     "--template-panel": visual.panel,
2955:     "--template-accent": visual.accent,
2956:     "--template-hover": visual.hover,
2957:     "--template-click": visual.click,
2958:     "--template-click-text": visual.clickText,
2959:     "--template-line": visual.line,
2960:     "--template-text": visual.text,
2961:     ...responsiveStyleValues(project)
2962:   };

```

```

2963:   Object.entries(values).forEach(([key, value]) => {
2964:     if (value) {
2965:       element.style.setProperty(key, value);
2966:     } else {
2967:       element.style.removeProperty(key);
2968:     }
2969:   });
2970: }

```

resourcesFrom - lines 2972-2988

Item	Explanation
Source location	Lines 2972-2988 (17 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function resourcesFrom(project, key) {
Touches	Mostly local calculations or local UI state.
Calls detected	map, parserItem
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 6 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

2972: function resourcesFrom(project, key) {
2973:   return (project[key] || []).map((item) => {
2974:     if (key === "tests") {
2975:       return parserItem(item.name, item.result || item.method, item.artifact, item.method || item.status
|| "Test", "test", item.richDescription);
2976:     }
2977:     if (key === "pcbs") {
2978:       return parserItem(item.name, item.revision || item.status, item.artifact, item.status || "PCB",
"pcb", item.richDescription);
2979:     }
2980:     if (key === "media") {
2981:       return parserItem(item.title, item.caption, item.url, item.status || "Image", "image",
item.richDescription);
2982:     }
2983:     if (key === "links") {
2984:       return parserItem(item.label, item.description || "", item.url, "Link", "link",
item.richDescription);
2985:     }
2986:     return parserItem(item.title || item.name || item.label, item.description || item.caption, item.url
|| item.artifact, item.type || item.status || "Document", "document", item.richDescription);
2987:   });
2988: }

```

resourcesFromItems - lines 2990-3003

Item	Explanation
Source location	Lines 2990-3003 (14 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function resourcesFromItems(items = [], fallbackKind = "document") {
Touches	Mostly local calculations or local UI state.
Calls detected	map, parserItem
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

2990: function resourcesFromItems(items = [], fallbackKind = "document") {
2991:   return (items || []).map((item) => {

```

```

2992:     const url = item.url || item.artifact || "";
2993:     const kind = item.kind || fallbackKind;
2994:     return parserItem(
2995:         item.title || item.name || item.label || "Untitled item",
2996:         item.description || item.caption || item.result || item.method || "",
2997:         url,
2998:         item.type || item.status || item.method || "",
2999:         kind,
3000:         item.richDescription
3001:     );
3002: });
3003: }

```

parsedSubsection - lines 3005-3007

Item	Explanation
Source location	Lines 3005-3007 (3 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function parsedSubsection(title, description = "", items = [], rich = null) {
Touches	Mostly local calculations or local UI state.
Calls detected	parserItem
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

3005: function parsedSubsection(title, description = "", items = [], rich = null) {
3006:   return parserItem(title, description, "", "Subsection", "subsection", rich, items);
3007: }

```

richHasContent - lines 3009-3018

Item	Explanation
Source location	Lines 3009-3018 (10 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function richHasContent(rich) {
Touches	filesystem
Calls detected	some
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

3009: function richHasContent(rich) {
3010:   return Boolean(rich?.blocks?.some((block) =>
3011:     block.type === "image" && block.url ||
3012:     block.type === "formula" && block.formula ||
3013:     block.type === "code" && block.code ||
3014:     block.text ||
3015:     block.title ||
3016:     block.caption
3017:   ));
3018: }

```

parsedItemHasContent - lines 3020-3031

Item	Explanation
Source location	Lines 3020-3031 (12 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function parsedItemHasContent(item) {
Touches	Mostly local calculations or local UI state.
Calls detected	richHasContent, some, includes
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 5 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

3020: function parsedItemHasContent(item) {
3021:   if (!item) return false;
3022:   if (item.kind === "summary") return Boolean(item.description || richHasContent(item.rich));
3023:   const children = item.children || item.items || [];
3024:   if (item.kind === "subsection") {
3025:     return Boolean(item.description || item.url || richHasContent(item.rich) ||
children.some(parsedItemHasContent));
3026:   }
3027:   if (["tool", "language"].includes(item.kind)) {
3028:     return Boolean(item.title || item.description || item.url || richHasContent(item.rich) ||
children.some(parsedItemHasContent));
3029:   }
3030:   return Boolean(item.description || item.url || richHasContent(item.rich) ||
children.some(parsedItemHasContent));
3031: }

```

parsedSectionHasContent - lines 3033-3035

Item	Explanation
Source location	Lines 3033-3035 (3 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function parsedSectionHasContent(section) {
Touches	Mostly local calculations or local UI state.
Calls detected	richHasContent, some
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

3033: function parsedSectionHasContent(section) {
3034:   return Boolean(section?.description || richHasContent(section?.rich) || (section?.items ||
[]).some(parsedItemHasContent));
3035: }

```

responsiveChildren - lines 3037-3039

Item	Explanation
Source location	Lines 3037-3039 (3 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function responsiveChildren(node) {
Touches	Mostly local calculations or local UI state.
Calls detected	No obvious named calls detected by the generator.

Item	Explanation
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

3037: function responsiveChildren(node) {
3038:   return node?.items || node?.children || [];
3039: }

```

richContainsMedia - lines 3041-3043

Item	Explanation
Source location	Lines 3041-3043 (3 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function richContainsMedia(rich) {
Touches	Mostly local calculations or local UI state.
Calls detected	some
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

3041: function richContainsMedia(rich) {
3042:   return Boolean(rich?.blocks?.some((block) => block.type === "image" && block.url));
3043: }

```

responsiveNodeHasMedia - lines 3045-3051

Item	Explanation
Source location	Lines 3045-3051 (7 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function responsiveNodeHasMedia(node) {
Touches	Mostly local calculations or local UI state.
Calls detected	richContainsMedia, responsiveChildren, some
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 4 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

3045: function responsiveNodeHasMedia(node) {
3046:   if (!node) return false;
3047:   if (node.kind === "image" || richContainsMedia(node.rich)) return true;
3048:   const url = String(node.url || "");
3049:   if (/\. (apng|avif|gif|jpe?g|png|svg|webp)(\?|#|$)/i.test(url)) return true;
3050:   return responsiveChildren(node).some(responsiveNodeHasMedia);
3051: }

```

responsiveNodeCount - lines 3053-3056

Item	Explanation
Source location	Lines 3053-3056 (4 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function responsiveNodeCount(node) {
Touches	Mostly local calculations or local UI state.
Calls detected	parsedItemHasContent, responsiveChildren, reduce
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

3053: function responsiveNodeCount(node) {
3054:   if (!node || !parsedItemHasContent(node)) return 0;
3055:   return 1 + responsiveChildren(node).reduce((count, child) => count + responsiveNodeCount(child), 0);
3056: }

```

responsiveNodeDepth - lines 3058-3063

Item	Explanation
Source location	Lines 3058-3063 (6 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function responsiveNodeDepth(node) {
Touches	Mostly local calculations or local UI state.
Calls detected	parsedItemHasContent, responsiveChildren, filter, max, map
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 3 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

3058: function responsiveNodeDepth(node) {
3059:   if (!node || !parsedItemHasContent(node)) return 0;
3060:   const children = responsiveChildren(node).filter(parsedItemHasContent);
3061:   if (!children.length) return 1;
3062:   return 1 + Math.max(...children.map(responsiveNodeDepth));
3063: }

```

buildResponsiveProfile - lines 3065-3096

Item	Explanation
Source location	Lines 3065-3096 (32 source lines).
Purpose	Creates an object, UI structure, process, payload, or generated artifact.
Declaration	function buildResponsiveProfile(project, parsedSections = []) {
Touches	Mostly local calculations or local UI state.
Calls detected	filter, parsedSectionHasContent, reduce, responsiveNodeCount, max, map, some, responsiveNodeHasMedia, hasPublicTemplate
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).

Item	Explanation
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

3065: function buildResponsiveProfile(project, parsedSections = []) {
3066:   const visibleSections = (parsedSections || []).filter((section) => section?.id !== "brief" &&
parsedSectionHasContent(section));
3067:   const itemCount = visibleSections.reduce(
3068:     (count, section) => count + (section.items || []).reduce((sum, item) => sum +
responsiveNodeCount(item), 0),
3069:     0
3070:   );
3071:   const maxDepth = visibleSections.reduce(
3072:     (depth, section) => Math.max(depth, ...(section.items || []).map(responsiveNodeDepth), 1),
3073:     1
3074:   );
3075:   const hasMedia = visibleSections.some((section) => responsiveNodeHasMedia(section));
3076:   const density = itemCount >= 18 || visibleSections.length >= 6 || maxDepth >= 4
3077:     ? "dense"
3078:     : itemCount <= 6 && visibleSections.length <= 2
3079:     ? "simple"
3080:     : "balanced";
3081:   return {
3082:     version: 1,
3083:     breakpoints: {
3084:       mobile: 640,
3085:       tablet: 900,
3086:       desktop: 1180
3087:     },
3088:     cardLayout: hasPublicTemplate(project) ? "single" : "split",
3089:     contentMax: hasMedia || maxDepth >= 3 ? "1180px" : "980px",
3090:     density,
3091:     mediaMax: hasMedia ? "860px" : "720px",
3092:     sectionCardMin: density === "dense" ? "210px" : density === "simple" ? "280px" : "240px",
3093:     touchTarget: "44px",
3094:     windowMode: "full-screen"
3095:   };
3096: }

```

visibleSections - lines 3066-3095

Item	Explanation
Source location	Lines 3066-3095 (30 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	const visibleSections = (parsedSections []).filter((section) => section?.id !== "brief" && parsedSectionHasContent(section));
Touches	Mostly local calculations or local UI state.
Calls detected	filter, parsedSectionHasContent, reduce, responsiveNodeCount, max, map, some, responsiveNodeHasMedia, hasPublicTemplate
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

3066:   const visibleSections = (parsedSections || []).filter((section) => section?.id !== "brief" &&
parsedSectionHasContent(section));
3067:   const itemCount = visibleSections.reduce(
3068:     (count, section) => count + (section.items || []).reduce((sum, item) => sum +
responsiveNodeCount(item), 0),
3069:     0
3070:   );
3071:   const maxDepth = visibleSections.reduce(
3072:     (depth, section) => Math.max(depth, ...(section.items || []).map(responsiveNodeDepth), 1),
3073:     1
3074:   );
3075:   const hasMedia = visibleSections.some((section) => responsiveNodeHasMedia(section));
3076:   const density = itemCount >= 18 || visibleSections.length >= 6 || maxDepth >= 4
3077:     ? "dense"
3078:     : itemCount <= 6 && visibleSections.length <= 2
3079:     ? "simple"
3080:     : "balanced";

```

```

3081:   return {
3082:     version: 1,
3083:     breakpoints: {
3084:       mobile: 640,
3085:       tablet: 900,
3086:       desktop: 1180
3087:     },
3088:     cardLayout: hasPublicTemplate(project) ? "single" : "split",
3089:     contentMax: hasMedia || maxDepth >= 3 ? "1180px" : "980px",
3090:     density,
3091:     mediaMax: hasMedia ? "860px" : "720px",
3092:     sectionCardMin: density === "dense" ? "210px" : density === "simple" ? "280px" : "240px",
3093:     touchTarget: "44px",
3094:     windowMode: "full-screen"
3095:   };

```

projectResponsiveProfile - lines 3098-3107

Item	Explanation
Source location	Lines 3098-3107 (10 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function projectResponsiveProfile(project) {
Touches	Mostly local calculations or local UI state.
Calls detected	buildResponsiveProfile
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

3098: function projectResponsiveProfile(project) {
3099:   const savedProfile = project?.portfolioView?.responsive;
3100:   if (savedProfile?.version) {
3101:     return {
3102:       ...buildResponsiveProfile(project, project?.portfolioView?.sections || []),
3103:       ...savedProfile
3104:     };
3105:   }
3106:   return buildResponsiveProfile(project, project?.portfolioView?.sections || []);
3107: }

```

parsedSection - lines 3109-3118

Item	Explanation
Source location	Lines 3109-3118 (10 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function parsedSection(id, title, items = [], description = "", rich = null) {
Touches	Mostly local calculations or local UI state.
Calls detected	filter, richHasContent, clone
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

3109: function parsedSection(id, title, items = [], description = "", rich = null) {
3110:   const section = {
3111:     description,
3112:     id,
3113:     items: items.filter(parsedItemHasContent),
3114:     title
3115:   };

```

```

3116:   if (richHasContent(rich)) section.rich = clone(rich);
3117:   return section;
3118: }

```

parsedDesignSection - lines 3120-3146

Item	Explanation
Source location	Lines 3120-3146 (27 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function parsedDesignSection(project) {
Touches	Mostly local calculations or local UI state.
Calls detected	isAnalogProject, ensureDesignModel, resourcesFrom, resourcesFromItems, parsedSubsection, parsedSection
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

3120: function parsedDesignSection(project) {
3121:   if (isAnalogProject(project)) {
3122:     const design = ensureDesignModel(project);
3123:     const documentationItems = [
3124:       ...resourcesFrom(project, "documents"),
3125:       ...resourcesFromItems(design.documentation.files, "document"),
3126:       parsedSubsection("References", "", resourcesFromItems(design.documentation.references,
"reference")),
3127:       parsedSubsection("Math / Analysis", "", resourcesFromItems(design.documentation.mathAnalysis,
"analysis"))
3128:     ];
3129:     const boardMediaItems = [
3130:       ...resourcesFrom(project, "pcbs"),
3131:       ...resourcesFrom(project, "media")
3132:     ];
3133:     return parsedSection("design", "Design", [
3134:       parsedSubsection("Design Overview", design.brief.summary || "", [
3135:         ...resourcesFromItems(design.brief.files, "document")
3136:       ], design.brief.summaryRich),
3137:       parsedSubsection("Documentation", design.documentation.summary || "", documentationItems,
design.documentation.summaryRich),
3138:       parsedSubsection("PCB and Visual Evidence", "", boardMediaItems)
3139:     ]);
3140:   }
3141:   return parsedSection("design", "Design", [
3142:     ...resourcesFrom(project, "documents"),
3143:     ...resourcesFrom(project, "pcbs"),
3144:     ...resourcesFrom(project, "media")
3145:   ]);
3146: }

```

parsedSimulationSection - lines 3148-3156

Item	Explanation
Source location	Lines 3148-3156 (9 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function parsedSimulationSection(project) {
Touches	Mostly local calculations or local UI state.
Calls detected	isAnalogProject, ensureDesignModel, parsedSection, parsedSubsection, resourcesFromItems
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

3148: function parsedSimulationSection(project) {
3149:   if (!isAnalogProject(project)) return null;
3150:   const design = ensureDesignModel(project);
3151:   return parsedSection("simulation", "Simulation", [
3152:     parsedSubsection("Simulation Overview", design.simulation.summary || "", [],
design.simulation.summaryRich),
3153:     parsedSubsection("Simulation Files", "", resourcesFromItems(design.simulation.files, "simulation-
file")),
3154:     parsedSubsection("Simulation Results", "", resourcesFromItems(design.simulation.results, "simulation-
result"))
3155:   ]);
3156: }

```

parsedToolsSection - lines 3158-3169

Item	Explanation
Source location	Lines 3158-3169 (12 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function parsedToolsSection(project) {
Touches	Mostly local calculations or local UI state.
Calls detected	map, parserItem, toolName, toolDescription, itemTitle, parsedSection, resourcesFrom
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

3158: function parsedToolsSection(project) {
3159:   const toolItems = (project.tools || []).map((tool) => parserItem(toolName(tool), toolDescription(tool),
"", "Tool", "tool", tool?.richDescription));
3160:   const languageItems = (project.languages || []).map((language) => {
3161:     const label = typeof language === "string" ? language : itemTitle(language);
3162:     return parserItem(label, "", "", "Language", "language");
3163:   });
3164:   return parsedSection("tools", "Tools", [
3165:     ...toolItems,
3166:     ...languageItems,
3167:     ...resourcesFrom(project, "links")
3168:   ]);
3169: }

```

toolItems - lines 3159-3163

Item	Explanation
Source location	Lines 3159-3163 (5 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	const toolItems = (project.tools []).map((tool) => parserItem(toolName(tool), toolDescription(tool), "", "Tool", "tool", tool?.richDescription));
Touches	Mostly local calculations or local UI state.
Calls detected	map, parserItem, toolName, toolDescription, itemTitle
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

3159:   const toolItems = (project.tools || []).map((tool) => parserItem(toolName(tool), toolDescription(tool),
"", "Tool", "tool", tool?.richDescription));
3160:   const languageItems = (project.languages || []).map((language) => {
3161:     const label = typeof language === "string" ? language : itemTitle(language);
3162:     return parserItem(label, "", "", "Language", "language");
3163:   });

```

languageItems - lines 3160-3163

Item	Explanation
Source location	Lines 3160-3163 (4 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	const languageItems = (project.languages []).map((language) => {
Touches	Mostly local calculations or local UI state.
Calls detected	map, itemTitle, parserItem
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
3160:   const languageItems = (project.languages || []).map((language) => {
3161:     const label = typeof language === "string" ? language : itemTitle(language);
3162:     return parserItem(label, "", "", "Language", "language");
3163:   });
```

parseProjectForPortfolio - lines 3171-3219

Item	Explanation
Source location	Lines 3171-3219 (49 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function parseProjectForPortfolio(project) {
Touches	Mostly local calculations or local UI state.
Calls detected	projectTemplateFor, sectionOptions, flatMap, parsedSection, parserItem, parsedDesignSection, parsedSimulationSection, resourcesFrom, parsedToolsSection, startsWith, replace, find
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 8 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
3171: function parseProjectForPortfolio(project) {
3172:   const template = projectTemplateFor(project);
3173:   const parsedSections = sectionOptions(project).flatMap((section) => {
3174:     if (section.id === "brief") {
3175:       return parsedSection("brief", "Overview", [
3176:         parserItem("Overview", project.summary || "", "", "Overview", "summary", project.summaryRich)
3177:       ]);
3178:     }
3179:     if (section.id === "design") return parsedDesignSection(project);
3180:     if (section.id === "simulation") return parsedSimulationSection(project);
3181:     if (section.id === "tests") return parsedSection("tests", "Tests", resourcesFrom(project, "tests"));
3182:     if (section.id === "tools") return parsedToolsSection(project);
3183:     if (section.id.startsWith("custom:")) {
3184:       const sectionId = section.id.replace("custom:", "");
3185:       const custom = (project.sections || []).find((item) => item.id === sectionId);
3186:       return parsedSection(
3187:         `custom:${sectionId}`,
3188:         custom?.title || section.label,
3189:         (custom?.items || []).map((item) => item.url
3190:           ? parserItem(item.title, item.description, item.url, item.type || item.status || "File",
3191:             "file", item.richDescription)
3192:           : parsedSubsection(
3193:             item.title,
3194:             item.description,
3195:             resourcesFromItems(item.children || item.items || item.files || [], "file"),
3196:             item.richDescription
3197:           )),
3198:         custom?.description || "",
```



```

3198:         custom?.richDescription
3199:     );
3200: }
3201: return null;
3202: }).filter(parsedSectionHasContent);
3203:
3204: return {
3205:     ...clone(project),
3206:     portfolioView: {
3207:         builtAt: new Date().toISOString(),
3208:         template: {
3209:             group: "Appearance",
3210:             id: projectTemplateId(project),
3211:             label: template.label || project.templateLabel || "",
3212:             visual: template.visual ? clone(template.visual) : projectTemplateVisual(project)
3213:         },
3214:         responsive: buildResponsiveProfile(project, parsedSections),
3215:         title: project.title,
3216:         sections: parsedSections
3217:     }
3218: };
3219: }

```

custom - lines 3185-3187

Item	Explanation
Source location	Lines 3185-3187 (3 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	<code>const custom = (project.sections []).find((item) => item.id === sectionId);</code>
Touches	Mostly local calculations or local UI state.
Calls detected	<code>find</code> , <code>parsedSection</code>
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

3185:     const custom = (project.sections || []).find((item) => item.id === sectionId);
3186:     return parsedSection(
3187:         `custom:${sectionId}`,

```

refreshOpenPreviews - lines 3221-3230

Item	Explanation
Source location	Lines 3221-3230 (10 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	<code>function refreshOpenPreviews() {</code>
Touches	Mostly local calculations or local UI state.
Calls detected	<code>selectedProject</code> , <code>find</code> , <code>renderProjectWebsitePreview</code> , <code>renderPreview</code>
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

3221: function refreshOpenPreviews() {
3222:     if (projectPreviewDialog?.open) {
3223:         const project = selectedProject();
3224:         const savedProject = (savedPortfolioCatalog.projects || []).find((item) => item.id === project?.id);
3225:         renderProjectWebsitePreview(savedProject || project);
3226:     }
3227:     if (portfolioPreviewDialog?.open) {

```

```

3228:     renderPreview();
3229:   }
3230: }

```

savedProject - lines 3224-3227

Item	Explanation
Source location	Lines 3224-3227 (4 source lines).
Purpose	Persists data to local state, disk, credentials, or browser storage.
Declaration	const savedProject = (savedPortfolioCatalog.projects []).find((item) => item.id === project?.id);
Touches	Mostly local calculations or local UI state.
Calls detected	find, renderProjectWebsitePreview
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

3224:     const savedProject = (savedPortfolioCatalog.projects || []).find((item) => item.id === project?.id);
3225:     renderProjectWebsitePreview(savedProject || project);
3226:   }
3227:   if (portfolioPreviewDialog?.open) {

```

saveSelectedProjectAndClose - lines 3232-3236

Item	Explanation
Source location	Lines 3232-3236 (5 source lines).
Purpose	Persists data to local state, disk, credentials, or browser storage.
Declaration	function saveSelectedProjectAndClose() {
Touches	Mostly local calculations or local UI state.
Calls detected	saveSelectedProjectToPortfolio, closeDialogElement
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

3232: function saveSelectedProjectAndClose() {
3233:   if (saveSelectedProjectToPortfolio()) {
3234:     closeDialogElement(projectDialog);
3235:   }
3236: }

```

commitActiveProjectEdits - lines 3238-3246

Item	Explanation
Source location	Lines 3238-3246 (9 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	async function commitActiveProjectEdits() {
Touches	Mostly local calculations or local UI state.
Calls detected	saveVisibleRichEditors, saveRichSummary
API endpoints	None detected inside this function body.

Item	Explanation
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

3238: async function commitActiveProjectEdits() {
3239:   saveVisibleRichEditors();
3240:
3241:   if (summaryEditorProjectId) {
3242:     return saveRichSummary();
3243:   }
3244:
3245:   return true;
3246: }

```

saveAllSections - lines 3248-3259

Item	Explanation
Source location	Lines 3248-3259 (12 source lines).
Purpose	Persists data to local state, disk, credentials, or browser storage.
Declaration	async function saveAllSections(options = {}) {
Touches	Mostly local calculations or local UI state.
Calls detected	commitActiveProjectEdits, syncParsedPortfolioGlobalsIntoSaved, map, parseProjectForPortfolio, markDraftNeedsSave, closeDialogElement, refreshOpenPreviews, setStatus, updateBuilderWorkflow
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	1 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

3248: async function saveAllSections(options = {}) {
3249:   const settings = options && options.currentTarget ? {} : options;
3250:   if (!(await commitActiveProjectEdits())) return false;
3251:   syncParsedPortfolioGlobalsIntoSaved();
3252:   savedPortfolioCatalog.projects = (catalog.projects || []).map((project) =>
parseProjectForPortfolio(project));
3253:   markDraftNeedsSave();
3254:   if (projectDialog?.open) closeDialogElement(projectDialog);
3255:   if (settings.refreshOpenPreviews !== false) refreshOpenPreviews();
3256:   setStatus(`Saved ${savedPortfolioCatalog.projects.length}
project${savedPortfolioCatalog.projects.length === 1 ? "" : "s"} into the portfolio preview.`);
3257:   updateBuilderWorkflow();
3258:   return true;
3259: }

```

selectedProject - lines 3261-3263

Item	Explanation
Source location	Lines 3261-3263 (3 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function selectedProject() {
Touches	Mostly local calculations or local UI state.
Calls detected	find
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).

Item	Explanation
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

3261: function selectedProject() {
3262:   return catalog.projects.find((project) => project.id === selectedProjectId) || catalog.projects[0];
3263: }

```

sectionWindowId - lines 3265-3269

Item	Explanation
Source location	Lines 3265-3269 (5 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function sectionWindowId(sectionId) {
Touches	Mostly local calculations or local UI state.
Calls detected	includes
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 3 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

3265: function sectionWindowId(sectionId) {
3266:   if (["documents", "pcbs", "media"].includes(sectionId)) return "design";
3267:   if (["links", "highlights", "tools", "languages"].includes(sectionId)) return "tools";
3268:   return sectionId || "brief";
3269: }

```

resetProjectWindowScroll - lines 3271-3279

Item	Explanation
Source location	Lines 3271-3279 (9 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function resetProjectWindowScroll() {
Touches	Mostly local calculations or local UI state.
Calls detected	requestAnimationFrame
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

3271: function resetProjectWindowScroll() {
3272:   projectWindowContent.scrollTop = 0;
3273:   requestAnimationFrame(() => {
3274:     projectWindowContent.scrollTop = 0;
3275:     requestAnimationFrame(() => {
3276:       projectWindowContent.scrollTop = 0;
3277:     });
3278:   });
3279: }

```

openProjectWindow - lines 3281-3302

Item	Explanation
Source location	Lines 3281-3302 (22 source lines).
Purpose	Controls a window, dialog, panel, menu, or visible state.
Declaration	function openProjectWindow(projectId, sectionId = "brief") {
Touches	browser/Electron DOM
Calls detected	sectionWindowId, removeAttribute, remove, renderAll, add, showModal, resetProjectWindowScroll, updateDialogWindowButtons
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

3281: function openProjectWindow(projectId, sectionId = "brief") {
3282:   selectedProjectId = projectId;
3283:   projectWindowTitle.dataset.projectId = projectId;
3284:   activeSectionId = sectionWindowId(sectionId);
3285:   projectWindowBackStack = [];
3286:   projectWindowForwardStack = [];
3287:   projectTitleMenu.hidden = true;
3288:   projectDialog.removeAttribute("style");
3289:   projectDialog.classList.remove(
3290:     "is-draggable-dialog",
3291:     "is-hidden-dialog",
3292:     "is-maximized-dialog",
3293:     "is-minimized-dialog",
3294:     "is-resized-dialog"
3295:   );
3296:   delete projectDialog.dataset.restoreWindowState;
3297:   renderAll();
3298:   document.body.classList.add("project-window-open");
3299:   if (!projectDialog.open) projectDialog.showModal();
3300:   resetProjectWindowScroll();
3301:   updateDialogWindowButtons(projectDialog);
3302: }

```

categoryById - lines 3304-3306

Item	Explanation
Source location	Lines 3304-3306 (3 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function categoryById(id) {
Touches	Mostly local calculations or local UI state.
Calls detected	find
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

3304: function categoryById(id) {
3305:   return catalog.categories.find((category) => category.id === id) || {};
3306: }

```

normalizeCategory - lines 3308-3318

Item	Explanation
Source location	Lines 3308-3318 (11 source lines).

Item	Explanation
Purpose	Validates or normalizes input so later code receives safe predictable values.
Declaration	function normalizeCategory(category = {}) {
Touches	filesystem
Calls detected	trim, slugify, normalizeTextColor
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

3308: function normalizeCategory(category = {}) {
3309:   const label = String(category.label || category.title || "New category").trim() || "New category";
3310:   const id = slugify(category.id || label) || "category";
3311:   const accent = normalizeTextColor(category.accent || "") || "#1677a8";
3312:   return {
3313:     accent,
3314:     description: String(category.description || "").trim(),
3315:     id,
3316:     label
3317:   };
3318: }

```

refreshCategoryControls - lines 3320-3327

Item	Explanation
Source location	Lines 3320-3327 (8 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function refreshCategoryControls() {
Touches	filesystem
Calls detected	map, clone, escapeHtml, join, renderCategoryDropdown
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

3320: function refreshCategoryControls() {
3321:   catalog.categories = (catalog.categories || []).map(normalizeCategory);
3322:   savedPortfolioCatalog.categories = clone(catalog.categories || []);
3323:   if (newCategorySelect) {
3324:     newCategorySelect.innerHTML = catalog.categories.map((category) => `<option
value="${escapeHtml(category.id)}">${escapeHtml(category.label)}</option>`).join("");
3325:   }
3326:   renderCategoryDropdown();
3327: }

```

linkAttributes - lines 3329-3332

Item	Explanation
Source location	Lines 3329-3332 (4 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function linkAttributes(url, item = {}) {
Touches	filesystem
Calls detected	normalizeLinkTarget, isWebsiteLinkItem

Item	Explanation
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

3329: function linkAttributes(url, item = {}) {
3330:   const target = normalizeLinkTarget(url, { assumeWeb: isWebsiteLinkItem(item, url) });
3331:   return /^https?:\/\//i.test(target) ? ' target="_blank" rel="noreferrer"' : "";
3332: }

```

isLocalDownloadTarget - lines 3334-3337

Item	Explanation
Source location	Lines 3334-3337 (4 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function isLocalDownloadTarget(target = "") {
Touches	filesystem
Calls detected	normalizeLinkTarget
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

3334: function isLocalDownloadTarget(target = "") {
3335:   const value = normalizeLinkTarget(target, { assumeWeb: true });
3336:   return Boolean(value) && !/^(https?:)?\/\//i.test(value) && !/^(mailto:|tel:|#)/i.test(value);
3337: }

```

downloadAttribute - lines 3339-3342

Item	Explanation
Source location	Lines 3339-3342 (4 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function downloadAttribute(target = "", item = {}) {
Touches	filesystem
Calls detected	normalizeLinkTarget, isWebsiteLinkItem, isLocalDownloadTarget
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

3339: function downloadAttribute(target = "", item = {}) {
3340:   const value = normalizeLinkTarget(target, { assumeWeb: isWebsiteLinkItem(item, target) });
3341:   return isLocalDownloadTarget(value) ? " download" : "";
3342: }

```

resourceLink - lines 3344-3353

Item	Explanation
Source location	Lines 3344-3353 (10 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function resourceLink(item, label = item.label item.title item.name) {
Touches	filesystem
Calls detected	normalizeLinkTarget, isWebsiteLinkItem, escapeHtml, linkAttributes, downloadAttribute
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

3344: function resourceLink(item, label = item.label || item.title || item.name) {
3345:   const rawTarget = item.url || item.artifact || item.href || item.file || item.path || item.src || "";
3346:   const target = normalizeLinkTarget(rawTarget, { assumeWeb: isWebsiteLinkItem(item, rawTarget) });
3347:
3348:   if (!target || item.status === "planned") {
3349:     return `<span class="resource-link muted">${label}</span>`;
3350:   }
3351:
3352:   return `<a class="resource-link" href="${escapeHtml(target)}"${linkAttributes(target,
item)}${downloadAttribute(target, item)}>${escapeHtml(label)}</a>`;
3353: }

```

pillList - lines 3355-3360

Item	Explanation
Source location	Lines 3355-3360 (6 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function pillList(items, className = "") {
Touches	Mostly local calculations or local UI state.
Calls detected	map, join
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

3355: function pillList(items, className = "") {
3356:   return (items || []).map((item) => {
3357:     const label = typeof item === "string" ? item : item.name || item.title || item.label || "";
3358:     return `<span class="tag ${className}">${label}</span>`;
3359:   }).join("");
3360: }

```

evidenceList - lines 3362-3365

Item	Explanation
Source location	Lines 3362-3365 (4 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function evidenceList(items, renderItem, emptyMessage) {
Touches	Mostly local calculations or local UI state.
Calls detected	map, join
API endpoints	None detected inside this function body.

Item	Explanation
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

3362: function evidenceList(items, renderItem, emptyMessage) {
3363:   if (!items || !items.length) return `

${emptyMessage}</p>`;
3364:   return `


```

detailBlock - lines 3367-3374

Item	Explanation
Source location	Lines 3367-3374 (8 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function detailBlock(title, className, content) {
Touches	Mostly local calculations or local UI state.
Calls detected	No obvious named calls detected by the generator.
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

3367: function detailBlock(title, className, content) {
3368:   return `
3369:     <details class="${className}">
3370:       <summary>${title}</summary>
3371:       ${content}
3372:     </details>
3373:   `;
3374: }

```

mediaGrid - lines 3376-3398

Item	Explanation
Source location	Lines 3376-3398 (23 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function mediaGrid(items) {
Touches	filesystem
Calls detected	map, escapeHtml, normalizeLinkTarget, isWebsiteLinkItem, linkAttributes, join
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

3376: function mediaGrid(items) {
3377:   if (!items || !items.length) return `

No project images have been added yet.</p>`;
3378:
3379:   return `
3380:     <div class="media-grid">
3381:       ${items.map((item) => `


```

```

3382:         <figure>
3383:             ${item.status === "planned" ? `
3384:                 <div class="media-placeholder">${item.title}</div>
3385:                 ` : `
3386:                 <a href="${escapeHtml(normalizeLinkTarget(item.url, { assumeWeb: isWebsiteLinkItem(item,
item.url) })))"${linkAttributes(item.url, item)}>
3387:                     
3388:                 </a>
3389:                 `}
3390:             <figcaption>
3391:                 <strong>${escapeHtml(item.title)}</strong>
3392:                 <span>${escapeHtml(item.caption || "")}</span>
3393:             </figcaption>
3394:         </figure>
3395:     `).join("")}
3396: </div>
3397: `;
3398: }

```

itemTitle - lines 3400-3403

Item	Explanation
Source location	Lines 3400-3403 (4 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function itemTitle(item) {
Touches	Mostly local calculations or local UI state.
Calls detected	No obvious named calls detected by the generator.
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

3400: function itemTitle(item) {
3401:   if (typeof item === "string") return item;
3402:   return item?.title || item?.name || item?.label || "Untitled item";
3403: }

```

itemUrl - lines 3405-3407

Item	Explanation
Source location	Lines 3405-3407 (3 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function itemUrl(item) {
Touches	Mostly local calculations or local UI state.
Calls detected	No obvious named calls detected by the generator.
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

3405: function itemUrl(item) {
3406:   return item?.url || item?.artifact || "";
3407: }

```

itemDescription - lines 3409-3411

Item	Explanation
Source location	Lines 3409-3411 (3 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function itemDescription(item) {
Touches	Mostly local calculations or local UI state.
Calls detected	No obvious named calls detected by the generator.
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
3409: function itemDescription(item) {
3410:   return item?.description || item?.caption || item?.result || item?.method || "";
3411: }
```

isImageUrl - lines 3413-3415

Item	Explanation
Source location	Lines 3413-3415 (3 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function isImageUrl(url) {
Touches	Mostly local calculations or local UI state.
Calls detected	No obvious named calls detected by the generator.
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
3413: function isImageUrl(url) {
3414:   return /\.(png|jpe?g|webp|gif|svg)(\?.*)?$/i.test(url || "");
3415: }
```

portfolioLink - lines 3417-3421

Item	Explanation
Source location	Lines 3417-3421 (5 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function portfolioLink(url, label) {
Touches	filesystem
Calls detected	normalizeLinkTarget, escapeHtml, linkAttributes, downloadAttribute
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

3417: function portfolioLink(url, label) {
3418:   if (!url) return `<span>${label}</span>`;
3419:   const target = normalizeLinkTarget(url, { assumeWeb: true });
3420:   return `<a
href="${escapeHtml(target)}"${linkAttributes(target)}${downloadAttribute(target)}>${escapeHtml(label)}</a>`;
3421: }

```

renderPortfolioResourceList - lines 3423-3444

Item	Explanation
Source location	Lines 3423-3444 (22 source lines).
Purpose	Renders UI, markup, or display output.
Declaration	function renderPortfolioResourceList(title, items, emptyMessage) {
Touches	Mostly local calculations or local UI state.
Calls detected	map, itemTitle, itemUrl, itemDescription, portfolioLink, join
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

3423: function renderPortfolioResourceList(title, items, emptyMessage) {
3424:   return `
3425:     <section class="portfolio-preview-section">
3426:       <h3>${title}</h3>
3427:       ${items || []}.length ? `
3428:         <div class="portfolio-resource-list">
3429:           ${items.map(item => {
3430:             const titleText = itemTitle(item);
3431:             const url = itemUrl(item);
3432:             const description = itemDescription(item);
3433:             return `
3434:               <article class="portfolio-resource">
3435:                 <strong>${portfolioLink(url, titleText)}</strong>
3436:                 ${description ? `<p>${description}</p>` : ""}
3437:               </article>
3438:             `;
3439:           }).join("")}
3440:         </div>
3441:       ` : `<p class="evidence-empty">${emptyMessage}</p>`;
3442:     </section>
3443:   `;
3444: }

```

renderPortfolioMedia - lines 3446-3468

Item	Explanation
Source location	Lines 3446-3468 (23 source lines).
Purpose	Renders UI, markup, or display output.
Declaration	function renderPortfolioMedia(items) {
Touches	Mostly local calculations or local UI state.
Calls detected	map, itemUrl, isImageUrl, itemTitle, itemDescription, join
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

3446: function renderPortfolioMedia(items) {
3447:   return `
3448:     <section class="portfolio-preview-section">
3449:       <h3>Images</h3>

```

```

3450:         ${items || []}.length ? `
3451:         <div class="portfolio-media-grid">
3452:             ${items.map((item) => {
3453:                 const url = itemUrl(item);
3454:                 return `
3455:                     <figure>
3456:                         ${isImageUrl(url) ? `` : `<div class="media-
placeholder">${itemTitle(item)}</div>`}
3457:                         <figcaption>
3458:                             <strong>${itemTitle(item)}</strong>
3459:                             ${itemDescription(item) ? `<span>${itemDescription(item)}</span>` : ""}
3460:                         </figcaption>
3461:                     </figure>
3462:                 `;
3463:             }).join("")}
3464:         </div>
3465:         : `<p class="evidence-empty">No images have been added yet.</p>`
3466:     </section>
3467: `;
3468: }

```

renderPortfolioTools - lines 3470-3486

Item	Explanation
Source location	Lines 3470-3486 (17 source lines).
Purpose	Renders UI, markup, or display output.
Declaration	function renderPortfolioTools(project) {
Touches	Mostly local calculations or local UI state.
Calls detected	map, toolName, toolDescription, join
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

3470: function renderPortfolioTools(project) {
3471:     return `
3472:     <section class="portfolio-preview-section">
3473:         <h3>Tools</h3>
3474:         ${project.tools || []}.length ? `
3475:         <div class="portfolio-tool-list">
3476:             ${project.tools.map((tool) => `
3477:                 <article>
3478:                     <strong>${toolName(tool)}</strong>
3479:                     ${toolDescription(tool) ? `<p>${toolDescription(tool)}</p>` : ""}
3480:                 </article>
3481:             `).join("")}
3482:         </div>
3483:         : `<p class="evidence-empty">No tools have been added yet.</p>`
3484:     </section>
3485: `;
3486: }

```

renderPortfolioCustomSections - lines 3488-3505

Item	Explanation
Source location	Lines 3488-3505 (18 source lines).
Purpose	Renders UI, markup, or display output.
Declaration	function renderPortfolioCustomSections(project) {
Touches	Mostly local calculations or local UI state.
Calls detected	map, portfolioLink, join
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).

Item	Explanation
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

3488: function renderPortfolioCustomSections(project) {
3489:   return (project.sections || []).map((section) => `
3490:     <section class="portfolio-preview-section">
3491:       <h3>${section.title}</h3>
3492:       ${section.description ? `<p>${section.description}</p>` : ""}
3493:       ${section.items || []}.length ? `
3494:       <div class="portfolio-resource-list">
3495:         ${section.items.map((item) => `
3496:           <article class="portfolio-resource">
3497:             <strong>${portfolioLink(item.url, item.title || "Untitled subsection")}</strong>
3498:             ${item.description ? `<p>${item.description}</p>` : ""}
3499:           </article>
3500:         `).join("")}
3501:       </div>
3502:       ` : `<p class="evidence-empty">No content has been added yet.</p>`}
3503:     </section>
3504:   `).join("");
3505: }

```

publicCustomSectionBlocks - lines 3507-3518

Item	Explanation
Source location	Lines 3507-3518 (12 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function publicCustomSectionBlocks(project) {
Touches	filesystem
Calls detected	map, detailBlock, renderMultilineInlineText, evidenceList, resourceLink, join
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

3507: function publicCustomSectionBlocks(project) {
3508:   return (project.sections || []).map((section) => detailBlock(section.title, "evidence-block evidence-
wide", `
3509:     ${section.description ? `<p class="evidence-
empty">${renderMultilineInlineText(section.description)}</p>` : ""}
3510:     ${evidenceList(section.items || [], (item) => `
3511:       <li>
3512:         ${item.url ? resourceLink(item, item.title) : `<strong>${item.title}</strong>`}
3513:         <span>${item.type || "Section item"} &middot; ${item.status || "tracked"}</span>
3514:         ${item.description ? `<p>${renderMultilineInlineText(item.description)}</p>` : ""}
3515:       </li>
3516:     `, "No content has been added yet.")}
3517:   `).join("");
3518: }

```

renderProjectPortfolioPreview - lines 3520-3556

Item	Explanation
Source location	Lines 3520-3556 (37 source lines).
Purpose	Renders UI, markup, or display output.
Declaration	function renderProjectPortfolioPreview(project) {
Touches	filesystem
Calls detected	categoryById, find, filter, previewSectionHasRenderableContent, projectTemplateClass, projectResponsiveClass, projectTemplateStyle, renderParsedBriefBlock, map, parsedPublicSection, join, renderMultilineInlineText
API endpoints	None detected inside this function body.

Item	Explanation
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 3 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

3520: function renderProjectPortfolioPreview(project) {
3521:   if (!project) return `
```

briefSection - lines 3525-3528

Item	Explanation
Source location	Lines 3525-3528 (4 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	const briefSection = (project.portfolioView.sections []).find((section) => section.id === "brief");
Touches	Mostly local calculations or local UI state.
Calls detected	find, filter, previewSectionHasRenderableContent, projectTemplateClass, projectResponsiveClass, projectTemplateStyle
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

3525:     const briefSection = (project.portfolioView.sections || []).find((section) => section.id ===
"brief");
3526:     const otherSections = (project.portfolioView.sections || []).filter((section) => section.id !==
"brief" && previewSectionHasRenderableContent(section));

```

```

3527:     return `
3528:       <article class="portfolio-preview-article ${projectTemplateClass(project)}
${projectResponsiveClass(project)}" style="${projectTemplateStyle(project, accent)}">

```

otherSections - lines 3526-3528

Item	Explanation
Source location	Lines 3526-3528 (3 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	const otherSections = (project.portfolioView.sections []).filter((section) => section.id !== "brief" && previewSectionHasRenderableContent(section));
Touches	Mostly local calculations or local UI state.
Calls detected	filter, previewSectionHasRenderableContent, projectTemplateClass, projectResponsiveClass, projectTemplateStyle
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

3526:     const otherSections = (project.portfolioView.sections || []).filter((section) => section.id !==
"brief" && previewSectionHasRenderableContent(section));
3527:     return `
3528:       <article class="portfolio-preview-article ${projectTemplateClass(project)}
${projectResponsiveClass(project)}" style="${projectTemplateStyle(project, accent)}">

```

fullProjectPreviewHtmlExact - lines 3558-3685

Item	Explanation
Source location	Lines 3558-3685 (128 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function fullProjectPreviewHtmlExact(project) {
Touches	filesystem
Calls detected	parseProjectForPortfolio, categoryById, normalizeCategory, displayTitle, parsedPortfolioGlobals, stringify, replaceAll, escapeHtml, var, min, calc, media
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

3558: function fullProjectPreviewHtmlExact(project) {
3559:   const baseHref = `${window.location.origin}/`;
3560:   if (!project) {
3561:     return `<!DOCTYPE html>
3562: <html lang="en">
3563: <head>
3564:   <meta charset="UTF-8" />
3565:   <meta name="viewport" content="width=device-width, initial-scale=1.0" />
3566:   <base href="${baseHref}" />
3567:   <link rel="stylesheet" href="styles.css" />
3568:   <title>Project preview</title>
3569: </head>
3570: <body class="project-isolated-preview">
3571:   <main class="project-isolated-shell">
3572:     <div class="empty-state">
3573:       <h3>No saved project preview</h3>
3574:       <p>Click Save project before viewing the parsed project preview.</p>
3575:     </div>
3576:   </main>
3577: </body>
3578: </html>`;
3579:   }

```



```

3580: const parsedProject = project.portfolioView ? project : parseProjectForPortfolio(project);
3581: const categorySource = categoryById(parsedProject.category);
3582: const previewCategory = normalizeCategory(categorySource.id ? categorySource : {
3583:   id: parsedProject.category || "project",
3584:   label: displayTitle(parsedProject.category || "Project", "Project")
3585: });
3586: const parsedGlobals = parsedPortfolioGlobals();
3587: const previewDataObject = {
3588:   categories: [previewCategory],
3589:   fieldStyles: parsedGlobals.fieldStyles,
3590:   funFacts: [],
3591:   funFactsRich: null,
3592:   profile: parsedGlobals.profile,
3593:   profileRich: parsedGlobals.profileRich,
3594:   projects: [parsedProject],
3595:   siteContent: parsedGlobals.siteContent,
3596:   siteContentRich: parsedGlobals.siteContentRich,
3597:   siteSections: []
3598: };
3599: const previewData = JSON.stringify(previewDataObject).replaceAll("</", "<\\");
3600: const title = parsedProject.portfolioView?.title || parsedProject.title || "Project preview";
3601: return `<!DOCTYPE html>
3602: <html lang="en">
3603: <head>
3604:   <meta charset="UTF-8" />
3605:   <meta name="viewport" content="width=device-width, initial-scale=1.0" />
3606:   <base href="${baseHref}" />
3607:   <title>${escapeHtml(title)} | Project preview</title>
3608:   <link rel="stylesheet" href="styles.css" />
3609:   <style>
3610:     body.project-isolated-preview {
3611:       min-height: 100vh;
3612:       margin: 0;
3613:       background: var(--paper, #f8fbff);
3614:       color: var(--ink, #172636);
3615:     }
3616:
3617:     .project-isolated-shell {
3618:       width: min(1180px, calc(100vw - 32px));
3619:       margin: 0 auto;
3620:       padding: 28px 0 36px;
3621:     }
3622:
3623:     .project-isolated-support {
3624:       display: none !important;
3625:     }
3626:
3627:     body.project-isolated-preview .project-grid {
3628:       display: block;
3629:     }
3630:
3631:     body.project-isolated-preview .category-section {
3632:       padding: 0;
3633:       border: 0;
3634:       background: transparent;
3635:     }
3636:
3637:     body.project-isolated-preview .category-heading,
3638:     body.project-isolated-preview .category-description {
3639:       display: none;
3640:     }
3641:
3642:     body.project-isolated-preview .category-projects {
3643:       display: block;
3644:     }
3645:
3646:     body.project-isolated-preview .project-card {
3647:       margin: 0;
3648:     }
3649:
3650:     @media (max-width: 720px) {
3651:       .project-isolated-shell {
3652:         width: min(100vw - 20px, 100%);
3653:         padding: 14px 0 24px;
3654:       }
3655:     }
3656:   </style>
3657: </head>
3658: <body class="project-isolated-preview">
3659:   <main class="project-isolated-shell" aria-label="Isolated project preview">
3660:     <div class="project-isolated-support" aria-hidden="true">
3661:       <label class="search-wrap" for="project-search">
3662:         <span>Search</span>
3663:         <input id="project-search" type="search" tabindex="-1" />
3664:       </label>
3665:       <div id="project-filters">
3666:         <button class="filter-button active" type="button" data-filter="all" tabindex="-1">All</button>
3667:       </div>
3668:       <span id="project-count">0</span>
3669:       <span id="project-track-count">0 Tracks</span>
3670:       <span id="project-track-labels">project categories</span>
3671:       <span id="year">${new Date().getFullYear()}</span>

```

```

3672:         </div>
3673:
3674:         <section class="section project-isolated-section" id="projects" aria-labelledby="project-isolated-
title">
3675:             <h1 class="sr-only" id="project-isolated-title">${escapeHtml(title)}</h1>
3676:             <div class="project-grid" id="project-grid" aria-live="polite"></div>
3677:         </section>
... excerpt truncated after 120 lines. Open the source file for the rest of this function.

```

renderProjectWebsitePreview - lines 3687-3705

Item	Explanation
Source location	Lines 3687-3705 (19 source lines).
Purpose	Renders UI, markup, or display output.
Declaration	function renderProjectWebsitePreview(project) {
Touches	browser/Electron DOM
Calls detected	categoryById, applyProjectTemplateToElement, updateProjectPreviewNavigation, add, escapeHtml, querySelector, fullProjectPreviewHtmlExact
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

3687: function renderProjectWebsitePreview(project) {
3688:   const category = categoryById(project?.category);
3689:   applyProjectTemplateToElement(projectPreviewDialog, project, category.accent || "#117c7a");
3690:   projectPreviewTitle.textContent = project?.portfolioView?.title || project?.title || "Project preview";
3691:   activeProjectPreviewProject = project || null;
3692:   activeProjectPreviewSectionIndex = -1;
3693:   activeProjectPreviewPath = [];
3694:   activeProjectPreviewForwardStack = [];
3695:   updateProjectPreviewNavigation();
3696:   projectPreviewContent.classList.add("is-website-preview");
3697:   projectPreviewContent.innerHTML = `
3698:     <iframe
3699:       class="portfolio-preview-frame project-preview-frame"
3700:       title="${escapeHtml(project?.title || "Project website preview")}"
3701:     ></iframe>
3702:   `;
3703:   const frame = projectPreviewContent.querySelector("iframe");
3704:   if (frame) frame.srcdoc = fullProjectPreviewHtmlExact(project);
3705: }

```

openProjectPortfolioPreview - lines 3707-3715

Item	Explanation
Source location	Lines 3707-3715 (9 source lines).
Purpose	Controls a window, dialog, panel, menu, or visible state.
Declaration	function openProjectPortfolioPreview() {
Touches	browser/Electron DOM
Calls detected	selectedProject, find, renderProjectWebsitePreview, add, showModal
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

3707: function openProjectPortfolioPreview() {
3708:   const project = selectedProject();
3709:   const savedProject = (savedPortfolioCatalog.projects || []).find((item) => item.id === project?.id);
3710:   const previewProject = savedProject || project;

```

```

3711:   renderProjectWebsitePreview(previewProject);
3712:   document.body.classList.add("full-window-open");
3713:   projectPreviewDialog.showModal();
3714:   projectPreviewDialog.scrollTop = 0;
3715: }

```

savedProject - lines 3709-3714

Item	Explanation
Source location	Lines 3709-3714 (6 source lines).
Purpose	Persists data to local state, disk, credentials, or browser storage.
Declaration	<code>const savedProject = (savedPortfolioCatalog.projects []).find((item) => item.id === project?.id);</code>
Touches	browser/Electron DOM
Calls detected	find, renderProjectWebsitePreview, add, showModal
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

3709:   const savedProject = (savedPortfolioCatalog.projects || []).find((item) => item.id === project?.id);
3710:   const previewProject = savedProject || project;
3711:   renderProjectWebsitePreview(previewProject);
3712:   document.body.classList.add("full-window-open");
3713:   projectPreviewDialog.showModal();
3714:   projectPreviewDialog.scrollTop = 0;

```

updateProjectPreviewNavigation - lines 3717-3721

Item	Explanation
Source location	Lines 3717-3721 (5 source lines).
Purpose	Updates UI, state, cache, or stored data.
Declaration	<code>function updateProjectPreviewNavigation() {</code>
Touches	Mostly local calculations or local UI state.
Calls detected	updateDialogWindowButtons
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

3717: function updateProjectPreviewNavigation() {
3718:   projectPreviewBack.hidden = activeProjectPreviewSectionIndex < 0;
3719:   projectPreviewForward.hidden = !activeProjectPreviewForwardStack.length;
3720:   updateDialogWindowButtons(projectPreviewDialog);
3721: }

```

openProjectPreviewNode - lines 3723-3745

Item	Explanation
Source location	Lines 3723-3745 (23 source lines).
Purpose	Controls a window, dialog, panel, menu, or visible state.
Declaration	<code>function openProjectPreviewNode(sectionIndex, path = [], options = {}) {</code>
Touches	Mostly local calculations or local UI state.

Item	Explanation
Calls detected	filter, previewSectionHasRenderableContent, previewNodeAtPath, parsedItemHasContent, updateProjectPreviewNavigation, displayTitle, parsedPublicSectionContent
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 4 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

3723: function openProjectPreviewNode(sectionIndex, path = [], options = {}) {
3724:   if (!activeProjectPreviewProject?.portfolioView) return;
3725:   const sections = (activeProjectPreviewProject.portfolioView.sections || []).filter((section) =>
section.id !== "brief" && previewSectionHasRenderableContent(section));
3726:   const section = sections[Number(sectionIndex)];
3727:   if (!section) return;
3728:   const node = path.length ? previewNodeAtPath(section, path) : section;
3729:   if (!node) return;
3730:   if (path.length ? !parsedItemHasContent(node) : !previewSectionHasRenderableContent(section)) return;
3731:
3732:   activeProjectPreviewSectionIndex = Number(sectionIndex);
3733:   activeProjectPreviewPath = path;
3734:   if (!options.preserveForward) activeProjectPreviewForwardStack = [];
3735:   updateProjectPreviewNavigation();
3736:   projectPreviewTitle.textContent = displayTitle(node.title || section.title, "Section");
3737:   projectPreviewContent.innerHTML = `
3738:     <article class="portfolio-preview-article">
3739:       <section class="portfolio-preview-section">
3740:         ${parsedPublicSectionContent(section, Number(sectionIndex), path)}
3741:       </section>
3742:     </article>
3743:   `;
3744:   projectPreviewDialog.scrollTop = 0;
3745: }

```

sections - lines 3725-3730

Item	Explanation
Source location	Lines 3725-3730 (6 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	const sections = (activeProjectPreviewProject.portfolioView.sections []).filter((section) => section.id !== "brief" && previewSectionHasRenderableContent(section));
Touches	Mostly local calculations or local UI state.
Calls detected	filter, previewSectionHasRenderableContent, previewNodeAtPath, parsedItemHasContent
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 3 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

3725:   const sections = (activeProjectPreviewProject.portfolioView.sections || []).filter((section) =>
section.id !== "brief" && previewSectionHasRenderableContent(section));
3726:   const section = sections[Number(sectionIndex)];
3727:   if (!section) return;
3728:   const node = path.length ? previewNodeAtPath(section, path) : section;
3729:   if (!node) return;
3730:   if (path.length ? !parsedItemHasContent(node) : !previewSectionHasRenderableContent(section)) return;

```

projectPreviewBackStep - lines 3747-3769

Item	Explanation
Source location	Lines 3747-3769 (23 source lines).
Purpose	Part of the private builder frontend and editing experience.

Item	Explanation
Declaration	function projectPreviewBackStep() {
Touches	Mostly local calculations or local UI state.
Calls detected	push, slice, renderProjectPortfolioPreview, updateProjectPreviewNavigation, openProjectPreviewNode
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

3747: function projectPreviewBackStep() {
3748:   if (!activeProjectPreviewProject) return;
3749:   if (!activeProjectPreviewPath.length) {
3750:     if (activeProjectPreviewSectionIndex >= 0) {
3751:       activeProjectPreviewForwardStack.push({
3752:         path: activeProjectPreviewPath.slice(),
3753:         sectionIndex: activeProjectPreviewSectionIndex
3754:       });
3755:     }
3756:     projectPreviewTitle.textContent = activeProjectPreviewProject.title || "Project preview";
3757:     projectPreviewContent.innerHTML = renderProjectPortfolioPreview(activeProjectPreviewProject);
3758:     activeProjectPreviewSectionIndex = -1;
3759:     activeProjectPreviewPath = [];
3760:     updateProjectPreviewNavigation();
3761:     return;
3762:   }
3763:   activeProjectPreviewForwardStack.push({
3764:     path: activeProjectPreviewPath.slice(),
3765:     sectionIndex: activeProjectPreviewSectionIndex
3766:   });
3767:   const nextPath = activeProjectPreviewPath.slice(0, -1);
3768:   openProjectPreviewNode(activeProjectPreviewSectionIndex, nextPath, { preserveForward: true });
3769: }

```

projectPreviewForwardStep - lines 3771-3775

Item	Explanation
Source location	Lines 3771-3775 (5 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function projectPreviewForwardStep() {
Touches	Mostly local calculations or local UI state.
Calls detected	pop, openProjectPreviewNode
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

3771: function projectPreviewForwardStep() {
3772:   if (!activeProjectPreviewProject || !activeProjectPreviewForwardStack.length) return;
3773:   const next = activeProjectPreviewForwardStack.pop();
3774:   openProjectPreviewNode(next.sectionIndex, next.path, { preserveForward: true });
3775: }

```

closeOrStepBackProjectPreview - lines 3777-3783

Item	Explanation
Source location	Lines 3777-3783 (7 source lines).
Purpose	Controls a window, dialog, panel, menu, or visible state.
Declaration	function closeOrStepBackProjectPreview() {

Item	Explanation
Touches	Mostly local calculations or local UI state.
Calls detected	projectPreviewBackStep, closeProjectPreviewWindow
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

3777: function closeOrStepBackProjectPreview() {
3778:   if (activeProjectPreviewProject && (activeProjectPreviewSectionIndex >= 0 ||
activeProjectPreviewPath.length)) {
3779:     projectPreviewBackStep();
3780:     return;
3781:   }
3782:   closeProjectPreviewWindow();
3783: }

```

closeProjectPreviewWindow - lines 3785-3791

Item	Explanation
Source location	Lines 3785-3791 (7 source lines).
Purpose	Controls a window, dialog, panel, menu, or visible state.
Declaration	function closeProjectPreviewWindow() {
Touches	Mostly local calculations or local UI state.
Calls detected	closeDialogElement
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

3785: function closeProjectPreviewWindow() {
3786:   activeProjectPreviewProject = null;
3787:   activeProjectPreviewSectionIndex = -1;
3788:   activeProjectPreviewPath = [];
3789:   activeProjectPreviewForwardStack = [];
3790:   closeDialogElement(projectPreviewDialog);
3791: }

```

publicProjectCard - lines 3793-3856

Item	Explanation
Source location	Lines 3793-3856 (64 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function publicProjectCard(project) {
Touches	Mostly local calculations or local UI state.
Calls detected	parsedPublicProjectCard, categoryById, projectTemplateClass, projectResponsiveClass, projectTemplateStyle, detailBlock, map, itemTitle, join, evidenceList, resourceLink, mediaGrid
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 2 return statement(s).

Item	Explanation
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

3793: function publicProjectCard(project) {
3794:   if (project.portfolioView) return parsedPublicProjectCard(project);
3795:   const category = categoryById(project.category);
3796:   const accent = category.accent || "#117c7a";
3797:
3798:   return `
3799:     <article class="project-card catalog-card ${projectTemplateClass(project)}
${projectResponsiveClass(project)}" id="${project.id}" style="${projectTemplateStyle(project, accent)}">
3800:       <div class="project-body">
3801:         <h3>${project.title}</h3>
3802:         <p>${project.summary || ""}</p>
3803:
3804:         ${project.highlights && project.highlights.length ? detailBlock("Project highlights", "project-
drawer", `
3805:           <ul class="highlight-list">
3806:             ${project.highlights.map((item) => `<li>${typeof item === "string" ? item :
itemTitle(item)}</li>`).join("")}
3807:           </ul>
3808:           `) : ""}
3809:
3810:         <div class="evidence-grid" aria-label="${project.title} evidence blocks">
3811:           ${detailBlock("Documents", "evidence-block", evidenceList(project.documents, (item) => `
3812:             <li>
3813:               ${resourceLink(item, item.title)}
3814:               <span>${item.type || "Document"} &middot; ${item.status || "tracked"}</span>
3815:             </li>
3816:             ` , "No document artifact has been added yet."))}
3817:
3818:           ${detailBlock("Tests and results", "evidence-block", evidenceList(project.tests, (item) => `
3819:             <li>
3820:               ${resourceLink({ url: item.artifact, status: item.status }, item.name)}
3821:               <span>${item.method || "Validation"} &middot; ${item.status || "tracked"}</span>
3822:               ${item.result ? `<p>${item.result}</p>` : ""}
3823:             </li>
3824:             ` , "No test artifact has been added yet."))}
3825:
3826:           ${detailBlock("PCBs built", "evidence-block", evidenceList(project.pcb, (item) => `
3827:             <li>
3828:               ${resourceLink({ url: item.artifact, status: item.status }, item.name)}
3829:               <span>${item.revision || "Revision"} &middot; ${item.status || "tracked"}</span>
3830:             </li>
3831:             ` , "No PCB build has been added yet."))}
3832:
3833:           ${detailBlock("Images", "evidence-block evidence-wide", mediaGrid(project.media))}
3834:           ${publicCustomSectionBlocks(project)}
3835:         </div>
3836:
3837:         ${detailBlock("Tools and implementation files", "project-drawer", `
3838:           <div class="project-tooling">
3839:             <div>
3840:               <h4>Tools Used</h4>
3841:               <div class="project-meta">${pillList(project.tools, "tool-tag")}</div>
3842:             </div>
3843:             <div>
3844:               <h4>Languages</h4>
3845:               <div class="project-meta">${pillList(project.languages, "language-tag")}</div>
3846:             </div>
3847:           </div>
3848:         `)}
3849:
3850:         <div class="resource-list">
3851:           ${project.links || []}.map((item) => resourceLink(item)).join("")
3852:         </div>
3853:       </div>
3854:     </article>
3855:   `;
3856: }

```

renderParsedBriefBlock - lines 3858-3868

Item	Explanation
Source location	Lines 3858-3868 (11 source lines).
Purpose	Renders UI, markup, or display output.
Declaration	function renderParsedBriefBlock(section, fallbackSummary = "") {
Touches	Mostly local calculations or local UI state.

Item	Explanation
Calls detected	renderRichContent
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

3858: function renderParsedBriefBlock(section, fallbackSummary = "") {
3859:   const briefItem = section?.items?.[0] || {};
3860:   const briefText = briefItem.description || fallbackSummary || "";
3861:   if (!briefItem.rich?.blocks?.length && !briefText) return "";
3862:   return `
3863:     <details class="project-brief-default parsed-summary" open>
3864:       <summary>Overview</summary>
3865:       ${renderRichContent(briefItem.rich, briefText)}
3866:     </details>
3867:   `;
3868: }

```

previewPathToString - lines 3870-3872

Item	Explanation
Source location	Lines 3870-3872 (3 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function previewPathToString(path = []) {
Touches	Mostly local calculations or local UI state.
Calls detected	join
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

3870: function previewPathToString(path = []) {
3871:   return path.join(".");
3872: }

```

previewPathFromString - lines 3874-3877

Item	Explanation
Source location	Lines 3874-3877 (4 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function previewPathFromString(value = "") {
Touches	Mostly local calculations or local UI state.
Calls detected	split, map, filter, isInteger
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

3874: function previewPathFromString(value = "") {

```



```

3875:   if (!value) return [];
3876:   return value.split(".").map((item) => Number(item)).filter((item) => Number.isInteger(item));
3877: }

```

previewNodeAtPath - lines 3879-3888

Item	Explanation
Source location	Lines 3879-3888 (10 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function previewNodeAtPath(section, path = []) {
Touches	Mostly local calculations or local UI state.
Calls detected	No obvious named calls detected by the generator.
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

3879: function previewNodeAtPath(section, path = []) {
3880:   let node = section;
3881:   let children = section?.items || [];
3882:   for (const index of path) {
3883:     node = children[index];
3884:     if (!node) return null;
3885:     children = node.children || [];
3886:   }
3887:   return node;
3888: }

```

previewNodeChildren - lines 3890-3892

Item	Explanation
Source location	Lines 3890-3892 (3 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function previewNodeChildren(node) {
Touches	Mostly local calculations or local UI state.
Calls detected	No obvious named calls detected by the generator.
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

3890: function previewNodeChildren(node) {
3891:   return node?.items || node?.children || [];
3892: }

```

previewSectionHasRenderableContent - lines 3894-3896

Item	Explanation
Source location	Lines 3894-3896 (3 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function previewSectionHasRenderableContent(section) {
Touches	Mostly local calculations or local UI state.

Item	Explanation
Calls detected	richHasContent, some
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

3894: function previewSectionHasRenderableContent(section) {
3895:   return Boolean(section?.description || richHasContent(section?.rich) || (section?.items ||
[]).some(parsedItemHasContent));
3896: }

```

previewNodeSummary - lines 3898-3906

Item	Explanation
Source location	Lines 3898-3906 (9 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function previewNodeSummary(title, rich, text, emptyMessage = "No summary has been added yet.") {
Touches	Mostly local calculations or local UI state.
Calls detected	escapeHtml, renderRichContent
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

3898: function previewNodeSummary(title, rich, text, emptyMessage = "No summary has been added yet.") {
3899:   if (!rich?.blocks?.length && !text) return "";
3900:   return `
3901:     <details class="parsed-summary" open>
3902:       <summary>${escapeHtml(title)}</summary>
3903:       ${renderRichContent(rich, text || "")}
3904:     </details>
3905:   `;
3906: }

```

previewNodeIsOverview - lines 3908-3911

Item	Explanation
Source location	Lines 3908-3911 (4 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function previewNodeIsOverview(item = {}) {
Touches	filesystem
Calls detected	normalize, displayTitle, endsWith
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

3908: function previewNodeIsOverview(item = {}) {
3909:   const title = normalize(displayTitle(item.title || item.label || ""));

```

```

3910:   return item.kind === "summary" || title === "overview" || title.endsWith(" overview");
3911: }

```

previewNodeOverviewDetails - lines 3913-3931

Item	Explanation
Source location	Lines 3913-3931 (19 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function previewNodeOverviewDetails(node, children = []) {
Touches	Mostly local calculations or local UI state.
Calls detected	filter, push, renderRichContent, forEach, previewFileList, previewNodeChildren, join
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

3913: function previewNodeOverviewDetails(node, children = []) {
3914:   const overviewItems = children.filter(previewNodeIsOverview);
3915:   const blocks = [];
3916:   if (node?.rich?.blocks?.length || node?.description) {
3917:     blocks.push(renderRichContent(node.rich, node.description || ""));
3918:   }
3919:   overviewItems.forEach((item) => {
3920:     if (item.rich?.blocks?.length || item.description) blocks.push(renderRichContent(item.rich,
item.description || ""));
3921:     const itemFiles = previewFileList(previewNodeChildren(item));
3922:     if (itemFiles) blocks.push(itemFiles);
3923:   });
3924:   if (!blocks.length) return "";
3925:   return
3926:     <details class="parsed-summary section-overview-default" open>
3927:       <summary>Overview</summary>
3928:       ${blocks.join("")}
3929:     </details>
3930:   `;
3931: }

```

previewNodeCard - lines 3933-3940

Item	Explanation
Source location	Lines 3933-3940 (8 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function previewNodeCard(node, sectionIndex, path) {
Touches	Mostly local calculations or local UI state.
Calls detected	displayTitle, previewPathToString, escapeHtml
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

3933: function previewNodeCard(node, sectionIndex, path) {
3934:   const title = displayTitle(node.title);
3935:   return `
3936:     <button class="section-open-card subsection-open-card" type="button" data-preview-section-
index="${sectionIndex}" data-preview-resource-path="${previewPathToString(path)}">
3937:       <span>${escapeHtml(title)}</span>
3938:     </button>
3939:   `;
3940: }

```

previewChildCards - lines 3942-3950

Item	Explanation
Source location	Lines 3942-3950 (9 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function previewChildCards(children, sectionIndex, basePath = []) {
Touches	Mostly local calculations or local UI state.
Calls detected	filter, parsedItemHasContent, previewNodesOverview, map, previewNodeCard, join
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
3942: function previewChildCards(children, sectionIndex, basePath = []) {
3943:   const visibleChildren = children.filter((child) => parsedItemHasContent(child) && !child.url &&
!previewNodeIsOverview(child));
3944:   if (!visibleChildren.length) return "";
3945:   return `
3946:     <div class="subsection-grid section-content-grid">
3947:       ${children.map((child, index) => parsedItemHasContent(child) && !child.url &&
!previewNodeIsOverview(child) ? previewNodeCard(child, sectionIndex, [...basePath, index]) : "").join("")}
3948:     </div>
3949:   `;
3950: }
```

previewFileList - lines 3952-3969

Item	Explanation
Source location	Lines 3952-3969 (18 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function previewFileList(children = []) {
Touches	filesystem
Calls detected	filter, map, escapeHtml, normalizeLinkTarget, isWebsiteLinkItem, linkAttributes, downloadAttribute, join
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
3952: function previewFileList(children = []) {
3953:   const files = children.filter((child) => child?.url);
3954:   if (!files.length) return "";
3955:   return `
3956:     <div class="section-file-list">
3957:       ${files.map((file) => `
3958:         <a
3959:           class="section-file-link"
3960:           href="${escapeHtml(normalizeLinkTarget(file.url, { assumeWeb: isWebsiteLinkItem(file, file.url)
3961:             ${linkAttributes(file.url, file)}
3962:             ${downloadAttribute(file.url, file)}
3963:           >
3964:             ${escapeHtml(file.title || "Open link")}
3965:           </a>
3966:         `).join("")}
3967:     </div>
3968:   `;
3969: }
```

previewNodeContent - lines 3971-3983

Item	Explanation
Source location	Lines 3971-3983 (13 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function previewNodeContent(node, sectionIndex, path = []) {
Touches	Mostly local calculations or local UI state.
Calls detected	previewFileList, previewNodeChildren, filter, previewNodeIsOverview, previewNodeOverviewDetails, previewChildCards
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

3971: function previewNodeContent(node, sectionIndex, path = []) {
3972:   const isContainer = !node.kind || node.kind === "subsection";
3973:   if (!isContainer && node.url) return previewFileList([node]);
3974:   const children = previewNodeChildren(node);
3975:   const contentChildren = children.filter((child) => !previewNodeIsOverview(child));
3976:   return `
3977:     <article class="parsed-window-panel section-directory">
3978:       ${previewNodeOverviewDetails(node, children)}
3979:       ${previewChildCards(children, sectionIndex, path)}
3980:       ${previewFileList(contentChildren)}
3981:     </article>
3982:   `;
3983: }
```

parsedPublicSectionContent - lines 3985-3989

Item	Explanation
Source location	Lines 3985-3989 (5 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function parsedPublicSectionContent(section, sectionIndex = 0, path = []) {
Touches	Mostly local calculations or local UI state.
Calls detected	previewNodeAtPath, previewNodeContent
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

3985: function parsedPublicSectionContent(section, sectionIndex = 0, path = []) {
3986:   const node = path.length ? previewNodeAtPath(section, path) : section;
3987:   if (!node) return `<p class="evidence-empty">This section could not be found.</p>`;
3988:   return previewNodeContent(node, sectionIndex, path);
3989: }
```

parsedPublicSection - lines 3991-3999

Item	Explanation
Source location	Lines 3991-3999 (9 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function parsedPublicSection(section, index = 0) {
Touches	Mostly local calculations or local UI state.
Calls detected	filter, some, escapeHtml, displayTitle

Item	Explanation
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

3991: function parsedPublicSection(section, index = 0) {
3992:   const visibleItems = (section.items || []).filter(parsedItemHasContent);
3993:   const hasImages = visibleItems.some((item) => item.kind === "image");
3994:   return `
3995:     <button class="section-open-card evidence-block ${hasImages ? "evidence-wide" : ""}" type="button"
data-preview-section-index="${index}">
3996:       <span>${escapeHtml(displayTitle(section.title, "Section"))}</span>
3997:     </button>
3998:   `;
3999: }

```

parsedPublicProjectCard - lines 4001-4019

Item	Explanation
Source location	Lines 4001-4019 (19 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function parsedPublicProjectCard(project) {
Touches	Mostly local calculations or local UI state.
Calls detected	categoryById, find, filter, previewSectionHasRenderableContent, projectTemplateClass, projectResponsiveClass, projectTemplateStyle, renderParsedBriefBlock, map, parsedPublicSection, join
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

4001: function parsedPublicProjectCard(project) {
4002:   const category = categoryById(project.category);
4003:   const accent = category.accent || "#117c7a";
4004:   const view = project.portfolioView;
4005:   const briefSection = (view.sections || []).find((section) => section.id === "brief");
4006:   const otherSections = (view.sections || []).filter((section) => section.id !== "brief" &&
previewSectionHasRenderableContent(section));
4007:
4008:   return `
4009:     <article class="project-card catalog-card ${projectTemplateClass(project)}
${projectResponsiveClass(project)}" id="${project.id}" style="${projectTemplateStyle(project, accent)}">
4010:       <div class="project-body">
4011:         <h3>${view.title || project.title}</h3>
4012:         ${renderParsedBriefBlock(briefSection, project.summary)}
4013:         <div class="evidence-grid" aria-label="${project.title} parsed project content">
4014:           ${otherSections.map((section, index) => parsedPublicSection(section, index)).join("")}
4015:         </div>
4016:       </div>
4017:     </article>
4018:   `;
4019: }

```

briefSection - lines 4005-4006

Item	Explanation
Source location	Lines 4005-4006 (2 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	const briefSection = (view.sections []).find((section) => section.id === "brief");
Touches	Mostly local calculations or local UI state.

Item	Explanation
Calls detected	find, filter, previewSectionHasRenderableContent
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

4005:   const briefSection = (view.sections || []).find((section) => section.id === "brief");
4006:   const otherSections = (view.sections || []).filter((section) => section.id !== "brief" &&
previewSectionHasRenderableContent(section));

```

otherSections - lines 4006-4006

Item	Explanation
Source location	Lines 4006-4006 (1 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	const otherSections = (view.sections []).filter((section) => section.id !== "brief" && previewSectionHasRenderableContent(section));
Touches	Mostly local calculations or local UI state.
Calls detected	filter, previewSectionHasRenderableContent
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

4006:   const otherSections = (view.sections || []).filter((section) => section.id !== "brief" &&
previewSectionHasRenderableContent(section));

```

publicCategorySection - lines 4021-4036

Item	Explanation
Source location	Lines 4021-4036 (16 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function publicCategorySection(category, visibleProjects) {
Touches	Mostly local calculations or local UI state.
Calls detected	map, join
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

4021: function publicCategorySection(category, visibleProjects) {
4022:   return `
4023:     <section class="category-section ${visibleProjects.length ? "" : "empty-category-section"}" aria-
labelledby="${category.id}-preview-title">
4024:       <div class="category-heading">
4025:         <div>
4026:           <h3 id="${category.id}-preview-title">${category.label}</h3>
4027:         </div>
4028:         ${visibleProjects.length ? `<span>${visibleProjects.length} project${visibleProjects.length === 1

```

```

? "" : "s"}</span>` : ""}
4029:     </div>
4030:     ${visibleProjects.length ? `<p class="category-description">${category.description}</p>` : ""}
4031:     <div class="category-projects">
4032:       ${visibleProjects.map(publicProjectCard).join("")}
4033:     </div>
4034:   </section>
4035: `;
4036: }

```

renderedPublicProjects - lines 4038-4051

Item	Explanation
Source location	Lines 4038-4051 (14 source lines).
Purpose	Renders UI, markup, or display output.
Declaration	function renderedPublicProjects() {
Touches	Mostly local calculations or local UI state.
Calls detected	map, filter, publicCategorySection, join
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 3 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

4038: function renderedPublicProjects() {
4039:   if (!savedPortfolioCatalog.projects.length) {
4040:     return `
4041:       <div class="empty-state">
4042:         <h3>No parsed projects saved yet.</h3>
4043:         <p>Open a project, edit it, then click Save to build it into this portfolio preview.</p>
4044:       </div>
4045:     `;
4046:   }
4047:   return savedPortfolioCatalog.categories.map((category) => {
4048:     const categoryProjects = savedPortfolioCatalog.projects.filter((project) => project.category ===
category.id);
4049:     return publicCategorySection(category, categoryProjects);
4050:   }).join("");
4051: }

```

sectionControls - lines 4053-4060

Item	Explanation
Source location	Lines 4053-4060 (8 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function sectionControls(sectionId, uploadKey = "", folder = "") {
Touches	Mostly local calculations or local UI state.
Calls detected	No obvious named calls detected by the generator.
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

4053: function sectionControls(sectionId, uploadKey = "", folder = "") {
4054:   return `
4055:     <div class="local-section-controls">
4056:       <button type="button" title="Edit this section" data-preview-section="${sectionId}">Edit</button>
4057:       ${uploadKey ? `<button type="button" title="Add file or image" data-preview-upload="${uploadKey}"
data-folder="${folder}">+</button>` : ""}
4058:     </div>
4059:   `;

```


4060: }

customSectionBlocks - lines 4062-4074

Item	Explanation
Source location	Lines 4062-4074 (13 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function customSectionBlocks(project) {
Touches	Mostly local calculations or local UI state.
Calls detected	map, detailBlock, sectionControls, evidenceList, resourceLink, join
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
4062: function customSectionBlocks(project) {
4063:   return (project.sections || []).map((section) => detailBlock(section.title, "evidence-block evidence-
wide", `
4064:     ${sectionControls(`custom:${section.id}`, "sections", section.id)}
4065:     ${section.description ? `
```

projectCard - lines 4076-4154

Item	Explanation
Source location	Lines 4076-4154 (79 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function projectCard(project, isSelected = false) {
Touches	Mostly local calculations or local UI state.
Calls detected	categoryById, projectTemplateClass, projectResponsiveClass, projectTemplateStyle, detailBlock, map, join, sectionControls, evidenceList, resourceLink, mediaGrid, customSectionBlocks
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
4076: function projectCard(project, isSelected = false) {
4077:   const category = categoryById(project.category);
4078:   const accent = category.accent || "#117c7a";
4079:
4080:   return `
4081:     <article class="project-card catalog-card ${projectTemplateClass(project)}
${projectResponsiveClass(project)} ${isSelected ? "preview-project-card" : ""}" id="${project.id}"
style="${projectTemplateStyle(project, accent)}">
4082:       <div class="project-body">
4083:         <div class="local-card-controls">
4084:           <button type="button" title="Open project" data-preview-project="${project.id}">Open</button>
4085:           <button type="button" title="Delete project" data-preview-
delete="${project.id}">Delete</button>
4086:         </div>
4087:         <h3>${project.title}</h3>
4088:         <p>${project.summary}</p>
```

```

4089:         ${project.highlights && project.highlights.length ? detailBlock("Project highlights", "project-
4090: drawer", `
4091:         <ul class="highlight-list">
4092:             ${project.highlights.map((item) => `<li>${item}</li>`).join("")}
4093:         </ul>
4094:         `) : ""}
4095:
4096:     <div class="evidence-grid" aria-label="${project.title} evidence blocks">
4097:         ${detailBlock("Documents", "evidence-block", `
4098:             ${sectionControls("documents", "documents", "documents")}
4099:             ${evidenceList(project.documents, (item) => `
4100:                 <li>
4101:                     ${resourceLink(item, item.title)}
4102:                     <span>${item.type || "Document"} &middot; ${item.status || "tracked"}</span>
4103:                 </li>
4104:             `), "No document artifact has been added yet.")}
4105:         `)}
4106:
4107:         ${detailBlock("Tests and results", "evidence-block", `
4108:             ${sectionControls("tests", "tests", "tests")}
4109:             ${evidenceList(project.tests, (item) => `
4110:                 <li>
4111:                     ${resourceLink({ url: item.artifact, status: item.status }, item.name)}
4112:                     <span>${item.method || "Validation"} &middot; ${item.status || "tracked"}</span>
4113:                     ${item.result ? `<p>${item.result}</p>` : ""}
4114:                 </li>
4115:             `), "No test artifact has been added yet.")}
4116:         `)}
4117:
4118:         ${detailBlock("PCBs built", "evidence-block", `
4119:             ${sectionControls("pcbs", "pcbs", "pcbs")}
4120:             ${evidenceList(project.pcbs, (item) => `
4121:                 <li>
4122:                     ${resourceLink({ url: item.artifact, status: item.status }, item.name)}
4123:                     <span>${item.revision || "Revision"} &middot; ${item.status || "tracked"}</span>
4124:                 </li>
4125:             `), "No PCB build has been added yet.")}
4126:         `)}
4127:
4128:         ${detailBlock("Images", "evidence-block evidence-wide", `
4129:             ${sectionControls("media", "media", "images")}
4130:             ${mediaGrid(project.media)}
4131:         `)}
4132:     ${customSectionBlocks(project)}
4133: </div>
4134:
4135:     ${detailBlock("Tools and implementation files", "project-drawer", `
4136:         <div class="project-tooling">
4137:             <div>
4138:                 <h4>Tools Used</h4>
4139:                 <div class="project-meta">${pillList(project.tools, "tool-tag")}</div>
4140:             </div>
4141:             <div>
4142:                 <h4>Languages</h4>
4143:                 <div class="project-meta">${pillList(project.languages, "language-tag")}</div>
4144:             </div>
4145:         </div>
4146:     `)}
4147:
4148:     <div class="resource-list">
4149:         ${project.links || []}.map((item) => resourceLink(item)).join("")
4150:     </div>
4151: </div>
4152: </article>
4153: `;
4154: }

```

buildTemplateProject - lines 4156-4186

Item	Explanation
Source location	Lines 4156-4186 (31 source lines).
Purpose	Creates an object, UI structure, process, payload, or generated artifact.
Declaration	function buildTemplateProject() {
Touches	filesystem
Calls detected	find, trim, slugify, clone, defaultDesignModel
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).

Item	Explanation
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

4156: function buildTemplateProject() {
4157:   const template = templates.find((item) => item.id === templateSelect.value) || null;
4158:   const title = newProjectTitle.value.trim() || "New Hardware Project";
4159:   const id = slugify(title);
4160:   const category = newCategorySelect.value || pendingCreateCategoryId || "analog-mixed-signal";
4161:
4162:   return {
4163:     id,
4164:     title,
4165:     templateId: template?.id || "",
4166:     templateLabel: template?.label || "",
4167:     templateGroup: "Appearance",
4168:     templateVisual: template?.visual ? clone(template.visual) : null,
4169:     category,
4170:     status: "Draft",
4171:     summary: "",
4172:     summaryRich: null,
4173:     focus: [],
4174:     highlights: [],
4175:     documents: [],
4176:     tests: [],
4177:     pcbs: [],
4178:     media: [],
4179:     tools: [],
4180:     languages: [],
4181:     links: [],
4182:     sections: [],
4183:     compileCode: { files: [], activeFileId: "", terminal: "" },
4184:     design: category === "analog-mixed-signal" ? defaultDesignModel() : undefined
4185:   };
4186: }

```

insertProject - lines 4188-4204

Item	Explanation
Source location	Lines 4188-4204 (17 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function insertProject(project) {
Touches	Mostly local calculations or local UI state.
Calls detected	filter, map, splice
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 4 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

4188: function insertProject(project) {
4189:   const nextProjects = catalog.projects.filter((item) => item.id !== project.id);
4190:   const placement = placementSelect.value;
4191:
4192:   if (placement === "site-start") return [project, ...nextProjects];
4193:   if (placement === "site-end") return [...nextProjects, project];
4194:
4195:   const indexes = nextProjects
4196:     .map((item, index) => ({ item, index }))
4197:     .filter(({ item }) => item.category === project.category)
4198:     .map(({ index }) => index);
4199:
4200:   if (!indexes.length) return [...nextProjects, project];
4201:   const targetIndex = placement === "category-start" ? indexes[0] : indexes[indexes.length - 1] + 1;
4202:   nextProjects.splice(targetIndex, 0, project);
4203:   return nextProjects;
4204: }

```

renderCategoryDropdown - lines 4206-4213

Item	Explanation
Source location	Lines 4206-4213 (8 source lines).
Purpose	Renders UI, markup, or display output.
Declaration	function renderCategoryDropdown() {
Touches	Mostly local calculations or local UI state.
Calls detected	map, escapeHtml, join
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

4206: function renderCategoryDropdown() {
4207:   categoryDropdown.innerHTML = (catalog.categories || []).map((category) => `
4208:     <button type="button" data-create-category="${escapeHtml(category.id)}" style="--category-accent:
${escapeHtml(category.accent || "#117c7a")}">
4209:       <span>${escapeHtml(category.label)}</span>
4210:       <small>${escapeHtml(category.description || "Project category")}</small>
4211:     </button>
4212:   `).join("");
4213: }

```

toggleCategoryDropdown - lines 4215-4219

Item	Explanation
Source location	Lines 4215-4219 (5 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function toggleCategoryDropdown(forceOpen = null) {
Touches	Mostly local calculations or local UI state.
Calls detected	setAttribute
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

4215: function toggleCategoryDropdown(forceOpen = null) {
4216:   const open = forceOpen === null ? categoryDropdown.hidden : forceOpen;
4217:   categoryDropdown.hidden = !open;
4218:   addProjectButton.setAttribute("aria-expanded", String(open));
4219: }

```

createProjectInCategory - lines 4221-4233

Item	Explanation
Source location	Lines 4221-4233 (13 source lines).
Purpose	Creates an object, UI structure, process, payload, or generated artifact.
Declaration	function createProjectInCategory(categoryId) {
Touches	Mostly local calculations or local UI state.
Calls detected	buildTemplateProject, insertProject, add, toggleCategoryDropdown, setStatus, categoryById, scheduleAutosave, renderAll, openProjectWindow
API endpoints	None detected inside this function body.

Item	Explanation
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

4221: function createProjectInCategory(categoryId) {
4222:   newCategorySelect.value = categoryId;
4223:   const project = buildTemplateProject();
4224:   catalog.projects = insertProject(project);
4225:   selectedProjectId = project.id;
4226:   activeSectionId = "brief";
4227:   expandedCategories.add(categoryId);
4228:   toggleCategoryDropdown(false);
4229:   setStatus(`Project added locally in ${categoryById(categoryId).label || categoryId}. Click Save project
to include it in the portfolio preview.`);
4230:   scheduleAutosave();
4231:   renderAll();
4232:   openProjectWindow(project.id, "brief");
4233: }

```

openProjectCreateDialog - lines 4235-4245

Item	Explanation
Source location	Lines 4235-4245 (11 source lines).
Purpose	Controls a window, dialog, panel, menu, or visible state.
Declaration	function openProjectCreateDialog(categoryId) {
Touches	Mostly local calculations or local UI state.
Calls detected	categoryById, groupedTemplateOptions, toggleCategoryDropdown, showModal
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

4235: function openProjectCreateDialog(categoryId) {
4236:   pendingCreateCategoryId = categoryId;
4237:   newCategorySelect.value = categoryId;
4238:   const category = categoryById(categoryId);
4239:   projectCreateCategory.textContent = category.label || "Project category";
4240:   templateSelect.innerHTML = groupedTemplateOptions();
4241:   templateSelect.value = "";
4242:   newProjectTitle.value = "";
4243:   toggleCategoryDropdown(false);
4244:   projectCreateDialog.showModal();
4245: }

```

groupedTemplateOptions - lines 4247-4253

Item	Explanation
Source location	Lines 4247-4253 (7 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function groupedTemplateOptions() {
Touches	Mostly local calculations or local UI state.
Calls detected	map, join
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).

Item	Explanation
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

4247: function groupedTemplateOptions() {
4248:   const emptyOption = `<option value="">No appearance - white project layout</option>`;
4249:   const options = templates
4250:     .map((template) => `<option value="${template.id}">${template.label}</option>`)
4251:     .join("");
4252:   return `${emptyOption}${options}`;
4253: }

```

showTemplatePreview - lines 4255-4276

Item	Explanation
Source location	Lines 4255-4276 (22 source lines).
Purpose	Controls a window, dialog, panel, menu, or visible state.
Declaration	function showTemplatePreview(template) {
Touches	Mostly local calculations or local UI state.
Calls detected	setProperty, map, join, showModal
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

4255: function showTemplatePreview(template) {
4256:   if (!template) return;
4257:   const visual = template.visual || {};
4258:   const palette = visual.palette || ["#0f4c5c", "#7dd3c7", "#ffffff"];
4259:
4260:   templatePreviewGroup.textContent = visual.style ? `Appearance / ${visual.style}` : "Appearance";
4261:   templatePreviewTitle.textContent = template.label;
4262:   templatePreviewDescription.textContent = template.description || "";
4263:   templatePreviewCardTitle.textContent = template.label;
4264:   templatePreviewInteraction.textContent = visual.interaction || "Hover and click styles are shown in the
local builder.";
4265:   templatePreviewCard.style.setProperty("--template-primary", palette[0]);
4266:   templatePreviewCard.style.setProperty("--template-hover", palette[1]);
4267:   templatePreviewCard.style.setProperty("--template-paper", palette[2] || "#ffffff");
4268:   templatePreviewCard.style.setProperty("--template-bg", visual.background || palette[0]);
4269:   templatePreviewCard.style.setProperty("--template-panel", visual.panel || palette[2] || "#ffffff");
4270:   templatePreviewCard.style.setProperty("--template-accent", visual.accent || palette[1]);
4271:   templatePreviewCard.style.setProperty("--template-click", visual.click || palette[2] || "#ffffff");
4272:   templatePreviewCard.style.setProperty("--template-click-text", visual.clickText || visual.text ||
"#172636");
4273:   templateVisual.className = `template-visual template-visual-${visual.style || "schematic"}`;
4274:   templatePalette.innerHTML = palette.map((color) => `<span style="background:
${color}">${color}</span>`).join("");
4275:   templateDialog.showModal();
4276: }

```

renderTree - lines 4278-4319

Item	Explanation
Source location	Lines 4278-4319 (42 source lines).
Purpose	Renders UI, markup, or display output.
Declaration	function renderTree() {
Touches	Mostly local calculations or local UI state.
Calls detected	projectSearchText, map, filter, includes, projectMatchesSearch, has, escapeHtml, highlightSearchText, join, updateProjectSearchStatus
API endpoints	None detected inside this function body.

Item	Explanation
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

4278: function renderTree() {
4279:   const cleanQuery = projectSearchText(projectSearchQuery);
4280:   let matchCount = 0;
4281:   const treeMarkup = catalog.categories.map((category) => {
4282:     const allCategoryProjects = catalog.projects.filter((project) => project.category === category.id);
4283:     const categoryHit = cleanQuery && projectSearchText(`${category.label}
${category.description}`).includes(cleanQuery);
4284:     const categoryProjects = cleanQuery
4285:       ? allCategoryProjects.filter((project) => categoryHit || projectMatchesSearch(project, category,
cleanQuery))
4286:       : allCategoryProjects;
4287:     if (cleanQuery && !categoryProjects.length && !categoryHit) return "";
4288:     matchCount += categoryProjects.length;
4289:     const isExpanded = cleanQuery ? true : expandedCategories.has(category.id);
4290:     return `
4291:       <section class="tree-category">
4292:         <div class="tree-category-heading">
4293:           <button class="tree-category-toggle" type="button" data-toggle-
category="${escapeHtml(category.id)}" aria-expanded="${isExpanded}">
4294:             <span>${highlightSearchText(category.label, projectSearchQuery)}</span>
4295:             <small>${allCategoryProjects.length} project${allCategoryProjects.length === 1 ? "" :
"s"}</small>
4296:           </button>
4297:         </div>
4298:         <div class="tree-projects" ${isExpanded ? "" : "hidden"}>
4299:           ${categoryProjects.length ? categoryProjects.map((project) => `
4300:             <div class="tree-project-row ${project.id === selectedProjectId ? "active" : ""}>
4301:               <button type="button" data-project-id="${escapeHtml(project.id)}">
4302:                 <span>${highlightSearchText(project.title, projectSearchQuery)}</span>
4303:               </button>
4304:               <button class="tree-project-delete" type="button" data-delete-
project="${escapeHtml(project.id)}" aria-label="Delete ${escapeHtml(project.title)}" title="Delete project">
4305:                 &times;
4306:               </button>
4307:             </div>
4308:           `).join("") : cleanQuery ? `<p class="tree-empty">Category matches, but no projects are in it
yet.</p>` : ""}
4309:         </div>
4310:       </section>
4311:     `;
4312:   }).join("");
4313:   projectTree.innerHTML = treeMarkup || `
4314:   <section class="tree-category tree-search-empty">
4315:     <p class="tree-empty">No project, file, tool, or section matched this search.</p>
4316:   </section>
4317:   `;
4318:   updateProjectSearchStatus(cleanQuery ? matchCount : catalog.projects.length);
4319: }

```

summaryWordCount - lines 4321-4323

Item	Explanation
Source location	Lines 4321-4323 (3 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function summaryWordCount(project) {
Touches	Mostly local calculations or local UI state.
Calls detected	wordCount, plainTextFromRich
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

4321: function summaryWordCount(project) {

```

```

4322:   return wordCount(project?.summaryRich?.blocks?.length ? plainTextFromRich(project.summaryRich) :
project?.summary || "");
4323: }

```

renderSummaryPreview - lines 4325-4329

Item	Explanation
Source location	Lines 4325-4329 (5 source lines).
Purpose	Renders UI, markup, or display output.
Declaration	function renderSummaryPreview(project) {
Touches	filesystem
Calls detected	renderRichContent, renderMultilineInlineText
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 3 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

4325: function renderSummaryPreview(project) {
4326:   if (project.summaryRich?.blocks?.length) return renderRichContent(project.summaryRich, project.summary
|| "");
4327:   if (project.summary) return `<p class="rich-
paragraph">${renderMultilineInlineText(project.summary)}</p>`;
4328:   return `<p class="evidence-empty">No project overview has been added yet.</p>`;
4329: }

```

renderSummaryBuilder - lines 4331-4359

Item	Explanation
Source location	Lines 4331-4359 (29 source lines).
Purpose	Renders UI, markup, or display output.
Declaration	function renderSummaryBuilder(project) {
Touches	filesystem
Calls detected	summaryWordCount, renderSummaryPreview
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

4331: function renderSummaryBuilder(project) {
4332:   const isEditing = summaryEditorProjectId === project.id;
4333:   const summaryWords = summaryWordCount(project);
4334:   const hasSummary = summaryWords > 0 || Boolean(project.summaryRich?.blocks?.length);
4335:   return `
4336:     <div class="summary-builder wide-field">
4337:       <div class="summary-builder-heading">
4338:         <div>
4339:           <span>Project overview <small>${summaryWords}/1000 words</small></span>
4340:         </div>
4341:         <div class="summary-builder-actions">
4342:           ${isEditing
4343:             ? `<button class="button secondary" type="button" data-summary-cancel="true">Cancel
overview</button>`
4344:             : `<button class="button primary" type="button" data-summary-create="true">${hasSummary ?
"Edit overview" : "Create overview"}</button>`}
4345:           ${hasSummary && !isEditing ? `<button class="button danger-control" type="button" data-summary-
delete="true">Delete overview</button>` : ""}
4346:         </div>
4347:       </div>
4348:       ${isEditing ? `
4349:         <div class="rich-summary-panel">

```



```

4350:         <p class="rich-editor-hint">Right-click text or an inserted block for formatting, images,
formulas, code blocks, alignment, and delete controls.</p>
4351:         <div class="rich-summary-editor" data-rich-editor="summary" data-placeholder="Click anywhere
and start writing the project overview." contenteditable="true" spellcheck="true" aria-label="Rich overview
editor"></div>
4352:         <div class="summary-save-actions">
4353:             <button class="button primary" type="button" data-summary-save="true">Save overview</button>
4354:         </div>
4355:     </div>
4356:     ` : `<div class="summary-rendered-preview">${renderSummaryPreview(project)}</div>`
4357: </div>
4358: `;
4359: }

```

applyRichTextBlockStyle - lines 4361-4372

Item	Explanation
Source location	Lines 4361-4372 (12 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function applyRichTextBlockStyle(block) {
Touches	browser/Electron DOM, filesystem
Calls detected	cleanFontFamily, normalizeFontPx, normalizeTextColor
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

4361: function applyRichTextBlockStyle(block) {
4362:     const fontFamily = cleanFontFamily(block.dataset.fontFamily || "Arial") || "Arial";
4363:     const fontPx = normalizeFontPx(block.dataset.fontPx || "");
4364:     const color = normalizeTextColor(block.dataset.color || "");
4365:
4366:     block.style.fontFamily = fontFamily || "";
4367:     block.style.fontSize = fontPx ? `${fontPx}px` : "";
4368:     block.style.color = color || "";
4369:     block.style.fontWeight = block.dataset.bold === "true" ? "700" : "";
4370:     block.style.fontStyle = block.dataset.italic === "true" ? "italic" : "";
4371:     block.style.textDecoration = block.dataset.underline === "true" ? "underline" : "";
4372: }

```

createRichTextBlock - lines 4374-4391

Item	Explanation
Source location	Lines 4374-4391 (18 source lines).
Purpose	Creates an object, UI structure, process, payload, or generated artifact.
Declaration	function createRichTextBlock(text = "", fontSize = "normal", align = "left", fontFamily = "Arial", fontPx = "", color = "", bold = false, html = "", italic = false, underline = false) {
Touches	browser/Electron DOM, filesystem
Calls detected	createElement, cleanFontFamily, normalizeFontPx, normalizeTextColor, sanitizeRichInlineHtml, applyRichTextBlockStyle
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

4374: function createRichTextBlock(text = "", fontSize = "normal", align = "left", fontFamily = "Arial", fontPx = "", color = "", bold = false, html = "", italic = false, underline = false) {
4375:     const block = document.createElement("p");
4376:     block.className = `rich-block rich-text-block rich-text-${fontSize} text-${align}`;
4377:     block.dataset.type = "paragraph";

```

```

4378:     block.dataset.fontSize = fontSize;
4379:     block.dataset.align = align;
4380:     block.dataset.fontFamily = cleanFontFamily(fontFamily || "Arial") || "Arial";
4381:     block.dataset.fontPx = normalizeFontPx(fontPx);
4382:     block.dataset.color = normalizeTextColor(color);
4383:     block.dataset.bold = bold ? "true" : "false";
4384:     block.dataset.italic = italic ? "true" : "false";
4385:     block.dataset.underline = underline ? "true" : "false";
4386:     block.spellcheck = true;
4387:     if (html) block.innerHTML = sanitizeRichInlineHtml(html);
4388:     else block.textContent = text;
4389:     applyRichTextBlockStyle(block);
4390:     return block;
4391: }

```

richBlockActions - lines 4393-4401

Item	Explanation
Source location	Lines 4393-4401 (9 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function richBlockActions(label = "block", options = {}) {
Touches	Mostly local calculations or local UI state.
Calls detected	escapeHtml
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

4393: function richBlockActions(label = "block", options = {}) {
4394:   return `
4395:     <div class="rich-block-actions">
4396:       ${options.movable ? `<button class="rich-drag-handle" type="button" draggable="true" data-rich-
drag-handle aria-label="Move ${escapeHtml(label)}">Move</button>` : ""}
4397:       <button type="button" data-rich-block-action="edit" aria-label="Edit
${escapeHtml(label)}">Edit</button>
4398:       <button class="danger-icon" type="button" data-rich-block-action="delete" aria-label="Delete
${escapeHtml(label)}">Delete</button>
4399:     </div>
4400:   `;
4401: }

```

createRichImageBlock - lines 4403-4431

Item	Explanation
Source location	Lines 4403-4431 (29 source lines).
Purpose	Creates an object, UI structure, process, payload, or generated artifact.
Declaration	function createRichImageBlock(blockData) {
Touches	browser/Electron DOM, filesystem
Calls detected	createElement, cleanRichImageTitle, normalizeCropAspect, normalizeCropZoom, normalizeCropPosition, richBlockActions, richImageCropStyle, escapeHtml, normalizeLinkTarget
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

4403: function createRichImageBlock(blockData) {
4404:   const figure = document.createElement("figure");
4405:   const align = blockData.align || "center";
4406:   const title = cleanRichImageTitle(blockData);
4407:   figure.className = `rich-block rich-image-block justify-${align}`;

```

```

4408:   figure.dataset.type = "image";
4409:   figure.dataset.url = blockData.url || "";
4410:   figure.dataset.title = blockData.title || "";
4411:   figure.dataset.caption = blockData.caption || "";
4412:   figure.dataset.align = align;
4413:   figure.dataset.display = blockData.display === "download" ? "download" : "show";
4414:   figure.dataset.source = blockData.source || "local";
4415:   figure.dataset.cropAspect = normalizeCropAspect(blockData.cropAspect);
4416:   figure.dataset.cropZoom = String(normalizeCropZoom(blockData.cropZoom));
4417:   figure.dataset.cropX = String(normalizeCropPosition(blockData.cropX));
4418:   figure.dataset.cropY = String(normalizeCropPosition(blockData.cropY));
4419:   figure.contentEditable = "false";
4420:   figure.draggable = true;
4421:   figure.title = "Drag image to move it between text.";
4422:   figure.innerHTML =
4423:     `${richBlockActions("overview image", { movable: true })}
4424:     ${figure.dataset.display === "download" ? `<span class="rich-download-badge">Download only</span>` :
""}
4425:     <span class="rich-image-viewport crop-${figure.dataset.cropAspect === "original" ? "original" :
"active"}">${richImageCropStyle(blockData)}>
4426:     
4427:     </span>
4428:     ${title || blockData.caption} ? `<figcaption>${title ? `<strong>${escapeHtml(title)}</strong>` :
""}${blockData.caption ? `<span>${escapeHtml(blockData.caption)}</span>` : ""}</figcaption>` : ""}
4429:   `;
4430:   return figure;
4431: }

```

createRichFormulaBlock - lines 4433-4446

Item	Explanation
Source location	Lines 4433-4446 (14 source lines).
Purpose	Creates an object, UI structure, process, payload, or generated artifact.
Declaration	function createRichFormulaBlock(blockData) {
Touches	browser/Electron DOM, filesystem
Calls detected	createElement, unwrapFormula, richBlockActions, escapeHtml
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

4433: function createRichFormulaBlock(blockData) {
4434:   const block = document.createElement("div");
4435:   const align = blockData.align || "center";
4436:   block.className = `rich-block rich-formula-block justify-${align}`;
4437:   block.dataset.type = "formula";
4438:   block.dataset.formula = unwrapFormula(blockData.formula || "");
4439:   block.dataset.align = align;
4440:   block.contentEditable = "false";
4441:   block.innerHTML = `
4442:     ${richBlockActions("formula")}
4443:     <span class="formula-text">${escapeHtml(unwrapFormula(blockData.formula || ""))}</span>
4444:   `;
4445:   return block;
4446: }

```

createRichCodeBlock - lines 4448-4465

Item	Explanation
Source location	Lines 4448-4465 (18 source lines).
Purpose	Creates an object, UI structure, process, payload, or generated artifact.
Declaration	function createRichCodeBlock(blockData = {}) {
Touches	browser/Electron DOM, filesystem, authentication/security data
Calls detected	createElement, normalizeCodePasteMode, normalizeCodeText, normalizeCodeLanguage, detectCodeLanguage, richBlockActions, escapeHtml, codeLanguageLabel, tokenizedCodeHtml

Item	Explanation
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

4448: function createRichCodeBlock(blockData = {}) {
4449:   const block = document.createElement("figure");
4450:   const pasteMode = normalizeCodePasteMode(blockData.pasteMode || "source");
4451:   const code = normalizeCodeText(blockData.code || "", pasteMode);
4452:   const language = normalizeCodeLanguage(blockData.language || detectCodeLanguage(code));
4453:   block.className = `rich-block rich-code-block rich-code-language-${language}`;
4454:   block.dataset.type = "code";
4455:   block.dataset.language = language;
4456:   block.dataset.pasteMode = pasteMode;
4457:   block.dataset.code = code;
4458:   block.contentEditable = "false";
4459:   block.innerHTML = `
4460:     ${richBlockActions("code block")}
4461:     <figcaption><span>&lt;/&gt;</span> ${escapeHtml(codeLanguageLabel(language))}<button type="button"
data-rich-block-action="copy-code">Copy code</button></figcaption>
4462:     <pre><code>${tokenizedCodeHtml(code, language)}</code></pre>
4463:   `;
4464:   return block;
4465: }

```

refreshRichImageBlock - lines 4467-4497

Item	Explanation
Source location	Lines 4467-4497 (31 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function refreshRichImageBlock(block, blockData) {
Touches	browser/Electron DOM, filesystem
Calls detected	normalizeCropAspect, normalizeCropZoom, normalizeCropPosition, remove, add, cleanRichImageTitle, richBlockActions, richImageCropStyle, escapeHtml, normalizeLinkTarget
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

4467: function refreshRichImageBlock(block, blockData) {
4468:   const align = blockData.align || block.dataset.align || "center";
4469:   block.dataset.url = blockData.url || block.dataset.url || "";
4470:   block.dataset.title = blockData.title || "";
4471:   block.dataset.caption = blockData.caption || "";
4472:   block.dataset.align = align;
4473:   block.dataset.display = blockData.display === "download" ? "download" : "show";
4474:   block.dataset.source = blockData.source || block.dataset.source || "local";
4475:   block.dataset.cropAspect = normalizeCropAspect(blockData.cropAspect || block.dataset.cropAspect);
4476:   block.dataset.cropZoom = String(normalizeCropZoom(blockData.cropZoom || block.dataset.cropZoom));
4477:   block.dataset.cropX = String(normalizeCropPosition(blockData.cropX ?? block.dataset.cropX));
4478:   block.dataset.cropY = String(normalizeCropPosition(blockData.cropY ?? block.dataset.cropY));
4479:   block.draggable = true;
4480:   block.title = "Drag image to move it between text.";
4481:   block.classList.remove("justify-left", "justify-center", "justify-right");
4482:   block.classList.add(`justify-${align}`);
4483:   const title = cleanRichImageTitle({ title: block.dataset.title });
4484:   block.innerHTML = `
4485:     ${richBlockActions("overview image", { movable: true })}
4486:     ${block.dataset.display === "download" ? "<span class='rich-download-badge'>Download only</span>" :
""}
4487:     <span class="rich-image-viewport crop-${block.dataset.cropAspect === "original" ? "original" :
"active"}">${richImageCropStyle({
4488:       cropAspect: block.dataset.cropAspect,
4489:       cropZoom: block.dataset.cropZoom,
4490:       cropX: block.dataset.cropX,
4491:       cropY: block.dataset.cropY

```

```

4492:     >>>
4493:     
4494:     </span>
4495:     ${title || block.dataset.caption} ? `<figcaption>${title ? `<strong>${escapeHtml(title)}</strong>` :
""}${block.dataset.caption ? `<span>${escapeHtml(block.dataset.caption)}</span>` : ""}</figcaption>` : ""}
4496: `;
4497: }

```

refreshRichFormulaBlock - lines 4499-4510

Item	Explanation
Source location	Lines 4499-4510 (12 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function refreshRichFormulaBlock(block, blockData) {
Touches	browser/Electron DOM, filesystem
Calls detected	unwrapFormula, remove, add, richBlockActions, escapeHtml
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

4499: function refreshRichFormulaBlock(block, blockData) {
4500:   const align = blockData.align || block.dataset.align || "center";
4501:   const formula = unwrapFormula(blockData.formula || block.dataset.formula || "");
4502:   block.dataset.formula = formula;
4503:   block.dataset.align = align;
4504:   block.classList.remove("justify-left", "justify-center", "justify-right");
4505:   block.classList.add(`justify-${align}`);
4506:   block.innerHTML = `
4507:     ${richBlockActions("formula")}
4508:     <span class="formula-text">${escapeHtml(formula)}</span>
4509: `;
4510: }

```

refreshRichCodeBlock - lines 4512-4525

Item	Explanation
Source location	Lines 4512-4525 (14 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function refreshRichCodeBlock(block, blockData = {}) {
Touches	browser/Electron DOM, filesystem, authentication/security data
Calls detected	normalizeCodePasteMode, normalizeCodeText, normalizeCodeLanguage, detectCodeLanguage, richBlockActions, escapeHtml, codeLanguageLabel, tokenizedCodeHtml
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

4512: function refreshRichCodeBlock(block, blockData = {}) {
4513:   const pasteMode = normalizeCodePasteMode(blockData.pasteMode || block.dataset.pasteMode || "source");
4514:   const code = normalizeCodeText(blockData.code || block.dataset.code || "", pasteMode);
4515:   const language = normalizeCodeLanguage(blockData.language || block.dataset.language ||
detectCodeLanguage(code));
4516:   block.dataset.language = language;
4517:   block.dataset.pasteMode = pasteMode;
4518:   block.dataset.code = code;
4519:   block.className = `rich-block rich-code-block rich-code-language-${language}`;
4520:   block.innerHTML = `
4521:     ${richBlockActions("code block")}

```

```

4522:     <figcaption><span>&lt;/span> ${escapeHtml(codeLanguageLabel(language))}<button type="button"
data-rich-block-action="copy-code">Copy code</button></figcaption>
4523:     <pre><code>${tokenizedCodeHtml(code, language)}</code></pre>
4524:     `;
4525: }

```

createRichBlockElement - lines 4527-4544

Item	Explanation
Source location	Lines 4527-4544 (18 source lines).
Purpose	Creates an object, UI structure, process, payload, or generated artifact.
Declaration	function createRichBlockElement(block) {
Touches	filesystem
Calls detected	createRichImageBlock, createRichFormulaBlock, createRichCodeBlock, createRichTextBlock
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 4 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

4527: function createRichBlockElement(block) {
4528:   if (block.type === "image") return createRichImageBlock(block);
4529:   if (block.type === "formula") return createRichFormulaBlock(block);
4530:   if (block.type === "code") return createRichCodeBlock(block);
4531:
4532:   return createRichTextBlock(
4533:     block.text || "",
4534:     block.fontSize || "normal",
4535:     block.align || "left",
4536:     block.fontFamily || "Arial",
4537:     block.fontPx || "",
4538:     block.color || "",
4539:     Boolean(block.bold),
4540:     block.html || "",
4541:     Boolean(block.italic),
4542:     Boolean(block.underline)
4543:   );
4544: }

```

populateSummaryEditor - lines 4546-4550

Item	Explanation
Source location	Lines 4546-4550 (5 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function populateSummaryEditor(project) {
Touches	browser/Electron DOM
Calls detected	querySelector, populateRichEditor
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

4546: function populateSummaryEditor(project) {
4547:   const editor = projectFields.querySelector("[data-rich-editor='summary']");
4548:   if (!editor) return;
4549:   populateRichEditor(editor, project);
4550: }

```

populateStandaloneRichEditor - lines 4552-4560

Item	Explanation
Source location	Lines 4552-4560 (9 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function populateStandaloneRichEditor(editor, rich, fallbackText = "") {
Touches	Mostly local calculations or local UI state.
Calls detected	richBlocksFromValue, forEach, append, createRichBlockElement, createRichTextBlock
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

4552: function populateStandaloneRichEditor(editor, rich, fallbackText = "") {
4553:   if (!editor) return;
4554:   editor.contentEditable = "true";
4555:   editor.tabIndex = 0;
4556:   editor.spellcheck = true;
4557:   editor.innerHTML = "";
4558:   richBlocksFromValue(rich, fallbackText).forEach((block) =>
editor.append(createRichBlockElement(block)));
4559:   if (!editor.children.length) editor.append(createRichTextBlock(""));
4560: }

```

populateRichEditor - lines 4562-4592

Item	Explanation
Source location	Lines 4562-4592 (31 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function populateRichEditor(editor, project = selectedProject()) {
Touches	browser/Electron DOM
Calls detected	selectedProject, richBlocksFromValue, forEach, append, createRichBlockElement, createRichTextBlock, richBlocksForProject, getByPath
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 3 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

4562: function populateRichEditor(editor, project = selectedProject()) {
4563:   if (!editor) return;
4564:   editor.contentEditable = "true";
4565:   editor.tabIndex = 0;
4566:   editor.spellcheck = true;
4567:
4568:   if (editor.dataset.richEditor === "pending-description") {
4569:     editor.innerHTML = "";
4570:     richBlocksFromValue(pendingEditor?.richDescription, pendingEditor?.description || "")
4571:       .forEach((block) => editor.append(createRichBlockElement(block)));
4572:     if (!editor.children.length) editor.append(createRichTextBlock(""));
4573:     return;
4574:   }
4575:
4576:   if (!project) return;
4577:
4578:   editor.innerHTML = "";
4579:
4580:   let blocks = [];
4581:
4582:   if (editor.dataset.richEditor === "summary" && !editor.dataset.richPath) {
4583:     blocks = richBlocksForProject(project);
4584:   } else {
4585:     const textPath = editor.dataset.richTextPath || "";
4586:     const richPath = editor.dataset.richPath || "";

```

```

4587:         blocks = richBlocksFromValue(getByPath(project, richPath), getByPath(project, textPath) || "");
4588:     }
4589:
4590:     blocks.forEach((block) => editor.append(createRichBlockElement(block)));
4591:     if (!editor.children.length) editor.append(createRichTextBlock(""));
4592: }

```

populateRichEditors - lines 4594-4596

Item	Explanation
Source location	Lines 4594-4596 (3 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function populateRichEditors(root = document) {
Touches	browser/Electron DOM
Calls detected	querySelectorAll, forEach, populateRichEditor
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

4594: function populateRichEditors(root = document) {
4595:     root.querySelectorAll("[data-rich-editor]").forEach((editor) => populateRichEditor(editor));
4596: }

```

saveRichEditorToProject - lines 4598-4658

Item	Explanation
Source location	Lines 4598-4658 (61 source lines).
Purpose	Persists data to local state, disk, credentials, or browser storage.
Declaration	function saveRichEditorToProject(editor) {
Touches	browser/Electron DOM, filesystem
Calls detected	syncFunFactsFromInput, setStatus, syncSiteRichField, syncProfileRichField, selectedProject, extractRichSummary, plainTextFromRich, wordCount, setByPath, scheduleAutosave, schedulePreviewRender
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 9 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

4598: function saveRichEditorToProject(editor) {
4599:     if (!editor) return true;
4600:
4601:     if (editor.dataset.richEditor === "fun-facts") {
4602:         syncFunFactsFromInput();
4603:         setStatus("Unsaved local changes.");
4604:         return true;
4605:     }
4606:
4607:     if (editor.dataset.richEditor === "site-field") {
4608:         syncSiteRichField(editor);
4609:         setStatus("Front page text formatting updated locally. Click Save draft before applying to
site.");
4610:         return true;
4611:     }
4612:
4613:     if (editor.dataset.richEditor === "profile-field") {
4614:         syncProfileRichField(editor);
4615:         setStatus("Profile and contact formatting updated locally. Click Save draft before applying to
site.");
4616:         return true;
4617:     }

```



```

4618:
4619:     if (editor.dataset.richEditor === "standalone") {
4620:         setStatus("Unsaved dialog changes. Use the dialog save button to keep them.");
4621:         return true;
4622:     }
4623:
4624:     const project = selectedProject();
4625:
4626:     if (editor.dataset.richEditor === "pending-description" && pendingEditor) {
4627:         const rich = extractRichSummary(editor);
4628:         pendingEditor.richDescription = rich;
4629:         pendingEditor.description = plainTextFromRich(rich);
4630:         setStatus("Unsaved local changes.");
4631:         return true;
4632:     }
4633:
4634:     if (!project) return true;
4635:
4636:     const rich = extractRichSummary(editor);
4637:     const plain = plainTextFromRich(rich);
4638:
4639:     if (wordCount(plain) > 1000) {
4640:         setStatus("Overview is limited to 1000 words. Shorten the text before saving.");
4641:         return false;
4642:     }
4643:
4644:     if (editor.dataset.richEditor === "summary" && !editor.dataset.richPath) {
4645:         project.summaryRich = rich;
4646:         project.summary = plain;
4647:     } else {
4648:         const textPath = editor.dataset.richTextPath;
4649:         const richPath = editor.dataset.richPath;
4650:         if (textPath) setByPath(project, textPath, plain);
4651:         if (richPath) setByPath(project, richPath, rich);
4652:     }
4653:
4654:     setStatus("Unsaved local changes.");
4655:     scheduleAutosave();
4656:     schedulePreviewRender();
4657:     return true;
4658: }

```

saveVisibleRichEditors - lines 4660-4664

Item	Explanation
Source location	Lines 4660-4664 (5 source lines).
Purpose	Persists data to local state, disk, credentials, or browser storage.
Declaration	function saveVisibleRichEditors() {
Touches	browser/Electron DOM
Calls detected	querySelectorAll, forEach, saveRichEditorToProject
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

4660: function saveVisibleRichEditors() {
4661:     document.querySelectorAll("[data-rich-editor]").forEach((editor) => {
4662:         saveRichEditorToProject(editor);
4663:     });
4664: }

```

currentRichBlock - lines 4666-4672

Item	Explanation
Source location	Lines 4666-4672 (7 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function currentRichBlock(editor) {
Touches	browser/Electron DOM

Item	Explanation
Calls detected	getSelection, contains, querySelector, closest
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

4666: function currentRichBlock(editor) {
4667:   const selection = window.getSelection();
4668:   let node = selection?.anchorNode;
4669:   if (!node || !editor.contains(node)) return editor.querySelector(".rich-block:last-child");
4670:   if (node.nodeType === Node.TEXT_NODE) node = node.parentElement;
4671:   return node.closest(".rich-block") || editor.querySelector(".rich-block:last-child");
4672: }

```

selectionRangeInsideEditor - lines 4674-4682

Item	Explanation
Source location	Lines 4674-4682 (9 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function selectionRangeInsideEditor(editor) {
Touches	Mostly local calculations or local UI state.
Calls detected	getSelection, getRangeAt, contains, cloneRange
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

4674: function selectionRangeInsideEditor(editor) {
4675:   const selection = window.getSelection();
4676:   if (!selection?.rangeCount) return null;
4677:   const range = selection.getRangeAt(0);
4678:   const container = range.commonAncestorContainer.nodeType === Node.TEXT_NODE
4679:     ? range.commonAncestorContainer.parentElement
4680:     : range.commonAncestorContainer;
4681:   return container && editor.contains(container) ? range.cloneRange() : null;
4682: }

```

captureRichSelection - lines 4684-4688

Item	Explanation
Source location	Lines 4684-4688 (5 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function captureRichSelection(editor) {
Touches	Mostly local calculations or local UI state.
Calls detected	selectionRangeInsideEditor
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
4684: function captureRichSelection(editor) {
4685:   const range = selectionRangeInsideEditor(editor);
4686:   if (range) activeRichSelectionRange = range;
4687:   return range;
4688: }
```

rememberRichFormattingSelection - lines 4690-4700

Item	Explanation
Source location	Lines 4690-4700 (11 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function rememberRichFormattingSelection(editor = activeSummaryEditor) {
Touches	filesystem
Calls detected	selectionRangeInsideEditor, rangeBelongsToEditor, cloneRange
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 3 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
4690: function rememberRichFormattingSelection(editor = activeSummaryEditor) {
4691:   if (!editor) return null;
4692:   const range = selectionRangeInsideEditor(editor)
4693:     || (rangeBelongsToEditor(activeTextSelectionRange, editor) ? activeTextSelectionRange.cloneRange() :
null)
4694:     || (rangeBelongsToEditor(activeRichSelectionRange, editor) ? activeRichSelectionRange.cloneRange() :
null);
4695:   if (!range) return null;
4696:   activeSummaryEditor = editor;
4697:   activeRichSelectionRange = range.cloneRange();
4698:   if (!range.collapsed) activeTextSelectionRange = range.cloneRange();
4699:   return range;
4700: }
```

restoreRichSelection - lines 4702-4713

Item	Explanation
Source location	Lines 4702-4713 (12 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function restoreRichSelection(editor) {
Touches	Mostly local calculations or local UI state.
Calls detected	contains, getSelection, removeAllRanges, addRange, cloneRange, showPersistentTextSelection
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 3 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
4702: function restoreRichSelection(editor) {
4703:   if (!activeRichSelectionRange) return false;
4704:   const container = activeRichSelectionRange.commonAncestorContainer.nodeType === Node.TEXT_NODE
4705:     ? activeRichSelectionRange.commonAncestorContainer.parentElement
4706:     : activeRichSelectionRange.commonAncestorContainer;
4707:   if (!container || !editor.contains(container)) return false;
4708:   const selection = window.getSelection();
4709:   selection.removeAllRanges();
4710:   selection.addRange(activeRichSelectionRange.cloneRange());
4711:   showPersistentTextSelection(activeRichSelectionRange);
4712:   return true;
4713: }
```

showPersistentTextSelection - lines 4715-4745

Item	Explanation
Source location	Lines 4715-4745 (31 source lines).
Purpose	Controls a window, dialog, panel, menu, or visible state.
Declaration	function showPersistentTextSelection(range) {
Touches	browser/Electron DOM
Calls detected	set, Highlight, cloneRange, closest, createElement, setAttribute, append, replaceChildren, getClientRects, forEach
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 3 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
4715: function showPersistentTextSelection(range) {
4716:   if (!range || range.collapsed) return;
4717:   if (window.CSS?.highlights && typeof window.Highlight === "function") {
4718:     window.CSS.highlights.set(persistentSelectionHighlightName, new
window.Highlight(range.cloneRange()));
4719:     if (persistentSelectionOverlay) persistentSelectionOverlay.hidden = true;
4720:     return;
4721:   }
4722:
4723:   const container = range.commonAncestorContainer.nodeType === Node.TEXT_NODE
4724:     ? range.commonAncestorContainer.parentElement
4725:     : range.commonAncestorContainer;
4726:   const editor = container?.closest?("[data-rich-editor]") || activeSummaryEditor;
4727:   const host = editor?.closest("dialog") || document.body;
4728:   if (!persistentSelectionOverlay) {
4729:     persistentSelectionOverlay = document.createElement("div");
4730:     persistentSelectionOverlay.className = "persistent-selection-overlay";
4731:     persistentSelectionOverlay.setAttribute("aria-hidden", "true");
4732:   }
4733:   if (persistentSelectionOverlay.parentElement !== host) host.append(persistentSelectionOverlay);
4734:   persistentSelectionOverlay.replaceChildren();
4735:   [...range.getClientRects()].forEach((rect) => {
4736:     if (rect.width < 1 || rect.height < 1) return;
4737:     const marker = document.createElement("span");
4738:     marker.style.left = `${rect.left}px`;
4739:     marker.style.top = `${rect.top}px`;
4740:     marker.style.width = `${rect.width}px`;
4741:     marker.style.height = `${rect.height}px`;
4742:     persistentSelectionOverlay.append(marker);
4743:   });
4744:   persistentSelectionOverlay.hidden = !persistentSelectionOverlay.childElementCount;
4745: }
```

clearPersistentTextSelection - lines 4747-4756

Item	Explanation
Source location	Lines 4747-4756 (10 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function clearPersistentTextSelection(clearRanges = false) {
Touches	Mostly local calculations or local UI state.
Calls detected	replaceChildren
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
4747: function clearPersistentTextSelection(clearRanges = false) {
```

```

4748: window.CSS?.highlights?.delete?.(persistentSelectionHighlightName);
4749: if (persistentSelectionOverlay) {
4750:   persistentSelectionOverlay.replaceChildren();
4751:   persistentSelectionOverlay.hidden = true;
4752: }
4753: if (!clearRanges) return;
4754: activeRichSelectionRange = null;
4755: activeTextSelectionRange = null;
4756: }

```

refreshPersistentSelectionHighlight - lines 4758-4765

Item	Explanation
Source location	Lines 4758-4765 (8 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function refreshPersistentSelectionHighlight() {
Touches	Mostly local calculations or local UI state.
Calls detected	rangeBelongsToEditor, showPersistentTextSelection
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

4758: function refreshPersistentSelectionHighlight() {
4759:   const range = rangeBelongsToEditor(activeTextSelectionRange, activeSummaryEditor)
4760:     ? activeTextSelectionRange
4761:     : rangeBelongsToEditor(activeRichSelectionRange, activeSummaryEditor)
4762:     ? activeRichSelectionRange
4763:     : null;
4764:   if (range) showPersistentTextSelection(range);
4765: }

```

rangeBelongsToEditor - lines 4767-4773

Item	Explanation
Source location	Lines 4767-4773 (7 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function rangeBelongsToEditor(range, editor) {
Touches	Mostly local calculations or local UI state.
Calls detected	contains
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

4767: function rangeBelongsToEditor(range, editor) {
4768:   if (!range || range.collapsed || !editor) return false;
4769:   const container = range.commonAncestorContainer.nodeType === Node.TEXT_NODE
4770:     ? range.commonAncestorContainer.parentElement
4771:     : range.commonAncestorContainer;
4772:   return Boolean(container && editor.contains(container));
4773: }

```

editorFromRange - lines 4775-4781

Item	Explanation
Source location	Lines 4775-4781 (7 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function editorFromRange(range) {
Touches	Mostly local calculations or local UI state.
Calls detected	No obvious named calls detected by the generator.
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

4775: function editorFromRange(range) {
4776:   if (!range || range.collapsed) return null;
4777:   const container = range.commonAncestorContainer.nodeType === Node.TEXT_NODE
4778:     ? range.commonAncestorContainer.parentElement
4779:     : range.commonAncestorContainer;
4780:   return container?.closest?("[data-rich-editor]") || null;
4781: }

```

pointInsideRange - lines 4783-4793

Item	Explanation
Source location	Lines 4783-4793 (11 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function pointInsideRange(range, x, y) {
Touches	Mostly local calculations or local UI state.
Calls detected	getClientRects, some
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

4783: function pointInsideRange(range, x, y) {
4784:   if (!range || range.collapsed) return false;
4785:   return [...range.getClientRects()].some((rect) =>
4786:     rect.width > 0 &&
4787:     rect.height > 0 &&
4788:     x >= rect.left &&
4789:     x <= rect.right &&
4790:     y >= rect.top &&
4791:     y <= rect.bottom
4792:   );
4793: }

```

richEditorFromContextEvent - lines 4795-4805

Item	Explanation
Source location	Lines 4795-4805 (11 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function richEditorFromContextEvent(event) {
Touches	Mostly local calculations or local UI state.
Calls detected	rangeBelongsToEditor, pointInsideRange, editorFromRange

Item	Explanation
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 3 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

4795: function richEditorFromContextEvent(event) {
4796:   const directEditor = event.target.closest?("[data-rich-editor]");
4797:   if (directEditor) return directEditor;
4798:   const savedRange = rangeBelongsToEditor(activeTextSelectionRange, activeSummaryEditor)
4799:     ? activeTextSelectionRange
4800:     : activeRichSelectionRange;
4801:   if (pointInsideRange(savedRange, event.clientX, event.clientY)) {
4802:     return editorFromRange(savedRange);
4803:   }
4804:   return null;
4805: }

```

displayRichFontName - lines 4807-4812

Item	Explanation
Source location	Lines 4807-4812 (6 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function displayRichFontName(value = "") {
Touches	Mostly local calculations or local UI state.
Calls detected	split, replace, trim
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

4807: function displayRichFontName(value = "") {
4808:   return String(value || "")
4809:     .split(",")[0]
4810:     .replace(/^[\'\\"]|[\\'\\"]$/g, "")
4811:     .trim() || "Arial";
4812: }

```

rgbColorToHex - lines 4814-4823

Item	Explanation
Source location	Lines 4814-4823 (10 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function rgbColorToHex(value = "") {
Touches	Mostly local calculations or local UI state.
Calls detected	trim, toLowerCase, slice, split, map, join, match, min, toString, padStart
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 4 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

4814: function rgbColorToHex(value = "") {
4815:   const color = String(value || "").trim();
4816:   if (/^#[0-9a-f]{6}$/i.test(color)) return color.toLowerCase();
4817:   if (/^#[0-9a-f]{3}$/i.test(color)) {
4818:     return `#${color.slice(1).split("").map((part) => part + part).join("")}`.toLowerCase();
4819:   }
4820:   const match = color.match(/^rgba?\s*(\d+)\D+(\d+)\D+(\d+)\D+(\d+)/i);
4821:   if (!match) return color;
4822:   return `#${match.slice(1, 4).map((part) => Math.min(255, Number(part)).toString(16).padStart(2,
"0")).join("")}`;
4823: }

```

textNodesInRichSelection - lines 4825-4844

Item	Explanation
Source location	Lines 4825-4844 (20 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function textNodesInRichSelection(range, editor, includeWhitespace = false) {
Touches	browser/Electron DOM
Calls detected	createTreeWalker, acceptNode, trim, closest, intersectsNode, nextNode, push
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 5 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

4825: function textNodesInRichSelection(range, editor, includeWhitespace = false) {
4826:   const walker = document.createTreeWalker(editor, NodeFilter.SHOW_TEXT, {
4827:     acceptNode(node) {
4828:       if (!node.textContent || (!includeWhitespace && !node.textContent.trim())) return
NodeFilter.FILTER_REJECT;
4829:       if (node.parentElement?.closest("[contenteditable='false']")) return NodeFilter.FILTER_REJECT;
4830:       try {
4831:         return range.intersectsNode(node) ? NodeFilter.FILTER_ACCEPT : NodeFilter.FILTER_REJECT;
4832:       } catch {
4833:         return NodeFilter.FILTER_REJECT;
4834:       }
4835:     }
4836:   });
4837:   const nodes = [];
4838:   let node = walker.nextNode();
4839:   while (node) {
4840:     nodes.push(node);
4841:     node = walker.nextNode();
4842:   }
4843:   return nodes;
4844: }

```

richBlockFromRange - lines 4846-4853

Item	Explanation
Source location	Lines 4846-4853 (8 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function richBlockFromRange(range, editor) {
Touches	Mostly local calculations or local UI state.
Calls detected	contains
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:


```

4846: function richBlockFromRange(range, editor) {
4847:   if (!range || !editor) return null;
4848:   const container = range.startContainer?.nodeType === Node.TEXT_NODE
4849:     ? range.startContainer.parentElement
4850:     : range.startContainer;
4851:   const block = container?.closest?(".rich-block");
4852:   return block && editor.contains(block) ? block : null;
4853: }

```

uniformSelectionValue - lines 4855-4861

Item	Explanation
Source location	Lines 4855-4861 (7 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function uniformSelectionValue(values) {
Touches	filesystem
Calls detected	map, trim, filter, every, toLowerCase
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

4855: function uniformSelectionValue(values) {
4856:   const cleanValues = values.map((value) => String(value || "").trim()).filter(Boolean);
4857:   if (!cleanValues.length) return "";
4858:   return cleanValues.every((value) => value.toLowerCase() === cleanValues[0].toLowerCase())
4859:     ? cleanValues[0]
4860:     : "";
4861: }

```

richSelectionStyleState - lines 4863-4871

Item	Explanation
Source location	Lines 4863-4871 (9 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function richSelectionStyleState(range, editor) {
Touches	filesystem
Calls detected	textNodesInRichSelection, map, getComputedStyle, uniformSelectionValue, rgbColorToHex, displayRichFontName, round
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

4863: function richSelectionStyleState(range, editor) {
4864:   const textNodes = textNodesInRichSelection(range, editor);
4865:   const styles = textNodes.map((node) => getComputedStyle(node.parentElement || editor));
4866:   return {
4867:     color: uniformSelectionValue(styles.map((style) => rgbColorToHex(style.color))),
4868:     font: uniformSelectionValue(styles.map((style) => displayRichFontName(style.fontFamily))),
4869:     size: uniformSelectionValue(styles.map((style) => String(Math.round(parseFloat(style.fontSize) ||
16))))
4870:   };
4871: }

```

closeSelectionInspectorOptions - lines 4873-4877

Item	Explanation
Source location	Lines 4873-4877 (5 source lines).
Purpose	Controls a window, dialog, panel, menu, or visible state.
Declaration	function closeSelectionInspectorOptions(except = "") {
Touches	browser/Electron DOM
Calls detected	querySelectorAll, forEach
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

4873: function closeSelectionInspectorOptions(except = "") {
4874:   textSelectionInspector?.querySelectorAll("[data-selection-options]").forEach((options) => {
4875:     if (options.dataset.selectionOptions !== except) options.hidden = true;
4876:   });
4877: }

```

positionSelectionInspector - lines 4879-4900

Item	Explanation
Source location	Lines 4879-4900 (22 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function positionSelectionInspector(range) {
Touches	Mostly local calculations or local UI state.
Calls detected	getBoundingClientRect, closest, min, max
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

4879: function positionSelectionInspector(range) {
4880:   if (!textSelectionInspector || selectionInspectorWasMoved) return;
4881:   const rect = range.getBoundingClientRect();
4882:   const hostDialog = textSelectionInspector.closest("dialog");
4883:   const hostRect = hostDialog?.getBoundingClientRect();
4884:   const bounds = {
4885:     bottom: Math.min(window.innerHeight - 8, (hostRect?.bottom ?? window.innerHeight) - 8),
4886:     left: Math.max(8, (hostRect?.left ?? 0) + 8),
4887:     right: Math.min(window.innerWidth - 8, (hostRect?.right ?? window.innerWidth) - 8),
4888:     top: Math.max(8, (hostRect?.top ?? 0) + 8)
4889:   };
4890:   const inspectorRect = textSelectionInspector.getBoundingClientRect();
4891:   const inspectorWidth = Math.min(inspectorRect.width || 310, bounds.right - bounds.left);
4892:   const inspectorHeight = Math.min(inspectorRect.height || 260, bounds.bottom - bounds.top);
4893:   const left = Math.max(bounds.left, Math.min(rect.left, bounds.right - inspectorWidth));
4894:   const preferredTop = rect.bottom + 10;
4895:   const top = preferredTop + inspectorHeight <= bounds.bottom
4896:     ? preferredTop
4897:     : Math.max(bounds.top, rect.top - inspectorHeight - 10);
4898:   textSelectionInspector.style.left = `${left}px`;
4899:   textSelectionInspector.style.top = `${top}px`;
4900: }

```

showTextSelectionInspector - lines 4902-4924

Item	Explanation
Source location	Lines 4902-4924 (23 source lines).

Item	Explanation
Purpose	Controls a window, dialog, panel, menu, or visible state.
Declaration	function showTextSelectionInspector(editor, range) {
Touches	browser/Electron DOM
Calls detected	toString, trim, closest, append, cloneRange, showPersistentTextSelection, richSelectionStyleState, slice, positionSelectionInspector
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

4902: function showTextSelectionInspector(editor, range) {
4903:   if (!textSelectionInspector || !editor || !range || range.collapsed) return;
4904:   const selectedText = range.toString().trim();
4905:   if (!selectedText) return;
4906:   const inspectorHost = editor.closest("dialog") || document.body;
4907:   if (textSelectionInspector.parentElement !== inspectorHost)
inspectorHost.append(textSelectionInspector);
4908:   activeSummaryEditor = editor;
4909:   activeRichSelectionRange = range.cloneRange();
4910:   activeTextSelectionRange = range.cloneRange();
4911:   showPersistentTextSelection(range);
4912:   const state = richSelectionStyleState(range, editor);
4913:   selectionTextPreview.textContent = selectedText.length > 120 ? `${selectedText.slice(0, 117)}...` :
selectedText;
4914:   selectionTextPreview.title = selectedText;
4915:   selectionFontInput.value = state.font;
4916:   selectionColorInput.value = state.color;
4917:   selectionSizeInput.value = state.size;
4918:   selectionFontInput.placeholder = state.font ? "" : "Mixed";
4919:   selectionColorInput.placeholder = state.color ? "" : "Mixed";
4920:   selectionSizeInput.placeholder = state.size ? "" : "Mixed";
4921:   if (state.color && /^[0-9a-f]{6}$/i.test(state.color)) selectionColorPicker.value = state.color;
4922:   textSelectionInspector.hidden = false;
4923:   positionSelectionInspector(range);
4924: }

```

hideTextSelectionInspector - lines 4926-4931

Item	Explanation
Source location	Lines 4926-4931 (6 source lines).
Purpose	Controls a window, dialog, panel, menu, or visible state.
Declaration	function hideTextSelectionInspector() {
Touches	Mostly local calculations or local UI state.
Calls detected	closeSelectionInspectorOptions
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

4926: function hideTextSelectionInspector() {
4927:   if (!textSelectionInspector) return;
4928:   textSelectionInspector.hidden = true;
4929:   closeSelectionInspectorOptions();
4930:   selectionInspectorWasMoved = false;
4931: }

```

refreshTextSelectionInspector - lines 4933-4953

Item	Explanation
Source location	Lines 4933-4953 (21 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function refreshTextSelectionInspector() {
Touches	Mostly local calculations or local UI state.
Calls detected	matches, getSelection, showTextSelectionInspector, hideTextSelectionInspector, getRangeAt
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 4 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

4933: function refreshTextSelectionInspector() {
4934:   if (!textSelectionInspector || selectionInspectorPointerInside ||
textSelectionInspector.matches(":focus-within")) return;
4935:   const selection = window.getSelection();
4936:   if (!selection?.rangeCount || selection.isCollapsed) {
4937:     if (activeTextSelectionRange && !activeTextSelectionRange.collapsed && activeSummaryEditor) {
4938:       showTextSelectionInspector(activeSummaryEditor, activeTextSelectionRange);
4939:       return;
4940:     }
4941:     hideTextSelectionInspector();
4942:     return;
4943:   }
4944:   const range = selection.getRangeAt(0);
4945:   let node = range.commonAncestorContainer;
4946:   if (node.nodeType === Node.TEXT_NODE) node = node.parentElement;
4947:   const editor = node?.closest?("[data-rich-editor]");
4948:   if (!editor) {
4949:     hideTextSelectionInspector();
4950:     return;
4951:   }
4952:   showTextSelectionInspector(editor, range);
4953: }

```

scheduleTextSelectionInspectorRefresh - lines 4955-4958

Item	Explanation
Source location	Lines 4955-4958 (4 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function scheduleTextSelectionInspectorRefresh(delay = 0) {
Touches	Mostly local calculations or local UI state.
Calls detected	No obvious named calls detected by the generator.
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

4955: function scheduleTextSelectionInspectorRefresh(delay = 0) {
4956:   clearTimeout(selectionInspectorRefreshTimer);
4957:   selectionInspectorRefreshTimer = setTimeout(refreshTextSelectionInspector, delay);
4958: }

```

applySelectionInspectorFormat - lines 4960-4990

Item	Explanation
Source location	Lines 4960-4990 (31 source lines).

Item	Explanation
Purpose	Part of the private builder frontend and editing experience.
Declaration	function applySelectionInspectorFormat(kind, rawValue) {
Touches	filesystem
Calls detected	cloneRange, cleanFontFamily, applyRichInlineCommand, normalizeTextColor, rgbColorToHex, normalizeFontPx, closeSelectionInspectorOptions, requestAnimationFrame, showTextSelectionInspector
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 4 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

4960: function applySelectionInspectorFormat(kind, rawValue) {
4961:   if (!activeSummaryEditor || !activeTextSelectionRange) return;
4962:   activeRichSelectionRange = activeTextSelectionRange.cloneRange();
4963:   if (kind === "font") {
4964:     const font = cleanFontFamily(rawValue);
4965:     if (!font) return;
4966:     selectionFontInput.value = font;
4967:     applyRichInlineCommand(activeSummaryEditor, "fontName", font);
4968:   }
4969:   if (kind === "color") {
4970:     const color = normalizeTextColor(rawValue);
4971:     if (!color) return;
4972:     const normalized = rgbColorToHex(color);
4973:     selectionColorInput.value = normalized;
4974:     if (/^[0-9a-f]{6}$/i.test(normalized)) selectionColorPicker.value = normalized;
4975:     applyRichInlineCommand(activeSummaryEditor, "foreColor", normalized);
4976:   }
4977:   if (kind === "size") {
4978:     const size = normalizeFontPx(rawValue);
4979:     if (!size) return;
4980:     selectionSizeInput.value = size;
4981:     applyRichInlineCommand(activeSummaryEditor, "fontSize", size);
4982:   }
4983:   closeSelectionInspectorOptions();
4984:   requestAnimationFrame(() => {
4985:     if (activeRichSelectionRange && activeSummaryEditor) {
4986:       activeTextSelectionRange = activeRichSelectionRange.cloneRange();
4987:       showTextSelectionInspector(activeSummaryEditor, activeRichSelectionRange);
4988:     }
4989:   });
4990: }

```

renderSelectionInspectorOptions - lines 4992-5007

Item	Explanation
Source location	Lines 4992-5007 (16 source lines).
Purpose	Renders UI, markup, or display output.
Declaration	function renderSelectionInspectorOptions() {
Touches	browser/Electron DOM
Calls detected	querySelector, map, escapeHtml, join
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

4992: function renderSelectionInspectorOptions() {
4993:   const fontOptions = textSelectionInspector?.querySelector("[data-selection-options='font']");
4994:   const sizeOptions = textSelectionInspector?.querySelector("[data-selection-options='size']");
4995:   if (fontOptions) {
4996:     fontOptions.innerHTML = commonRichFonts.map((font) => `<button type="button" data-selection-
value="font" data-value="${escapeHtml(font)}" style="font-family:

```

```

    ${escapeHtml(font)}">${escapeHtml(font)}</button>`).join("");
4997:   }
4998:   if (sizeOptions) {
4999:     sizeOptions.innerHTML = commonRichSizes.map((size) => `<button type="button" data-selection-
value="size" data-value="${size}">${size} px</button>`).join("");
5000:   }
5001:   if (selectionColorSwatches) {
5002:     selectionColorSwatches.innerHTML = commonRichColors.map((color) => `<button type="button" data-
selection-value="color" data-value="${color}" style="--selection-swatch: ${color}" aria-label="Use
${color}"></button>`).join("");
5003:   }
5004:   if (richContextColorSwatches) {
5005:     richContextColorSwatches.innerHTML = commonRichColors.map((color) => `<button type="button" data-
rich-color-value="${color}" style="--selection-swatch: ${color}" aria-label="Use ${color}"></button>`).join("");
5006:   }
5007: }

```

applyStyleToRichSelection - lines 5009-5055

Item	Explanation
Source location	Lines 5009-5055 (47 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function applyStyleToRichSelection(editor, range, command, value = "") {
Touches	browser/Electron DOM, filesystem
Calls detected	textNodesInRichSelection, every, getComputedStyle, includes, forEach, max, min, splitText, createElement, cleanFontFamily, normalizeTextColor, normalizeFontPx
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 4 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

5009: function applyStyleToRichSelection(editor, range, command, value = "") {
5010:   const nodes = textNodesInRichSelection(range, editor, true);
5011:   if (!nodes.length) return null;
5012:   const makeBold = command === "bold"
5013:     ? !nodes.every((node) => Number.parseInt(getComputedStyle(node.parentElement || editor).fontWeight,
10) >= 600)
5014:     : false;
5015:   const makeItalic = command === "italic"
5016:     ? !nodes.every((node) => getComputedStyle(node.parentElement || editor).fontStyle === "italic")
5017:     : false;
5018:   const makeUnderline = command === "underline"
5019:     ? !nodes.every((node) => getComputedStyle(node.parentElement ||
editor).textDecorationLine.includes("underline"))
5020:     : false;
5021:   const formattedNodes = [];
5022:
5023:   nodes.forEach((node) => {
5024:     const length = node.textContent?.length || 0;
5025:     let start = node === range.startContainer ? range.startOffset : 0;
5026:     let end = node === range.endContainer ? range.endOffset : length;
5027:     start = Math.max(0, Math.min(start, length));
5028:     end = Math.max(start, Math.min(end, length));
5029:     if (start === end) return;
5030:
5031:     if (end < length) node.splitText(end);
5032:     const selectedNode = start > 0 ? node.splitText(start) : node;
5033:     const wrapper = document.createElement("span");
5034:     wrapper.className = "rich-inline-style";
5035:     wrapper.style.display = "inline";
5036:     if (command === "fontName") wrapper.style.fontFamily = cleanFontFamily(value);
5037:     if (command === "foreColor") wrapper.style.color = normalizeTextColor(value);
5038:     if (command === "fontSize") wrapper.style.fontSize = `${normalizeFontPx(value)}px`;
5039:     if (command === "bold") wrapper.style.fontWeight = makeBold ? "700" : "400";
5040:     if (command === "italic") wrapper.style.fontStyle = makeItalic ? "italic" : "normal";
5041:     if (command === "underline") wrapper.style.textDecoration = makeUnderline ? "underline" : "none";
5042:     selectedNode.replaceWith(wrapper);
5043:     wrapper.append(selectedNode);
5044:     formattedNodes.push(selectedNode);
5045:   });
5046:
5047:   if (!formattedNodes.length) return null;
5048:   normalizeInlineStyleSpans(editor);
5049:   const nextRange = document.createRange();
5050:   const firstNode = formattedNodes[0];
5051:   const lastNode = formattedNodes[formattedNodes.length - 1];

```

```

5052:   nextRange.setStart(firstNode, 0);
5053:   nextRange.setEnd(lastNode, lastNode.textContent?.length || 0);
5054:   return nextRange;
5055: }

```

applyRichInlineCommand - lines 5057-5103

Item	Explanation
Source location	Lines 5057-5103 (47 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function applyRichInlineCommand(editor, command, value = "") {
Touches	browser/Electron DOM, filesystem
Calls detected	selectionRangeInsideEditor, cloneRange, rangeBelongsToEditor, applyStyleToRichSelection, showPersistentTextSelection, richBlockFromRange, selectedRichBlock, currentRichBlock, setStatus, selectRichBlock, updateCurrentTextBlock, saveRichEditorToProject
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 3 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

5057: function applyRichInlineCommand(editor, command, value = "") {
5058:   if (!editor) return;
5059:   const liveRange = selectionRangeInsideEditor(editor);
5060:   const savedRange = liveRange && !liveRange.collapsed
5061:     ? liveRange.cloneRange()
5062:     : rangeBelongsToEditor(activeTextSelectionRange, editor)
5063:       ? activeTextSelectionRange.cloneRange()
5064:       : rangeBelongsToEditor(activeRichSelectionRange, editor)
5065:         ? activeRichSelectionRange.cloneRange()
5066:         : null;
5067:
5068:   let appliedToSelection = false;
5069:   if (savedRange) {
5070:     const nextRange = applyStyleToRichSelection(editor, savedRange, command, value);
5071:     if (nextRange) {
5072:       activeRichSelectionRange = nextRange.cloneRange();
5073:       activeTextSelectionRange = nextRange.cloneRange();
5074:       showPersistentTextSelection(nextRange);
5075:       appliedToSelection = true;
5076:     }
5077:   }
5078:
5079:   if (!appliedToSelection) {
5080:     const block = richBlockFromRange(savedRange, editor) || selectedRichBlock(editor) ||
currentRichBlock(editor);
5081:     if (!block || block.dataset.type !== "paragraph") {
5082:       setStatus("Highlight text or click inside a text line before formatting.");
5083:       return;
5084:     }
5085:     selectRichBlock(block);
5086:     if (command === "fontName") updateCurrentTextBlock(editor, { fontFamily: value });
5087:     if (command === "foreColor") updateCurrentTextBlock(editor, { color: value });
5088:     if (command === "fontSize") updateCurrentTextBlock(editor, { fontPx: value });
5089:     if (command === "bold") updateCurrentTextBlock(editor, { bold: block.dataset.bold !== "true" });
5090:     if (command === "italic") {
5091:       updateCurrentTextBlock(editor, { italic: block.dataset.italic !== "true" });
5092:     }
5093:     if (command === "underline") {
5094:       updateCurrentTextBlock(editor, { underline: block.dataset.underline !== "true" });
5095:     }
5096:     saveRichEditorToProject(editor);
5097:     setStatus("Formatting applied and saved locally for the current text line.");
5098:     return;
5099:   }
5100:
5101:   saveRichEditorToProject(editor);
5102:   setStatus("Unsaved local changes. Click the section save button to keep this formatting.");
5103: }

```

selectRichBlock - lines 5105-5113

Item	Explanation
Source location	Lines 5105-5113 (9 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function selectRichBlock(block) {
Touches	browser/Electron DOM
Calls detected	querySelectorAll, forEach, remove, add, closest, indexOf
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

5105: function selectRichBlock(block) {
5106:   document.querySelectorAll(".rich-block.selected").forEach((item) => item.classList.remove("selected"));
5107:   activeSummaryBlock = block || null;
5108:   if (block) {
5109:     block.classList.add("selected");
5110:     const editor = block.closest("[data-rich-editor]");
5111:     if (editor) editor.dataset.activeRichBlockIndex = String([...editor.children].indexOf(block));
5112:   }
5113: }

```

selectedRichBlock - lines 5115-5123

Item	Explanation
Source location	Lines 5115-5123 (9 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function selectedRichBlock(editor) {
Touches	browser/Electron DOM
Calls detected	isInteger, matches, contains, currentRichBlock
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 4 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

5115: function selectedRichBlock(editor) {
5116:   if (!editor) return null;
5117:   const index = Number(editor.dataset.activeRichBlockIndex);
5118:   if (Number.isInteger(index) && index >= 0 && editor.children[index]?.matches(".rich-block")) {
5119:     return editor.children[index];
5120:   }
5121:   if (activeSummaryBlock && editor.contains(activeSummaryBlock)) return activeSummaryBlock;
5122:   return currentRichBlock(editor);
5123: }

```

syncRichContextMenuControls - lines 5125-5144

Item	Explanation
Source location	Lines 5125-5144 (20 source lines).
Purpose	Keeps two places aligned, such as local data and target files.
Declaration	function syncRichContextMenuControls(block) {
Touches	browser/Electron DOM, filesystem
Calls detected	getComputedStyle, displayRichFontName, round, normalizeTextColor, restoreRichSelection, closest, queryCommandState, some, setAttribute

Item	Explanation
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

5125: function syncRichContextMenuControls(block) {
5126:   if (!block || block.dataset.type !== "paragraph") return;
5127:
5128:   const computed = getComputedStyle(block);
5129:   const family = displayRichFontName(computed.fontFamily) || "Arial";
5130:   const size = String(Math.round(parseFloat(computed.fontSize) || 16));
5131:   const color = normalizeTextColor(block.dataset.color || "") || "#17202a";
5132:   restoreRichSelection(block.closest("[data-rich-editor]"));
5133:   const bold = document.queryCommandState("bold") || block.dataset.bold === "true";
5134:
5135:   if (richFontSelect) {
5136:     richFontSelect.value = [...richFontSelect.options].some((option) => option.value === family ||
option.textContent === family)
5137:       ? family
5138:       : "Arial";
5139:   }
5140:
5141:   if (richFontSize) richFontSize.value = size;
5142:   if (richColorInput) richColorInput.value = color;
5143:   if (richBoldButton) richBoldButton.setAttribute("aria-pressed", bold ? "true" : "false");
5144: }

```

focusTextBlock - lines 5146-5158

Item	Explanation
Source location	Lines 5146-5158 (13 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function focusTextBlock(block) {
Touches	browser/Electron DOM
Calls detected	selectRichBlock, closest, focus, createRange, selectNodeContents, collapse, getSelection, removeAllRanges, addRange, captureRichSelection
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

5146: function focusTextBlock(block) {
5147:   if (!block) return;
5148:   selectRichBlock(block);
5149:   const editor = block.closest("[data-rich-editor]");
5150:   editor?.focus();
5151:   const range = document.createRange();
5152:   range.selectNodeContents(block);
5153:   range.collapse(false);
5154:   const selection = window.getSelection();
5155:   selection.removeAllRanges();
5156:   selection.addRange(range);
5157:   captureRichSelection(editor);
5158: }

```

setTextBlockCaretOffset - lines 5160-5174

Item	Explanation
Source location	Lines 5160-5174 (15 source lines).
Purpose	Part of the private builder frontend and editing experience.

Item	Explanation
Declaration	function setTextBlockCaretOffset(block, offset) {
Touches	browser/Electron DOM
Calls detected	selectRichBlock, closest, focus, max, min, createRange, setStart, collapse, getSelection, removeAllRanges, addRange, captureRichSelection
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

5160: function setTextBlockCaretOffset(block, offset) {
5161:   if (!block) return;
5162:   selectRichBlock(block);
5163:   const editor = block.closest("[data-rich-editor]");
5164:   editor?.focus();
5165:   const textNode = block.firstChild || block;
5166:   const safeOffset = Math.max(0, Math.min(offset, textNode.textContent?.length || 0));
5167:   const range = document.createRange();
5168:   range.setStart(textNode, safeOffset);
5169:   range.collapse(true);
5170:   const selection = window.getSelection();
5171:   selection.removeAllRanges();
5172:   selection.addRange(range);
5173:   captureRichSelection(editor);
5174: }

```

textBlockCaretOffset - lines 5176-5183

Item	Explanation
Source location	Lines 5176-5183 (8 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function textBlockCaretOffset(block) {
Touches	Mostly local calculations or local UI state.
Calls detected	getSelection, contains, getRangeAt, cloneRange, selectNodeContents, setEnd, toString
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

5176: function textBlockCaretOffset(block) {
5177:   const selection = window.getSelection();
5178:   if (!selection?.rangeCount || !block.contains(selection.anchorNode)) return block.textContent.length;
5179:   const range = selection.getRangeAt(0).cloneRange();
5180:   range.selectNodeContents(block);
5181:   range.setEnd(selection.anchorNode, selection.anchorOffset);
5182:   return range.toString().length;
5183: }

```

matchingTextBlock - lines 5185-5198

Item	Explanation
Source location	Lines 5185-5198 (14 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function matchingTextBlock(block, text = "") {
Touches	browser/Electron DOM, filesystem

Item	Explanation
Calls detected	createRichTextBlock
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

5185: function matchingTextBlock(block, text = "") {
5186:     return createRichTextBlock(
5187:         text,
5188:         block?.dataset.fontSize || "normal",
5189:         block?.dataset.align || "left",
5190:         block?.dataset.fontFamily || "Arial",
5191:         block?.dataset.fontPx || "",
5192:         block?.dataset.color || "",
5193:         block?.dataset.bold === "true",
5194:         "",
5195:         block?.dataset.italic === "true",
5196:         block?.dataset.underline === "true"
5197:     );
5198: }

```

richInsertionRange - lines 5200-5221

Item	Explanation
Source location	Lines 5200-5221 (22 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function richInsertionRange(editor) {
Touches	browser/Electron DOM
Calls detected	selectionRangeInsideEditor, collapse, restoreRichSelection, querySelectorAll, at, createRange, selectNodeContents
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 4 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

5200: function richInsertionRange(editor) {
5201:     const currentRange = selectionRangeInsideEditor(editor);
5202:     if (currentRange) {
5203:         currentRange.collapse(true);
5204:         return currentRange;
5205:     }
5206:     if (restoreRichSelection(editor)) {
5207:         const restored = selectionRangeInsideEditor(editor);
5208:         if (restored) {
5209:             restored.collapse(true);
5210:             return restored;
5211:         }
5212:     }
5213:     const lastText = [...editor.querySelectorAll(".rich-text-block")].at(-1);
5214:     if (lastText) {
5215:         const range = document.createRange();
5216:         range.selectNodeContents(lastText);
5217:         range.collapse(false);
5218:         return range;
5219:     }
5220:     return null;
5221: }

```

insertRichBlockAtCursor - lines 5223-5257

Item	Explanation
Source location	Lines 5223-5257 (35 source lines).

Item	Explanation
Purpose	Part of the private builder frontend and editing experience.
Declaration	function insertRichBlockAtCursor(editor, blockElement, insertionRange = null) {
Touches	browser/Electron DOM
Calls detected	cloneRange, richInsertionRange, selectedRichBlock, contains, collapse, createRange, setStart, setEnd, extractContents, after, matchingTextBlock, append
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

5223: function insertRichBlockAtCursor(editor, blockElement, insertionRange = null) {
5224:   const range = insertionRange?.cloneRange() || richInsertionRange(editor);
5225:   const rangeNode = range?.startContainer?.nodeType === Node.TEXT_NODE
5226:     ? range.startContainer.parentElement
5227:     : range?.startContainer;
5228:   const current = rangeNode?.closest?(".rich-block") || selectedRichBlock(editor);
5229:   let nextText = null;
5230:
5231:   if (range && current?.dataset.type === "paragraph" && current.contains(range.startContainer)) {
5232:     range.collapse(true);
5233:     const tail = document.createRange();
5234:     tail.setStart(range.startContainer, range.startOffset);
5235:     tail.setEnd(current, current.childNodes.length);
5236:     const trailingContent = tail.extractContents();
5237:     current.after(blockElement);
5238:     nextText = matchingTextBlock(current, "");
5239:     nextText.append(trailingContent);
5240:     blockElement.after(nextText);
5241:   } else if (range?.startContainer === editor) {
5242:     const reference = editor.children[range.startOffset] || null;
5243:     editor.insertBefore(blockElement, reference);
5244:     nextText = createRichTextBlock("");
5245:     blockElement.after(nextText);
5246:   } else if (current) {
5247:     current.after(blockElement);
5248:     nextText = matchingTextBlock(current, "");
5249:     blockElement.after(nextText);
5250:   } else {
5251:     editor.append(blockElement);
5252:     nextText = createRichTextBlock("");
5253:     editor.append(nextText);
5254:   }
5255:
5256:   focusTextBlock(nextText);
5257: }

```

insertRichBlockAfterCursor - lines 5259-5261

Item	Explanation
Source location	Lines 5259-5261 (3 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function insertRichBlockAfterCursor(editor, blockElement) {
Touches	Mostly local calculations or local UI state.
Calls detected	insertRichBlockAtCursor
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

5259: function insertRichBlockAfterCursor(editor, blockElement) {
5260:   insertRichBlockAtCursor(editor, blockElement);

```

5261: }

cleanupEmptyRichTextBlocks - lines 5263-5275

Item	Explanation
Source location	Lines 5263-5275 (13 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function cleanupEmptyRichTextBlocks(editor) {
Touches	browser/Electron DOM, authentication/security data
Calls detected	querySelectorAll, forEach, richElementPlainText, remove, querySelector, append, createRichTextBlock
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
5263: function cleanupEmptyRichTextBlocks(editor) {
5264:   if (!editor) return;
5265:   [...editor.querySelectorAll(":scope > .rich-text-block")].forEach((block) => {
5266:     const hasVisibleText = richElementPlainText(block);
5267:     if (hasVisibleText) return;
5268:     const previous = block.previousElementSibling;
5269:     const next = block.nextElementSibling;
5270:     const keepTypingBlockAfterImage = previous?.dataset.type === "image" && !next;
5271:     const isOnlyBlock = editor.children.length <= 1;
5272:     if (!isOnlyBlock && !keepTypingBlockAfterImage) block.remove();
5273:   });
5274:   if (!editor.querySelector(":scope > .rich-block")) editor.append(createRichTextBlock(""));
5275: }
```

normalizeRichEditorStructure - lines 5277-5307

Item	Explanation
Source location	Lines 5277-5307 (31 source lines).
Purpose	Validates or normalizes input so later code receives safe predictable values.
Declaration	function normalizeRichEditorStructure(editor) {
Touches	browser/Electron DOM, filesystem, authentication/security data
Calls detected	forEach, trim, remove, createRichTextBlock, replaceWith, matches, sanitizeRichInlineHtml, querySelectorAll, add, removeAttribute, applyRichTextBlockStyle, querySelector
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 4 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
5277: function normalizeRichEditorStructure(editor) {
5278:   if (!editor) return;
5279:   [...editor.childNodes].forEach((node) => {
5280:     if (node.nodeType === Node.TEXT_NODE) {
5281:       if (!node.textContent.trim()) {
5282:         node.remove();
5283:         return;
5284:       }
5285:       const paragraph = createRichTextBlock(node.textContent);
5286:       node.replaceWith(paragraph);
5287:       return;
5288:     }
5289:
5290:     if (node.nodeType !== Node.ELEMENT_NODE || node.matches(".rich-block")) return;
5291:     const paragraph = createRichTextBlock("");
5292:     paragraph.innerHTML = sanitizeRichInlineHtml(node.innerHTML || node.textContent || "");
5293:     node.replaceWith(paragraph);
5294:   });
5295: }
```

```

5294: });
5295:
5296: editor.querySelectorAll(".rich-text-block").forEach((block) => {
5297:   block.dataset.type = "paragraph";
5298:   block.dataset.fontSize ||= "normal";
5299:   block.dataset.align ||= "left";
5300:   block.dataset.fontFamily ||= "Arial";
5301:   block.classList.add("rich-block");
5302:   block.removeAttribute("contenteditable");
5303:   applyRichTextBlockStyle(block);
5304: });
5305:
5306: if (!editor.querySelector(":scope > .rich-block")) editor.append(createRichTextBlock(""));
5307: }

```

richElementPlainText - lines 5309-5313

Item	Explanation
Source location	Lines 5309-5313 (5 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function richElementPlainText(element) {
Touches	browser/Electron DOM
Calls detected	cloneNode, querySelectorAll, forEach, replaceWith, trim
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

5309: function richElementPlainText(element) {
5310:   const cloneElement = element.cloneNode(true);
5311:   cloneElement.querySelectorAll("br").forEach((lineBreak) => lineBreak.replaceWith("\n"));
5312:   return String(cloneElement.textContent || "").trim();
5313: }

```

updateCurrentTextBlock - lines 5315-5357

Item	Explanation
Source location	Lines 5315-5357 (43 source lines).
Purpose	Updates UI, state, cache, or stored data.
Declaration	function updateCurrentTextBlock(editor, updates) {
Touches	browser/Electron DOM, filesystem
Calls detected	selectedRichBlock, remove, add, call, cleanFontFamily, normalizeFontPx, normalizeTextColor, applyRichTextBlockStyle, saveRichEditorToProject
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

5315: function updateCurrentTextBlock(editor, updates) {
5316:   const block = selectedRichBlock(editor);
5317:   if (!block || block.dataset.type !== "paragraph") return;
5318:
5319:   if (updates.fontSize) {
5320:     block.dataset.fontSize = updates.fontSize;
5321:     block.classList.remove("rich-text-small", "rich-text-normal", "rich-text-large");
5322:     block.classList.add(`rich-text-${updates.fontSize}`);
5323:   }
5324:
5325:   if (updates.align) {
5326:     block.dataset.align = updates.align;

```

```

5327:         block.classList.remove("text-left", "text-center", "text-right");
5328:         block.classList.add(`text-${updates.align}`);
5329:     }
5330:
5331:     if (Object.prototype.hasOwnProperty.call(updates, "fontFamily")) {
5332:         block.dataset.fontFamily = cleanFontFamily(updates.fontFamily);
5333:     }
5334:
5335:     if (Object.prototype.hasOwnProperty.call(updates, "fontPx")) {
5336:         block.dataset.fontPx = normalizeFontPx(updates.fontPx);
5337:     }
5338:
5339:     if (Object.prototype.hasOwnProperty.call(updates, "color")) {
5340:         block.dataset.color = normalizeTextColor(updates.color);
5341:     }
5342:
5343:     if (Object.prototype.hasOwnProperty.call(updates, "bold")) {
5344:         block.dataset.bold = updates.bold ? "true" : "false";
5345:     }
5346:
5347:     if (Object.prototype.hasOwnProperty.call(updates, "italic")) {
5348:         block.dataset.italic = updates.italic ? "true" : "false";
5349:     }
5350:
5351:     if (Object.prototype.hasOwnProperty.call(updates, "underline")) {
5352:         block.dataset.underline = updates.underline ? "true" : "false";
5353:     }
5354:
5355:     applyRichTextBlockStyle(block);
5356:     saveRichEditorToProject(editor);
5357: }

```

extractRichSummary - lines 5359-5416

Item	Explanation
Source location	Lines 5359-5416 (58 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function extractRichSummary(editor) {
Touches	browser/Electron DOM, filesystem
Calls detected	normalizeRichEditorStructure, normalizeInlineStyleSpans, querySelectorAll, map, normalizeCropAspect, normalizeCropPosition, normalizeCropZoom, unwrapFormula, normalizeCodeText, querySelector, normalizeCodeLanguage, detectCodeLanguage
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 8 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

5359: function extractRichSummary(editor) {
5360:     normalizeRichEditorStructure(editor);
5361:     normalizeInlineStyleSpans(editor);
5362:     const blocks = [...editor.querySelectorAll(".rich-block")].map((element) => {
5363:         const align = element.dataset.align || "left";
5364:         if (element.dataset.type === "image") {
5365:             return {
5366:                 align,
5367:                 caption: element.dataset.caption || "",
5368:                 cropAspect: normalizeCropAspect(element.dataset.cropAspect),
5369:                 cropX: normalizeCropPosition(element.dataset.cropX),
5370:                 cropY: normalizeCropPosition(element.dataset.cropY),
5371:                 cropZoom: normalizeCropZoom(element.dataset.cropZoom),
5372:                 display: element.dataset.display === "download" ? "download" : "show",
5373:                 source: element.dataset.source || "local",
5374:                 title: element.dataset.title || "",
5375:                 type: "image",
5376:                 url: element.dataset.url || ""
5377:             };
5378:         }
5379:         if (element.dataset.type === "formula") {
5380:             return {
5381:                 align,
5382:                 formula: unwrapFormula(element.dataset.formula || element.textContent || ""),
5383:                 type: "formula"
5384:             };
5385:         }
5386:         if (element.dataset.type === "code") {
5387:             const code = normalizeCodeText(element.dataset.code || element.querySelector("code")?.textContent

```

```

|| "", element.dataset.pasteMode || "source");
5388:     if (!code) return null;
5389:     return {
5390:       code,
5391:       language: normalizeCodeLanguage(element.dataset.language || detectCodeLanguage(code)),
5392:       pasteMode: normalizeCodePasteMode(element.dataset.pasteMode),
5393:       type: "code"
5394:     };
5395:   }
5396:   const text = richElementPlainText(element);
5397:   if (!text) return null;
5398:   if (isFormulaOnly(text)) {
5399:     return { align: element.dataset.align || "center", formula: unwrapFormula(text), type: "formula" };
5400:   }
5401:   return {
5402:     align,
5403:     bold: element.dataset.bold === "true",
5404:     color: element.dataset.color || "",
5405:     fontFamily: element.dataset.fontFamily || "",
5406:     fontPx: element.dataset.fontPx || "",
5407:     fontSize: element.dataset.fontSize || "normal",
5408:     html: sanitizeRichInlineHtml(element.innerHTML),
5409:     italic: element.dataset.italic === "true",
5410:     text,
5411:     underline: element.dataset.underline === "true",
5412:     type: "paragraph"
5413:   };
5414: }).filter(Boolean);
5415: return { blocks };
5416: }

```

uploadSummaryImageFile - lines 5418-5458

Item	Explanation
Source location	Lines 5418-5458 (41 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	async function uploadSummaryImageFile(file, options = {}) {
Touches	filesystem, network/API requests
Calls detected	selectedProject, isAllowedUpload, setStatus, call, withExtension, readFileAsDataURL, fetch, stringify, json, normalizeCropAspect, normalizeCropPosition, normalizeCropZoom
API endpoints	/api/upload
Storage keys	No local/session storage keys detected.
Async/return clues	3 await expression(s), 4 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

5418: async function uploadSummaryImageFile(file, options = {}) {
5419:   const project = selectedProject();
5420:   const projectId = options.projectId || project?.id;
5421:   if (!projectId || !file) return null;
5422:   const validation = isAllowedUpload("media", file.name, file.type);
5423:   if (!validation.ok) {
5424:     setStatus(validation.message);
5425:     return null;
5426:   }
5427:   const displayTitle = Object.prototype.hasOwnProperty.call(options, "title") ? options.title :
file.name;
5428:   const fileName = withExtension(displayTitle || file.name, file.name);
5429:   const data = await readFileAsDataURL(file);
5430:   const response = await fetch("/api/upload", {
5431:     method: "POST",
5432:     headers: { "Content-Type": "application/json" },
5433:     body: JSON.stringify({
5434:       projectId,
5435:       section: options.folder || "summary",
5436:       fileName,
5437:       data
5438:     })
5439:   });
5440:   const result = await response.json();
5441:   if (!response.ok) {
5442:     setStatus(result.error || "Image upload failed.");
5443:     return null;
5444:   }
5445:   return {
5446:     align: options.align || "center",
5447:     caption: options.caption || "",

```



```

5448:     cropAspect: normalizeCropAspect(options.cropAspect),
5449:     cropX: normalizeCropPosition(options.cropX),
5450:     cropY: normalizeCropPosition(options.cropY),
5451:     cropZoom: normalizeCropZoom(options.cropZoom),
5452:     display: options.display === "download" ? "download" : "show",
5453:     source: "local",
5454:     title: displayTitle || "",
5455:     type: "image",
5456:     url: result.url
5457:   });
5458: }

```

updateSummaryImageDialogVisibility - lines 5460-5465

Item	Explanation
Source location	Lines 5460-5465 (6 source lines).
Purpose	Updates UI, state, cache, or stored data.
Declaration	function updateSummaryImageDialogVisibility() {
Touches	browser/Electron DOM
Calls detected	querySelector
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

5460: function updateSummaryImageDialogVisibility() {
5461:   const source = summaryImageSource?.value || "local";
5462:   const isLocal = source === "local";
5463:   document.querySelector(".summary-image-local-field").hidden = !isLocal;
5464:   document.querySelector(".summary-image-url-field").hidden = isLocal;
5465: }

```

openSummaryImageDialog - lines 5467-5546

Item	Explanation
Source location	Lines 5467-5546 (80 source lines).
Purpose	Controls a window, dialog, panel, menu, or visible state.
Declaration	async function openSummaryImageDialog(existingBlock = null, options = {}) {
Touches	browser/Electron DOM, filesystem
Calls detected	querySelector, normalizeCropAspect, normalizeCropZoom, updateSummaryImageDialogVisibility, dialogValue, normalizeLinkTarget, trim, setStatus, urlLooksLikeDirectImage, normalizeCropPosition, displayNameFromUrl, uploadSummaryImageFile
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	1 await expression(s), 7 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

5467: async function openSummaryImageDialog(existingBlock = null, options = {}) {
5468:   const heading = summaryImageDialog.querySelector("h2");
5469:   const submitButton = summaryImageDialog.querySelector("button[type='submit']");
5470:   if (heading) heading.textContent = existingBlock ? "Edit image" : "Add image to overview";
5471:   if (submitButton) submitButton.textContent = existingBlock ? "Save image" : "Add image";
5472:   summaryImageSource.value = existingBlock?.dataset.source || "local";
5473:   summaryImageFile.value = "";
5474:   summaryImageUrl.value = existingBlock?.dataset.source && existingBlock.dataset.source !== "local" ?
existingBlock.dataset.url || "" : "";
5475:   summaryImageTitle.value = existingBlock?.dataset.title || "";
5476:   summaryImageCaption.value = existingBlock?.dataset.caption || "";
5477:   summaryImageAlign.value = existingBlock?.dataset.align || "center";
5478:   summaryImageDisplay.value = existingBlock?.dataset.display || "show";

```

```

5479: summaryImageCrop.value = normalizeCropAspect(existingBlock?.dataset.cropAspect);
5480: summaryImageZoom.value = String(normalizeCropZoom(existingBlock?.dataset.cropZoom));
5481: updateSummaryImageDialogVisibility();
5482: const saved = await dialogValue(summaryImageDialog);
5483: if (!saved) return null;
5484: const file = summaryImageFile.files[0];
5485: const source = summaryImageSource.value || "local";
5486: const url = normalizeLinkTarget(summaryImageUrl.value.trim(), { assumeWeb: true });
5487:
5488: if (source !== "local") {
5489:   const existingUrl = existingBlock?.dataset.url || "";
5490:   const nextUrl = url || existingUrl;
5491:   if (!nextUrl) {
5492:     setStatus("Paste an image URL or share link first.");
5493:     return null;
5494:   }
5495:   const display = summaryImageDisplay.value === "download" ? "download" : "show";
5496:   if (display === "show" && !urlLooksLikeDirectImage(nextUrl)) {
5497:     setStatus("Use a direct image URL for visible images, or choose Downloadable file only for share
links and non-image URLs.");
5498:     return null;
5499:   }
5500:   return {
5501:     align: summaryImageAlign.value,
5502:     caption: summaryImageCaption.value.trim(),
5503:     cropAspect: summaryImageCrop.value,
5504:     cropX: normalizeCropPosition(existingBlock?.dataset.cropX),
5505:     cropY: normalizeCropPosition(existingBlock?.dataset.cropY),
5506:     cropZoom: normalizeCropZoom(summaryImageZoom.value),
5507:     display,
5508:     source,
5509:     title: summaryImageTitle.value.trim() || displayNameFromUrl(nextUrl, source === "drive" ? "Google
Drive image" : "Web image"),
5510:     type: "image",
5511:     url: nextUrl
5512:   };
5513: }
5514:
5515: if (!file && !existingBlock) {
5516:   setStatus("Choose an image first.");
5517:   return null;
5518: }
5519: if (!file && existingBlock) {
5520:   return {
5521:     align: summaryImageAlign.value,
5522:     caption: summaryImageCaption.value.trim(),
5523:     cropAspect: summaryImageCrop.value,
5524:     cropX: normalizeCropPosition(existingBlock.dataset.cropX),
5525:     cropY: normalizeCropPosition(existingBlock.dataset.cropY),
5526:     cropZoom: normalizeCropZoom(summaryImageZoom.value),
5527:     display: summaryImageDisplay.value === "download" ? "download" : "show",
5528:     source: existingBlock.dataset.source || "local",
5529:     title: summaryImageTitle.value.trim() || existingBlock.dataset.title || "",
5530:     type: "image",
5531:     url: existingBlock.dataset.url || ""
5532:   };
5533: }
5534: return uploadSummaryImageFile(file, {
5535:   align: summaryImageAlign.value,
5536:   caption: summaryImageCaption.value.trim(),
5537:   cropAspect: summaryImageCrop.value,
5538:   cropX: 50,
5539:   cropY: 50,
5540:   cropZoom: normalizeCropZoom(summaryImageZoom.value),
5541:   display: summaryImageDisplay.value === "download" ? "download" : "show",
5542:   folder: options.folder || "summary",
5543:   projectId: options.projectId || "",
5544:   title: summaryImageTitle.value.trim() || file.name
5545: });
5546: }

```

openSummaryFormulaDialog - lines 5548-5563

Item	Explanation
Source location	Lines 5548-5563 (16 source lines).
Purpose	Controls a window, dialog, panel, menu, or visible state.
Declaration	async function openSummaryFormulaDialog(existingBlock = null) {
Touches	browser/Electron DOM, filesystem
Calls detected	querySelector, dialogValue, unwrapFormula, trim, setStatus
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.

Item	Explanation
Async/return clues	1 await expression(s), 3 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

5548: async function openSummaryFormulaDialog(existingBlock = null) {
5549:   const heading = summaryFormulaDialog.querySelector("h2");
5550:   const submitButton = summaryFormulaDialog.querySelector("button[type='submit']");
5551:   if (heading) heading.textContent = existingBlock ? "Edit formula" : "Add formula";
5552:   if (submitButton) submitButton.textContent = existingBlock ? "Save formula" : "Add formula";
5553:   summaryFormulaInput.value = existingBlock?.dataset.formula || "";
5554:   summaryFormulaAlign.value = existingBlock?.dataset.align || "center";
5555:   const saved = await dialogValue(summaryFormulaDialog);
5556:   if (!saved) return null;
5557:   const formula = unwrapFormula(summaryFormulaInput.value.trim());
5558:   if (!formula) {
5559:     setStatus("Paste or type a formula first.");
5560:     return null;
5561:   }
5562:   return { align: summaryFormulaAlign.value, formula, type: "formula" };
5563: }

```

refreshSummaryCodeDialogPreview - lines 5565-5575

Item	Explanation
Source location	Lines 5565-5575 (11 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function refreshSummaryCodeDialogPreview() {
Touches	filesystem
Calls detected	detectCodeLanguage, normalizeCodeLanguage, normalizeCodeText, renderRichCodeBlock
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

5565: function refreshSummaryCodeDialogPreview() {
5566:   if (!summaryCodePreview || !summaryCodeInput) return;
5567:   const rawCode = summaryCodeInput.value || "";
5568:   const language = summaryCodeLanguage?.value === "auto"
5569:     ? detectCodeLanguage(rawCode)
5570:     : normalizeCodeLanguage(summaryCodeLanguage?.value || "javascript");
5571:   const code = normalizeCodeText(rawCode, summaryCodePasteMode?.value || "source");
5572:   summaryCodePreview.innerHTML = code
5573:     ? renderRichCodeBlock({ code, language, pasteMode: summaryCodePasteMode?.value || "source" })
5574:     : `<p class="evidence-empty">Code preview appears here.</p>`;
5575: }

```

beautifyCodeClient - lines 5577-5592

Item	Explanation
Source location	Lines 5577-5592 (16 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function beautifyCodeClient(code = "", language = "javascript") {
Touches	filesystem
Calls detected	replace, normalizeCodeLanguage, includes, split, map, trim, max, repeat, match, join, trimEnd
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 3 return statement(s).

Item	Explanation
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

5577: function beautifyCodeClient(code = "", language = "javascript") {
5578:   const normalized = String(code || "").replace(/\r\n?/g, "\n").replace(/\t/g, " ");
5579:   const lang = normalizeCodeLanguage(language);
5580:   if (["c", "cpp", "java", "javascript", "verilog", "systemverilog"].includes(lang)) {
5581:     let depth = 0;
5582:     return normalized.split("\n").map((rawLine) => {
5583:       const line = rawLine.trim();
5584:       if (!line) return "";
5585:       if (/^[{}]\]/.test(line)) depth = Math.max(0, depth - 1);
5586:       const nextLine = `${" ".repeat(depth)}${line}`;
5587:       const opens = (line.match(/[{}]/g) || []).length;
5588:       const closes = (line.match(/[{}]\]/g) || []).length;
5589:       depth = Math.max(0, depth + opens - closes);
5590:       return nextLine;
5591:     }).join("\n").trimEnd();
5592:   }

```

beautifySummaryCodeDialog - lines 5599-5618

Item	Explanation
Source location	Lines 5599-5618 (20 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	async function beautifySummaryCodeDialog() {
Touches	filesystem, network/API requests
Calls detected	detectCodeLanguage, normalizeCodeLanguage, fetch, now, stringify, json, Error, beautifyCodeClient, refreshSummaryCodeDialogPreview
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	2 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

5599: async function beautifySummaryCodeDialog() {
5600:   if (!summaryCodeInput) return;
5601:   const language = summaryCodeLanguage?.value === "auto"
5602:     ? detectCodeLanguage(summaryCodeInput.value || "")
5603:     : normalizeCodeLanguage(summaryCodeLanguage?.value || "javascript");
5604:   try {
5605:     const response = await fetch(`/api/code/beautify?t=${Date.now()}`, {
5606:       method: "POST",
5607:       cache: "no-store",
5608:       headers: { "Content-Type": "application/json" },
5609:       body: JSON.stringify({ language, code: summaryCodeInput.value || "" })
5610:     });
5611:     const result = await response.json();
5612:     if (!response.ok || !result.ok) throw new Error(result.error || "Beautifier failed.");
5613:     summaryCodeInput.value = result.code || summaryCodeInput.value;
5614:   } catch {
5615:     summaryCodeInput.value = beautifyCodeClient(summaryCodeInput.value || "", language);
5616:   }
5617:   refreshSummaryCodeDialogPreview();
5618: }

```

openSummaryCodeDialog - lines 5620-5640

Item	Explanation
Source location	Lines 5620-5640 (21 source lines).
Purpose	Controls a window, dialog, panel, menu, or visible state.
Declaration	async function openSummaryCodeDialog(existingBlock = null, options = {}) {
Touches	browser/Electron DOM, filesystem

Item	Explanation
Calls detected	querySelector, detectCodeLanguage, refreshSummaryCodeDialogPreview, dialogValue, normalizeCodePasteMode, normalizeCodeText, setStatus, normalizeCodeLanguage
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	1 await expression(s), 3 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

5620: async function openSummaryCodeDialog(existingBlock = null, options = {}) {
5621:   const heading = summaryCodeDialog.querySelector("h2");
5622:   const submitButton = summaryCodeDialog.querySelector("button[type='submit']");
5623:   if (heading) heading.textContent = existingBlock ? "Edit code block" : "Add code block";
5624:   if (submitButton) submitButton.textContent = existingBlock ? "Save code" : "Add code";
5625:   const defaultCode = options.code || "";
5626:   summaryCodeLanguage.value = existingBlock?.dataset.language || (defaultCode ?
detectCodeLanguage(defaultCode) : "auto");
5627:   summaryCodePasteMode.value = existingBlock?.dataset.pasteMode || options.pasteMode || "source";
5628:   summaryCodeInput.value = existingBlock?.dataset.code || defaultCode;
5629:   refreshSummaryCodeDialogPreview();
5630:   const saved = await dialogValue(summaryCodeDialog);
5631:   if (!saved) return null;
5632:   const pasteMode = normalizeCodePasteMode(summaryCodePasteMode.value);
5633:   const code = normalizeCodeText(summaryCodeInput.value, pasteMode);
5634:   if (!code) {
5635:     setStatus("Paste or type code before saving the block.");
5636:     return null;
5637:   }
5638:   const language = summaryCodeLanguage.value === "auto" ? detectCodeLanguage(code) :
normalizeCodeLanguage(summaryCodeLanguage.value);
5639:   return { code, language, pasteMode, type: "code" };
5640: }

```

configureSummaryContextMenu - lines 5642-5665

Item	Explanation
Source location	Lines 5642-5665 (24 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function configureSummaryContextMenu(mode = "rich", options = {}) {
Touches	browser/Electron DOM, filesystem
Calls detected	Set, querySelectorAll, forEach, has
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

5642: function configureSummaryContextMenu(mode = "rich", options = {}) {
5643:   const plainActions = new Set([
5644:     "copy-text",
5645:     "paste-text",
5646:     "cut-text",
5647:     "select-all-text",
5648:     "toggle-bold",
5649:     "toggle-italic",
5650:     "toggle-underline"
5651:   ]);
5652:   summaryContextMenu.querySelectorAll("[data-rich-action]").forEach((button) => {
5653:     const action = button.dataset.richAction;
5654:     button.hidden = mode === "plain" ? !plainActions.has(action) : false;
5655:   });
5656:   summaryContextMenu.querySelectorAll(".rich-menu-field").forEach((field) => {
5657:     field.hidden = false;
5658:   });
5659:   if (mode === "rich") {
5660:     const textOnly = Boolean(options.textOnly);
5661:     summaryContextMenu.querySelectorAll("[data-rich-action='add-image'], [data-rich-action='add-
formula'], [data-rich-action='add-code'], [data-rich-action='paste-code']").forEach((button) => {

```

```

5662:         button.hidden = textOnly;
5663:     });
5664: }
5665: }

```

applyPlainTextControlFormat - lines 5667-5701

Item	Explanation
Source location	Lines 5667-5701 (35 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function applyPlainTextControlFormat(control, updates = {}) {
Touches	browser/Electron DOM, filesystem
Calls detected	call, cleanFontFamily, normalizeFontPx, normalizeTextColor, rgbColorToHex, storePlainControlStyle, setStatus
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

5667: function applyPlainTextControlFormat(control, updates = {}) {
5668:   if (!control) return;
5669:   if (Object.prototype.hasOwnProperty.call(updates, "fontFamily")) {
5670:     const font = cleanFontFamily(updates.fontFamily);
5671:     control.dataset.builderFontFamily = font;
5672:     control.style.fontFamily = font || "";
5673:   }
5674:   if (Object.prototype.hasOwnProperty.call(updates, "fontPx")) {
5675:     const size = normalizeFontPx(updates.fontPx);
5676:     control.dataset.builderFontPx = size;
5677:     control.style.fontSize = size ? `${size}px` : "";
5678:   }
5679:   if (Object.prototype.hasOwnProperty.call(updates, "color")) {
5680:     const color = normalizeTextColor(updates.color);
5681:     control.dataset.builderColor = color;
5682:     control.style.color = color || "";
5683:     if (richColorInput && color && /^[0-9a-f]{6}$/i.test(rgbColorToHex(color))) {
5684:       richColorInput.value = rgbColorToHex(color);
5685:     }
5686:   }
5687:   if (Object.prototype.hasOwnProperty.call(updates, "bold")) {
5688:     control.dataset.builderBold = updates.bold ? "true" : "false";
5689:     control.style.fontWeight = updates.bold ? "700" : "";
5690:   }
5691:   if (Object.prototype.hasOwnProperty.call(updates, "italic")) {
5692:     control.dataset.builderItalic = updates.italic ? "true" : "false";
5693:     control.style.fontStyle = updates.italic ? "italic" : "";
5694:   }
5695:   if (Object.prototype.hasOwnProperty.call(updates, "underline")) {
5696:     control.dataset.builderUnderline = updates.underline ? "true" : "false";
5697:     control.style.textDecoration = updates.underline ? "underline" : "";
5698:   }
5699:   storePlainControlStyle(control);
5700:   setStatus("Formatting applied and saved locally for this field.");
5701: }

```

syncPlainContextMenuControls - lines 5703-5719

Item	Explanation
Source location	Lines 5703-5719 (17 source lines).
Purpose	Keeps two places aligned, such as local data and target files.
Declaration	function syncPlainContextMenuControls(control) {
Touches	browser/Electron DOM
Calls detected	getComputedStyle, displayRichFontName, round, rgbColorToHex, some, setAttribute
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.

Item	Explanation
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

5703: function syncPlainContextMenuControls(control) {
5704:   if (!control) return;
5705:   const computed = getComputedStyle(control);
5706:   const family = displayRichFontName(control.dataset.builderFontFamily || computed.fontFamily) ||
"Arial";
5707:   const size = String(Math.round(parseFloat(control.dataset.builderFontPx || computed.fontSize) || 16));
5708:   const color = rgbColorToHex(control.dataset.builderColor || computed.color || "#17202a");
5709:   if (richFontSelect) {
5710:     richFontSelect.value = [...richFontSelect.options].some((option) => option.value === family ||
option.textContent === family)
5711:       ? family
5712:       : "Arial";
5713:   }
5714:   if (richFontSize) richFontSize.value = [...richFontSize.options].some((option) => option.value === size
|| option.textContent === size)
5715:     ? size
5716:     : "16";
5717:   if (richColorInput && /^[0-9a-f]{6}$/i.test(color)) richColorInput.value = color;
5718:   if (richBoldButton) richBoldButton.setAttribute("aria-pressed", control.dataset.builderBold === "true"
? "true" : "false");
5719: }

```

plainTextControlFromContextEvent - lines 5721-5728

Item	Explanation
Source location	Lines 5721-5728 (8 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function plainTextControlFromContextEvent(event) {
Touches	Mostly local calculations or local UI state.
Calls detected	closest, toLowerCase, includes
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 4 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

5721: function plainTextControlFromContextEvent(event) {
5722:   const control = event.target.closest?(".textarea, input");
5723:   if (!control || control.closest("#summary-context-menu")) return null;
5724:   const type = String(control.type || "text").toLowerCase();
5725:   if (control.disabled || control.readOnly) return null;
5726:   if (["button", "checkbox", "color", "file", "hidden", "image", "radio", "range", "reset",
"submit"].includes(type)) return null;
5727:   return control;
5728: }

```

handlePlainTextAction - lines 5730-5778

Item	Explanation
Source location	Lines 5730-5778 (49 source lines).
Purpose	Handles an event, request, command, or user action.
Declaration	async function handlePlainTextAction(action) {
Touches	browser/Electron DOM, filesystem
Calls detected	focus, max, min, slice, writeText, setRangeText, dispatchEvent, Event, readText, select, applyPlainTextControlFormat
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.

Item	Explanation
Async/return clues	2 await expression(s), 7 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

5730: async function handlePlainTextAction(action) {
5731:   const control = activePlainTextControl;
5732:   if (!control) return;
5733:   control.focus({ preventScroll: true });
5734:   const selection = activePlainTextSelection || {
5735:     start: control.selectionStart ?? 0,
5736:     end: control.selectionEnd ?? control.value.length
5737:   };
5738:   const start = Math.max(0, Math.min(selection.start, control.value.length));
5739:   const end = Math.max(start, Math.min(selection.end, control.value.length));
5740:
5741:   if (action === "copy-text" || action === "cut-text") {
5742:     const text = control.value.slice(start, end) || control.value;
5743:     if (text) await navigator.clipboard.writeText(text);
5744:     if (action === "cut-text" && end > start) {
5745:       control.setRangeText("", start, end, "start");
5746:       control.dispatchEvent(new Event("input", { bubbles: true }));
5747:     }
5748:     return;
5749:   }
5750:
5751:   if (action === "paste-text") {
5752:     const text = await navigator.clipboard.readText();
5753:     if (text) {
5754:       control.setRangeText(text, start, end, "end");
5755:       control.dispatchEvent(new Event("input", { bubbles: true }));
5756:     }
5757:     return;
5758:   }
5759:
5760:   if (action === "select-all-text") {
5761:     control.select();
5762:     activePlainTextSelection = { start: 0, end: control.value.length };
5763:     return;
5764:   }
5765:
5766:   if (action === "toggle-bold") {
5767:     applyPlainTextControlFormat(control, { bold: control.dataset.builderBold !== "true" });
5768:     return;
5769:   }
5770:   if (action === "toggle-italic") {
5771:     applyPlainTextControlFormat(control, { italic: control.dataset.builderItalic !== "true" });
5772:     return;
5773:   }
5774:   if (action === "toggle-underline") {
5775:     applyPlainTextControlFormat(control, { underline: control.dataset.builderUnderline !== "true" });
5776:     return;
5777:   }
5778: }

```

handleRichAction - lines 5780-5878

Item	Explanation
Source location	Lines 5780-5878 (99 source lines).
Purpose	Handles an event, request, command, or user action.
Declaration	async function handleRichAction(action, editor) {
Touches	browser/Electron DOM, filesystem
Calls detected	applyRichInlineCommand, updateCurrentTextBlock, restoreRichSelection, getSelection, toString, selectedRichBlock, writeText, readText, insertPlainTextIntoRichEditor, saveRichEditorToProject, getRangeAt, deleteContents
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	9 await expression(s), 5 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

5780: async function handleRichAction(action, editor) {

```



```

5781:   if (!editor) return;
5782:   activeSummaryEditor = editor;
5783:   if (action === "toggle-bold") {
5784:     applyRichInlineCommand(editor, "bold");
5785:   }
5786:   if (action === "toggle-italic") {
5787:     applyRichInlineCommand(editor, "italic");
5788:   }
5789:   if (action === "toggle-underline") {
5790:     applyRichInlineCommand(editor, "underline");
5791:   }
5792:   if (action === "align-left") updateCurrentTextBlock(editor, { align: "left" });
5793:   if (action === "align-center") updateCurrentTextBlock(editor, { align: "center" });
5794:   if (action === "align-right") updateCurrentTextBlock(editor, { align: "right" });
5795:   if (action === "copy-text") {
5796:     restoreRichSelection(editor);
5797:     const selectedText = window.getSelection()?.toString() || "";
5798:     const block = selectedRichBlock(editor);
5799:     const text = selectedText || block?.textContent || "";
5800:     if (text) await navigator.clipboard.writeText(text);
5801:     return;
5802:   }
5803:
5804:   if (action === "paste-text") {
5805:     const text = await navigator.clipboard.readText();
5806:     if (text) {
5807:       insertPlainTextIntoRichEditor(editor, text);
5808:       saveRichEditorToProject(editor);
5809:     }
5810:     return;
5811:   }
5812:   if (action === "cut-text") {
5813:     restoreRichSelection(editor);
5814:     const selection = window.getSelection();
5815:     const selectedText = selection?.toString() || "";
5816:     if (selectedText && selection.rangeCount) {
5817:       await navigator.clipboard.writeText(selectedText);
5818:       const range = selection.getRangeAt(0);
5819:       range.deleteContents();
5820:       captureRichSelection(editor);
5821:       cleanupEmptyRichTextBlocks(editor);
5822:       saveRichEditorToProject(editor);
5823:     }
5824:     return;
5825:   }
5826:   if (action === "select-all-text") {
5827:     const block = selectedRichBlock(editor);
5828:     const target = block?.dataset.type === "paragraph" ? block : editor;
5829:     const range = document.createRange();
5830:     range.selectNodeContents(target);
5831:     const selection = window.getSelection();
5832:     selection.removeAllRanges();
5833:     selection.addRange(range);
5834:     activeRichSelectionRange = range.cloneRange();
5835:     activeTextSelectionRange = range.cloneRange();
5836:     showPersistentTextSelection(range);
5837:     showTextSelectionInspector(editor, range);
5838:     return;
5839:   }
5840:   if (action === "add-image") {
5841:     const imageBlock = await openSummaryImageDialog(null, {
5842:       folder: editor.dataset.richFolder || "summary",
5843:       projectId: editor.dataset.richProjectId || ""
5844:     });
5845:     if (imageBlock) {
5846:       insertRichBlockAfterCursor(editor, createRichImageBlock(imageBlock));
5847:       saveRichEditorToProject(editor);
5848:     }
5849:   }
5850:   if (action === "add-formula") {
5851:     const formulaBlock = await openSummaryFormulaDialog();
5852:     if (formulaBlock) {
5853:       insertRichBlockAfterCursor(editor, createRichFormulaBlock(formulaBlock));
5854:       saveRichEditorToProject(editor);
5855:     }
5856:   }
5857:   if (action === "add-code") {
5858:     const selectedText = window.getSelection()?.toString() || "";
5859:     const codeBlock = await openSummaryCodeDialog(null, { code: selectedText, pasteMode: "source" });
5860:     if (codeBlock) {
5861:       insertRichBlockAfterCursor(editor, createRichCodeBlock(codeBlock));
5862:       saveRichEditorToProject(editor);
5863:     }
5864:   }
5865:   if (action === "paste-code") {
5866:     const text = await navigator.clipboard.readText();
5867:     const codeBlock = await openSummaryCodeDialog(null, { code: text, pasteMode: "source" });
5868:     if (codeBlock) {
5869:       insertRichBlockAfterCursor(editor, createRichCodeBlock(codeBlock));
5870:       saveRichEditorToProject(editor);
5871:     }
5872:   }

```

```

5873:   if (action === "edit-block") {
5874:     await editSelectedRichBlock(editor);
5875:     saveRichEditorToProject(editor);
5876:   }
5877:   if (action === "delete-block") deleteSelectedRichBlock(editor);
5878: }

```

hideSummaryContextMenu - lines 5880-5884

Item	Explanation
Source location	Lines 5880-5884 (5 source lines).
Purpose	Controls a window, dialog, panel, menu, or visible state.
Declaration	function hideSummaryContextMenu() {
Touches	Mostly local calculations or local UI state.
Calls detected	No obvious named calls detected by the generator.
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

5880: function hideSummaryContextMenu() {
5881:   summaryContextMenu.hidden = true;
5882:   activePlainTextControl = null;
5883:   activePlainTextSelection = null;
5884: }

```

editSelectedRichBlock - lines 5886-5909

Item	Explanation
Source location	Lines 5886-5909 (24 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	async function editSelectedRichBlock(editor) {
Touches	browser/Electron DOM, filesystem
Calls detected	selectedRichBlock, selectRichBlock, openSummaryImageDialog, refreshRichImageBlock, openSummaryFormulaDialog, refreshRichFormulaBlock, openSummaryCodeDialog, refreshRichCodeBlock, focusTextBlock, setStatus
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	3 await expression(s), 4 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

5886: async function editSelectedRichBlock(editor) {
5887:   const block = selectedRichBlock(editor);
5888:   if (!block) return;
5889:   selectRichBlock(block);
5890:   if (block.dataset.type === "image") {
5891:     const updated = await openSummaryImageDialog(block);
5892:     if (updated) refreshRichImageBlock(block, updated);
5893:     return;
5894:   }
5895:   if (block.dataset.type === "formula") {
5896:     const updated = await openSummaryFormulaDialog(block);
5897:     if (updated) refreshRichFormulaBlock(block, updated);
5898:     return;
5899:   }
5900:   if (block.dataset.type === "code") {
5901:     const updated = await openSummaryCodeDialog(block);
5902:     if (updated) refreshRichCodeBlock(block, updated);
5903:     return;
5904:   }

```

```

5905:   if (block.dataset.type === "paragraph") {
5906:     focusTextBlock(block);
5907:     setStatus("Text block selected. Type directly in the block to edit it.");
5908:   }
5909: }

```

copyRichCodeBlock - lines 5911-5918

Item	Explanation
Source location	Lines 5911-5918 (8 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	async function copyRichCodeBlock(block) {
Touches	browser/Electron DOM
Calls detected	querySelector, writeText, setStatus
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	1 await expression(s), 3 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

5911: async function copyRichCodeBlock(block) {
5912:   if (!block || block.dataset.type !== "code") return false;
5913:   const code = block.dataset.code || block.querySelector("code")?.textContent || "";
5914:   if (!code) return false;
5915:   await navigator.clipboard.writeText(code);
5916:   setStatus("Code copied to clipboard.");
5917:   return true;
5918: }

```

removeRichBlock - lines 5920-5931

Item	Explanation
Source location	Lines 5920-5931 (12 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function removeRichBlock(block, editor) {
Touches	browser/Electron DOM
Calls detected	matches, remove, querySelector, append, createRichTextBlock, focusTextBlock
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

5920: function removeRichBlock(block, editor) {
5921:   if (!block) return;
5922:   const nextFocus = block.previousElementSibling?.matches(".rich-text-block")
5923:     ? block.previousElementSibling
5924:     : block.nextElementSibling?.matches(".rich-text-block")
5925:     ? block.nextElementSibling
5926:     : null;
5927:   block.remove();
5928:   activeSummaryBlock = null;
5929:   if (!editor.querySelector(".rich-block")) editor.append(createRichTextBlock(""));
5930:   focusTextBlock(nextFocus || editor.querySelector(".rich-text-block"));
5931: }

```

deleteSelectedRichBlock - lines 5933-5948

Item	Explanation
Source location	Lines 5933-5948 (16 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function deleteSelectedRichBlock(editor) {
Touches	browser/Electron DOM, filesystem
Calls detected	selectedRichBlock, openDeleteConfirm, removeRichBlock
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

5933: function deleteSelectedRichBlock(editor) {
5934:   const block = selectedRichBlock(editor);
5935:   if (!block) return;
5936:   const label = block.dataset.type === "image"
5937:     ? "this overview image"
5938:     : block.dataset.type === "formula"
5939:     ? "this formula"
5940:     : block.dataset.type === "code"
5941:     ? "this code block"
5942:     : "this text block";
5943:   openDeleteConfirm({
5944:     title: "Delete overview block",
5945:     message: `Are you sure you want to delete ${label}?`,
5946:     onConfirm: () => removeRichBlock(block, editor)
5947:   });
5948: }

```

deleteProjectSummary - lines 5950-5966

Item	Explanation
Source location	Lines 5950-5966 (17 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function deleteProjectSummary(project) {
Touches	filesystem
Calls detected	openDeleteConfirm, setStatus, scheduleAutosave, renderAll
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

5950: function deleteProjectSummary(project) {
5951:   if (!project) return;
5952:   openDeleteConfirm({
5953:     title: "Delete project overview",
5954:     message: "Are you sure you want to delete the current project overview?",
5955:     onConfirm: () => {
5956:       project.summary = "";
5957:       project.summaryRich = null;
5958:       summaryEditorProjectId = "";
5959:       activeSummaryEditor = null;
5960:       activeSummaryBlock = null;
5961:       setStatus("Project overview deleted in the editor. Click Save project to update the portfolio
5962: preview.");
5962:       scheduleAutosave();
5963:       renderAll();
5964:     }
5965:   });
5966: }

```

saveRichSummary - lines 5968-5987

Item	Explanation
Source location	Lines 5968-5987 (20 source lines).
Purpose	Persists data to local state, disk, credentials, or browser storage.
Declaration	async function saveRichSummary() {
Touches	browser/Electron DOM
Calls detected	selectedProject, querySelector, extractRichSummary, plainTextFromRich, wordCount, setStatus, scheduleAutosave, renderAll
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 3 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
5968: async function saveRichSummary() {
5969:   const project = selectedProject();
5970:   const editor = projectFields.querySelector("[data-rich-editor='summary']");
5971:   if (!project || !editor) return true;
5972:   const rich = extractRichSummary(editor);
5973:   const plain = plainTextFromRich(rich);
5974:   if (wordCount(plain) > 1000) {
5975:     setStatus("Project overview is limited to 1000 words. Shorten the text before saving.");
5976:     return false;
5977:   }
5978:   project.summaryRich = rich;
5979:   project.summary = plain;
5980:   summaryEditorProjectId = "";
5981:   activeSummaryEditor = null;
5982:   activeSummaryBlock = null;
5983:   setStatus("Overview saved in the editor. Click Save project to include this version in the portfolio
preview.");
5984:   scheduleAutosave();
5985:   renderAll();
5986:   return true;
5987: }
```

renderFields - lines 5989-6004

Item	Explanation
Source location	Lines 5989-6004 (16 source lines).
Purpose	Renders UI, markup, or display output.
Declaration	function renderFields(project) {
Touches	Mostly local calculations or local UI state.
Calls detected	renderSummaryBuilder, requestAnimationFrame, populateSummaryEditor
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
5989: function renderFields(project) {
5990:   if (!project) {
5991:     projectFields.hidden = false;
5992:     projectFields.innerHTML = `<p class="evidence-empty">Add or select a project to begin editing.</p>`;
5993:     return;
5994:   }
5995:
5996:   projectFields.hidden = activeSectionId !== "brief";
5997:   projectWindowTitle.textContent = project.title;
5998:   projectFields.innerHTML = `
5999:     ${renderSummaryBuilder(project)}`
```

```

6000: `;
6001:   if (summaryEditorProjectId === project.id) {
6002:     requestAnimationFrame(() => populateSummaryEditor(project));
6003:   }
6004: }

```

sectionOptions - lines 6006-6016

Item	Explanation
Source location	Lines 6006-6016 (11 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function sectionOptions(project) {
Touches	Mostly local calculations or local UI state.
Calls detected	map, filter, isAnalogProject
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

6006: function sectionOptions(project) {
6007:   const custom = (project.sections || []).map((section) => ({
6008:     id: `custom:${section.id}`,
6009:     label: section.title,
6010:     folder: section.id
6011:   }));
6012:   return [
6013:     ...standardSections.filter((section) => !section.analogOnly || isAnalogProject(project)),
6014:     ...custom
6015:   ];
6016: }

```

custom - lines 6007-6011

Item	Explanation
Source location	Lines 6007-6011 (5 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	const custom = (project.sections []).map((section) => ({
Touches	Mostly local calculations or local UI state.
Calls detected	map
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

6007:   const custom = (project.sections || []).map((section) => ({
6008:     id: `custom:${section.id}`,
6009:     label: section.title,
6010:     folder: section.id
6011:   }));

```

ensureSiteSections - lines 6018-6021

Item	Explanation
Source location	Lines 6018-6021 (4 source lines).
Purpose	Part of the private builder frontend and editing experience.

Item	Explanation
Declaration	function ensureSiteSections() {
Touches	Mostly local calculations or local UI state.
Calls detected	clone
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

6018: function ensureSiteSections() {
6019:   catalog.siteSections = catalog.siteSections || [];
6020:   savedPortfolioCatalog.siteSections = savedPortfolioCatalog.siteSections || clone(catalog.siteSections
|| []);
6021: }

```

siteSectionHasContent - lines 6023-6032

Item	Explanation
Source location	Lines 6023-6032 (10 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function siteSectionHasContent(section) {
Touches	Mostly local calculations or local UI state.
Calls detected	richHasContent, some
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

6023: function siteSectionHasContent(section) {
6024:   return Boolean(
6025:     section?.description ||
6026:     richHasContent(section?.richDescription) ||
6027:     section?.backgroundImage ||
6028:     (section?.links || []).some((link) => link.label || link.url) ||
6029:     (section?.assets || []).some((asset) => asset.title || asset.url) ||
6030:     (section?.subsections || []).some((item) => item.title || item.description ||
richHasContent(item.richDescription) || (item.links || []).length)
6031:   );
6032: }

```

siteSectionRenderable - lines 6034-6036

Item	Explanation
Source location	Lines 6034-6036 (3 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function siteSectionRenderable(section) {
Touches	Mostly local calculations or local UI state.
Calls detected	siteSectionHasContent
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).

Item	Explanation
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
6034: function siteSectionRenderable(section) {
6035:   return section?.visible !== false && siteSectionHasContent(section);
6036: }
```

siteSectionLinkCount - lines 6038-6044

Item	Explanation
Source location	Lines 6038-6044 (7 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function siteSectionLinkCount(section) {
Touches	Mostly local calculations or local UI state.
Calls detected	flatMap, filter
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
6038: function siteSectionLinkCount(section) {
6039:   return [
6040:     ...(section.links || []),
6041:     ...(section.assets || []),
6042:     ...(section.subsections || []).flatMap((item) => item.links || [])
6043:   ].filter((item) => item.label || item.url).length;
6044: }
```

renderSiteSectionList - lines 6046-6094

Item	Explanation
Source location	Lines 6046-6094 (49 source lines).
Purpose	Renders UI, markup, or display output.
Declaration	function renderSiteSectionList() {
Touches	Mostly local calculations or local UI state.
Calls detected	ensureSiteSections, map, url, siteSectionLinkCount, richHasContent, renderRichContent, join
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
6046: function renderSiteSectionList() {
6047:   ensureSiteSections();
6048:   const sections = catalog.siteSections || [];
6049:   siteSectionList.innerHTML = sections.length ? sections.map((section) => `
6050:     <article class="site-section-item ${section.visible === false ? "is-hidden-section" : ""}"
style="${section.backgroundImage ? `--site-section-bg: url('${section.backgroundImage}');` : ""}">
6051:       <div>
6052:         <strong>${section.title || "Untitled section"}</strong>
6053:         <span>${section.visible === false ? "Hidden from website" : "Visible on website"} .
${(section.subsections || []).length} subsection${(section.subsections || []).length === 1 ? "" : "s"} .
${siteSectionLinkCount(section)} link${siteSectionLinkCount(section) === 1 ? "" : "s"}</span>
6054:         ${section.description || richHasContent(section.richDescription) ?
renderRichContent(section.richDescription, section.description || "") : ""}

```



```

6055:         </div>
6056:         ${section.subsections || []}.length ? `
6057:         <div class="site-subsection-list">
6058:             ${section.subsections || []}.map((item, index) => `
6059:             <article class="site-subsection-item">
6060:                 <div>
6061:                     <strong>${item.title || "Untitled subsection"}</strong>
6062:                     ${item.description || richHasContent(item.richDescription) ?
renderRichContent(item.richDescription, item.description || "") : ""}
6063:                 </div>
6064:                 <div class="site-section-actions">
6065:                     <button type="button" data-site-subsection-edit="${section.id}" data-
index="${index}">Edit</button>
6066:                     <button class="danger-icon" type="button" data-site-subsection-delete="${section.id}"
data-index="${index}">Delete</button>
6067:                 </div>
6068:             </article>
6069:             `).join("")}
6070:         </div>
6071:         ` : ""}
6072:         ${section.links || []}.length ? `
6073:         <div class="site-link-list">
6074:             ${section.links || []}.map((link, index) => `
6075:             <span>
6076:                 ${link.label || "Untitled link"}
6077:                 <button type="button" data-site-link-edit="${section.id}" data-
index="${index}">Edit</button>
6078:                 <button type="button" data-site-link-delete="${section.id}" data-
index="${index}">Delete</button>
6079:             </span>
6080:             `).join("")}
6081:         </div>
6082:         ` : ""}
6083:         <div class="site-section-actions">
6084:             <button type="button" data-site-section-toggle="${section.id}">${section.visible === false ?
"Show" : "Hide"}</button>
6085:             <button type="button" data-site-section-edit="${section.id}">Edit</button>
6086:             <button type="button" data-site-subsection-add="${section.id}">Add subsection</button>
6087:             <button type="button" data-site-link-add="${section.id}">Add link</button>
6088:             <button type="button" data-site-section-bg="${section.id}">Background</button>
6089:             ${section.backgroundImage ? `<button type="button" data-site-section-bg-
clear="${section.id}">Clear bg</button>` : ""}
6090:             <button class="danger-icon" type="button" data-site-section-
delete="${section.id}">Delete</button>
6091:         </div>
6092:         </article>
6093:         `).join("") : `<p class="evidence-empty">No extra portfolio areas added yet.</p>`;
6094:     }

```

siteSectionDialogValue - lines 6096-6118

Item	Explanation
Source location	Lines 6096-6118 (23 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	async function siteSectionDialogValue(section = {}) {
Touches	browser/Electron DOM
Calls detected	populateStandaloneRichEditor, dialogValue, trim, setStatus, extractRichSummary, plainTextFromRich, slugify
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	1 await expression(s), 3 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

6096: async function siteSectionDialogValue(section = {}) {
6097:     sectionDialogTitle.textContent = section.id ? "Edit section" : "Create section";
6098:     sectionTitle.placeholder = "Professional profile, Personal background, Sports...";
6099:     sectionDescription.dataset.placeholder = "Short section overview shown on the website";
6100:     sectionDescription.dataset.richFolder = section.id || "new-portfolio-section";
6101:     sectionDescription.dataset.richProjectId = "_site-sections";
6102:     sectionTitle.value = section.title || "";
6103:     populateStandaloneRichEditor(sectionDescription, section.richDescription, section.description || "");
6104:     const saved = await dialogValue(sectionDialog);
6105:     if (!saved) return null;
6106:     const title = sectionTitle.value.trim();
6107:     if (!title) {
6108:         setStatus("Type a section title before saving.");
6109:         return null;

```

```

6110:   }
6111:   const richDescription = extractRichSummary(sectionDescription);
6112:   return {
6113:     description: plainTextFromRich(richDescription, "\n").trim(),
6114:     id: section.id || slugify(title),
6115:     richDescription,
6116:     title
6117:   };
6118: }

```

siteSubsectionDialogValue - lines 6120-6143

Item	Explanation
Source location	Lines 6120-6143 (24 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	async function siteSubsectionDialogValue(subsection = {}) {
Touches	browser/Electron DOM
Calls detected	populateStandaloneRichEditor, dialogValue, trim, setStatus, extractRichSummary, plainTextFromRich, slugify
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	1 await expression(s), 3 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

6120: async function siteSubsectionDialogValue(subsection = {}) {
6121:   sectionDialogTitle.textContent = subsection.id ? "Edit subsection" : "Create subsection";
6122:   sectionTitle.placeholder = "Sports, Hobbies, Resume links, Origin...";
6123:   sectionDescription.dataset.placeholder = "Short subsection text shown inside the portfolio area";
6124:   sectionDescription.dataset.richFolder = subsection.id || "new-portfolio-subsection";
6125:   sectionDescription.dataset.richProjectId = "_site-sections";
6126:   sectionTitle.value = subsection.title || "";
6127:   populateStandaloneRichEditor(sectionDescription, subsection.richDescription, subsection.description ||
""");
6128:   const saved = await dialogValue(sectionDialog);
6129:   if (!saved) return null;
6130:   const title = sectionTitle.value.trim();
6131:   if (!title) {
6132:     setStatus("Type a subsection title before saving.");
6133:     return null;
6134:   }
6135:   const richDescription = extractRichSummary(sectionDescription);
6136:   return {
6137:     description: plainTextFromRich(richDescription, "\n").trim(),
6138:     id: subsection.id || slugify(title),
6139:     links: subsection.links || [],
6140:     richDescription,
6141:     title
6142:   };
6143: }

```

dialogLinkValue - lines 6145-6169

Item	Explanation
Source location	Lines 6145-6169 (25 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function dialogLinkValue(link = {}) {
Touches	filesystem
Calls detected	removeEventListener, resolve, trim, setStatus, normalizeLinkTarget, addEventListener, showModal
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 3 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
6145: function dialogLinkValue(link = {}) {
6146:   return new Promise((resolve) => {
6147:     siteLinkEyebrow.textContent = link.url ? "Edit portfolio link" : "Add portfolio link";
6148:     siteLinkTitle.textContent = link.url ? "Edit link" : "Add link";
6149:     siteLinkLabel.value = link.label || "";
6150:     siteLinkUrl.value = link.url || "";
6151:     const onClose = () => {
6152:       siteLinkDialog.removeEventListener("close", onClose);
6153:       if (siteLinkDialog.returnValue !== "save") {
6154:         resolve(null);
6155:         return;
6156:       }
6157:       const label = siteLinkLabel.value.trim();
6158:       const url = siteLinkUrl.value.trim();
6159:       if (!label || !url) {
6160:         setStatus("Type both a link label and URL before saving.");
6161:         resolve(null);
6162:         return;
6163:       }
6164:       resolve({ ...link, label, url: normalizeLinkTarget(url, { assumeWeb: true }) });
6165:     };
6166:     siteLinkDialog.addEventListener("close", onClose);
6167:     siteLinkDialog.showModal();
6168:   });
6169: }
```

onClose - lines 6151-6165

Item	Explanation
Source location	Lines 6151-6165 (15 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	const onClose = () => {
Touches	filesystem
Calls detected	removeEventListener, resolve, trim, setStatus, normalizeLinkTarget
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
6151:   const onClose = () => {
6152:     siteLinkDialog.removeEventListener("close", onClose);
6153:     if (siteLinkDialog.returnValue !== "save") {
6154:       resolve(null);
6155:       return;
6156:     }
6157:     const label = siteLinkLabel.value.trim();
6158:     const url = siteLinkUrl.value.trim();
6159:     if (!label || !url) {
6160:       setStatus("Type both a link label and URL before saving.");
6161:       resolve(null);
6162:       return;
6163:     }
6164:     resolve({ ...link, label, url: normalizeLinkTarget(url, { assumeWeb: true }) });
6165:   };
```

addSiteSection - lines 6171-6191

Item	Explanation
Source location	Lines 6171-6191 (21 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	async function addSiteSection() {
Touches	Mostly local calculations or local UI state.
Calls detected	ensureSiteSections, siteSectionDialogValue, slugify, push, markDraftNeedsSave, setStatus, scheduleAutosave, renderAll

Item	Explanation
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	1 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

6171: async function addSiteSection() {
6172:   ensureSiteSections();
6173:   const input = await siteSectionDialogValue();
6174:   if (!input) return;
6175:   const id = input.id || slugify(input.title);
6176:   catalog.siteSections.push({
6177:     id,
6178:     title: input.title,
6179:     description: input.description,
6180:     richDescription: input.richDescription,
6181:     visible: false,
6182:     backgroundImage: "",
6183:     links: [],
6184:     assets: [],
6185:     subsections: []
6186:   });
6187:   markDraftNeedsSave();
6188:   setStatus("Portfolio area added locally.");
6189:   scheduleAutosave();
6190:   renderAll();
6191: }

```

editSiteSection - lines 6193-6206

Item	Explanation
Source location	Lines 6193-6206 (14 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	async function editSiteSection(sectionId) {
Touches	Mostly local calculations or local UI state.
Calls detected	ensureSiteSections, find, siteSectionDialogValue, markDraftNeedsSave, setStatus, scheduleAutosave, renderAll
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	1 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

6193: async function editSiteSection(sectionId) {
6194:   ensureSiteSections();
6195:   const section = catalog.siteSections.find((item) => item.id === sectionId);
6196:   if (!section) return;
6197:   const input = await siteSectionDialogValue(section);
6198:   if (!input) return;
6199:   section.title = input.title;
6200:   section.description = input.description;
6201:   section.richDescription = input.richDescription;
6202:   markDraftNeedsSave();
6203:   setStatus("Portfolio area edited locally.");
6204:   scheduleAutosave();
6205:   renderAll();
6206: }

```

addSiteSubsection - lines 6208-6219

Item	Explanation
Source location	Lines 6208-6219 (12 source lines).
Purpose	Part of the private builder frontend and editing experience.

Item	Explanation
Declaration	async function addSiteSubsection(sectionId) {
Touches	Mostly local calculations or local UI state.
Calls detected	find, siteSubsectionDialogValue, push, markDraftNeedsSave, setStatus, scheduleAutosave, renderAll
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	1 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

6208: async function addSiteSubsection(sectionId) {
6209:   const section = (catalog.siteSections || []).find((item) => item.id === sectionId);
6210:   if (!section) return;
6211:   const input = await siteSubsectionDialogValue();
6212:   if (!input) return;
6213:   section.subsections = section.subsections || [];
6214:   section.subsections.push(input);
6215:   markDraftNeedsSave();
6216:   setStatus("Portfolio subsection added locally.");
6217:   scheduleAutosave();
6218:   renderAll();
6219: }

```

section - lines 6209-6218

Item	Explanation
Source location	Lines 6209-6218 (10 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	const section = (catalog.siteSections []).find((item) => item.id === sectionId);
Touches	Mostly local calculations or local UI state.
Calls detected	find, siteSubsectionDialogValue, push, markDraftNeedsSave, setStatus, scheduleAutosave, renderAll
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	1 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

6209:   const section = (catalog.siteSections || []).find((item) => item.id === sectionId);
6210:   if (!section) return;
6211:   const input = await siteSubsectionDialogValue();
6212:   if (!input) return;
6213:   section.subsections = section.subsections || [];
6214:   section.subsections.push(input);
6215:   markDraftNeedsSave();
6216:   setStatus("Portfolio subsection added locally.");
6217:   scheduleAutosave();
6218:   renderAll();

```

editSiteSubsection - lines 6221-6232

Item	Explanation
Source location	Lines 6221-6232 (12 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	async function editSiteSubsection(sectionId, index) {
Touches	Mostly local calculations or local UI state.
Calls detected	find, siteSubsectionDialogValue, markDraftNeedsSave, setStatus, scheduleAutosave, renderAll

Item	Explanation
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	1 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

6221: async function editSiteSubsection(sectionId, index) {
6222:   const section = (catalog.siteSections || []).find((item) => item.id === sectionId);
6223:   const subsection = section?.subsections?.[Number(index)];
6224:   if (!section || !subsection) return;
6225:   const input = await siteSubsectionDialogValue(subsection);
6226:   if (!input) return;
6227:   section.subsections[Number(index)] = input;
6228:   markDraftNeedsSave();
6229:   setStatus("Portfolio subsection edited locally.");
6230:   scheduleAutosave();
6231:   renderAll();
6232: }

```

section - lines 6222-6231

Item	Explanation
Source location	Lines 6222-6231 (10 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	const section = (catalog.siteSections []).find((item) => item.id === sectionId);
Touches	Mostly local calculations or local UI state.
Calls detected	find, siteSubsectionDialogValue, markDraftNeedsSave, setStatus, scheduleAutosave, renderAll
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	1 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

6222:   const section = (catalog.siteSections || []).find((item) => item.id === sectionId);
6223:   const subsection = section?.subsections?.[Number(index)];
6224:   if (!section || !subsection) return;
6225:   const input = await siteSubsectionDialogValue(subsection);
6226:   if (!input) return;
6227:   section.subsections[Number(index)] = input;
6228:   markDraftNeedsSave();
6229:   setStatus("Portfolio subsection edited locally.");
6230:   scheduleAutosave();
6231:   renderAll();

```

deleteSiteSubsection - lines 6234-6248

Item	Explanation
Source location	Lines 6234-6248 (15 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function deleteSiteSubsection(sectionId, index) {
Touches	filesystem
Calls detected	find, openDeleteConfirm, splice, markDraftNeedsSave, scheduleAutosave, renderAll
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).

Item	Explanation
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

6234: function deleteSiteSubsection(sectionId, index) {
6235:   const section = (catalog.siteSections || []).find((item) => item.id === sectionId);
6236:   const subsection = section?.subsections?.[Number(index)];
6237:   if (!section || !subsection) return;
6238:   openDeleteConfirm({
6239:     title: "Delete subsection",
6240:     message: `Are you sure you want to delete "${subsection.title}"?`,
6241:     onConfirm: () => {
6242:       section.subsections.splice(Number(index), 1);
6243:       markDraftNeedsSave();
6244:       scheduleAutosave();
6245:       renderAll();
6246:     }
6247:   });
6248: }

```

section - lines 6235-6247

Item	Explanation
Source location	Lines 6235-6247 (13 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	const section = (catalog.siteSections []).find((item) => item.id === sectionId);
Touches	filesystem
Calls detected	find, openDeleteConfirm, splice, markDraftNeedsSave, scheduleAutosave, renderAll
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

6235: const section = (catalog.siteSections || []).find((item) => item.id === sectionId);
6236: const subsection = section?.subsections?.[Number(index)];
6237: if (!section || !subsection) return;
6238: openDeleteConfirm({
6239:   title: "Delete subsection",
6240:   message: `Are you sure you want to delete "${subsection.title}"?`,
6241:   onConfirm: () => {
6242:     section.subsections.splice(Number(index), 1);
6243:     markDraftNeedsSave();
6244:     scheduleAutosave();
6245:     renderAll();
6246:   }
6247: });

```

addSiteSectionLink - lines 6250-6261

Item	Explanation
Source location	Lines 6250-6261 (12 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	async function addSiteSectionLink(sectionId) {
Touches	Mostly local calculations or local UI state.
Calls detected	find, dialogLinkValue, push, markDraftNeedsSave, setStatus, scheduleAutosave, renderAll
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	1 await expression(s), 2 return statement(s).

Item	Explanation
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

6250: async function addSiteSectionLink(sectionId) {
6251:   const section = (catalog.siteSections || []).find((item) => item.id === sectionId);
6252:   if (!section) return;
6253:   const input = await dialogLinkValue();
6254:   if (!input) return;
6255:   section.links = section.links || [];
6256:   section.links.push(input);
6257:   markDraftNeedsSave();
6258:   setStatus("Portfolio link added locally.");
6259:   scheduleAutosave();
6260:   renderAll();
6261: }

```

section - lines 6251-6260

Item	Explanation
Source location	Lines 6251-6260 (10 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	const section = (catalog.siteSections []).find((item) => item.id === sectionId);
Touches	Mostly local calculations or local UI state.
Calls detected	find, dialogLinkValue, push, markDraftNeedsSave, setStatus, scheduleAutosave, renderAll
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	1 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

6251:   const section = (catalog.siteSections || []).find((item) => item.id === sectionId);
6252:   if (!section) return;
6253:   const input = await dialogLinkValue();
6254:   if (!input) return;
6255:   section.links = section.links || [];
6256:   section.links.push(input);
6257:   markDraftNeedsSave();
6258:   setStatus("Portfolio link added locally.");
6259:   scheduleAutosave();
6260:   renderAll();

```

editSiteSectionLink - lines 6263-6274

Item	Explanation
Source location	Lines 6263-6274 (12 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	async function editSiteSectionLink(sectionId, index) {
Touches	Mostly local calculations or local UI state.
Calls detected	find, dialogLinkValue, markDraftNeedsSave, setStatus, scheduleAutosave, renderAll
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	1 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

6263: async function editSiteSectionLink(sectionId, index) {

```



```

6264:   const section = (catalog.siteSections || []).find((item) => item.id === sectionId);
6265:   const link = section?.links?.[Number(index)];
6266:   if (!section || !link) return;
6267:   const input = await dialogLinkValue(link);
6268:   if (!input) return;
6269:   section.links[Number(index)] = input;
6270:   markDraftNeedsSave();
6271:   setStatus("Portfolio link edited locally.");
6272:   scheduleAutosave();
6273:   renderAll();
6274: }

```

section - lines 6264-6273

Item	Explanation
Source location	Lines 6264-6273 (10 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	<code>const section = (catalog.siteSections []).find((item) => item.id === sectionId);</code>
Touches	Mostly local calculations or local UI state.
Calls detected	<code>find</code> , <code>dialogLinkValue</code> , <code>markDraftNeedsSave</code> , <code>setStatus</code> , <code>scheduleAutosave</code> , <code>renderAll</code>
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	1 <code>await</code> expression(s), 2 <code>return</code> statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

6264:   const section = (catalog.siteSections || []).find((item) => item.id === sectionId);
6265:   const link = section?.links?.[Number(index)];
6266:   if (!section || !link) return;
6267:   const input = await dialogLinkValue(link);
6268:   if (!input) return;
6269:   section.links[Number(index)] = input;
6270:   markDraftNeedsSave();
6271:   setStatus("Portfolio link edited locally.");
6272:   scheduleAutosave();
6273:   renderAll();

```

deleteSiteSectionLink - lines 6276-6284

Item	Explanation
Source location	Lines 6276-6284 (9 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	<code>function deleteSiteSectionLink(sectionId, index) {</code>
Touches	Mostly local calculations or local UI state.
Calls detected	<code>find</code> , <code>splice</code> , <code>markDraftNeedsSave</code> , <code>setStatus</code> , <code>scheduleAutosave</code> , <code>renderAll</code>
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 <code>await</code> expression(s), 1 <code>return</code> statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

6276: function deleteSiteSectionLink(sectionId, index) {
6277:   const section = (catalog.siteSections || []).find((item) => item.id === sectionId);
6278:   if (!section?.links?.[Number(index)]) return;
6279:   section.links.splice(Number(index), 1);
6280:   markDraftNeedsSave();
6281:   setStatus("Portfolio link deleted locally.");
6282:   scheduleAutosave();
6283:   renderAll();
6284: }

```

section - lines 6277-6283

Item	Explanation
Source location	Lines 6277-6283 (7 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	<code>const section = (catalog.siteSections []).find((item) => item.id === sectionId);</code>
Touches	Mostly local calculations or local UI state.
Calls detected	<code>find</code> , <code>splice</code> , <code>markDraftNeedsSave</code> , <code>setStatus</code> , <code>scheduleAutosave</code> , <code>renderAll</code>
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 <code>await</code> expression(s), 1 <code>return</code> statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
6277: const section = (catalog.siteSections || []).find((item) => item.id === sectionId);
6278: if (!section?.links?.[Number(index)]) return;
6279: section.links.splice(Number(index), 1);
6280: markDraftNeedsSave();
6281: setStatus("Portfolio link deleted locally.");
6282: scheduleAutosave();
6283: renderAll();
```

toggleSiteSectionVisibility - lines 6286-6294

Item	Explanation
Source location	Lines 6286-6294 (9 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	<code>function toggleSiteSectionVisibility(sectionId) {</code>
Touches	Mostly local calculations or local UI state.
Calls detected	<code>find</code> , <code>markDraftNeedsSave</code> , <code>setStatus</code> , <code>scheduleAutosave</code> , <code>renderAll</code>
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 <code>await</code> expression(s), 1 <code>return</code> statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
6286: function toggleSiteSectionVisibility(sectionId) {
6287:   const section = (catalog.siteSections || []).find((item) => item.id === sectionId);
6288:   if (!section) return;
6289:   section.visible = section.visible === false;
6290:   markDraftNeedsSave();
6291:   setStatus(section.visible ? "Portfolio area will show on the website preview." : "Portfolio area hidden
from the website preview.");
6292:   scheduleAutosave();
6293:   renderAll();
6294: }
```

section - lines 6287-6293

Item	Explanation
Source location	Lines 6287-6293 (7 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	<code>const section = (catalog.siteSections []).find((item) => item.id === sectionId);</code>
Touches	Mostly local calculations or local UI state.

Item	Explanation
Calls detected	find, markDraftNeedsSave, setStatus, scheduleAutosave, renderAll
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

6287:   const section = (catalog.siteSections || []).find((item) => item.id === sectionId);
6288:   if (!section) return;
6289:   section.visible = section.visible === false;
6290:   markDraftNeedsSave();
6291:   setStatus(section.visible ? "Portfolio area will show on the website preview." : "Portfolio area hidden
from the website preview.");
6292:   scheduleAutosave();
6293:   renderAll();

```

deleteSiteSection - lines 6296-6310

Item	Explanation
Source location	Lines 6296-6310 (15 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function deleteSiteSection(sectionId) {
Touches	filesystem
Calls detected	find, openDeleteConfirm, filter, markDraftNeedsSave, scheduleAutosave, renderAll
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

6296: function deleteSiteSection(sectionId) {
6297:   const section = (catalog.siteSections || []).find((item) => item.id === sectionId);
6298:   if (!section) return;
6299:   openDeleteConfirm({
6300:     title: "Delete portfolio area",
6301:     message: `Are you sure you want to delete "${section.title}"?`,
6302:     onConfirm: () => {
6303:       catalog.siteSections = (catalog.siteSections || []).filter((item) => item.id !== sectionId);
6304:       savedPortfolioCatalog.siteSections = (savedPortfolioCatalog.siteSections || []).filter((item) =>
item.id !== sectionId);
6305:       markDraftNeedsSave();
6306:       scheduleAutosave();
6307:       renderAll();
6308:     }
6309:   });
6310: }

```

section - lines 6297-6309

Item	Explanation
Source location	Lines 6297-6309 (13 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	const section = (catalog.siteSections []).find((item) => item.id === sectionId);
Touches	filesystem
Calls detected	find, openDeleteConfirm, filter, markDraftNeedsSave, scheduleAutosave, renderAll
API endpoints	None detected inside this function body.

Item	Explanation
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

6297:   const section = (catalog.siteSections || []).find((item) => item.id === sectionId);
6298:   if (!section) return;
6299:   openDeleteConfirm({
6300:     title: "Delete portfolio area",
6301:     message: `Are you sure you want to delete "${section.title}"?`,
6302:     onConfirm: () => {
6303:       catalog.siteSections = (catalog.siteSections || []).filter((item) => item.id !== sectionId);
6304:       savedPortfolioCatalog.siteSections = (savedPortfolioCatalog.siteSections || []).filter((item) =>
item.id !== sectionId);
6305:       markDraftNeedsSave();
6306:       scheduleAutosave();
6307:       renderAll();
6308:     }
6309:   });

```

setSiteSectionBackground - lines 6312-6343

Item	Explanation
Source location	Lines 6312-6343 (32 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	async function setSiteSectionBackground(sectionId) {
Touches	filesystem, network/API requests
Calls detected	find, chooseFile, startsWith, setStatus, readFileAsDataURL, fetch, now, stringify, json, markDraftNeedsSave, scheduleAutosave, renderAll
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	4 await expression(s), 4 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

6312: async function setSiteSectionBackground(sectionId) {
6313:   const section = (catalog.siteSections || []).find((item) => item.id === sectionId);
6314:   if (!section) return;
6315:   const file = await chooseFile();
6316:   if (!file) return;
6317:   if (!file.type.startsWith("image/")) {
6318:     setStatus("Choose an image file for a section background.");
6319:     return;
6320:   }
6321:   const data = await readFileAsDataURL(file);
6322:   const response = await fetch(`/api/upload?t=${Date.now()}`, {
6323:     method: "POST",
6324:     cache: "no-store",
6325:     headers: { "Content-Type": "application/json" },
6326:     body: JSON.stringify({
6327:       projectId: "_site-sections",
6328:       section: section.id,
6329:       fileName: file.name,
6330:       data
6331:     })
6332:   });
6333:   const result = await response.json();
6334:   if (!response.ok) {
6335:     setStatus(result.error || "Background upload failed.");
6336:     return;
6337:   }
6338:   section.backgroundImage = result.url;
6339:   markDraftNeedsSave();
6340:   setStatus("Portfolio area background saved locally.");
6341:   scheduleAutosave();
6342:   renderAll();
6343: }

```

section - lines 6313-6320

Item	Explanation
Source location	Lines 6313-6320 (8 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	<code>const section = (catalog.siteSections []).find((item) => item.id === sectionId);</code>
Touches	Mostly local calculations or local UI state.
Calls detected	<code>find</code> , <code>chooseFile</code> , <code>startsWith</code> , <code>setStatus</code>
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	1 <code>await</code> expression(s), 3 <code>return</code> statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
6313: const section = (catalog.siteSections || []).find((item) => item.id === sectionId);
6314: if (!section) return;
6315: const file = await chooseFile();
6316: if (!file) return;
6317: if (!file.type.startsWith("image/")) {
6318:   setStatus("Choose an image file for a section background.");
6319:   return;
6320: }
```

clearSiteSectionBackground - lines 6345-6353

Item	Explanation
Source location	Lines 6345-6353 (9 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	<code>function clearSiteSectionBackground(sectionId) {</code>
Touches	Mostly local calculations or local UI state.
Calls detected	<code>find</code> , <code>markDraftNeedsSave</code> , <code>setStatus</code> , <code>scheduleAutosave</code> , <code>renderAll</code>
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 <code>await</code> expression(s), 1 <code>return</code> statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
6345: function clearSiteSectionBackground(sectionId) {
6346:   const section = (catalog.siteSections || []).find((item) => item.id === sectionId);
6347:   if (!section) return;
6348:   section.backgroundImage = "";
6349:   markDraftNeedsSave();
6350:   setStatus("Portfolio area background cleared locally.");
6351:   scheduleAutosave();
6352:   renderAll();
6353: }
```

section - lines 6346-6352

Item	Explanation
Source location	Lines 6346-6352 (7 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	<code>const section = (catalog.siteSections []).find((item) => item.id === sectionId);</code>
Touches	Mostly local calculations or local UI state.

Item	Explanation
Calls detected	find, markDraftNeedsSave, setStatus, scheduleAutosave, renderAll
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

6346: const section = (catalog.siteSections || []).find((item) => item.id === sectionId);
6347: if (!section) return;
6348: section.backgroundImage = "";
6349: markDraftNeedsSave();
6350: setStatus("Portfolio area background cleared locally.");
6351: scheduleAutosave();
6352: renderAll();

```

renderSectionTabs - lines 6355-6369

Item	Explanation
Source location	Lines 6355-6369 (15 source lines).
Purpose	Renders UI, markup, or display output.
Declaration	function renderSectionTabs(project) {
Touches	Mostly local calculations or local UI state.
Calls detected	sectionOptions, map, join
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

6355: function renderSectionTabs(project) {
6356:   if (!project) {
6357:     sectionTabs.innerHTML = "";
6358:     return;
6359:   }
6360:
6361:   sectionTabs.innerHTML = `
6362:     ${sectionOptions(project).map((section) => `
6363:       <button class="${section.id === activeSectionId ? "active" : ""}" type="button" data-section-
id="${section.id}">
6364:         ${section.label}
6365:       </button>
6366:     `).join("")}
6367:     <button type="button" data-add-section="true">Add section</button>
6368:   `;
6369: }

```

renderTextArray - lines 6371-6389

Item	Explanation
Source location	Lines 6371-6389 (19 source lines).
Purpose	Renders UI, markup, or display output.
Declaration	function renderTextArray(project, key, label) {
Touches	Mostly local calculations or local UI state.
Calls detected	map, join, toLowerCase
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.

Item	Explanation
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

6371: function renderTextArray(project, key, label) {
6372:   const items = project[key] || [];
6373:   return
6374:     <div class="section-actions">
6375:       <button class="button primary" type="button" data-open-editor="text-array" data-key="${key}" data-
mode="add">Add ${label}</button>
6376:     </div>
6377:     <div class="builder-items">
6378:       ${items.map((item, index) => `
6379:         <article class="builder-item">
6380:           <div>
6381:             <strong>${typeof item === "string" ? item : item.title || item.name || item.label ||
""}</strong>
6382:           </div>
6383:           <button type="button" data-open-editor="text-array" data-key="${key}" data-mode="edit" data-
index="${index}">Edit</button>
6384:           <button class="danger-icon" type="button" data-delete-array="${key}" data-
index="${index}">Delete</button>
6385:         </article>
6386:       `).join("")} || `<p class="evidence-empty">No ${label.toLowerCase()} added yet.</p>`}
6387:     </div>
6388:   `;
6389: }

```

toolName - lines 6391-6393

Item	Explanation
Source location	Lines 6391-6393 (3 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function toolName(item) {
Touches	Mostly local calculations or local UI state.
Calls detected	No obvious named calls detected by the generator.
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

6391: function toolName(item) {
6392:   return typeof item === "string" ? item : item.name || item.title || item.label || "";
6393: }

```

toolDescription - lines 6395-6397

Item	Explanation
Source location	Lines 6395-6397 (3 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function toolDescription(item) {
Touches	Mostly local calculations or local UI state.
Calls detected	No obvious named calls detected by the generator.
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).

Item	Explanation
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

6395: function toolDescription(item) {
6396:   return typeof item === "string" ? "" : item.description || item.usedFor || "";
6397: }

```

renderToolsSection - lines 6399-6420

Item	Explanation
Source location	Lines 6399-6420 (22 source lines).
Purpose	Renders UI, markup, or display output.
Declaration	function renderToolsSection(project) {
Touches	Mostly local calculations or local UI state.
Calls detected	map, toolName, richHasContent, renderRichContent, toolDescription, join
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

6399: function renderToolsSection(project) {
6400:   const items = project.tools || [];
6401:   return
6402:     <div class="section-actions">
6403:       <button class="button primary" type="button" data-open-editor="tool" data-mode="add">Add
tool</button>
6404:     </div>
6405:     <div class="builder-items tool-items">
6406:       ${items.map((item, index) => `
6407:         <article class="builder-item tool-item">
6408:           <div>
6409:             <strong>${toolName(item)} || "Untitled tool"</strong>
6410:           </div>
6411:           ${richHasContent(item?.richDescription)
6412:             ? renderRichContent(item.richDescription, toolDescription(item))
6413:             : `<p>${toolDescription(item)} || "No usage description added yet."</p>`}
6414:           <button type="button" data-open-editor="tool" data-mode="edit" data-
index="${index}">Edit</button>
6415:           <button class="danger-icon" type="button" data-delete-array="tools" data-
index="${index}">Delete</button>
6416:         </article>
6417:       `).join("")} || `<p class="evidence-empty">No tools added yet.</p>`}
6418:     </div>
6419:   `;
6420: }

```

renderObjectSection - lines 6422-6450

Item	Explanation
Source location	Lines 6422-6450 (29 source lines).
Purpose	Renders UI, markup, or display output.
Declaration	function renderObjectSection(project, key, folder, label) {
Touches	filesystem
Calls detected	map, richHasContent, renderRichContent, renderMultilineInlineText, join, toLowerCase
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 2 return statement(s).

Item	Explanation
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

6422: function renderObjectSection(project, key, folder, label) {
6423:   const items = project[key] || [];
6424:   return `
6425:     <div class="section-actions">
6426:       <button class="button primary" type="button" data-open-editor="object" data-key="${key}" data-
mode="add">Add text item</button>
6427:       ${folder ? `<button class="button secondary" type="button" data-upload="${key}" data-
folder="${folder}">Add file or image</button>` : ""}
6428:     </div>
6429:     <div class="builder-items">
6430:       ${items.map((item, index) => {
6431:         const title = item.title || item.name || item.label || "Untitled item";
6432:         const url = item.url || item.artifact || "";
6433:         const description = item.description || item.caption || item.result || item.method || "";
6434:         return `
6435:           <article class="builder-item">
6436:             <div>
6437:               <strong>${title}</strong>
6438:               <span>${url || "No file attached"}</span>
6439:               ${richHasContent(item.richDescription)
6440:                 ? renderRichContent(item.richDescription, description)
6441:                 : description ? `<p>${renderMultilineInlineText(description)}</p>` : ""}
6442:             </div>
6443:             <button type="button" data-open-editor="object" data-key="${key}" data-mode="edit" data-
index="${index}">Edit</button>
6444:             <button class="danger-icon" type="button" data-delete-item="${key}" data-
index="${index}">Delete</button>
6445:           </article>
6446:         `;
6447:       }).join("") || `<p class="evidence-empty">No ${label.toLowerCase()} added yet.</p>`}
6448:     </div>
6449:   `;
6450: }

```

designArray - lines 6452-6458

Item	Explanation
Source location	Lines 6452-6458 (7 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function designArray(project, pathValue) {
Touches	Mostly local calculations or local UI state.
Calls detected	ensureDesignModel, getByPath, isArray, setByPath
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

6452: function designArray(project, pathValue) {
6453:   ensureDesignModel(project);
6454:   const items = getByPath(project, pathValue);
6455:   if (Array.isArray(items)) return items;
6456:   setByPath(project, pathValue, []);
6457:   return getByPath(project, pathValue);
6458: }

```

makeDesignItem - lines 6460-6466

Item	Explanation
Source location	Lines 6460-6466 (7 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function makeDesignItem(kind, title, description = "", url = "") {

Item	Explanation
Touches	Mostly local calculations or local UI state.
Calls detected	No obvious named calls detected by the generator.
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 5 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

6460: function makeDesignItem(kind, title, description = "", url = "") {
6461:   if (kind === "reference") return { title, description, url, type: "Reference", status: url ? "linked" :
"draft" };
6462:   if (kind === "analysis") return { title, description, type: "Math / Analysis", status: "draft" };
6463:   if (kind === "simulation-file") return { title, description, url, type: "Simulation file", status: url
? "uploaded" : "draft" };
6464:   if (kind === "simulation-result") return { title, description, url, type: "Simulation result", status:
url ? "uploaded" : "draft" };
6465:   return { title, description, url, type: "Document", status: url ? "uploaded" : "draft" };
6466: }

```

renderDesignObjectSection - lines 6468-6496

Item	Explanation
Source location	Lines 6468-6496 (29 source lines).
Purpose	Renders UI, markup, or display output.
Declaration	function renderDesignObjectSection(project, pathValue, folder, label, kind = "document") {
Touches	filesystem
Calls detected	designArray, map, richHasContent, renderRichContent, renderMultilineInlineText, join, toLowerCase
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

6468: function renderDesignObjectSection(project, pathValue, folder, label, kind = "document") {
6469:   const items = designArray(project, pathValue);
6470:   return `
6471:     <div class="section-actions">
6472:       <button class="button primary" type="button" data-open-editor="design-object" data-design-
path="${pathValue}" data-design-kind="${kind}" data-mode="add">Add ${label}</button>
6473:       ${folder ? `<button class="button secondary" type="button" data-upload-design="${pathValue}" data-
folder="${folder}" data-design-kind="${kind}">Add file</button>` : ""}
6474:     </div>
6475:     <div class="builder-items">
6476:       ${items.map((item, index) => {
6477:         const title = item.title || item.name || item.label || "Untitled item";
6478:         const url = item.url || item.artifact || "";
6479:         const description = item.description || item.caption || item.result || item.method || "";
6480:         return `
6481:           <article class="builder-item">
6482:             <div>
6483:               <strong>${title}</strong>
6484:               <span>${url || "No file attached"}</span>
6485:               ${richHasContent(item.richDescription)
6486:                 ? renderRichContent(item.richDescription, description)
6487:                 : description ? `<p>${renderMultilineInlineText(description)}</p>` : ""}
6488:             </div>
6489:             <button type="button" data-open-editor="design-object" data-design-path="${pathValue}" data-
design-kind="${kind}" data-mode="edit" data-index="${index}">Edit</button>
6490:             <button class="danger-icon" type="button" data-delete-design-item="${pathValue}" data-
index="${index}">Delete</button>
6491:           </article>
6492:         `;
6493:       }).join("") || `<p class="evidence-empty">No ${label.toLowerCase()} added yet.</p>`}
6494:     </div>
6495:   `;
6496: }

```

renderDesignSummaryField - lines 6498-6530

Item	Explanation
Source location	Lines 6498-6530 (33 source lines).
Purpose	Renders UI, markup, or display output.
Declaration	function renderDesignSummaryField(project, pathValue, label, placeholder) {
Touches	filesystem
Calls detected	getByPath, summaryRichPathFromTextPath, escapeHtml, overviewFolderFromPath
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
6498: function renderDesignSummaryField(project, pathValue, label, placeholder) {
6499:   const textValue = getByPath(project, pathValue) || "";
6500:   const richPath = summaryRichPathFromTextPath(pathValue);
6501:   const richValue = getByPath(project, richPath);
6502:   const hasValue = Boolean(textValue || richValue?.blocks?.length);
6503:
6504:   return `
6505:     <div class="wide-field design-summary-field rich-design-summary-field">
6506:       <div class="summary-builder-heading">
6507:         <span>
6508:           ${label}
6509:           ${hasValue ? `<button class="inline-danger-button" type="button" data-clear-design-
summary="${pathValue}">Clear overview</button>` : ""}
6510:         </span>
6511:       </div>
6512:
6513:       <div class="rich-summary-panel">
6514:         <p class="rich-editor-hint">Right-click to add images, code blocks, apply bold, color, font,
numeric size, alignment, or formulas.</p>
6515:
6516:         <div
6517:           class="rich-summary-editor"
6518:           data-rich-editor="section-overview"
6519:           data-rich-text-path="${escapeHtml(pathValue)}"
6520:           data-rich-path="${escapeHtml(richPath)}"
6521:           data-rich-folder="${escapeHtml(overviewFolderFromPath(pathValue))}"
6522:           data-placeholder="${escapeHtml(placeholder || "")}"
6523:           contenteditable="true"
6524:           spellcheck="true"
6525:           aria-label="${escapeHtml(label)} rich editor"
6526:         ></div>
6527:       </div>
6528:     </div>
6529:   `;
6530: }
```

designPathHasContent - lines 6532-6536

Item	Explanation
Source location	Lines 6532-6536 (5 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function designPathHasContent(project, pathValue) {
Touches	Mostly local calculations or local UI state.
Calls detected	getByPath, isArray, some, itemTitle, richHasContent, summaryRichPathFromTextPath
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
6532: function designPathHasContent(project, pathValue) {
6533:   const value = getByPath(project, pathValue);
6534:   if (Array.isArray(value)) return value.some((item) => itemTitle(item) || item?.description || item?.url
|| item?.artifact || richHasContent(item?.richDescription));
6535:   return Boolean(value || richHasContent(getByPath(project, summaryRichPathFromTextPath(pathValue))));
6536: }
```

renderEmptyDynamicSectionPrompt - lines 6538-6548

Item	Explanation
Source location	Lines 6538-6548 (11 source lines).
Purpose	Renders UI, markup, or display output.
Declaration	function renderEmptyDynamicSectionPrompt(label = "section") {
Touches	Mostly local calculations or local UI state.
Calls detected	escapeHtml
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
6538: function renderEmptyDynamicSectionPrompt(label = "section") {
6539:   return `
6540:     <div class="compile-code-empty">
6541:       <h3>No ${escapeHtml(label)} content added yet</h3>
6542:       <p>Create your own sections and subsections with the Add section button. Empty predefined groups
stay hidden so the project structure is controlled by you.</p>
6543:       <div class="section-actions">
6544:         <button class="button primary" type="button" data-add-section="true">Add section</button>
6545:       </div>
6546:     </div>
6547:   `;
6548: }
```

renderAnalogDesignWorkspace - lines 6550-6593

Item	Explanation
Source location	Lines 6550-6593 (44 source lines).
Purpose	Renders UI, markup, or display output.
Declaration	function renderAnalogDesignWorkspace(project) {
Touches	Mostly local calculations or local UI state.
Calls detected	ensureDesignModel, designPathHasContent, push, renderDesignSummaryField, renderDesignObjectSection, renderObjectSection, join, renderEmptyDynamicSectionPrompt, renderPendingEditor
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
6550: function renderAnalogDesignWorkspace(project) {
6551:   ensureDesignModel(project);
6552:   const cards = [];
6553:   if (designPathHasContent(project, "design.brief.summary") || designPathHasContent(project,
"design.brief.files")) {
6554:     cards.push(`
6555:       <section class="design-workspace-card">
6556:         <h3>Design Overview</h3>
6557:         ${renderDesignSummaryField(project, "design.brief.summary", "Design overview", "State the analog
design objective, topology, constraints, and design decisions.")}
6558:         ${renderDesignObjectSection(project, "design.brief.files", "design-brief", "brief file",
```

```

"document"))}
6559:     </section>
6560:   `);
6561: }
6562: if (
6563:   designPathHasContent(project, "design.documentation.summary") ||
6564:   designPathHasContent(project, "design.documentation.files") ||
6565:   designPathHasContent(project, "design.documentation.references") ||
6566:   designPathHasContent(project, "design.documentation.mathAnalysis")
6567: ) {
6568:   cards.push(`
6569:     <section class="design-workspace-card">
6570:       <h3>Documentation</h3>
6571:       ${renderDesignSummaryField(project, "design.documentation.summary", "Documentation overview",
"Summarize schematics, notes, datasheets, calculations, and supporting documents.")}
6572:       ${renderDesignObjectSection(project, "design.documentation.files", "documentation", "document",
"document")}
6573:       ${designPathHasContent(project, "design.documentation.references") ?
`<h4>References</h4>${renderDesignObjectSection(project, "design.documentation.references", "", "reference",
"reference")}` : ""}
6574:       ${designPathHasContent(project, "design.documentation.mathAnalysis") ? `<h4>Math /
Analysis</h4>${renderDesignObjectSection(project, "design.documentation.mathAnalysis", "", "analysis item",
"analysis")}` : ""}
6575:     </section>
6576:   `);
6577: }
6578: if ((project.pcbns || []).length || (project.media || []).length) {
6579:   cards.push(`
6580:     <section class="design-workspace-card">
6581:       <h3>PCB and Visual Evidence</h3>
6582:       ${((project.pcbns || []).length ? renderObjectSection(project, "pcbns", "pcbns", "PCBs") : "")}
6583:       ${((project.media || []).length ? renderObjectSection(project, "media", "images", "Images") : "")}
6584:     </section>
6585:   `);
6586: }
6587: return `
6588:   <div class="section-stack analog-design-workspace">
6589:     ${cards.join("")} || renderEmptyDynamicSectionPrompt("design")}
6590:     ${renderPendingEditor()}
6591:   </div>
6592: `;
6593: }

```

renderAnalogSimulationWorkspace - lines 6595-6628

Item	Explanation
Source location	Lines 6595-6628 (34 source lines).
Purpose	Renders UI, markup, or display output.
Declaration	function renderAnalogSimulationWorkspace(project) {
Touches	Mostly local calculations or local UI state.
Calls detected	ensureDesignModel, designPathHasContent, push, renderDesignSummaryField, renderDesignObjectSection, join, renderEmptyDynamicSectionPrompt, renderPendingEditor
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

6595: function renderAnalogSimulationWorkspace(project) {
6596:   ensureDesignModel(project);
6597:   const cards = [];
6598:   if (designPathHasContent(project, "design.simulation.summary")) {
6599:     cards.push(`
6600:       <section class="design-workspace-card">
6601:         <h3>Simulation Overview</h3>
6602:         ${renderDesignSummaryField(project, "design.simulation.summary", "Simulation overview", "Explain
what was simulated, why it mattered, and what the results proved.")}
6603:       </section>
6604:     `);
6605:   }
6606:   if (designPathHasContent(project, "design.simulation.files")) {
6607:     cards.push(`
6608:       <section class="design-workspace-card">
6609:         <h3>Simulation Files</h3>
6610:         ${renderDesignObjectSection(project, "design.simulation.files", "simulations", "simulation file",
"simulation-file")}
6611:       </section>

```

```

6612:     `);
6613:   }
6614:   if (designPathHasContent(project, "design.simulation.results")) {
6615:     cards.push(`
6616:       <section class="design-workspace-card">
6617:         <h3>Simulation Results</h3>
6618:         ${renderDesignObjectSection(project, "design.simulation.results", "simulation-results",
"simulation result", "simulation-result")}
6619:       </section>
6620:     `);
6621:   }
6622:   return `
6623:     <div class="section-stack analog-design-workspace">
6624:       ${cards.join("")} || renderEmptyDynamicSectionPrompt("simulation")}
6625:       ${renderPendingEditor()}
6626:     </div>
6627:   `;
6628: }

```

pendingEditorUsesRichDescription - lines 6630-6632

Item	Explanation
Source location	Lines 6630-6632 (3 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function pendingEditorUsesRichDescription(editor = pendingEditor) {
Touches	Mostly local calculations or local UI state.
Calls detected	No obvious named calls detected by the generator.
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

6630: function pendingEditorUsesRichDescription(editor = pendingEditor) {
6631:   return Boolean(editor && editor.type !== "text-array");
6632: }

```

pendingEditorFolder - lines 6634-6641

Item	Explanation
Source location	Lines 6634-6641 (8 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function pendingEditorFolder(editor = pendingEditor) {
Touches	Mostly local calculations or local UI state.
Calls detected	slugify
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 6 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

6634: function pendingEditorFolder(editor = pendingEditor) {
6635:   if (!editor) return "content";
6636:   if (editor.type === "custom-section") return `custom-${slugify(editor.sectionId ||
"section")}-overview`;
6637:   if (editor.type === "custom") return `custom-${slugify(editor.sectionId || "section")}-subsection`;
6638:   if (editor.type === "design-object") return `design-${slugify(editor.kind || "content")}`;
6639:   if (editor.type === "object") return slugify(editor.key || "content");
6640:   return slugify(editor.type || "content");
6641: }

```

renderPendingRichDescriptionEditor - lines 6643-6666

Item	Explanation
Source location	Lines 6643-6666 (24 source lines).
Purpose	Renders UI, markup, or display output.
Declaration	function renderPendingRichDescriptionEditor(editor = pendingEditor) {
Touches	filesystem
Calls detected	escapeHtml, pendingEditorFolder
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
6643: function renderPendingRichDescriptionEditor(editor = pendingEditor) {
6644:   const label = editor?.type === "tool"
6645:     ? "What it was used for"
6646:     : editor?.type === "custom-section"
6647:       ? "Section content"
6648:       : "Content";
6649:   return `
6650:     <div class="pending-rich-description rich-design-summary-field">
6651:       <div class="summary-builder-heading"><span>${escapeHtml(label)}</span></div>
6652:       <div class="rich-summary-panel">
6653:         <p class="rich-editor-hint">Right-click to add images, code blocks, apply bold, color, font,
numeric size, alignment, or formulas.</p>
6654:         <div
6655:           class="rich-summary-editor"
6656:           data-rich-editor="pending-description"
6657:           data-rich-folder="${escapeHtml(pendingEditorFolder(editor))}"
6658:           data-placeholder="Type or paste the complete content here."
6659:           contenteditable="true"
6660:           spellcheck="true"
6661:           aria-label="${escapeHtml(label)} rich editor"
6662:         ></div>
6663:       </div>
6664:     </div>
6665:   `;
6666: }
```

renderPendingEditor - lines 6668-6692

Item	Explanation
Source location	Lines 6668-6692 (25 source lines).
Purpose	Renders UI, markup, or display output.
Declaration	function renderPendingEditor() {
Touches	filesystem
Calls detected	renderPendingRichDescriptionEditor
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
6668: function renderPendingEditor() {
6669:   if (!pendingEditor) return "";
6670:   const title = pendingEditor.title || "";
6671:   const isTool = pendingEditor.type === "tool";
6672:   const isSimpleText = pendingEditor.type === "text-array";
6673:   const isSectionDetails = pendingEditor.type === "custom-section";
6674:   return `
6675:     <form class="pending-editor" id="pending-editor">
```

```

6676:         <div>
6677:             <h3>${pendingEditor.mode === "edit" ? "Edit" : "Add"} ${isTool ? "tool" : isSimpleText ? "item" :
isSectionDetails ? "section" : "subsection"}</h3>
6678:         </div>
6679:         <div class="${isTool ? "tool-editor-grid" : "pending-editor-grid"}">
6680:             <label>
6681:                 <span>${isTool ? "Tool name" : "Title"}</span>
6682:                 <input name="title" type="text" value="${title}" required>
6683:             </label>
6684:             ${isSimpleText ? "" : renderPendingRichDescriptionEditor(pendingEditor)}
6685:         </div>
6686:         <div class="pending-editor-actions">
6687:             <button class="button primary" type="submit">Save</button>
6688:             <button class="button secondary" type="button" data-cancel-pending="true">Cancel</button>
6689:         </div>
6690:     </form>
6691: `;
6692: }

```

openPendingEditor - lines 6694-6701

Item	Explanation
Source location	Lines 6694-6701 (8 source lines).
Purpose	Controls a window, dialog, panel, menu, or visible state.
Declaration	function openPendingEditor(config) {
Touches	browser/Electron DOM
Calls detected	renderSectionContent, selectedProject, requestAnimationFrame, populateRichEditors, querySelector, scrollIntoView
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

6694: function openPendingEditor(config) {
6695:     pendingEditor = config;
6696:     renderSectionContent(selectedProject());
6697:     requestAnimationFrame(() => {
6698:         populateRichEditors(sectionContent);
6699:         document.querySelector("#pending-editor")?.scrollIntoView({ behavior: "smooth", block: "nearest" });
6700:     });
6701: }

```

savePendingEditor - lines 6703-6785

Item	Explanation
Source location	Lines 6703-6785 (83 source lines).
Purpose	Persists data to local state, disk, credentials, or browser storage.
Declaration	function savePendingEditor(form) {
Touches	browser/Electron DOM, filesystem
Calls detected	selectedProject, trim, querySelector, extractRichSummary, plainTextFromRich, setStatus, push, makeItemForSection, designArray, makeDesignItem, find, scheduleAutosave
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 4 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

6703: function savePendingEditor(form) {
6704:     const project = selectedProject();
6705:     if (!project || !pendingEditor) return;
6706:     const title = form.elements.title.value.trim();
6707:     const richEditor = form.querySelector("[data-rich-editor='pending-description']");

```



```

6708: const richDescription = richEditor ? extractRichSummary(richEditor) : null;
6709: const description = richEditor ? plainTextFromRich(richDescription) : "";
6710: if (!title) {
6711:   setStatus("Type a title before saving.");
6712:   return;
6713: }
6714:
6715: if (pendingEditor.type === "tool") {
6716:   project.tools = project.tools || [];
6717:   const nextTool = { name: title, description, richDescription };
6718:   if (pendingEditor.mode === "edit") {
6719:     project.tools[Number(pendingEditor.index)] = nextTool;
6720:   } else {
6721:     project.tools.push(nextTool);
6722:   }
6723: }
6724:
6725: if (pendingEditor.type === "object") {
6726:   const key = pendingEditor.key;
6727:   project[key] = project[key] || [];
6728:   if (pendingEditor.mode === "edit") {
6729:     const existing = project[key][Number(pendingEditor.index)] || {};
6730:     const existingUrl = existing.url || existing.artifact || "";
6731:     const nextItem = makeItemForSection(key, title, description, existingUrl);
6732:     project[key][Number(pendingEditor.index)] = { ...existing, ...nextItem, richDescription };
6733:   } else {
6734:     const nextItem = makeItemForSection(key, title, description);
6735:     project[key].push({ ...nextItem, richDescription });
6736:   }
6737: }
6738:
6739: if (pendingEditor.type === "design-object") {
6740:   const items = designArray(project, pendingEditor.path);
6741:   const kind = pendingEditor.kind || "document";
6742:   if (pendingEditor.mode === "edit") {
6743:     const existing = items[Number(pendingEditor.index)] || {};
6744:     const existingUrl = existing.url || existing.artifact || "";
6745:     items[Number(pendingEditor.index)] = { ...existing, ...makeDesignItem(kind, title, description,
existingUrl), richDescription };
6746:   } else {
6747:     items.push({ ...makeDesignItem(kind, title, description), richDescription });
6748:   }
6749: }
6750:
6751: if (pendingEditor.type === "text-array") {
6752:   const key = pendingEditor.key;
6753:   project[key] = project[key] || [];
6754:   if (pendingEditor.mode === "edit") {
6755:     project[key][Number(pendingEditor.index)] = title;
6756:   } else {
6757:     project[key].push(title);
6758:   }
6759: }
6760:
6761: if (pendingEditor.type === "custom") {
6762:   const section = (project.sections || []).find((item) => item.id === pendingEditor.sectionId);
6763:   if (!section) return;
6764:   section.items = section.items || [];
6765:   const nextItem = { title, description, richDescription, type: "Text", status: "draft" };
6766:   if (pendingEditor.mode === "edit") {
6767:     section.items[Number(pendingEditor.index)] = { ...section.items[Number(pendingEditor.index)],
...nextItem };
6768:   } else {
6769:     section.items.push(nextItem);
6770:   }
6771: }
6772:
6773: if (pendingEditor.type === "custom-section") {
6774:   const section = (project.sections || []).find((item) => item.id === pendingEditor.sectionId);
6775:   if (!section) return;
6776:   section.title = title;
6777:   section.description = description;
6778:   section.richDescription = richDescription;
6779: }
6780:
6781: pendingEditor = null;
6782: setStatus("Saved in the project editor. Click Save project to include this version in the portfolio
preview.");
6783: scheduleAutosave();
6784: renderAll();
6785: }

```

section - lines 6762-6765

Item	Explanation
Source location	Lines 6762-6765 (4 source lines).
Purpose	Part of the private builder frontend and editing experience.

Item	Explanation
Declaration	const section = (project.sections []).find((item) => item.id === pendingEditor.sectionId);
Touches	Mostly local calculations or local UI state.
Calls detected	find
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

6762:     const section = (project.sections || []).find((item) => item.id === pendingEditor.sectionId);
6763:     if (!section) return;
6764:     section.items = section.items || [];
6765:     const nextItem = { title, description, richDescription, type: "Text", status: "draft" };

```

section - lines 6774-6779

Item	Explanation
Source location	Lines 6774-6779 (6 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	const section = (project.sections []).find((item) => item.id === pendingEditor.sectionId);
Touches	Mostly local calculations or local UI state.
Calls detected	find
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

6774:     const section = (project.sections || []).find((item) => item.id === pendingEditor.sectionId);
6775:     if (!section) return;
6776:     section.title = title;
6777:     section.description = description;
6778:     section.richDescription = richDescription;
6779: }

```

openEditorFromButton - lines 6787-6879

Item	Explanation
Source location	Lines 6787-6879 (93 source lines).
Purpose	Controls a window, dialog, panel, menu, or visible state.
Declaration	function openEditorFromButton(button) {
Touches	browser/Electron DOM
Calls detected	selectedProject, openPendingEditor, toolName, toolDescription, designArray, find
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 6 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

6787: function openEditorFromButton(button) {
6788:   const project = selectedProject();
6789:   if (!project) return;
6790:   const type = button.dataset.openEditor;
6791:   const mode = button.dataset.mode || "add";
6792:   const index = button.dataset.index;
6793:
6794:   if (type === "tool") {
6795:     const item = mode === "edit" ? project.tools?.[Number(index)] : {};
6796:     openPendingEditor({
6797:       type,
6798:       mode,
6799:       index,
6800:       title: toolName(item),
6801:       description: toolDescription(item),
6802:       richDescription: item?.richDescription || null
6803:     });
6804:     return;
6805:   }
6806:
6807:   if (type === "object") {
6808:     const key = button.dataset.key;
6809:     const item = mode === "edit" ? project[key]?.[Number(index)] : {};
6810:     openPendingEditor({
6811:       type,
6812:       mode,
6813:       key,
6814:       index,
6815:       title: item?.title || item?.name || item?.label || "",
6816:       description: item?.description || item?.caption || item?.result || item?.method || "",
6817:       richDescription: item?.richDescription || null
6818:     });
6819:     return;
6820:   }
6821:
6822:   if (type === "design-object") {
6823:     const pathValue = button.dataset.designPath;
6824:     const items = designArray(project, pathValue);
6825:     const item = mode === "edit" ? items?.[Number(index)] : {};
6826:     openPendingEditor({
6827:       type,
6828:       mode,
6829:       path: pathValue,
6830:       kind: button.dataset.designKind || "document",
6831:       index,
6832:       title: item?.title || item?.name || item?.label || "",
6833:       description: item?.description || item?.caption || item?.result || item?.method || "",
6834:       richDescription: item?.richDescription || null
6835:     });
6836:     return;
6837:   }
6838:
6839:
6840:   if (type === "custom") {
6841:     const section = (project.sections || []).find((item) => item.id === button.dataset.sectionId);
6842:     const item = mode === "edit" ? section?.items?.[Number(index)] : {};
6843:     openPendingEditor({
6844:       type,
6845:       mode,
6846:       sectionId: button.dataset.sectionId,
6847:       index,
6848:       title: item?.title || "",
6849:       description: item?.description || "",
6850:       richDescription: item?.richDescription || null
6851:     });
6852:     return;
6853:   }
6854:
6855:   if (type === "text-array") {
6856:     const key = button.dataset.key;
6857:     const item = mode === "edit" ? project[key]?.[Number(index)] : "";
6858:     openPendingEditor({
6859:       type,
6860:       mode,
6861:       key,
6862:       index,
6863:       title: typeof item === "string" ? item : item?.title || item?.name || item?.label || ""
6864:     });
6865:     return;
6866:   }
6867:
6868:   if (type === "custom-section") {
6869:     const section = (project.sections || []).find((item) => item.id === button.dataset.sectionId);
6870:     openPendingEditor({
6871:       type,
6872:       mode: "edit",
6873:       sectionId: button.dataset.sectionId,
6874:       title: section?.title || "",
6875:       description: section?.description || "",
6876:       richDescription: section?.richDescription || null
6877:     });
6878:   }

```

6879: }

section - lines 6841-6842

Item	Explanation
Source location	Lines 6841-6842 (2 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	<code>const section = (project.sections []).find((item) => item.id === button.dataset.sectionId);</code>
Touches	browser/Electron DOM
Calls detected	find
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
6841:     const section = (project.sections || []).find((item) => item.id === button.dataset.sectionId);
6842:     const item = mode === "edit" ? section?.items?.[Number(index)] : {};
```

section - lines 6869-6877

Item	Explanation
Source location	Lines 6869-6877 (9 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	<code>const section = (project.sections []).find((item) => item.id === button.dataset.sectionId);</code>
Touches	browser/Electron DOM
Calls detected	find, openPendingEditor
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
6869:     const section = (project.sections || []).find((item) => item.id === button.dataset.sectionId);
6870:     openPendingEditor({
6871:       type,
6872:       mode: "edit",
6873:       sectionId: button.dataset.sectionId,
6874:       title: section?.title || "",
6875:       description: section?.description || "",
6876:       richDescription: section?.richDescription || null
6877:     });
```

builderPreviewContent - lines 6881-6888

Item	Explanation
Source location	Lines 6881-6888 (8 source lines).
Purpose	Creates an object, UI structure, process, payload, or generated artifact.
Declaration	<code>function builderPreviewContent(rich, text = "", emptyText = "No content has been added yet.") {</code>
Touches	filesystem
Calls detected	richHasContent, renderRichContent, renderMultilineInlineText, escapeHtml
API endpoints	None detected inside this function body.

Item	Explanation
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

6881: function builderPreviewContent(rich, text = "", emptyText = "No content has been added yet.") {
6882:   const content = richHasContent(rich)
6883:   ? renderRichContent(rich, text || "")
6884:   : text
6885:   ? `<p class="rich-paragraph">${renderMultilineInlineText(text)}</p>`
6886:   : `<p class="evidence-empty">${escapeHtml(emptyText)}</p>`;
6887:   return `<div class="builder-item-preview">${content}</div>`;
6888: }

```

renderCustomSection - lines 6890-6933

Item	Explanation
Source location	Lines 6890-6933 (44 source lines).
Purpose	Renders UI, markup, or display output.
Declaration	function renderCustomSection(project, sectionId) {
Touches	Mostly local calculations or local UI state.
Calls detected	find, builderPreviewContent, map, escapeHtml, richHasContent, join, renderPendingEditor
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

6890: function renderCustomSection(project, sectionId) {
6891:   const section = (project.sections || []).find((item) => item.id === sectionId);
6892:   if (!section) return `<p class="evidence-empty">Section not found.</p>`;
6893:
6894:   return `
6895:     <div class="section-window-heading">
6896:       <h3>${section.title || "Custom section"}</h3>
6897:       <div>
6898:         <button type="button" data-open-editor="custom-section" data-section-id="${section.id}">Edit
6899:       </div>
6900:     </div>
6901:     <div>
6902:       ${builderPreviewContent(section.richDescription, section.description || "", "No section content has
6903:       been added yet.")}
6904:     </div>
6905:     <div class="section-actions">
6906:       <button class="button primary" type="button" data-add-custom-item="${section.id}">Add
6907:     </div>
6908:     <div>
6909:       <button class="button secondary" type="button" data-upload-custom="${section.id}">Add file or
6910:       image</button>
6911:     </div>
6912:     <div class="builder-items">
6913:       ${((section.items || []).map((item, index) => `
6914:         <article class="builder-item">
6915:           <div>
6916:             <strong>${escapeHtml(item.title || (item.url ? "Untitled file" : "Untitled
6917:             subsection"))}</strong>
6918:             ${item.url ? `<span>${escapeHtml(item.url)}</span>` : ""}
6919:             ${richHasContent(item.richDescription) || item.description} ?
6920:             builderPreviewContent(item.richDescription, item.description || "", "") : ""
6921:             ${item.children || []}.length ? `
6922:               <div class="builder-child-files">
6923:                 ${item.children || []}.map((child, childIndex) => `
6924:                   <div>
6925:                     <span>${escapeHtml(child.title || "Untitled file")}</span>
6926:                     <button class="danger-icon" type="button" data-delete-custom-
6927:                     child="${escapeHtml(section.id)}" data-index="${index}" data-child-index="${childIndex}">Delete</button>
6928:                   </div>
6929:                 `).join("")
6930:               </div>
6931:             `
6932:           </div>
6933:         `

```

```

6923:         ` : ""}
6924:     </div>
6925:     ${item.url ? "" : `<button type="button" data-upload-custom-
subsection="${escapeHtml(section.id)}" data-index="${index}">Add file</button>`}
6926:     ${item.url ? "" : `<button type="button" data-open-editor="custom" data-mode="edit" data-
section-id="${escapeHtml(section.id)}" data-index="${index}">Edit</button>`}
6927:     <button class="danger-icon" type="button" data-delete-custom-item="${section.id}" data-
index="${index}">Delete</button>
6928: </article>
6929: `).join("") || `<p class="evidence-empty">No subsections added yet.</p>`}
6930: </div>
6931: ${renderPendingEditor()}
6932: `;
6933: }

```

section - lines 6891-6892

Item	Explanation
Source location	Lines 6891-6892 (2 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	const section = (project.sections []).find((item) => item.id === sectionId);
Touches	Mostly local calculations or local UI state.
Calls detected	find
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

6891:     const section = (project.sections || []).find((item) => item.id === sectionId);
6892:     if (!section) return `<p class="evidence-empty">Section not found.</p>`;

```

compileLanguageOptions - lines 6935-6940

Item	Explanation
Source location	Lines 6935-6940 (6 source lines).
Purpose	Part of the Compile Code language/tool/build/run flow.
Declaration	function compileLanguageOptions(selected = "javascript") {
Touches	filesystem
Calls detected	normalizeCodeLanguage, map, escapeHtml, join
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

6935: function compileLanguageOptions(selected = "javascript") {
6936:     const normalized = normalizeCodeLanguage(selected);
6937:     return supportedCodeLanguages.map((language) => `
6938:         <option value="${escapeHtml(language.id)}"${language.id === normalized ? " selected" :
""}>${escapeHtml(language.label)}</option>
6939:     `).join("");
6940: }

```

compileRoleOptions - lines 6942-6947

Item	Explanation
Source location	Lines 6942-6947 (6 source lines).

Item	Explanation
Purpose	Part of the Compile Code language/tool/build/run flow.
Declaration	function compileRoleOptions(selected = "design") {
Touches	filesystem
Calls detected	normalizeCompileFileRole, map, escapeHtml, compileFileRoleLabel, join
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

6942: function compileRoleOptions(selected = "design") {
6943:   const normalized = normalizeCompileFileRole(selected, "verilog");
6944:   return ["design", "testbench"].map((role) => `
6945:     <option value="${escapeHtml(role)}"${role === normalized ? " selected" :
""}>${escapeHtml(compileFileRoleLabel(role))}</option>
6946:   `).join("");
6947: }

```

hdlTestbenchTemplate - lines 6949-6973

Item	Explanation
Source location	Lines 6949-6973 (25 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function hdlTestbenchTemplate(language = "systemverilog", designName = "design") {
Touches	filesystem
Calls detected	slugify, replace, dut, clk, reset_n, done, dumpfile, dumpvars, join
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

6949: function hdlTestbenchTemplate(language = "systemverilog", designName = "design") {
6950:   const tbName = `tb_${slugify(designName || "design").replace(/-/g, "_")} || "design"`;
6951:   const dutName = slugify(designName || "design").replace(/-/g, "_") || "design";
6952:   const commentPrefix = language === "verilog" ? "/*" : "/*";
6953:   return [
6954:     `timescale 1ns/1ps`,
6955:     ``,
6956:     `module ${tbName};`,
6957:     `  ${commentPrefix} Declare testbench signals here.`,
6958:     `  ${commentPrefix} Example: reg clk = 0; reg reset_n = 0; wire done;`,
6959:     ``,
6960:     `  ${commentPrefix} Instantiate the design under test and connect the signals.`,
6961:     `  ${commentPrefix} Example: ${dutName} dut (.clk(clk), .reset_n(reset_n), .done(done));`,
6962:     ``,
6963:     `  initial begin`,
6964:     `    $dumpfile("waveform.vcd");`,
6965:     `    $dumpvars(0, ${tbName});`,
6966:     ``,
6967:     `    ${commentPrefix} Drive stimulus over time here.`,
6968:     `    #100;`,
6969:     `    $finish;`,
6970:     `  end`,
6971:     `endmodule`
6972:   ].join("\n");
6973: }

```

firstHdlModuleName - lines 6975-6978

Item	Explanation
Source location	Lines 6975-6978 (4 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function firstHdlModuleName(code = "") {
Touches	Mostly local calculations or local UI state.
Calls detected	match
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
6975: function firstHdlModuleName(code = "") {
6976:   const match = String(code || "").match(/bmodule\s+([A-Za-z_$][\w$]*)/);
6977:   return match?.[1] || "design";
6978: }
```

compileAppendDestinations - lines 6980-7026

Item	Explanation
Source location	Lines 6980-7026 (47 source lines).
Purpose	Part of the Compile Code language/tool/build/run flow.
Declaration	function compileAppendDestinations(project) {
Touches	Mostly local calculations or local UI state.
Calls detected	isAnalogProject, ensureDesignModel, forEach, push, summaryRichPathFromTextPath, walkItems
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
6980: function compileAppendDestinations(project) {
6981:   if (!project) return [];
6982:   const destinations = [
6983:     id: "project-overview",
6984:     label: "Project overview",
6985:     richPath: "summaryRich",
6986:     textPath: "summary"
6987:   ];
6988:
6989:   if (isAnalogProject(project)) {
6990:     ensureDesignModel(project);
6991:     [
6992:       ["design-overview", "Design overview", "design.brief.summary"],
6993:       ["documentation-overview", "Documentation overview", "design.documentation.summary"],
6994:       ["simulation-overview", "Simulation overview", "design.simulation.summary"]
6995:     ].forEach(([id, label, textPath]) => {
6996:       destinations.push({ id, label, richPath: summaryRichPathFromTextPath(textPath), textPath });
6997:     });
6998:   }
6999:
7000:   (project.sections || []).forEach((section, index) => {
7001:     destinations.push({
7002:       id: `custom-${section.id || index}`,
7003:       label: `${section.title || `Custom section ${index + 1}`} overview`,
7004:       richPath: `sections.${index}.richDescription`,
7005:       textPath: `sections.${index}.description`
7006:     });
7007:     const walkItems = (items = [], prefix = `sections.${index}.items`, labelPrefix = section.title || `Custom section ${index + 1}`) => {
7008:       (items || []).forEach((item, itemIndex) => {
7009:         const itemPath = `${prefix}.${itemIndex}`;
7010:         const itemTitle = item.title || `Subsection ${itemIndex + 1}`;
```



```

7011:         if (!item.url) {
7012:             destinations.push({
7013:                 id: `custom-${section.id || index}-${itemPath}`,
7014:                 label: `${labelPrefix} / ${itemTitle} overview`,
7015:                 richPath: `${itemPath}.richDescription`,
7016:                 textPath: `${itemPath}.description`
7017:             });
7018:         }
7019:         walkItems(item.children || item.items || [], `${itemPath}.children`, `${labelPrefix} /
${itemTitle}`);
7020:     });
7021:     };
7022:     walkItems(section.items || []);
7023: });
7024:
7025: return destinations;
7026: }

```

compileAppendDestinationOptions - lines 7028-7036

Item	Explanation
Source location	Lines 7028-7036 (9 source lines).
Purpose	Part of the Compile Code language/tool/build/run flow.
Declaration	function compileAppendDestinationOptions(project, selectedId = "") {
Touches	Mostly local calculations or local UI state.
Calls detected	compileAppendDestinations, some, map, escapeHtml, join
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

7028: function compileAppendDestinationOptions(project, selectedId = "") {
7029:     const destinations = compileAppendDestinations(project);
7030:     const selected = destinations.some((destination) => destination.id === selectedId)
7031:     ? selectedId
7032:     : destinations[0]?.id || "";
7033:     return destinations.map((destination) => `
7034:         <option value="${escapeHtml(destination.id)}"${destination.id === selected ? " selected" :
""}>${escapeHtml(destination.label)}</option>
7035:     `).join("");
7036: }

```

activeCompileFile - lines 7038-7041

Item	Explanation
Source location	Lines 7038-7041 (4 source lines).
Purpose	Part of the Compile Code language/tool/build/run flow.
Declaration	function activeCompileFile(project) {
Touches	Mostly local calculations or local UI state.
Calls detected	ensureCompileCode, find
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

7038: function activeCompileFile(project) {
7039:     const workspace = ensureCompileCode(project);
7040:     return workspace.files.find((file) => file.id === workspace.activeFileId) || workspace.files[0] ||
null;
7041: }

```

compileLogTimestamp - lines 7043-7047

Item	Explanation
Source location	Lines 7043-7047 (5 source lines).
Purpose	Part of the Compile Code language/tool/build/run flow.
Declaration	function compileLogTimestamp(value = new Date().toISOString()) {
Touches	Mostly local calculations or local UI state.
Calls detected	toISOString, isNaN, getTime, toLocaleTimeString
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
7043: function compileLogTimestamp(value = new Date().toISOString()) {
7044:   const date = new Date(value);
7045:   if (Number.isNaN(date.getTime())) return "";
7046:   return date.toLocaleTimeString([], { hour: "2-digit", minute: "2-digit", second: "2-digit" });
7047: }
```

addCompileMessage - lines 7049-7062

Item	Explanation
Source location	Lines 7049-7062 (14 source lines).
Purpose	Part of the Compile Code language/tool/build/run flow.
Declaration	function addCompileMessage(project, text, level = "info") {
Touches	Mostly local calculations or local UI state.
Calls detected	ensureCompileCode, slugify, now, random, toString, slice, toISOString, includes, updateCompileMessagesPanel
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
7049: function addCompileMessage(project, text, level = "info") {
7050:   const workspace = ensureCompileCode(project);
7051:   if (!workspace || !text) return;
7052:   workspace.messages = [
7053:     {
7054:       id: slugify(`message-${Date.now()}-${Math.random().toString(16).slice(2)}`),
7055:       at: new Date().toISOString(),
7056:       level: ["info", "success", "warning", "error"].includes(level) ? level : "info",
7057:       text: String(text)
7058:     },
7059:     ...(workspace.messages || [])
7060:   ].slice(0, 80);
7061:   updateCompileMessagesPanel(project);
7062: }
```

renderCompileMessages - lines 7064-7073

Item	Explanation
Source location	Lines 7064-7073 (10 source lines).
Purpose	Renders UI, markup, or display output.
Declaration	function renderCompileMessages(workspace) {

Item	Explanation
Touches	Mostly local calculations or local UI state.
Calls detected	isArray, map, escapeHtml, compileLogTimestamp, join
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

7064: function renderCompileMessages(workspace) {
7065:   const messages = Array.isArray(workspace?.messages) ? workspace.messages : [];
7066:   if (!messages.length) return `
```

updateCompileTerminalPanel - lines 7075-7084

Item	Explanation
Source location	Lines 7075-7084 (10 source lines).
Purpose	Updates UI, state, cache, or stored data.
Declaration	function updateCompileTerminalPanel(project, file = activeCompileFile(project)) {
Touches	browser/Electron DOM, filesystem
Calls detected	activeCompileFile, ensureCompileCode, querySelector, requestAnimationFrame
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

7075: function updateCompileTerminalPanel(project, file = activeCompileFile(project)) {
7076:   const workspace = ensureCompileCode(project);
7077:   const terminal = file?.lastResult?.terminal || workspace?.terminal || compileTerminalStatus ||
"Compiler output will appear here after you save or run a source file.";
7078:   const terminalElement = sectionContent.querySelector("[data-compile-terminal]");
7079:   if (!terminalElement) return;
7080:   terminalElement.textContent = terminal;
7081:   requestAnimationFrame(() => {
7082:     terminalElement.scrollTop = terminalElement.scrollHeight;
7083:   });
7084: }

```

updateCompileMessagesPanel - lines 7086-7091

Item	Explanation
Source location	Lines 7086-7091 (6 source lines).
Purpose	Updates UI, state, cache, or stored data.
Declaration	function updateCompileMessagesPanel(project) {
Touches	browser/Electron DOM
Calls detected	ensureCompileCode, querySelector, renderCompileMessages
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.

Item	Explanation
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

7086: function updateCompileMessagesPanel(project) {
7087:   const workspace = ensureCompileCode(project);
7088:   const log = sectionContent.querySelector("[data-compile-messages]");
7089:   if (!log) return;
7090:   log.innerHTML = renderCompileMessages(workspace);
7091: }

```

showCompileOutput - lines 7093-7105

Item	Explanation
Source location	Lines 7093-7105 (13 source lines).
Purpose	Part of the Compile Code language/tool/build/run flow.
Declaration	function showCompileOutput(project, file = activeCompileFile(project)) {
Touches	browser/Electron DOM, filesystem
Calls detected	activeCompileFile, updateCompileTerminalPanel, querySelector, add, scrollIntoView, requestAnimationFrame, remove
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

7093: function showCompileOutput(project, file = activeCompileFile(project)) {
7094:   updateCompileTerminalPanel(project, file);
7095:   const panel = sectionContent.querySelector("[data-compile-terminal-panel]");
7096:   const terminal = sectionContent.querySelector("[data-compile-terminal]");
7097:   if (!panel || !terminal) return;
7098:   panel.classList.add("is-focused");
7099:   panel.scrollIntoView({ behavior: "smooth", block: "center" });
7100:   requestAnimationFrame(() => {
7101:     terminal.focus?({ preventScroll: true });
7102:     terminal.scrollTop = terminal.scrollHeight;
7103:   });
7104:   window.setTimeout(() => panel.classList.remove("is-focused"), 1400);
7105: }

```

activeCompileWaveform - lines 7107-7110

Item	Explanation
Source location	Lines 7107-7110 (4 source lines).
Purpose	Part of the Compile Code language/tool/build/run flow.
Declaration	function activeCompileWaveform(project, file = activeCompileFile(project)) {
Touches	filesystem
Calls detected	activeCompileFile, ensureCompileCode
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

7107: function activeCompileWaveform(project, file = activeCompileFile(project)) {
7108:   const workspace = ensureCompileCode(project);

```

```

7109:   return file?.lastResult?.waveform || file?.waveform || workspace?.waveform || null;
7110: }

```

scopeSignalValueAt - lines 7112-7117

Item	Explanation
Source location	Lines 7112-7117 (6 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function scopeSignalValueAt(changes = [], index = 0) {
Touches	authentication/security data
Calls detected	toLowerCase
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 3 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

7112: function scopeSignalValueAt(changes = [], index = 0) {
7113:   const value = String(changes[index]?.value ?? "x").toLowerCase();
7114:   if (value === "1") return "1";
7115:   if (value === "0") return "0";
7116:   return "x";
7117: }

```

renderScalarWavePath - lines 7119-7140

Item	Explanation
Source location	Lines 7119-7140 (22 source lines).
Purpose	Renders UI, markup, or display output.
Declaration	function renderScalarWavePath(changes = [], maxTime = 1, rowY = 0, left = 150, width = 680) {
Touches	filesystem, authentication/security data
Calls detected	isArray, map, sort, max, min, forEach, xFor, yFor, scopeSignalValueAt, toFixed
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

7119: function renderScalarWavePath(changes = [], maxTime = 1, rowY = 0, left = 150, width = 680) {
7120:   const normalized = (Array.isArray(changes) && changes.length ? changes : [{ time: 0, value: "x" }])
7121:     .map((change) => ({ time: Number(change.time) || 0, value: String(change.value ?? "x") }))
7122:     .sort((a, b) => a.time - b.time);
7123:   const top = rowY + 5;
7124:   const bottom = rowY + 23;
7125:   const middle = rowY + 14;
7126:   const yFor = (value) => value === "1" ? top : value === "0" ? bottom : middle;
7127:   const xFor = (time) => left + Math.max(0, Math.min(1, (Number(time) || 0) / Math.max(1, maxTime))) *
width;
7128:   let path = "";
7129:   normalized.forEach((change, index) => {
7130:     const x = xFor(change.time);
7131:     const y = yFor(scopeSignalValueAt(normalized, index));
7132:     if (index === 0) {
7133:       path += `M ${x.toFixed(1)} ${y.toFixed(1)}`;
7134:     } else {
7135:       path += `H ${x.toFixed(1)} V ${y.toFixed(1)}`;
7136:     }
7137:   });
7138:   path += `H ${(left + width).toFixed(1)}`;
7139:   return path;
7140: }

```

yFor - lines 7126-7137

Item	Explanation
Source location	Lines 7126-7137 (12 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	const yFor = (value) => value === "1" ? top : value === "0" ? bottom : middle;
Touches	filesystem, authentication/security data
Calls detected	max, min, forEach, xFor, scopeSignalValueAt, toFixed
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
7126: const yFor = (value) => value === "1" ? top : value === "0" ? bottom : middle;
7127: const xFor = (time) => left + Math.max(0, Math.min(1, (Number(time) || 0) / Math.max(1, maxTime))) *
width;
7128: let path = "";
7129: normalized.forEach((change, index) => {
7130:   const x = xFor(change.time);
7131:   const y = yFor(scopeSignalValueAt(normalized, index));
7132:   if (index === 0) {
7133:     path += `M ${x.toFixed(1)} ${y.toFixed(1)}`;
7134:   } else {
7135:     path += `H ${x.toFixed(1)} V ${y.toFixed(1)}`;
7136:   }
7137: });
```

xFor - lines 7127-7137

Item	Explanation
Source location	Lines 7127-7137 (11 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	const xFor = (time) => left + Math.max(0, Math.min(1, (Number(time) 0) / Math.max(1, maxTime))) * width;
Touches	filesystem, authentication/security data
Calls detected	max, min, forEach, yFor, scopeSignalValueAt, toFixed
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
7127: const xFor = (time) => left + Math.max(0, Math.min(1, (Number(time) || 0) / Math.max(1, maxTime))) *
width;
7128: let path = "";
7129: normalized.forEach((change, index) => {
7130:   const x = xFor(change.time);
7131:   const y = yFor(scopeSignalValueAt(normalized, index));
7132:   if (index === 0) {
7133:     path += `M ${x.toFixed(1)} ${y.toFixed(1)}`;
7134:   } else {
7135:     path += `H ${x.toFixed(1)} V ${y.toFixed(1)}`;
7136:   }
7137: });
```

renderBusWaveLabels - lines 7142-7152

Item	Explanation
Source location	Lines 7142-7152 (11 source lines).

Item	Explanation
Purpose	Renders UI, markup, or display output.
Declaration	function renderBusWaveLabels(changes = [], maxTime = 1, rowY = 0, left = 150, width = 680) {
Touches	filesystem, authentication/security data
Calls detected	isArray, map, sort, slice, max, min, toFixed, escapeHtml, join
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

7142: function renderBusWaveLabels(changes = [], maxTime = 1, rowY = 0, left = 150, width = 680) {
7143:   const normalized = (Array.isArray(changes) && changes.length ? changes : [{ time: 0, value: "x" }])
7144:   .map((change) => ({ time: Number(change.time) || 0, value: String(change.value ?? "x") }))
7145:   .sort((a, b) => a.time - b.time)
7146:   .slice(0, 12);
7147:   return normalized.map((change, index) => {
7148:     const x = left + Math.max(0, Math.min(1, change.time / Math.max(1, maxTime))) * width;
7149:     const value = change.value.length > 12 ? `${change.value.slice(0, 12)}...` : change.value;
7150:     return `

```

renderHdlScope - lines 7154-7203

Item	Explanation
Source location	Lines 7154-7203 (50 source lines).
Purpose	Renders UI, markup, or display output.
Declaration	function renderHdlScope(waveform) {
Touches	filesystem, authentication/security data
Calls detected	isArray, slice, max, flatMap, map, round, toFixed, join, escapeHtml, renderScalarWavePath, renderBusWaveLabels
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 4 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

7154: function renderHdlScope(waveform) {
7155:   const signals = Array.isArray(waveform?.signals) ? waveform.signals.slice(0, 16) : [];
7156:   if (!signals.length) {
7157:     return `
7158:       <div class="compile-scope-empty">
7159:         <strong>No waveform scope yet.</strong>
7160:         <span>Run a Verilog/SystemVerilog design with a testbench that calls <code>$dumpfile</code> and
<code>$dumpvars</code>.</span>
7161:       </div>
7162:     `;
7163:   }
7164:   const left = 165;
7165:   const width = 760;
7166:   const rowHeight = 34;
7167:   const top = 28;
7168:   const height = top + signals.length * rowHeight + 28;
7169:   const maxTime = Math.max(1, Number(waveform.maxTime) || Math.max(...signals.flatMap((signal) =>
(signal.changes || []).map((change) => Number(change.time) || 0)), 1));
7170:   const grid = [0, 0.25, 0.5, 0.75, 1].map((ratio) => {
7171:     const x = left + ratio * width;
7172:     const label = Math.round(maxTime * ratio);
7173:     return `

```

```

7176:     const y = top + index * rowHeight;
7177:     const name = String(signal.name || signal.reference || `signal_${index + 1}`);
7178:     const changes = Array.isArray(signal.changes) ? signal.changes : [];
7179:     const scalar = Number(signal.width || 1) <= 1;
7180:     return `
7181:       <g class="scope-row">
7182:         <text x="12" y="${y + 20}" class="scope-signal-label">${escapeHtml(name.length > 24 ?
` ${name.slice(0, 24)}...` : name)}</text>
7183:         <line x1="${left}" x2="${left + width}" y1="${y + 27}" y2="${y + 27}" class="scope-row-line" />
7184:         ${scalar
7185:           ? `<path d="${renderScalarWavePath(changes, maxTime, y, left, width)}" class="scope-wave" />`
7186:           : `<line x1="${left}" x2="${left + width}" y1="${y + 15}" y2="${y + 15}" class="scope-bus"
/>${renderBusWaveLabels(changes, maxTime, y, left, width)}`}
7187:       </g>
7188:     `;
7189:   }).join("");
7190:   return `
7191:     <div class="compile-scope-meta">
7192:       <span>${escapeHtml(waveform.source || "waveform.vcd")}</span>
7193:       <span>${signals.length} signal${signals.length === 1 ? "" : "s"}</span>
7194:       <span>0 to ${escapeHtml(maxTime)} ${escapeHtml(waveform.timeScale || "ticks")}</span>
7195:     </div>
7196:     <div class="compile-scope-scroll">
7197:       <svg class="compile-scope-svg" viewBox="0 0 ${left + width + 20} ${height}" role="img" aria-
label="HDL waveform scope">
7198:         ${grid}
7199:         ${rows}
7200:       </svg>
7201:     </div>
7202:   `;
7203: }

```

renderCompileScopePanel - lines 7205-7217

Item	Explanation
Source location	Lines 7205-7217 (13 source lines).
Purpose	Renders UI, markup, or display output.
Declaration	function renderCompileScopePanel(project, file = activeCompileFile(project)) {
Touches	filesystem, authentication/security data
Calls detected	activeCompileFile, activeCompileWaveform, isHdlLanguage, renderHdlScope
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

7205: function renderCompileScopePanel(project, file = activeCompileFile(project)) {
7206:   const waveform = activeCompileWaveform(project, file);
7207:   if (!isHdlLanguage(file?.language) && !waveform) return "";
7208:   return `
7209:     <section class="compile-scope-panel" data-compile-scope-panel aria-label="HDL signal scope">
7210:       <div class="compile-terminal-heading">
7211:         <span class="compile-terminal-title"><span aria-hidden="true" class="scope-dot"></span> Signal
scope</span>
7212:         <button type="button" data-compile-show-scope>Show scope</button>
7213:       </div>
7214:       ${renderHdlScope(waveform)}
7215:     </section>
7216:   `;
7217: }

```

showCompileScope - lines 7219-7228

Item	Explanation
Source location	Lines 7219-7228 (10 source lines).
Purpose	Part of the Compile Code language/tool/build/run flow.
Declaration	function showCompileScope(project, file = activeCompileFile(project)) {
Touches	browser/Electron DOM, authentication/security data
Calls detected	activeCompileFile, querySelector, addCompileMessage, add, scrollIntoView, remove

Item	Explanation
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

7219: function showCompileScope(project, file = activeCompileFile(project)) {
7220:   const panel = sectionContent.querySelector("[data-compile-scope-panel]");
7221:   if (!panel) {
7222:     addCompileMessage(project, "No HDL scope is available for the active source yet.", "warning");
7223:     return;
7224:   }
7225:   panel.classList.add("is-focused");
7226:   panel.scrollIntoView({ behavior: "smooth", block: "center" });
7227:   window.setTimeout(() => panel.classList.remove("is-focused"), 1400);
7228: }

```

copyCompileOutput - lines 7230-7245

Item	Explanation
Source location	Lines 7230-7245 (16 source lines).
Purpose	Part of the Compile Code language/tool/build/run flow.
Declaration	async function copyCompileOutput(project, file = activeCompileFile(project)) {
Touches	filesystem
Calls detected	activeCompileFile, ensureCompileCode, trim, addCompileMessage, writeText, setStatus
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	1 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

7230: async function copyCompileOutput(project, file = activeCompileFile(project)) {
7231:   const workspace = ensureCompileCode(project);
7232:   const text = file?.lastResult?.terminal || workspace?.terminal || compileTerminalStatus || "";
7233:   if (!text.trim()) {
7234:     addCompileMessage(project, "There is no terminal output to copy yet.", "warning");
7235:     return;
7236:   }
7237:   try {
7238:     await navigator.clipboard.writeText(text);
7239:     addCompileMessage(project, "Terminal output copied to the clipboard.", "success");
7240:     setStatus("Terminal output copied.");
7241:   } catch {
7242:     addCompileMessage(project, "Terminal output could not be copied by this browser.", "warning");
7243:     setStatus("Terminal output could not be copied.");
7244:   }
7245: }

```

clearCompileOutput - lines 7247-7256

Item	Explanation
Source location	Lines 7247-7256 (10 source lines).
Purpose	Part of the Compile Code language/tool/build/run flow.
Declaration	function clearCompileOutput(project, file = activeCompileFile(project)) {
Touches	filesystem
Calls detected	activeCompileFile, ensureCompileCode, updateCompileTerminalPanel, addCompileMessage, setStatus, scheduleAutosave
API endpoints	None detected inside this function body.

Item	Explanation
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

7247: function clearCompileOutput(project, file = activeCompileFile(project)) {
7248:   const workspace = ensureCompileCode(project);
7249:   if (file?.lastResult) file.lastResult.terminal = "";
7250:   if (workspace) workspace.terminal = "";
7251:   compileTerminalStatus = "";
7252:   updateCompileTerminalPanel(project, file);
7253:   addCompileMessage(project, "Terminal output cleared.", "info");
7254:   setStatus("Terminal output cleared.");
7255:   scheduleAutosave();
7256: }

```

richBlocksForCompileAppend - lines 7258-7265

Item	Explanation
Source location	Lines 7258-7265 (8 source lines).
Purpose	Part of the Compile Code language/tool/build/run flow.
Declaration	function richBlocksForCompileAppend(project, destination) {
Touches	Mostly local calculations or local UI state.
Calls detected	getByPath, richBlocksFromValue, filter, trim
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 3 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

7258: function richBlocksForCompileAppend(project, destination) {
7259:   const existingRich = getByPath(project, destination.richPath);
7260:   const existingText = getByPath(project, destination.textPath) || "";
7261:   return richBlocksFromValue(existingRich, existingText).filter((block) => {
7262:     if (block.type !== "paragraph") return true;
7263:     return Boolean(String(block.text || "").trim() || block.html);
7264:   });
7265: }

```

codeBlockForCompileFile - lines 7267-7274

Item	Explanation
Source location	Lines 7267-7274 (8 source lines).
Purpose	Part of the Compile Code language/tool/build/run flow.
Declaration	function codeBlockForCompileFile(file) {
Touches	filesystem
Calls detected	normalizeCodeText, normalizeCodeLanguage, detectCodeLanguage
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

7267: function codeBlockForCompileFile(file) {

```

```

7268:   return {
7269:     code: normalizeCodeText(file?.code || "", "source"),
7270:     language: normalizeCodeLanguage(file?.language || detectCodeLanguage(file?.code || "", file?.fileName)
|| "")),
7271:     pasteMode: "source",
7272:     type: "code"
7273:   };
7274: }

```

compileFileDirtyLabel - lines 7276-7283

Item	Explanation
Source location	Lines 7276-7283 (8 source lines).
Purpose	Part of the Compile Code language/tool/build/run flow.
Declaration	function compileFileDirtyLabel(file) {
Touches	Mostly local calculations or local UI state.
Calls detected	No obvious named calls detected by the generator.
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 6 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

7276: function compileFileDirtyLabel(file) {
7277:   if (!file) return "";
7278:   if (file.dirty) return "Unsaved";
7279:   if (file.lastResult?.ok === true) return "Compiled";
7280:   if (file.lastResult?.ok === false) return "Needs fix";
7281:   if (file.savedAt) return "Saved";
7282:   return "Draft";
7283: }

```

renderCompileCodeSection - lines 7285-7408

Item	Explanation
Source location	Lines 7285-7408 (124 source lines).
Purpose	Renders UI, markup, or display output.
Declaration	function renderCompileCodeSection(project) {
Touches	filesystem, authentication/security data
Calls detected	ensureCompileCode, activeCompileFile, escapeHtml, map, match, replace, compileFileDirtyLabel, join, defaultCodeFileName, compileLanguageOptions, isHdlLanguage, compileRoleOptions
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

7285: function renderCompileCodeSection(project) {
7286:   const workspace = ensureCompileCode(project);
7287:   const activeFile = activeCompileFile(project);
7288:   const terminal = activeFile?.lastResult?.terminal || workspace.terminal || compileTerminalStatus ||
"Compiler output will appear here after you save or run a source file.";
7289:   return
7290:     <div class="compile-code-workspace">
7291:       <aside class="compile-code-sidebar" aria-label="Compile code files">
7292:         <div class="compile-code-sidebar-heading">
7293:           <h3>Compile Code</h3>
7294:           <small>${workspace.files.length} source file${workspace.files.length === 1 ? "" : "s"}</small>
7295:         </div>
7296:         <div class="compile-code-workspace-tree" aria-label="Project workspace tree">
7297:           <details open>
7298:             <summary>${escapeHtml(project.title || "Project workspace")}</summary>

```

```

7299:         <div class="compile-code-tree-files">
7300:             ${workspace.files.map((file) => `
7301:                 <button class="compile-code-tree-file${file.id} === workspace.activeFileId ? " is-active"
: "}" type="button" data-compile-select="${escapeHtml(file.id)}">
7302:                     <span class="compile-file-extension">${escapeHtml((file.fileName.match(/\.[^.]?$/)[0]
|| "src").replace(".", ""))}</span>
7303:                     <span>${escapeHtml(file.fileName)}</span>
7304:                     <small>${escapeHtml(compileFileDirtyLabel(file))}</small>
7305:                 </button>
7306:             `).join("") || `<p>No source files yet.</p>`}
7307:         </div>
7308:     </details>
7309: </div>
7310: <div class="compile-code-actions">
7311:     <button type="button" data-compile-add>Add code file</button>
7312:     <button type="button" data-compile-add-testbench>Add testbench</button>
7313:     <button type="button" data-compile-import>Import file</button>
7314:     <button type="button" data-compile-tools>Check compilers</button>
7315: </div>
7316: </aside>
7317:
7318: <section class="compile-code-main" aria-label="Compile code editor">
7319:     ${activeFile ?
7320:     <div class="compile-code-meta-grid">
7321:         <label>
7322:             <span>Title</span>
7323:             <input type="text" data-compile-field="title" value="${escapeHtml(activeFile.title || "")}"
placeholder="Source title" />
7324:         </label>
7325:         <label>
7326:             <span>File name</span>
7327:             <input type="text" data-compile-field="fileName" value="${escapeHtml(activeFile.fileName ||
"" )}" placeholder="${escapeHtml(defaultCodeFileName(activeFile.language))}" />
7328:         </label>
7329:         <label>
7330:             <span>Language</span>
7331:             <select data-compile-
field="language">${compileLanguageOptions(activeFile.language)}</select>
7332:         </label>
7333:         <label>
7334:             <span>${isHdlLanguage(activeFile.language) ? "HDL role" : "Source role"}</span>
7335:             ${isHdlLanguage(activeFile.language)
? `<select data-compile-field="role">${compileRoleOptions(activeFile.role)}</select>`
: `<input type="text" value="Program source" disabled />`}
7336:         </label>
7337:     </div>
7338:     <div class="compile-code-editor-grid compile-code-editor-grid-single">
7339:         <section class="compile-code-preview-panel compile-code-active-panel" aria-label="Active
syntax highlighted code editor">
7340:             <div class="compile-code-preview-heading">
7341:                 <span>${escapeHtml(codeLanguageLabel(activeFile.language))} active editor</span>
7342:             </div>
7343:             <div class="compile-code-active-editor">
7344:                 <pre class="compile-code-preview" aria-hidden="true"><code data-compile-
preview>${tokenizedCodeHtml(activeFile.code || "", activeFile.language)}</code></pre>
7345:                 <textarea data-compile-field="code" data-compile-active-editor spellcheck="false"
wrap="off" rows="18" placeholder="Type or paste code here">${escapeHtml(activeFile.code || "")}</textarea>
7346:             </div>
7347:         </section>
7348:     </div>
7349:     <div class="compile-stdin-field">
7350:         <span>Program input, optional</span>
7351:         <textarea data-compile-field="stdin" rows="4" placeholder="Input passed to stdin when the
code runs">${escapeHtml(activeFile.stdin || "")}</textarea>
7352:     </div>
7353:     <div class="compile-code-command-bar">
7354:         <button type="button" data-compile-save>Save source</button>
7355:         <button type="button" data-compile-beautify>Beautify</button>
7356:         <button type="button" data-compile-compile>Compile</button>
7357:         <button type="button" data-compile-build>Build project</button>
7358:         ${isHdlLanguage(activeFile.language)
? `<button type="button" data-compile-simulate>Simulate</button>`
: `<button type="button" data-compile-run>Run</button>`}
7359:         <button class="compile-output-button" type="button" data-compile-show-output title="Open the
terminal output for this source"><span class="compile-command-icon" aria-hidden="true">&gt;</span><span>Show
output</span></button>
7360:         ${isHdlLanguage(activeFile.language) ? `<button class="compile-output-button" type="button"
data-compile-show-scope title="Open the HDL waveform scope"><span class="compile-command-icon" aria-
hidden="true"></span><span>Show scope</span></button>` : ""}
7361:         <button type="button" data-compile-install>Install tools</button>
7362:         <button class="danger-icon" type="button" data-compile-delete>Delete source</button>
7363:     </div>
7364:     <section class="compile-append-panel" aria-label="Append code to project section">
7365:         <div>
7366:             <strong>Append code to project</strong>
7367:             <span>Insert the active source as a formatted code block into this project.</span>
7368:         </div>
7369:         <label>
7370:             <span>Destination</span>
7371:             <select data-compile-append-target>${compileAppendDestinationOptions(project,
workspace.appendTargetId)}</select>
7372:         </label>

```

```

7377:         <div class="compile-append-actions">
7378:             <button type="button" data-compile-append-code>Append code to project</button>
7379:         </div>
7380:     </section>
7381:     `
7382:     <div class="compile-code-empty">
7383:         <h3>No compile file selected</h3>
7384:         <p>Add a source file for C, C++, SystemVerilog, LTspice, Java, JavaScript, Python, or
HTML.</p>
7385:     </div>
7386: `
7387: <section class="compile-terminal-panel" data-compile-terminal-panel aria-label="Compiler terminal
output">
7388:     <div class="compile-terminal-heading">
7389:         <span class="compile-terminal-title"><span aria-hidden="true" class="terminal-dot"></span>
OMB Local Terminal</span>
7390:         <span class="compile-terminal-controls">
7391:             <button type="button" data-compile-copy-output>Copy output</button>
7392:             <button type="button" data-compile-clear-output>Clear output</button>
7393:         </span>
7394:     </div>
7395:     <pre class="compile-terminal" data-compile-terminal tabindex="0" role="log" aria-
live="polite">${escapeHtml(terminal)}</pre>
7396: </section>
7397: ${renderCompileScopePanel(project, activeFile)}
7398: <section class="compile-messages-panel" aria-label="Compile messages log">
7399:     <div class="compile-terminal-heading">
7400:         <span>Messages log</span>
7401:         <button type="button" data-compile-clear-messages>Clear log</button>
7402:     </div>
7403:     <div class="compile-messages-log" data-compile-
messages>${renderCompileMessages(workspace)}</div>
7404: </section>
... excerpt truncated after 120 lines. Open the source file for the rest of this function.

```

renderSectionContent - lines 7410-7472

Item	Explanation
Source location	Lines 7410-7472 (63 source lines).
Purpose	Renders UI, markup, or display output.
Declaration	function renderSectionContent(project) {
Touches	Mostly local calculations or local UI state.
Calls detected	sectionOptions, find, startsWith, renderCustomSection, replace, requestAnimationFrame, populateRichEditors, isAnalogProject, renderAnalogDesignWorkspace, renderObjectSection, filter, join
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 5 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

7410: function renderSectionContent(project) {
7411:     if (!project) {
7412:         sectionContent.hidden = true;
7413:         sectionContent.innerHTML = "";
7414:         return;
7415:     }
7416:
7417:     const section = sectionOptions(project).find((item) => item.id === activeSectionId) ||
standardSections[0];
7418:     const isOverview = section.id === "brief";
7419:     projectFields.hidden = !isOverview;
7420:     sectionContent.hidden = isOverview;
7421:     if (isOverview) {
7422:         sectionContent.innerHTML = "";
7423:         return;
7424:     }
7425:
7426:     if (section.id.startsWith("custom:")) {
7427:         sectionContent.innerHTML = renderCustomSection(project, section.id.replace("custom:", ""));
7428:         requestAnimationFrame(() => populateRichEditors(sectionContent));
7429:         return;
7430:     }
7431:
7432:     const renderers = {
7433:         brief: () => "",
7434:         design: () => {
7435:             if (isAnalogProject(project)) return renderAnalogDesignWorkspace(project);

```

```

7436:     const cards = [
7437:       (project.documents || []).length ? `<section><h3>Documents</h3>${renderObjectSection(project,
"documents", "documents", "Documents")}</section>` : "",
7438:       (project.pcbcs || []).length ? `<section><h3>PCBs Built</h3>${renderObjectSection(project, "pcbcs",
"pcbcs", "PCBs")}</section>` : "",
7439:       (project.media || []).length ? `<section><h3>Images</h3>${renderObjectSection(project, "media",
"images", "Images")}</section>` : ""
7440:     ].filter(Boolean);
7441:     return `
7442:       <div class="section-stack">
7443:         ${cards.join("") || renderEmptyDynamicSectionPrompt("design")}
7444:         ${renderPendingEditor()}
7445:       </div>
7446:     `;
7447:   },
7448:   simulation: () => isAnalogProject(project) ? renderAnalogSimulationWorkspace(project) : "",
7449:   "compile-code": () => renderCompileCodeSection(project),
7450:   tests: () => `${renderObjectSection(project, "tests", "tests", "Tests")}${renderPendingEditor()}`,
7451:   tools: () => `
7452:     <div class="section-stack">
7453:       <section>
7454:         <h3>Tools Used</h3>
7455:         ${renderToolsSection(project)}
7456:       </section>
7457:       <section>
7458:         <h3>Languages</h3>
7459:         ${renderTextArray(project, "languages", "Language")}
7460:       </section>
7461:       <section>
7462:         <h3>Links</h3>
7463:         ${renderObjectSection(project, "links", "", "Links")}
7464:       </section>
7465:       ${renderPendingEditor()}
7466:     </div>
7467:   `;
7468:   });
7469:
7470:   sectionContent.innerHTML = renderers[section.id]();
7471:   requestAnimationFrame(() => populateRichEditors(sectionContent));
7472: }

```

fullPortfolioPreviewHtml - lines 7474-7476

Item	Explanation
Source location	Lines 7474-7476 (3 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function fullPortfolioPreviewHtml() {
Touches	Mostly local calculations or local UI state.
Calls detected	fullPortfolioPreviewHtmlExact
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

7474: function fullPortfolioPreviewHtml() {
7475:   return fullPortfolioPreviewHtmlExact();
7476: }

```

previewValue - lines 7478-7480

Item	Explanation
Source location	Lines 7478-7480 (3 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function previewValue(source, key, fallback) {
Touches	Mostly local calculations or local UI state.
Calls detected	call

Item	Explanation
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

7478: function previewValue(source, key, fallback) {
7479:   return source && Object.prototype.hasOwnProperty.call(source, key) ? source[key] : fallback;
7480: }

```

fullPortfolioPreviewHtmlExact - lines 7482-7702

Item	Explanation
Source location	Lines 7482-7702 (221 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function fullPortfolioPreviewHtmlExact(dataOverride = null) {
Touches	filesystem, network/API requests, authentication/security data
Calls detected	parsedPortfolioGlobals, normalizeSiteContent, previewValue, normalizeProfile, normalizeFieldStyles, stringify, replaceAll, escapeHtml, plainFieldStyleAttribute, renderRichFieldContent, getFullYear
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

7482: function fullPortfolioPreviewHtmlExact(dataOverride = null) {
7483:   const baseHref = `${window.location.origin}/`;
7484:   const parsedGlobals = dataOverride ? null : parsedPortfolioGlobals();
7485:   const siteContent = normalizeSiteContent(previewValue(dataOverride, "siteContent",
parsedGlobals?.siteContent || {}));
7486:   const profile = normalizeProfile(previewValue(dataOverride, "profile", parsedGlobals?.profile || {}));
7487:   const fieldStyles = normalizeFieldStyles(previewValue(dataOverride, "fieldStyles",
parsedGlobals?.fieldStyles || {}));
7488:   const profileRich = previewValue(dataOverride, "profileRich", parsedGlobals?.profileRich || {});
7489:   const siteContentRich = previewValue(dataOverride, "siteContentRich", parsedGlobals?.siteContentRich ||
{});
7490:   const ownerName = profile.displayName || "Portfolio";
7491:   const portfolioLabel = profile.portfolioLabel || "Portfolio";
7492:   const previewDataObject = {
7493:     categories: previewValue(dataOverride, "categories", parsedGlobals?.categories || []),
7494:     fieldStyles,
7495:     funFacts: previewValue(dataOverride, "funFacts", parsedGlobals?.funFacts || []),
7496:     funFactsRich: previewValue(dataOverride, "funFactsRich", parsedGlobals?.funFactsRich || null),
7497:     profile,
7498:     profileRich,
7499:     projects: previewValue(dataOverride, "projects", savedPortfolioCatalog.projects || []),
7500:     siteContent,
7501:     siteContentRich,
7502:     siteSections: previewValue(dataOverride, "siteSections", parsedGlobals?.siteSections || [])
7503:   };
7504:   const previewData = JSON.stringify(previewDataObject).replaceAll("</", "<\\");
7505:
7506:   return `<!DOCTYPE html>
7507: <html lang="en">
7508:   <head>
7509:     <meta charset="UTF-8" />
7510:     <meta name="viewport" content="width=device-width, initial-scale=1.0" />
7511:     <base href="${baseHref}" />
7512:     <meta name="description" content="A portfolio website built with OMB Portfolio Builder for projects,
documents, diagrams, source code, tests, links, and profile information." />
7513:     <meta name="robots" content="index, follow" />
7514:     <meta name="author" content="${escapeHtml(ownerName)}" />
7515:     <meta name="portfolio-ai-endpoint" content="https://omb-portfolio-ai.maurice-baraza-
otieno.workers.dev/api/portfolio-ai" />
7516:     <title>${escapeHtml(ownerName)} | ${escapeHtml(portfolioLabel)}</title>
7517:     <link rel="stylesheet" href="styles.css" />
7518:   </head>
7519:   <body>

```

```

7520:     <header class="site-header" id="top">
7521:         <a class="brand" href="#top" aria-label="Go to top">
7522:             <span class="brand-lockup" aria-hidden="true">
7523:                 <img class="brand-icon" alt="" hidden />
7524:                 <span class="omb-engraving">${escapeHtml(profile.brandText || "Portfolio")}</span>
7525:             </span>
7526:             <span>
7527:                 <strong${plainFieldStyleAttribute(fieldStyles, "profile-display-
name")}>${renderRichFieldContent(profileRich.displayName, ownerName)}</strong>
7528:                 <small${plainFieldStyleAttribute(fieldStyles, "profile-portfolio-
label")}>${renderRichFieldContent(profileRich.portfolioLabel, portfolioLabel)}</small>
7529:             </span>
7530:         </a>
7531:
7532:         <div class="header-right">
7533:             <nav class="site-nav" aria-label="Primary navigation">
7534:                 <a href="#ask-ai">Ask AI</a>
7535:                 <a href="#projects">Projects</a>
7536:                 <a href="#resume">Resume</a>
7537:                 <a href="#process">Process</a>
7538:                 <a href="#contact">Contact</a>
7539:             </nav>
7540:             <a class="header-avatar" href="#contact" aria-label="Go to contact information" hidden>
7541:                 <img alt="" />
7542:             </a>
7543:         </div>
7544:     </header>
7545:
7546:     <main>
7547:         <section class="hero" aria-labelledby="hero-title">
7548:             <img class="hero-image" alt="" hidden />
7549:             <div class="hero-shade" aria-hidden="true"></div>
7550:             <div class="hero-content">
7551:                 <p class="eyebrow">${plainFieldStyleAttribute(fieldStyles, "site-hero-
eyebrow")}>${renderRichFieldContent(siteContentRich.heroEyebrow, siteContent.heroEyebrow)}</p>
7552:                 <h1 id="hero-title">${plainFieldStyleAttribute(fieldStyles, "site-hero-
title")}>${renderRichFieldContent(siteContentRich.heroTitle, siteContent.heroTitle)}</h1>
7553:                 <p class="hero-copy">${plainFieldStyleAttribute(fieldStyles, "site-hero-
copy")}>${renderRichFieldContent(siteContentRich.heroCopy, siteContent.heroCopy)}</p>
7554:                 <div class="hero-actions">
7555:                     <a class="button primary" href="#projects">View projects</a>
7556:                     <a class="button secondary" href="#resume" hidden>View resume</a>
7557:                     <a class="button secondary" href="#" target="_blank" rel="noreferrer" hidden>GitHub
profile</a>
7558:                     <a class="ai-jump-link" href="#ask-ai" aria-label="Jump to the portfolio AI assistant">
7559:                         <svg viewBox="0 0 24 24" aria-hidden="true">
7560:                             <path d="M12 3v3m-5 7H4m16 0h-3M7.5 7.5 5.4 5.4m11.1 2.1 2.1-2.1M8 10h8v7a2 2 0 0 1-2
2h-4a2 2 0 0 1-2-2z" />
7561:                             <path d="M10 13h.01M14 13h.01M10 16h4" />
7562:                         </svg>
7563:                         <span>Ask AI</span>
7564:                     </a>
7565:                 </div>
7566:             </div>
7567:         </section>
7568:
7569:         <section class="fun-facts-section" id="fun-facts-callout" aria-label="Fun facts" hidden></section>
7570:
7571:         <section class="builder-download-section" aria-label="Download portfolio builder">
7572:             <a class="builder-download-link" href="https://github.com/otienomaurice/omb-portfolio-
builder/releases/latest/download/OMB-Portfolio-Builder-Setup-latest-x64.exe" download>
7573:                 <svg viewBox="0 0 24 24" aria-hidden="true">
7574:                     <path d="M12 3v11m0 0 4-4m-4 4-4-4" />
7575:                     <path d="M5 17v2h14v-2" />
7576:                 </svg>
7577:                 <span>Download builder application</span>
7578:             </a>
7579:         </section>
7580:
7581:         <section class="summary-band" aria-label="Portfolio highlights">
7582:             <div class="metric">
7583:                 <strong id="project-count">0</strong>
7584:                 <span>presented projects</span>
7585:             </div>
7586:             <div class="metric">
7587:                 <strong id="project-track-count">0 Tracks</strong>
7588:                 <span id="project-track-labels">project categories</span>
7589:             </div>
7590:             <div class="metric">
7591:                 <strong>Artifacts</strong>
7592:                 <span>source, diagrams, and reports</span>
7593:             </div>
7594:         </section>
7595:
7596:         <section class="section" id="projects" aria-labelledby="projects-title">
7597:             <div class="section-heading">
7598:                 <div>
7599:                     <p class="eyebrow">Selected work</p>
7600:                     <h2 id="projects-title">Engineering Projects</h2>
7601:                 </div>

```

... excerpt truncated after 120 lines. Open the source file for the rest of this function.

renderPreview - lines 7704-7706

Item	Explanation
Source location	Lines 7704-7706 (3 source lines).
Purpose	Renders UI, markup, or display output.
Declaration	function renderPreview() {
Touches	Mostly local calculations or local UI state.
Calls detected	fullPortfolioPreviewHtmlExact
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
7704: function renderPreview() {
7705:   portfolioPreviewFrame.srcdoc = fullPortfolioPreviewHtmlExact();
7706: }
```

openPortfolioPreview - lines 7708-7713

Item	Explanation
Source location	Lines 7708-7713 (6 source lines).
Purpose	Controls a window, dialog, panel, menu, or visible state.
Declaration	async function openPortfolioPreview() {
Touches	browser/Electron DOM
Calls detected	saveAllSections, renderPreview, add, showModal
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	1 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
7708: async function openPortfolioPreview() {
7709:   if (!(await saveAllSections({ refreshOpenPreviews: false }))) return;
7710:   renderPreview();
7711:   document.body.classList.add("full-window-open");
7712:   portfolioPreviewDialog.showModal();
7713: }
```

renderAll - lines 7715-7732

Item	Explanation
Source location	Lines 7715-7732 (18 source lines).
Purpose	Renders UI, markup, or display output.
Declaration	function renderAll() {
Touches	Mostly local calculations or local UI state.
Calls detected	ensureSiteSections, selectedProject, sectionOptions, some, renderFunFactsEditor, renderSiteContentEditor, renderProfileEditor, renderSiteSectionList, renderTree, renderFields, renderSectionTabs, renderSectionContent
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 0 return statement(s).

Item	Explanation
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

7715: function renderAll() {
7716:   ensureSiteSections();
7717:   if (!selectedProjectId && catalog.projects.length) selectedProjectId = catalog.projects[0].id;
7718:   const project = selectedProject();
7719:   if (project && !sectionOptions(project).some((section) => section.id === activeSectionId)) {
7720:     activeSectionId = "brief";
7721:   }
7722:   renderFunFactsEditor();
7723:   renderSiteContentEditor();
7724:   renderProfileEditor();
7725:   renderSiteSectionList();
7726:   renderTree();
7727:   renderFields(project);
7728:   renderSectionTabs(project);
7729:   renderSectionContent(project);
7730:   if (portfolioPreviewDialog?.open) renderPreview();
7731:   updateBuilderWorkflow();
7732: }

```

updateProjectField - lines 7734-7768

Item	Explanation
Source location	Lines 7734-7768 (35 source lines).
Purpose	Updates UI, state, cache, or stored data.
Declaration	function updateProjectField(field, value) {
Touches	browser/Electron DOM
Calls detected	selectedProject, split, map, trim, filter, limitWords, slugify, includes, setStatus, scheduleAutosave, scheduleChromeRender, schedulePreviewRender
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

7734: function updateProjectField(field, value) {
7735:   const project = selectedProject();
7736:   if (!project) return;
7737:
7738:   let needsChromeRender = false;
7739:   let statusMessage = "Unsaved local changes.";
7740:   if (field === "focus") {
7741:     project.focus = value.split(",").map((item) => item.trim()).filter(Boolean);
7742:   } else if (field === "summary") {
7743:     const limited = limitWords(value, 1000);
7744:     project.summary = limited;
7745:     if (limited !== value && document.activeElement?.dataset?.field === "summary") {
7746:       document.activeElement.value = limited;
7747:       statusMessage = "Project overview is limited to 1000 words.";
7748:     }
7749:   } else if (field === "id") {
7750:     const nextId = slugify(value);
7751:     if (!nextId) return;
7752:     project.id = nextId;
7753:     selectedProjectId = nextId;
7754:     needsChromeRender = true;
7755:   } else {
7756:     project[field] = value;
7757:     needsChromeRender = ["title", "category", "status"].includes(field);
7758:   }
7759:
7760:   if (field === "title") projectWindowTitle.textContent = value || "Untitled project";
7761:   setStatus(statusMessage);
7762:   scheduleAutosave();
7763:   if (needsChromeRender) {
7764:     scheduleChromeRender();
7765:   } else {
7766:     schedulePreviewRender();
7767:   }

```

7768: }

renameSelectedProject - lines 7770-7775

Item	Explanation
Source location	Lines 7770-7775 (6 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function renameSelectedProject() {
Touches	Mostly local calculations or local UI state.
Calls detected	selectedProject, beginTitleRename
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
7770: function renameSelectedProject() {
7771:   const project = selectedProject();
7772:   if (!project) return;
7773:   projectTitleMenu.hidden = true;
7774:   beginTitleRename();
7775: }
```

projectMetaRows - lines 7777-7813

Item	Explanation
Source location	Lines 7777-7813 (37 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function projectMetaRows(project) {
Touches	Mostly local calculations or local UI state.
Calls detected	categoryById, projectTemplateFor, find, flatMap, filter, toLocaleString
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
7777: function projectMetaRows(project) {
7778:   if (!project) return [];
7779:   const category = categoryById(project.category);
7780:   const template = projectTemplateFor(project);
7781:   const savedProject = (savedPortfolioCatalog.projects || []).find((item) => item.id === project.id);
7782:   const design = project.design || {};
7783:   const designFileCount = [
7784:     ...(design.brief?.files || []),
7785:     ...(design.documentation?.files || []),
7786:     ...(design.documentation?.references || []),
7787:     ...(design.documentation?.mathAnalysis || []),
7788:     ...(design.simulation?.files || []),
7789:     ...(design.simulation?.results || [])
7790:   ].length;
7791:   const uploadedFileCount = [
7792:     ...(project.documents || []),
7793:     ...(project.tests || []),
7794:     ...(project.pcbbs || []),
7795:     ...(project.media || []),
7796:     ...(project.links || []),
7797:     ...(project.sections || []).flatMap((section) => section.items || [])
7798:   ].filter((item) => item.url || item.artifact).length + designFileCount;
7799:   return [
7800:     ["Title", project.title || "Untitled project"],
7801:     ["Project ID / folder", project.id || "Not assigned"],
```

```

7802:    ["Category", category.label || project.category || "Not assigned"],
7803:    ["Status", project.status || "Draft"],
7804:    ["Appearance", template.label || project.templateLabel || "No appearance - white project layout"],
7805:    ["Appearance ID", project.templateId || "None"],
7806:    ["Portfolio preview", savedProject ? "Saved into parsed portfolio preview" : "Not yet saved into
portfolio preview"],
7807:    ["Custom sections", String((project.sections || []).length)],
7808:    ["Project files / links", String(uploadedFileCount)],
7809:    ["Tools", String((project.tools || []).length)],
7810:    ["Languages", String((project.languages || []).length)],
7811:    ["Last parsed", savedProject?.portfolioView?.builtAt ? new
Date(savedProject.portfolioView.builtAt).toLocaleString() : "Not parsed yet"]
7812:  ];
7813: }

```

savedProject - lines 7781-7782

Item	Explanation
Source location	Lines 7781-7782 (2 source lines).
Purpose	Persists data to local state, disk, credentials, or browser storage.
Declaration	<code>const savedProject = (savedPortfolioCatalog.projects []).find((item) => item.id === project.id);</code>
Touches	Mostly local calculations or local UI state.
Calls detected	<code>find</code>
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

7781:    const savedProject = (savedPortfolioCatalog.projects || []).find((item) => item.id === project.id);
7782:    const design = project.design || {};

```

openProjectMetaDetails - lines 7815-7827

Item	Explanation
Source location	Lines 7815-7827 (13 source lines).
Purpose	Controls a window, dialog, panel, menu, or visible state.
Declaration	<code>function openProjectMetaDetails() {</code>
Touches	Mostly local calculations or local UI state.
Calls detected	<code>selectedProject</code> , <code>projectMetaRows</code> , <code>map</code> , <code>join</code> , <code>showModal</code>
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

7815: function openProjectMetaDetails() {
7816:   const project = selectedProject();
7817:   if (!project) return;
7818:   projectTitleMenu.hidden = true;
7819:   projectMetaTitle.textContent = project.title || "Project details";
7820:   projectMetaGrid.innerHTML = projectMetaRows(project).map(([label, value]) => `
7821:     <article>
7822:       <span>${label}</span>
7823:       <strong>${value}</strong>
7824:     </article>
7825: `).join("");
7826:   if (!projectMetaDialog.open) projectMetaDialog.showModal();
7827: }

```

selectTitleText - lines 7829-7835

Item	Explanation
Source location	Lines 7829-7835 (7 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function selectTitleText() {
Touches	browser/Electron DOM
Calls detected	getSelection, createRange, selectNodeContents, removeAllRanges, addRange
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
7829: function selectTitleText() {
7830:   const selection = window.getSelection();
7831:   const range = document.createRange();
7832:   range.selectNodeContents(projectWindowTitle);
7833:   selection.removeAllRanges();
7834:   selection.addRange(range);
7835: }
```

beginTitleRename - lines 7837-7846

Item	Explanation
Source location	Lines 7837-7846 (10 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function beginTitleRename() {
Touches	Mostly local calculations or local UI state.
Calls detected	selectedProject, focus, selectTitleText, setStatus
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
7837: function beginTitleRename() {
7838:   const project = selectedProject();
7839:   if (!project) return;
7840:   originalTitleDraft = project.title || "";
7841:   projectWindowTitle.contentEditable = "true";
7842:   projectWindowTitle.focus();
7843:   selectTitleText();
7844:   titleRenameActions.hidden = false;
7845:   setStatus("Edit the title, then click Save title.");
7846: }
```

endTitleRename - lines 7848-7865

Item	Explanation
Source location	Lines 7848-7865 (18 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function endTitleRename(saveChanges) {
Touches	Mostly local calculations or local UI state.

Item	Explanation
Calls detected	selectedProject, trim, getSelection, removeAllRanges, setStatus, scheduleAutosave, scheduleChromeRender
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

7848: function endTitleRename(saveChanges) {
7849:   const project = selectedProject();
7850:   if (!project) return;
7851:   const nextTitle = projectWindowTitle.textContent.trim();
7852:   projectWindowTitle.contentEditable = "false";
7853:   titleRenameActions.hidden = true;
7854:   window.getSelection()?.removeAllRanges();
7855:   if (saveChanges && nextTitle) {
7856:     project.title = nextTitle;
7857:     projectWindowTitle.textContent = nextTitle;
7858:     setStatus("Project title saved in the editor. Click Save project to include this version in the
portfolio preview.");
7859:     scheduleAutosave();
7860:     scheduleChromeRender();
7861:   } else {
7862:     projectWindowTitle.textContent = originalTitleDraft || project.title || "Untitled project";
7863:     setStatus("Title edit canceled.");
7864:   }
7865: }

```

makeItemForSection - lines 7867-7874

Item	Explanation
Source location	Lines 7867-7874 (8 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function makeItemForSection(key, title, description = "", url = "") {
Touches	Mostly local calculations or local UI state.
Calls detected	No obvious named calls detected by the generator.
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 6 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

7867: function makeItemForSection(key, title, description = "", url = "") {
7868:   if (key === "documents") return { title, type: "Document", url, status: url ? "uploaded" : "draft" };
7869:   if (key === "tests") return { name: title, method: description || "Validation notes", status: url ?
"uploaded" : "draft", artifact: url, result: description };
7870:   if (key === "pcbs") return { name: title, revision: "Rev A", status: url ? "uploaded" : "draft",
artifact: url };
7871:   if (key === "media") return { title, url, status: url ? "uploaded" : "draft", caption: description };
7872:   if (key === "links") return { label: title, url };
7873:   return { title, description, url, status: url ? "uploaded" : "draft" };
7874: }

```

addTextItem - lines 7876-7891

Item	Explanation
Source location	Lines 7876-7891 (16 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function addTextItem(key) {
Touches	Mostly local calculations or local UI state.

Item	Explanation
Calls detected	selectedProject, prompt, includes, push, makeItemForSection, setStatus, scheduleAutosave, renderAll
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

7876: function addTextItem(key) {
7877:   const project = selectedProject();
7878:   const title = prompt("Type a title for this item.");
7879:   if (!project || !title) return;
7880:
7881:   project[key] = project[key] || [];
7882:   if (["highlights", "tools", "languages"].includes(key)) {
7883:     project[key].push(title);
7884:   } else {
7885:     const description = prompt("Add a short description if you want.") || "";
7886:     project[key].push(makeItemForSection(key, title, description));
7887:   }
7888:   setStatus("Item added in the editor. Click Save project to include this version in the portfolio
preview.");
7889:   scheduleAutosave();
7890:   renderAll();
7891: }

```

editObjectItem - lines 7893-7914

Item	Explanation
Source location	Lines 7893-7914 (22 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function editObjectItem(key, index) {
Touches	Mostly local calculations or local UI state.
Calls detected	selectedProject, prompt, setStatus, scheduleAutosave, renderAll
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

7893: function editObjectItem(key, index) {
7894:   const project = selectedProject();
7895:   const item = project?.[key]?.[index];
7896:   if (!item) return;
7897:
7898:   const currentTitle = item.title || item.name || item.label || "";
7899:   const nextTitle = prompt("Edit title.", currentTitle);
7900:   if (!nextTitle) return;
7901:   const nextDescription = prompt("Edit description.", item.description || item.caption || item.result ||
item.method || "") || "";
7902:
7903:   if ("title" in item) item.title = nextTitle;
7904:   if ("name" in item) item.name = nextTitle;
7905:   if ("label" in item) item.label = nextTitle;
7906:   if ("caption" in item) item.caption = nextDescription;
7907:   if ("result" in item) item.result = nextDescription;
7908:   if ("description" in item) item.description = nextDescription;
7909:   if (!("caption" in item) && !("result" in item) && !("description" in item)) item.description =
nextDescription;
7910:
7911:   setStatus("Item edited in the editor. Click Save project to include this version in the portfolio
preview.");
7912:   scheduleAutosave();
7913:   renderAll();
7914: }

```

chooseFile - lines 7916-7922

Item	Explanation
Source location	Lines 7916-7922 (7 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function chooseFile() {
Touches	Mostly local calculations or local UI state.
Calls detected	resolve, click
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
7916: function chooseFile() {
7917:   return new Promise((resolve) => {
7918:     filePicker.value = "";
7919:     filePicker.onchange = () => resolve(filePicker.files[0] || null);
7920:     filePicker.click();
7921:   });
7922: }
```

newCompileFile - lines 7924-7942

Item	Explanation
Source location	Lines 7924-7942 (19 source lines).
Purpose	Part of the Compile Code language/tool/build/run flow.
Declaration	function newCompileFile(language = "javascript", seed = {}) {
Touches	filesystem
Calls detected	normalizeCodeLanguage, safeClientCodeFileName, defaultCodeFileName, normalizeCompileFileRole, inferCompileFileRole, slugify, now, random, toString, slice
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
7924: function newCompileFile(language = "javascript", seed = {}) {
7925:   const normalized = normalizeCodeLanguage(language);
7926:   const fileName = safeClientCodeFileName(seed.fileName || defaultCodeFileName(normalized), normalized);
7927:   const role = normalizeCompileFileRole(seed.role || inferCompileFileRole(fileName, seed.code || "",
normalized), normalized);
7928:   return {
7929:     id: slugify(`${fileName}-${Date.now()}-${Math.random().toString(16).slice(2)}`),
7930:     title: seed.title || fileName,
7931:     fileName,
7932:     language: normalized,
7933:     role,
7934:     code: String(seed.code || ""),
7935:     stdin: "",
7936:     savedPath: "",
7937:     savedAt: "",
7938:     waveform: null,
7939:     lastResult: null,
7940:     dirty: true
7941:   };
7942: }
```

activeCompileWorkspaceAndFile - lines 7944-7947

Item	Explanation
Source location	Lines 7944-7947 (4 source lines).
Purpose	Part of the Compile Code language/tool/build/run flow.
Declaration	function activeCompileWorkspaceAndFile(project = selectedProject()) {
Touches	Mostly local calculations or local UI state.
Calls detected	selectedProject, ensureCompileCode, activeCompileFile
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
7944: function activeCompileWorkspaceAndFile(project = selectedProject()) {
7945:   const workspace = ensureCompileCode(project);
7946:   return { workspace, file: activeCompileFile(project) };
7947: }
```

updateCompilePreview - lines 7949-7952

Item	Explanation
Source location	Lines 7949-7952 (4 source lines).
Purpose	Updates UI, state, cache, or stored data.
Declaration	function updateCompilePreview(file) {
Touches	browser/Electron DOM, authentication/security data
Calls detected	querySelector, tokenizedCodeHtml
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
7949: function updateCompilePreview(file) {
7950:   const preview = sectionContent.querySelector("[data-compile-preview]");
7951:   if (preview && file) preview.innerHTML = tokenizedCodeHtml(file.code || "", file.language);
7952: }
```

syncCompileEditorScroll - lines 7954-7960

Item	Explanation
Source location	Lines 7954-7960 (7 source lines).
Purpose	Part of the Compile Code language/tool/build/run flow.
Declaration	function syncCompileEditorScroll(textarea) {
Touches	browser/Electron DOM
Calls detected	closest, querySelector
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
7954: function syncCompileEditorScroll(textarea) {
7955:   const editor = textarea?.closest(".compile-code-active-editor");
7956:   const preview = editor?.querySelector(".compile-code-preview");
7957:   if (!preview || !textarea) return;
7958:   preview.scrollTop = textarea.scrollTop;
7959:   preview.scrollLeft = textarea.scrollLeft;
7960: }
```

compilePayload - lines 7962-7984

Item	Explanation
Source location	Lines 7962-7984 (23 source lines).
Purpose	Part of the Compile Code language/tool/build/run flow.
Declaration	function compilePayload(project, file, options = {}) {
Touches	Mostly local calculations or local UI state.
Calls detected	ensureCompileCode, inferCompileFileRole, map
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
7962: function compilePayload(project, file, options = {}) {
7963:   const workspace = ensureCompileCode(project);
7964:   return {
7965:     projectId: project.id,
7966:     fileId: file.id,
7967:     title: file.title,
7968:     fileName: file.fileName,
7969:     language: file.language,
7970:     role: file.role || inferCompileFileRole(file.fileName, file.code, file.language),
7971:     code: file.code,
7972:     stdin: file.stdin || "",
7973:     workspaceFiles: (workspace.files || []).map((workspaceFile) => ({
7974:       id: workspaceFile.id,
7975:       title: workspaceFile.title,
7976:       fileName: workspaceFile.fileName,
7977:       language: workspaceFile.language,
7978:       role: workspaceFile.role || inferCompileFileRole(workspaceFile.fileName, workspaceFile.code,
workspaceFile.language),
7979:       code: workspaceFile.code || ""
7980:     })),
7981:     action: options.action || "",
7982:     forceRebuild: options.forceRebuild === true
7983:   };
7984: }
```

selectedCompileAppendDestination - lines 7986-7990

Item	Explanation
Source location	Lines 7986-7990 (5 source lines).
Purpose	Part of the Compile Code language/tool/build/run flow.
Declaration	function selectedCompileAppendDestination(project) {
Touches	Mostly local calculations or local UI state.
Calls detected	ensureCompileCode, compileAppendDestinations, find
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
7986: function selectedCompileAppendDestination(project) {
7987:   const workspace = ensureCompileCode(project);
7988:   const destinations = compileAppendDestinations(project);
7989:   return destinations.find((destination) => destination.id === workspace.appendTargetId) ||
destinations[0] || null;
7990: }
```

appendCompileCodeToProject - lines 7992-8019

Item	Explanation
Source location	Lines 7992-8019 (28 source lines).
Purpose	Part of the Compile Code language/tool/build/run flow.
Declaration	function appendCompileCodeToProject(project, file) {
Touches	Mostly local calculations or local UI state.
Calls detected	codeBlockForCompileFile, trim, setStatus, addCompileMessage, selectedCompileAppendDestination, richBlocksForCompileAppend, push, setByPath, plainTextFromRich, toISOString, scheduleAutosave, schedulePreviewRender
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 3 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
7992: function appendCompileCodeToProject(project, file) {
7993:   const block = codeBlockForCompileFile(file);
7994:   if (!block.code.trim()) {
7995:     setStatus("Type or import code before appending it to the project.");
7996:     addCompileMessage(project, "Append stopped because the active source is empty.", "warning");
7997:     return false;
7998:   }
7999:
8000:   const destination = selectedCompileAppendDestination(project);
8001:   if (!destination) {
8002:     setStatus("No project destination is available for this code block.");
8003:     addCompileMessage(project, "Append stopped because no destination section is available.", "warning");
8004:     return false;
8005:   }
8006:
8007:   const blocks = richBlocksForCompileAppend(project, destination);
8008:   blocks.push(block);
8009:   setByPath(project, destination.richPath, { blocks });
8010:   setByPath(project, destination.textPath, plainTextFromRich({ blocks }));
8011:
8012:   file.lastAppendedAt = new Date().toISOString();
8013:   addCompileMessage(project, `${file.fileName || file.title || "Source code"} appended to
${destination.label}.`, "success");
8014:   setStatus(`Code appended to ${destination.label}. Save project to include it in the portfolio
preview.`);
8015:   scheduleAutosave();
8016:   schedulePreviewRender();
8017:   updateCompileMessagesPanel(project);
8018:   return true;
8019: }
```

saveCompileFile - lines 8021-8038

Item	Explanation
Source location	Lines 8021-8038 (18 source lines).
Purpose	Persists data to local state, disk, credentials, or browser storage.
Declaration	async function saveCompileFile(project, file) {
Touches	network/API requests
Calls detected	fetch, now, stringify, compilePayload, json, Error, toISOString, addCompileMessage
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.

Item	Explanation
Async/return clues	2 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

8021: async function saveCompileFile(project, file) {
8022:   if (!project || !file) return null;
8023:   const response = await fetch(`/api/code/save?t=${Date.now()}`, {
8024:     method: "POST",
8025:     cache: "no-store",
8026:     headers: { "Content-Type": "application/json" },
8027:     body: JSON.stringify(compilePayload(project, file))
8028:   });
8029:   const result = await response.json();
8030:   if (!response.ok || !result.ok) throw new Error(result.error || "Code could not be saved.");
8031:   file.savedPath = result.saved?.sourcePath || "";
8032:   file.savedAt = result.saved?.savedAt || new Date().toISOString();
8033:   file.fileName = result.saved?.fileName || file.fileName;
8034:   file.language = result.saved?.language || file.language;
8035:   file.dirty = false;
8036:   addCompileMessage(project, `Saved ${file.fileName} || "source file"`, "info");
8037:   return result.saved;
8038: }

```

importCompileFile - lines 8040-8063

Item	Explanation
Source location	Lines 8040-8063 (24 source lines).
Purpose	Part of the Compile Code language/tool/build/run flow.
Declaration	async function importCompileFile(project) {
Touches	Mostly local calculations or local UI state.
Calls detected	ensureCompileCode, chooseFile, flatMap, split, pop, toLowerCase, includes, setStatus, text, detectCodeLanguage, codeLanguageProfile, newCompileFile
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	2 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

8040: async function importCompileFile(project) {
8041:   const workspace = ensureCompileCode(project);
8042:   const file = await chooseFile();
8043:   if (!file) return;
8044:   const allowedExtensions = supportedCodeLanguages.flatMap(item => item.extensions || []);
8045:   const extension = `.${String(file.name || "").split(".").pop().toLowerCase()}`;
8046:   if (!allowedExtensions.includes(extension)) {
8047:     setStatus("Choose a supported source file: C, C++, Verilog, SystemVerilog, LTspice, Java, JavaScript,
Python, or HTML.");
8048:     return;
8049:   }
8050:   const code = await file.text();
8051:   const language = detectCodeLanguage(code, file.name);
8052:   const profile = codeLanguageProfile(language);
8053:   const sourceFile = newCompileFile(profile.id, {
8054:     fileName: file.name,
8055:     title: file.name.replace(/\.[^.]*/g, ""),
8056:     code
8057:   });
8058:   workspace.files.push(sourceFile);
8059:   workspace.activeFileId = sourceFile.id;
8060:   setStatus(`Imported ${file.name}. Click Save source before compiling.`);
8061:   scheduleAutosave();
8062:   renderSectionContent(project);
8063: }

```

beautifyCompileFile - lines 8065-8085

Item	Explanation
Source location	Lines 8065-8085 (21 source lines).
Purpose	Part of the Compile Code language/tool/build/run flow.
Declaration	async function beautifyCompileFile(project, file) {
Touches	network/API requests
Calls detected	fetch, now, stringify, compilePayload, json, Error, setStatus, scheduleAutosave, renderSectionContent
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	2 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

8065: async function beautifyCompileFile(project, file) {
8066:   if (!file) return;
8067:   try {
8068:     const response = await fetch(`/api/code/beautify?t=${Date.now()}`, {
8069:       method: "POST",
8070:       cache: "no-store",
8071:       headers: { "Content-Type": "application/json" },
8072:       body: JSON.stringify(compilePayload(project, file))
8073:     });
8074:     const result = await response.json();
8075:     if (!response.ok || !result.ok) throw new Error(result.error || "Beautifier failed.");
8076:     file.code = result.code || file.code;
8077:     file.language = result.language || file.language;
8078:     file.dirty = true;
8079:     setStatus("Code beautified. Review it, then save source.");
8080:     scheduleAutosave();
8081:     renderSectionContent(project);
8082:   } catch (error) {
8083:     setStatus(error.message || "Code beautifier failed.");
8084:   }
8085: }

```

compileActiveFile - lines 8087-8145

Item	Explanation
Source location	Lines 8087-8145 (59 source lines).
Purpose	Part of the Compile Code language/tool/build/run flow.
Declaration	async function compileActiveFile(project, file, options = {}) {
Touches	filesystem, network/API requests
Calls detected	ensureCompileCode, isHdlLanguage, addCompileMessage, updateCompileTerminalPanel, updateCompileMessagesPanel, saveCompileFile, fetch, now, stringify, compilePayload, json, toISOString
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	3 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

8087: async function compileActiveFile(project, file, options = {}) {
8088:   if (!file) return;
8089:   const workspace = ensureCompileCode(project);
8090:   const action = options.action || (isHdlLanguage(file.language) ? "simulate" : "run");
8091:   const actionLabel = action === "compile" ? "Compile" : action === "build" ? "Build project" : action
=== "simulate" ? "Simulate" : "Run";
8092:   file.lastResult = {
8093:     ok: null,
8094:     terminal: `${actionLabel} started. Please wait...`
8095:   };
8096:   workspace.terminal = file.lastResult.terminal;
8097:   addCompileMessage(project, `${actionLabel} started for ${file.fileName || file.title || "source
file"}`, "info");
8098:   updateCompileTerminalPanel(project, file);

```

```

8099:   updateCompileMessagesPanel(project);
8100:   try {
8101:     await saveCompileFile(project, file);
8102:     const response = await fetch(`/api/code/compile?t=${Date.now()}`, {
8103:       method: "POST",
8104:       cache: "no-store",
8105:       headers: { "Content-Type": "application/json" },
8106:       body: JSON.stringify(compilePayload(project, file, {
8107:         action,
8108:         forceRebuild: options.forceRebuild === true || action === "build"
8109:       })))
8110:   };
8111:   const result = await response.json();
8112:   const compileResult = result.result || {};
8113:   file.lastResult = {
8114:     ok: Boolean(result.ok),
8115:     language: compileResult.language || file.language,
8116:     terminal: compileResult.terminal || result.error || "No compiler output was returned.",
8117:     waveform: compileResult.waveform || null,
8118:     finishedAt: new Date().toISOString()
8119:   };
8120:   file.waveform = compileResult.waveform || null;
8121:   ensureCompileCode(project).waveform = compileResult.waveform || null;
8122:   if (compileResult.saved) {
8123:     file.savedPath = compileResult.saved.sourcePath || file.savedPath;
8124:     file.savedAt = compileResult.saved.savedAt || file.savedAt;
8125:   }
8126:   file.dirty = false;
8127:   ensureCompileCode(project).terminal = file.lastResult.terminal;
8128:   updateCompileTerminalPanel(project, file);
8129:   addCompileMessage(project, result.ok ? `${actionLabel} succeeded for ${file.fileName}.` :
`${actionLabel} completed with errors for ${file.fileName}.`, result.ok ? "success" : "error");
8130:   setStatus(result.ok ? `${actionLabel} succeeded.` : `${actionLabel} completed with errors.`);
8131:   scheduleAutosave();
8132:   renderSectionContent(project);
8133: } catch (error) {
8134:   file.lastResult = {
8135:     ok: false,
8136:     terminal: error.message || "Compile/run failed.",
8137:     finishedAt: new Date().toISOString()
8138:   };
8139:   ensureCompileCode(project).terminal = file.lastResult.terminal;
8140:   updateCompileTerminalPanel(project, file);
8141:   addCompileMessage(project, `${actionLabel} failed for ${file.fileName || "source file"}.`, "error");
8142:   setStatus(error.message || `${actionLabel} failed.`);
8143:   renderSectionContent(project);
8144: }
8145: }

```

checkCompileTools - lines 8147-8162

Item	Explanation
Source location	Lines 8147-8162 (16 source lines).
Purpose	Part of the Compile Code language/tool/build/run flow.
Declaration	async function checkCompileTools(project) {
Touches	filesystem, network/API requests
Calls detected	fetch, now, json, Error, entries, map, join, ensureCompileCode, renderSectionContent, setStatus
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	2 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

8147: async function checkCompileTools(project) {
8148:   try {
8149:     const response = await fetch(`/api/code/tools?t=${Date.now()}`, { cache: "no-store" });
8150:     const result = await response.json();
8151:     if (!response.ok || !result.ok) throw new Error(result.error || "Compiler status failed.");
8152:     const lines = Object.entries(result.tools?.languages || {}).map(([id, status]) => {
8153:       const toolText = Object.entries(status.tools || {}).map(([tool, value]) => `${tool}:
${value}`).join(", ") || "no compiler required";
8154:       return `${status.ready ? "READY" : "MISSING"} ${status.label} (${id}) - ${toolText}`;
8155:     });
8156:     compileTerminalStatus = [`Compile workspace: ${result.tools?.compileRoot || ""}`,
...lines].join("\n");
8157:     ensureCompileCode(project).terminal = compileTerminalStatus;
8158:     renderSectionContent(project);

```

```

8159:   } catch (error) {
8160:     setStatus(error.message || "Compiler status failed.");
8161:   }
8162: }

```

installCompileTools - lines 8164-8187

Item	Explanation
Source location	Lines 8164-8187 (24 source lines).
Purpose	Part of the Compile Code language/tool/build/run flow.
Declaration	async function installCompileTools(project, file) {
Touches	filesystem, network/API requests
Calls detected	codeLanguageLabel, renderSectionContent, fetch, now, stringify, json, toISOString, setStatus
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	2 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

8164: async function installCompileTools(project, file) {
8165:   if (!file) return;
8166:   file.lastResult = { ok: null, terminal: `Installing tools for ${codeLanguageLabel(file.language)}. This
can take several minutes...`; };
8167:   renderSectionContent(project);
8168:   try {
8169:     const response = await fetch(`/api/code/install-tools?t=${Date.now()}`, {
8170:       method: "POST",
8171:       cache: "no-store",
8172:       headers: { "Content-Type": "application/json" },
8173:       body: JSON.stringify({ language: file.language })
8174:     });
8175:     const result = await response.json();
8176:     file.lastResult = {
8177:       ok: Boolean(result.ok),
8178:       terminal: result.terminal || result.error || "Tool installation finished.",
8179:       finishedAt: new Date().toISOString()
8180:     };
8181:     setStatus(result.ok ? "Compiler tool installation finished." : "Compiler tool installation could not
complete.");
8182:     renderSectionContent(project);
8183:   } catch (error) {
8184:     file.lastResult = { ok: false, terminal: error.message || "Tool installation failed." };
8185:     renderSectionContent(project);
8186:   }
8187: }

```

readFileAsDataURL - lines 8189-8196

Item	Explanation
Source location	Lines 8189-8196 (8 source lines).
Purpose	Loads data from disk, network, browser storage, or a service.
Declaration	function readFileAsDataURL(file) {
Touches	filesystem
Calls detected	FileReader, resolve, readAsDataURL
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

8189: function readFileAsDataURL(file) {
8190:   return new Promise((resolve, reject) => {

```

```

8191:     const reader = new FileReader();
8192:     reader.onload = () => resolve(reader.result);
8193:     reader.onerror = reject;
8194:     reader.readAsDataURL(file);
8195:   });
8196: }

```

uploadProfileAsset - lines 8198-8242

Item	Explanation
Source location	Lines 8198-8242 (45 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	async function uploadProfileAsset(kind) {
Touches	filesystem, network/API requests
Calls detected	syncProfileFromInputs, chooseFile, Set, has, startsWith, setStatus, includes, extensionFor, toLowerCase, readFileAsDataURL, fetch, now
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	4 await expression(s), 4 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

8198: async function uploadProfileAsset(kind) {
8199:   const profile = syncProfileFromInputs();
8200:   const file = await chooseFile();
8201:   if (!file) return;
8202:
8203:   const imageKinds = new Set(["profileImage", "heroImage", "brandImage"]);
8204:   if (imageKinds.has(kind) && !file.type.startsWith("image/")) {
8205:     setStatus("Choose an image file for this profile asset.");
8206:     return;
8207:   }
8208:
8209:   if (kind === "resumeUrl") {
8210:     const allowedResume = [".pdf", ".doc", ".docx"].includes(extensionFor(file.name).toLowerCase());
8211:     if (!allowedResume) {
8212:       setStatus("Choose a PDF, DOC, or DOCX resume file.");
8213:       return;
8214:     }
8215:   }
8216:
8217:   const data = await readFileAsDataURL(file);
8218:   const response = await fetch(`/api/upload?t=${Date.now()}`, {
8219:     method: "POST",
8220:     cache: "no-store",
8221:     headers: { "Content-Type": "application/json" },
8222:     body: JSON.stringify({
8223:       projectId: "_site-profile",
8224:       section: kind,
8225:       fileName: file.name,
8226:       data
8227:     })
8228:   });
8229:   const result = await response.json();
8230:   if (!response.ok) {
8231:     setStatus(result.error || "Profile asset upload failed.");
8232:     return;
8233:   }
8234:
8235:   profile[kind] = result.url;
8236:   catalog.profile = normalizeProfile(profile);
8237:   markDraftNeedsSave();
8238:   setStatus("Profile asset saved locally. Click Save draft before applying to site.");
8239:   scheduleAutosave();
8240:   schedulePreviewRender();
8241:   renderProfileEditor();
8242: }

```

extensionFor - lines 8244-8247

Item	Explanation
Source location	Lines 8244-8247 (4 source lines).

Item	Explanation
Purpose	Part of the private builder frontend and editing experience.
Declaration	function extensionFor(fileName) {
Touches	Mostly local calculations or local UI state.
Calls detected	match
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

8244: function extensionFor(fileName) {
8245:   const match = String(fileName || "").match(/(\.[a-z0-9]+)$/i);
8246:   return match ? match[1] : "";
8247: }

```

withExtension - lines 8249-8253

Item	Explanation
Source location	Lines 8249-8253 (5 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function withExtension(name, fallbackFileName) {
Touches	Mostly local calculations or local UI state.
Calls detected	trim, extensionFor
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

8249: function withExtension(name, fallbackFileName) {
8250:   const trimmed = String(name || "").trim();
8251:   if (!trimmed) return fallbackFileName;
8252:   return /\.[a-z0-9]+$/i.test(trimmed) ? trimmed : `${trimmed}${extensionFor(fallbackFileName)}`;
8253: }

```

uploadProfile - lines 8255-8286

Item	Explanation
Source location	Lines 8255-8286 (32 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function uploadProfile(key) {
Touches	Mostly local calculations or local UI state.
Calls detected	No obvious named calls detected by the generator.
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

8255: function uploadProfile(key) {
8256:   const anyProjectFile = {
8257:     label: "project file",
8258:     accept: "",
8259:     extensions: [],
8260:     mimeStarts: [],
8261:     allFiles: true
8262:   };
8263:   const profiles = {
8264:     media: {
8265:       ...anyProjectFile,
8266:       label: "image or file"
8267:     },
8268:     pcbs: {
8269:       ...anyProjectFile,
8270:       label: "PCB artifact"
8271:     },
8272:     tests: {
8273:       ...anyProjectFile,
8274:       label: "test artifact"
8275:     },
8276:     documents: {
8277:       ...anyProjectFile,
8278:       label: "document"
8279:     },
8280:     sections: {
8281:       ...anyProjectFile,
8282:       label: "section file"
8283:     }
8284:   };
8285:   return profiles[key] || profiles.sections;
8286: }

```

allowedFileMessage - lines 8288-8291

Item	Explanation
Source location	Lines 8288-8291 (4 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function allowedFileMessage(profile) {
Touches	Mostly local calculations or local UI state.
Calls detected	join
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

8288: function allowedFileMessage(profile) {
8289:   if (profile.allFiles) return `Add any local file type for this ${profile.label}.`;
8290:   return `Add a ${profile.label}: ${profile.extensions.join(", ")}`;
8291: }

```

isAllowedUpload - lines 8293-8306

Item	Explanation
Source location	Lines 8293-8306 (14 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function isAllowedUpload(key, fileName, mimeType = "") {
Touches	Mostly local calculations or local UI state.
Calls detected	uploadProfile, extensionFor, toLowerCase, includes, startsWith, some, allowedFileMessage
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 3 return statement(s).

Item	Explanation
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

8293: function isAllowedUpload(key, fileName, mimeType = "") {
8294:   const profile = uploadProfile(key);
8295:   if (profile.allFiles) return { ok: true, message: "" };
8296:   const extension = extensionFor(fileName.toLowerCase());
8297:   if (key === "media" && !profile.extensions.includes(extension) && !mimeType.startsWith("image/")) {
8298:     return { ok: false, message: "Images can only accept image files. Add PDFs under Documents, Tests, or
a custom section instead." };
8299:   }
8300:   const allowedByExtension = profile.extensions.includes(extension);
8301:   const allowedByMime = profile.mimeStarts.some((prefix) => mimeType.startsWith(prefix));
8302:   return {
8303:     ok: allowedByExtension || allowedByMime,
8304:     message: `${fileName} || "This file" is not a supported ${profile.label}.
${allowedFileMessage(profile)}.`
8305:   };
8306: }

```

validateAssetForSection - lines 8308-8317

Item	Explanation
Source location	Lines 8308-8317 (10 source lines).
Purpose	Validates or normalizes input so later code receives safe predictable values.
Declaration	function validateAssetForSection(key, asset) {
Touches	filesystem
Calls detected	isAllowedUpload, normalizeLinkTarget, extensionFromUrl, extensionFor
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 3 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

8308: function validateAssetForSection(key, asset) {
8309:   if (asset.source === "local") {
8310:     return isAllowedUpload(key, asset.fileName || asset.file?.name || "", asset.file?.type || "");
8311:   }
8312:   const url = normalizeLinkTarget(asset.url || "", { assumeWeb: true });
8313:   if (!extensionFromUrl(url) && !extensionFor(asset.title || "")) {
8314:     return { ok: true, message: "" };
8315:   }
8316:   return isAllowedUpload(key, url || asset.title || "", "");
8317: }

```

dialogValue - lines 8319-8328

Item	Explanation
Source location	Lines 8319-8328 (10 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function dialogValue(dialog) {
Touches	Mostly local calculations or local UI state.
Calls detected	removeEventListener, resolve, addEventListener, showModal
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
8319: function dialogValue(dialog) {
8320:   return new Promise((resolve) => {
8321:     const onClose = () => {
8322:       dialog.removeEventListener("close", onClose);
8323:       resolve(dialog.returnValue === "save");
8324:     };
8325:     dialog.addEventListener("close", onClose);
8326:     dialog.showModal();
8327:   });
8328: }
```

onClose - lines 8321-8324

Item	Explanation
Source location	Lines 8321-8324 (4 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	const onClose = () => {
Touches	Mostly local calculations or local UI state.
Calls detected	removeEventListener, resolve
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
8321:   const onClose = () => {
8322:     dialog.removeEventListener("close", onClose);
8323:     resolve(dialog.returnValue === "save");
8324:   };
```

openAssetDialog - lines 8330-8392

Item	Explanation
Source location	Lines 8330-8392 (63 source lines).
Purpose	Controls a window, dialog, panel, menu, or visible state.
Declaration	async function openAssetDialog(key) {
Touches	filesystem
Calls detected	uploadProfile, updateAssetDialogVisibility, dialogValue, trim, setStatus, withExtension, isAllowedUpload, normalizeLinkTarget, displayNameFromUrl, validateAssetForSection, text, inferUrlAssetKind
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	2 await expression(s), 6 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
8330: async function openAssetDialog(key) {
8331:   const profile = uploadProfile(key);
8332:   assetDialogTitle.textContent = `Add ${profile.label}`;
8333:   assetSource.value = "local";
8334:   assetFile.value = "";
8335:   assetFile.accept = profile.accept;
8336:   assetUrl.value = "";
8337:   assetTitle.value = "";
8338:   assetCaption.value = "";
8339:   captionSource.value = "text";
8340:   captionFile.value = "";
8341:   updateAssetDialogVisibility();
8342:
8343:   const saved = await dialogValue(assetDialog);
8344:   if (!saved) return null;
```

```

8345:
8346:   let file = null;
8347:   let url = assetUrl.value.trim();
8348:   let fileName = "";
8349:   let title = assetTitle.value.trim();
8350:
8351:   if (assetSource.value === "local") {
8352:     file = assetFile.files[0];
8353:     if (!file) {
8354:       setStatus("Choose a local file first.");
8355:       return null;
8356:     }
8357:     fileName = withExtension(title, file.name);
8358:     title = title || file.name;
8359:     const validation = isAllowedUpload(key, fileName, file.type);
8360:     if (!validation.ok) {
8361:       setStatus(validation.message);
8362:       return null;
8363:     }
8364:   } else {
8365:     if (!url) {
8366:       setStatus("Paste a web or Google Drive link first.");
8367:       return null;
8368:     }
8369:     url = normalizeLinkTarget(url, { assumeWeb: true });
8370:     title = title || displayNameFromUrl(url, "Linked asset");
8371:     const validation = validateAssetForSection(key, { source: assetSource.value, url, title });
8372:     if (!validation.ok) {
8373:       setStatus(validation.message);
8374:       return null;
8375:     }
8376:   }
8377:
8378:   let caption = assetCaption.value.trim();
8379:   if (captionSource.value === "file" && captionFile.files[0]) {
8380:     caption = await captionFile.files[0].text();
8381:   }
8382:
8383:   return {
8384:     caption,
8385:     file,
8386:     fileName,
8387:     kind: file ? "file" : inferUrlAssetKind(url, assetSource.value),
8388:     source: assetSource.value,
8389:     title,
8390:     url
8391:   };
8392: }

```

uploadToSection - lines 8394-8454

Item	Explanation
Source location	Lines 8394-8454 (61 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	async function uploadToSection(key, folder, customSectionId = "", customItemIndex = null) {
Touches	filesystem, network/API requests
Calls detected	selectedProject, openAssetDialog, validateAssetForSection, setStatus, labelForUrlAssetKind, readFileAsDataURL, fetch, stringify, json, find, push, makeItemForSection
API endpoints	/api/upload
Storage keys	No local/session storage keys detected.
Async/return clues	4 await expression(s), 5 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

8394: async function uploadToSection(key, folder, customSectionId = "", customItemIndex = null) {
8395:   const project = selectedProject();
8396:   if (!project) return;
8397:
8398:   const asset = await openAssetDialog(key);
8399:   if (!asset) return;
8400:
8401:   const validation = validateAssetForSection(key, asset);
8402:   if (!validation.ok) {
8403:     setStatus(validation.message);
8404:     return;
8405:   }
8406:

```

```

8407: const sectionFolder = customSectionId || folder || key;
8408: let url = asset.url;
8409: let type = asset.source === "local" ? "File" : labelForUrlAssetKind(asset.kind);
8410:
8411: if (asset.file) {
8412:   const data = await readFileAsDataURL(asset.file);
8413:   const response = await fetch("/api/upload", {
8414:     method: "POST",
8415:     headers: { "Content-Type": "application/json" },
8416:     body: JSON.stringify({
8417:       projectId: project.id,
8418:       section: sectionFolder,
8419:       fileName: asset.fileName,
8420:       data
8421:     })
8422:   });
8423:   const result = await response.json();
8424:   if (!response.ok) {
8425:     setStatus(result.error || "Upload failed.");
8426:     return;
8427:   }
8428:   url = result.url;
8429:   type = asset.file.type || "File";
8430: }
8431:
8432: if (customSectionId) {
8433:   const section = (project.sections || []).find((item) => item.id === customSectionId);
8434:   const uploadedItem = { title: asset.title, description: asset.caption, type, url, status:
asset.source === "local" ? "uploaded" : "linked" };
8435:   if (customItemIndex !== null && customItemIndex !== "") {
8436:     const parent = section?.items?.[Number(customItemIndex)];
8437:     if (!parent || parent.url) return;
8438:     parent.children = parent.children || [];
8439:     parent.children.push(uploadedItem);
8440:   } else {
8441:     section.items = section.items || [];
8442:     section.items.push(uploadedItem);
8443:   }
8444: } else {
8445:   project[key] = project[key] || [];
8446:   project[key].push(makeItemForSection(key, asset.title, asset.caption, url));
8447: }
8448:
8449: setStatus(asset.source === "local"
8450:   ? `Saved ${asset.fileName} to ${url}. Click Save project to include it in the portfolio preview.`
8451:   : `Linked ${asset.title}. Click Save project to include it in the portfolio preview.`);
8452: scheduleAutosave();
8453: renderAll();
8454: }

```

section - lines 8433-8434

Item	Explanation
Source location	Lines 8433-8434 (2 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	<code>const section = (project.sections []).find((item) => item.id === customSectionId);</code>
Touches	Mostly local calculations or local UI state.
Calls detected	find
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

8433:   const section = (project.sections || []).find((item) => item.id === customSectionId);
8434:   const uploadedItem = { title: asset.title, description: asset.caption, type, url, status:
asset.source === "local" ? "uploaded" : "linked" };

```

uploadToDesignSection - lines 8456-8502

Item	Explanation
Source location	Lines 8456-8502 (47 source lines).

Item	Explanation
Purpose	Part of the private builder frontend and editing experience.
Declaration	async function uploadToDesignSection(pathValue, folder, kind = "document") {
Touches	filesystem, network/API requests
Calls detected	selectedProject, openAssetDialog, validateAssetForSection, setStatus, labelForUrlAssetKind, readFileAsDataURL, fetch, stringify, json, designArray, push, makeDesignItem
API endpoints	/api/upload
Storage keys	No local/session storage keys detected.
Async/return clues	4 await expression(s), 4 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

8456: async function uploadToDesignSection(pathValue, folder, kind = "document") {
8457:   const project = selectedProject();
8458:   if (!project) return;
8459:
8460:   const asset = await openAssetDialog(kind === "simulation-result" ? "tests" : "sections");
8461:   if (!asset) return;
8462:
8463:   const validation = validateAssetForSection(kind === "simulation-result" ? "tests" : "sections", asset);
8464:   if (!validation.ok) {
8465:     setStatus(validation.message);
8466:     return;
8467:   }
8468:
8469:   let url = asset.url;
8470:   let type = asset.source === "local" ? "File" : labelForUrlAssetKind(asset.kind);
8471:   if (asset.file) {
8472:     const data = await readFileAsDataURL(asset.file);
8473:     const response = await fetch("/api/upload", {
8474:       method: "POST",
8475:       headers: { "Content-Type": "application/json" },
8476:       body: JSON.stringify({
8477:         projectId: project.id,
8478:         section: folder || "design",
8479:         fileName: asset.fileName,
8480:         data
8481:       })
8482:     });
8483:     const result = await response.json();
8484:     if (!response.ok) {
8485:       setStatus(result.error || "Upload failed.");
8486:       return;
8487:     }
8488:     url = result.url;
8489:     type = asset.file.type || "File";
8490:   }
8491:
8492:   const items = designArray(project, pathValue);
8493:   items.push({
8494:     ...makeDesignItem(kind, asset.title, asset.caption, url),
8495:     type
8496:   });
8497:   setStatus(asset.source === "local"
8498:     ? `Saved ${asset.fileName} to ${url}. Click Save project to include it in the portfolio preview.`
8499:     : `Linked ${asset.title}. Click Save project to include it in the portfolio preview.`);
8500:   scheduleAutosave();
8501:   renderAll();
8502: }

```

openSectionDialog - lines 8504-8523

Item	Explanation
Source location	Lines 8504-8523 (20 source lines).
Purpose	Controls a window, dialog, panel, menu, or visible state.
Declaration	async function openSectionDialog() {
Touches	filesystem
Calls detected	dialogValue, trim, setStatus
API endpoints	None detected inside this function body.

Item	Explanation
Storage keys	No local/session storage keys detected.
Async/return clues	1 await expression(s), 3 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

8504: async function openSectionDialog() {
8505:   sectionDialogTitle.textContent = "Create a section";
8506:   sectionTitle.placeholder = "Design, Measurements, Firmware...";
8507:   sectionDescription.placeholder = "Optional description shown to recruiters";
8508:   sectionTitle.value = "";
8509:   sectionDescription.value = "";
8510:   const saved = await dialogValue(sectionDialog);
8511:   if (!saved) return null;
8512:
8513:   const title = sectionTitle.value.trim();
8514:   if (!title) {
8515:     setStatus("Type a section title before saving.");
8516:     return null;
8517:   }
8518:
8519:   return {
8520:     description: sectionDescription.value.trim(),
8521:     title
8522:   };
8523: }

```

openCategoryDialog - lines 8525-8531

Item	Explanation
Source location	Lines 8525-8531 (7 source lines).
Purpose	Controls a window, dialog, panel, menu, or visible state.
Declaration	function openCategoryDialog() {
Touches	Mostly local calculations or local UI state.
Calls detected	showModal, focus
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

8525: function openCategoryDialog() {
8526:   categoryTitle.value = "";
8527:   categoryDescription.value = "";
8528:   categoryAccent.value = "#1677a8";
8529:   categoryDialog.showModal();
8530:   setTimeout(() => categoryTitle.focus(), 0);
8531: }

```

saveCategoryFromDialog - lines 8533-8559

Item	Explanation
Source location	Lines 8533-8559 (27 source lines).
Purpose	Persists data to local state, disk, credentials, or browser storage.
Declaration	function saveCategoryFromDialog() {
Touches	filesystem
Calls detected	trim, setStatus, slugify, Set, map, has, normalizeCategory, add, refreshCategoryControls, renderAll, scheduleAutosave
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.

Item	Explanation
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

8533: function saveCategoryFromDialog() {
8534:   const label = categoryTitle.value.trim();
8535:   if (!label) {
8536:     setStatus("Type a category name before saving.");
8537:     return;
8538:   }
8539:   const baseId = slugify(label);
8540:   const ids = new Set((catalog.categories || []).map((category) => category.id));
8541:   let id = baseId;
8542:   let suffix = 2;
8543:   while (ids.has(id)) {
8544:     id = `${baseId}-${suffix}`;
8545:     suffix += 1;
8546:   }
8547:   const category = normalizeCategory({
8548:     accent: categoryAccent.value,
8549:     description: categoryDescription.value,
8550:     id,
8551:     label
8552:   });
8553:   catalog.categories = [...(catalog.categories || []), category];
8554:   expandedCategories.add(category.id);
8555:   refreshCategoryControls();
8556:   renderAll();
8557:   scheduleAutosave();
8558:   setStatus(`Category "${category.label}" added locally. It is now available under Add new project.`);
8559: }

```

addCustomSection - lines 8561-8582

Item	Explanation
Source location	Lines 8561-8582 (22 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	async function addCustomSection() {
Touches	Mostly local calculations or local UI state.
Calls detected	selectedProject, openSectionDialog, slugify, Set, map, has, push, setStatus, scheduleAutosave, renderAll
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	1 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

8561: async function addCustomSection() {
8562:   const project = selectedProject();
8563:   if (!project) return;
8564:
8565:   const sectionInput = await openSectionDialog();
8566:   if (!sectionInput) return;
8567:
8568:   let id = slugify(sectionInput.title);
8569:   const existingIds = new Set((project.sections || []).map((section) => section.id));
8570:   let suffix = 2;
8571:   while (existingIds.has(id)) {
8572:     id = `${slugify(sectionInput.title)}-${suffix}`;
8573:     suffix += 1;
8574:   }
8575:
8576:   project.sections = project.sections || [];
8577:   project.sections.push({ id, title: sectionInput.title, description: sectionInput.description, items: [] });
8578:   activeSectionId = `custom:${id}`;
8579:   setStatus("New section added in the editor. Click Save project to include this version in the portfolio preview.");
8580:   scheduleAutosave();
8581:   renderAll();
8582: }

```

addCustomItem - lines 8584-8596

Item	Explanation
Source location	Lines 8584-8596 (13 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function addCustomItem(sectionId) {
Touches	Mostly local calculations or local UI state.
Calls detected	selectedProject, find, prompt, push, setStatus, scheduleAutosave, renderAll
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
8584: function addCustomItem(sectionId) {
8585:   const project = selectedProject();
8586:   const section = (project.sections || []).find((item) => item.id === sectionId);
8587:   const title = prompt("Type a subsection title.");
8588:   if (!section || !title) return;
8589:   const description = prompt("Paste or type a text description if you want.") || "";
8590:
8591:   section.items = section.items || [];
8592:   section.items.push({ title, description, type: "Text", status: "draft" });
8593:   setStatus("Subsection added locally.");
8594:   scheduleAutosave();
8595:   renderAll();
8596: }
```

section - lines 8586-8589

Item	Explanation
Source location	Lines 8586-8589 (4 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	const section = (project.sections []).find((item) => item.id === sectionId);
Touches	Mostly local calculations or local UI state.
Calls detected	find, prompt
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
8586:   const section = (project.sections || []).find((item) => item.id === sectionId);
8587:   const title = prompt("Type a subsection title.");
8588:   if (!section || !title) return;
8589:   const description = prompt("Paste or type a text description if you want.") || "";
```

editCustomItem - lines 8598-8609

Item	Explanation
Source location	Lines 8598-8609 (12 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function editCustomItem(sectionId, index) {
Touches	Mostly local calculations or local UI state.

Item	Explanation
Calls detected	selectedProject, find, prompt, setStatus, scheduleAutosave, renderAll
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

8598: function editCustomItem(sectionId, index) {
8599:   const project = selectedProject();
8600:   const item = (project.sections || []).find((section) => section.id === sectionId)?.items?.[index];
8601:   if (!item) return;
8602:   const title = prompt("Edit subsection title.", item.title || "");
8603:   if (!title) return;
8604:   item.title = title;
8605:   item.description = prompt("Edit description.", item.description || "") || "";
8606:   setStatus("Subsection edited locally.");
8607:   scheduleAutosave();
8608:   renderAll();
8609: }

```

item - lines 8600-8608

Item	Explanation
Source location	Lines 8600-8608 (9 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	const item = (project.sections []).find((section) => section.id === sectionId)?.items?.[index];
Touches	Mostly local calculations or local UI state.
Calls detected	find, prompt, setStatus, scheduleAutosave, renderAll
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

8600:   const item = (project.sections || []).find((section) => section.id === sectionId)?.items?.[index];
8601:   if (!item) return;
8602:   const title = prompt("Edit subsection title.", item.title || "");
8603:   if (!title) return;
8604:   item.title = title;
8605:   item.description = prompt("Edit description.", item.description || "") || "";
8606:   setStatus("Subsection edited locally.");
8607:   scheduleAutosave();
8608:   renderAll();

```

catalogForSave - lines 8611-8633

Item	Explanation
Source location	Lines 8611-8633 (23 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function catalogForSave(endpoint) {
Touches	network/API requests
Calls detected	parsedPortfolioGlobals, clone
API endpoints	/api/apply-catalog
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 2 return statement(s).

Item	Explanation
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

8611: function catalogForSave(endpoint) {
8612:   const parsedGlobals = parsedPortfolioGlobals();
8613:   if (endpoint === "/api/apply-catalog") {
8614:     return {
8615:       ...savedPortfolioCatalog,
8616:       ...parsedGlobals,
8617:       projects: clone(savedPortfolioCatalog.projects || [])
8618:     };
8619:   }
8620:   return {
8621:     ...catalog,
8622:     categories: parsedGlobals.categories,
8623:     fieldStyles: parsedGlobals.fieldStyles,
8624:     funFacts: parsedGlobals.funFacts,
8625:     funFactsRich: parsedGlobals.funFactsRich,
8626:     profile: parsedGlobals.profile,
8627:     profileRich: parsedGlobals.profileRich,
8628:     siteContent: parsedGlobals.siteContent,
8629:     siteContentRich: parsedGlobals.siteContentRich,
8630:     siteSections: clone(catalog.siteSections || []),
8631:     projects: clone(catalog.projects || [])
8632:   };
8633: }

```

saveCatalog - lines 8635-8697

Item	Explanation
Source location	Lines 8635-8697 (63 source lines).
Purpose	Persists data to local state, disk, credentials, or browser storage.
Declaration	async function saveCatalog(endpoint, message) {
Touches	network/API requests
Calls detected	catalogForSave, showBuilderError, setStatus, setSaveState, fetch, now, stringify, json, showPublishResult
API endpoints	/api/apply-catalog, /api/save-draft
Storage keys	No local/session storage keys detected.
Async/return clues	2 await expression(s), 4 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

8635: async function saveCatalog(endpoint, message) {
8636:   const targetCatalog = catalogForSave(endpoint);
8637:   if (endpoint === "/api/apply-catalog" && !draftSavedSinceChanges) {
8638:     showBuilderError(
8639:       "Save draft required",
8640:       "Please click Save draft before applying this portfolio to the live site.",
8641:       "Apply to site was stopped before commit and push. Save the draft, then click Apply to site again."
8642:     );
8643:     setStatus("Apply to site stopped. Save draft first.");
8644:     setSaveState("dirty", "Save draft needed");
8645:     return;
8646:   }
8647:   if (endpoint === "/api/apply-catalog" && !(targetCatalog.projects || []).length) {
8648:     showBuilderError(
8649:       "No saved project preview",
8650:       "Save a project or use Save all sections before applying to the live site.",
8651:       "Apply to site was stopped before commit and push."
8652:     );
8653:     setStatus("Apply to site stopped. Save a project or use Save all sections first.");
8654:     setSaveState("dirty", "Preview needs projects");
8655:     return;
8656:   }
8657:   try {
8658:     clearTimeout(autosaveTimer);
8659:     saveDraftButton.disabled = true;
8660:     applyCatalogButton.disabled = true;
8661:     const applying = endpoint === "/api/apply-catalog";
8662:     setStatus(applying ? "Applying site changes..." : "Saving draft...");
8663:     setSaveState(applying ? "publishing" : "saving", applying ? "Publishing..." : "Saving...");

```

```

8665:     const response = await fetch(`${endpoint}?t=${Date.now()}`, {
8666:       method: "POST",
8667:       cache: "no-store",
8668:       headers: { "Content-Type": "application/json" },
8669:       body: JSON.stringify({ catalog: targetCatalog })
8670:     });
8671:     const result = await response.json();
8672:     if (!response.ok) {
8673:       setStatus(result.error || "Save failed.");
8674:       setSaveState("error", endpoint === "/api/apply-catalog" ? "Publish failed" : "Save failed");
8675:       return;
8676:     }
8677:     if (result.publish) {
8678:       setStatus(result.publish.pushed
8679:         ? `${message}: ${result.file}. Pushed to ${result.publish.branch}.`
8680:         : `${message}: ${result.file}. Git push failed: ${result.publish.error}`);
8681:       setSaveState(result.publish.pushed ? "published" : "error", result.publish.pushed ? "Site applied"
: "Push failed");
8682:       showPublishResult(result);
8683:       return;
8684:     }
8685:     if (endpoint === "/api/save-draft") {
8686:       draftSavedSinceChanges = true;
8687:       setSaveState("saved", "Draft saved");
8688:     }
8689:     setStatus(`${message}: ${result.file}`);
8690:   } catch {
8691:     setStatus("Save failed. Make sure the local server window is still running.");
8692:     setSaveState("error", "Save failed");
8693:   } finally {
8694:     saveDraftButton.disabled = false;
8695:     applyCatalogButton.disabled = false;
8696:   }
8697: }

```

loadData - lines 8699-8745

Item	Explanation
Source location	Lines 8699-8745 (47 source lines).
Purpose	Loads data from disk, network, browser storage, or a service.
Declaration	async function loadData() {
Touches	filesystem, network/API requests
Calls detected	all, fetch, now, json, call, normalizeFunFacts, clone, normalizeFieldStyles, normalizeProfile, normalizeSiteContent, filter, has
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	3 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

8699: async function loadData() {
8700:   const [catalogResponse, templateResponse] = await Promise.all([
8701:     fetch(`/api/catalog?t=${Date.now()}`, { cache: "no-store" }),
8702:     fetch(`/api/templates?t=${Date.now()}`, { cache: "no-store" })
8703:   ]);
8704:   const catalogData = await catalogResponse.json();
8705:   const templateData = await templateResponse.json();
8706:
8707:   catalog = catalogData.catalog;
8708:   catalog.funFacts = Object.prototype.hasOwnProperty.call(catalog, "funFacts")
8709:     ? normalizeFunFacts(catalog.funFacts)
8710:     : clone(defaultFunFacts);
8711:   catalog.funFactsRich = catalog.funFactsRich || null;
8712:   catalog.fieldStyles = normalizeFieldStyles(catalog.fieldStyles || {});
8713:   catalog.siteContentRich = catalog.siteContentRich || {};
8714:   catalog.profileRich = catalog.profileRich || {};
8715:   catalog.profile = normalizeProfile(catalog.profile || {});
8716:   catalog.siteContent = normalizeSiteContent(catalog.siteContent || {});
8717:   catalog.siteSections = catalog.siteSections || [];
8718:   catalog.projects = (catalog.projects || []).filter((project) => !starterProjectIds.has(project.id));
8719:   catalog.projects.forEach((project) => {
8720:     if (isAnalogProject(project)) ensureDesignModel(project);
8721:     ensureCompileCode(project);
8722:   });
8723:   savedPortfolioCatalog = {
8724:     categories: clone(catalog.categories || []),
8725:     fieldStyles: clone(catalog.fieldStyles || {}),

```

```

8726:     funFacts: clone(catalog.funFacts || []),
8727:     funFactsRich: clone(catalog.funFactsRich || null),
8728:     profile: clone(normalizeProfile(catalog.profile || {})),
8729:     profileRich: clone(catalog.profileRich || {}),
8730:     siteContent: clone(catalog.siteContent || defaultSiteContent),
8731:     siteContentRich: clone(catalog.siteContentRich || {}),
8732:     projects: [],
8733:     siteSections: clone(catalog.siteSections || [])
8734: };
8735: templates = templateData.templates || [];
8736: setStatus(`Loaded ${catalogData.source} catalog. Builder controls are local-only.`);
8737: draftSavedSinceChanges = true;
8738: setSaveState("loaded", "Loaded");
8739:
8740: templateSelect.innerHTML = groupedTemplateOptions();
8741: refreshCategoryControls();
8742: selectedProjectId = catalog.projects[0]?.id || "";
8743: expandedCategories = new Set(catalog.categories.map((category) => category.id));
8744: renderAll();
8745: }

```

prepareBuilderGuideTopicLinks - lines 8758-8768

Item	Explanation
Source location	Lines 8758-8768 (11 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function prepareBuilderGuideTopicLinks() {
Touches	browser/Electron DOM
Calls detected	querySelectorAll, forEach, querySelector, setAttribute, trim
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

8758: function prepareBuilderGuideTopicLinks() {
8759:   builderGuideSections?.querySelectorAll("[data-guide-section]").forEach((section) => {
8760:     const summary = section.querySelector("summary");
8761:     if (!summary) return;
8762:     summary.setAttribute("role", "link");
8763:     summary.setAttribute("tabindex", "0");
8764:     summary.setAttribute("aria-label", `Open ${summary.textContent.trim()}`);
8765:     summary.title = "Open this guide topic";
8766:     section.open = false;
8767:   });
8768: }

```

updateBuilderGuideStats - lines 8770-8849

Item	Explanation
Source location	Lines 8770-8849 (80 source lines).
Purpose	Updates UI, state, cache, or stored data.
Declaration	function updateBuilderGuideStats() {
Touches	browser/Electron DOM
Calls detected	replace, entries, forEach, querySelector, guideSectionText, join, toLowerCase, filterBuilderGuide, trim, querySelectorAll, split, includes
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 3 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

8770: function updateBuilderGuideStats() {
8771:   if (!builderGuideDialog) return;
8772:   const projectCount = catalog.projects?.length || 0;
8773:   const categoryCount = catalog.categories?.length || 0;
8774:   const parsedCount = savedPortfolioCatalog.projects?.length || 0;
8775:   const targetLabel = currentPublishTarget?.remote
8776:     ? `Target: ${currentPublishTarget.remote.replace(/^https:\/\/github\.com\/\//i, "")}`
8777:     : "Target: not connected";
8778:   const values = {
8779:     projects: `Projects: ${projectCount} (${parsedCount} parsed)`,
8780:     categories: `Categories: ${categoryCount}`,
8781:     draft: draftSavedSinceChanges ? "Draft: saved" : "Draft: unsaved changes",
8782:     target: targetLabel
8783:   };
8784:   Object.entries(values).forEach(([key, value]) => {
8785:     const node = builderGuideDialog.querySelector(`[data-guide-stat='${key}']`);
8786:     if (node) node.textContent = value;
8787:   });
8788: }
8789:
8790: function guideSectionText(section) {
8791:   return [
8792:     section.querySelector("summary")?.textContent || "",
8793:     section.textContent || "",
8794:     section.dataset.guideSection || ""
8795:   ].join(" ").toLowerCase();
8796: }
8797:
8798: function filterBuilderGuide() {
8799:   if (!builderGuideSections) return;
8800:   const query = String(builderGuideSearch?.value || "").trim().toLowerCase();
8801:   const sections = [...builderGuideSections.querySelectorAll("[data-guide-section]")];
8802:   let visibleCount = 0;
8803:   sections.forEach((section) => {
8804:     const topicMatch = activeBuilderGuideTopic === "all" || (section.dataset.guideSection ||
8805:       "").split(/\s+/).includes(activeBuilderGuideTopic);
8806:     const queryMatch = !query || guideSectionText(section).includes(query);
8807:     const visible = topicMatch && queryMatch;
8808:     section.hidden = !visible;
8809:     if (visible) {
8810:       visibleCount += 1;
8811:       section.open = false;
8812:     }
8813:   });
8814:   if (builderGuideResults) {
8815:     const topicText = activeBuilderGuideTopic === "all" ? "all topics" : activeBuilderGuideTopic;
8816:     builderGuideResults.textContent = visibleCount
8817:       ? `Showing ${visibleCount} guide topic${visibleCount === 1 ? "" : "s"} for ${topicText}${query ? `
8818:         matching "${query}"` : ""}` : `No guide topics match "${query}"`;
8819:   }
8820: }
8821:
8822: function setBuilderGuideTopic(topic = "all") {
8823:   activeBuilderGuideTopic = topic || "all";
8824:   builderGuideDialog?.querySelectorAll("[data-guide-topic]").forEach((button) => {
8825:     button.setAttribute("aria-pressed", button.dataset.guideTopic === activeBuilderGuideTopic ? "true" :
8826:       "false");
8827:   });
8828:   filterBuilderGuide();
8829: }
8830:
8831: function closeGuideAndFocus(target) {
8832:   closeDialogElement(builderGuideDialog);
8833:   requestAnimationFrame(() => {
8834:     target?.scrollIntoView?({ behavior: "smooth", block: "center" });
8835:     if (target?.focus) target.focus({ preventScroll: true });
8836:   });
8837: }
8838:
8839: function handleBuilderGuideAction(action = "") {
8840:   if (action === "profile") closeGuideAndFocus(profileDisplayNameInput);
8841:   if (action === "site-content") closeGuideAndFocus(siteHeroTitleInput);
8842:   if (action === "fun-facts") closeGuideAndFocus(funFactsInput);
8843:   if (action === "projects") closeGuideAndFocus(projectTree);
8844:   if (action === "preview") {
8845:     closeDialogElement(builderGuideDialog);
8846:     openPortfolioPreviewButton?.click();
8847:   }
8848:   if (action === "target") {
8849:     closeDialogElement(builderGuideDialog);
8850:     publishTargetOpen?.click();
8851:   }
8852: }

```

guideSectionText - lines 8790-8796

Item	Explanation
Source location	Lines 8790-8796 (7 source lines).

Item	Explanation
Purpose	Part of the private builder frontend and editing experience.
Declaration	function guideSectionText(section) {
Touches	browser/Electron DOM
Calls detected	querySelector, join, toLowerCase
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

8790: function guideSectionText(section) {
8791:   return [
8792:     section.querySelector("summary")?.textContent || "",
8793:     section.textContent || "",
8794:     section.dataset.guideSection || ""
8795:   ].join(" ").toLowerCase();
8796: }

```

filterBuilderGuide - lines 8798-8819

Item	Explanation
Source location	Lines 8798-8819 (22 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function filterBuilderGuide() {
Touches	browser/Electron DOM
Calls detected	trim, toLowerCase, querySelectorAll, forEach, split, includes, guideSectionText
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

8798: function filterBuilderGuide() {
8799:   if (!builderGuideSections) return;
8800:   const query = String(builderGuideSearch?.value || "").trim().toLowerCase();
8801:   const sections = [...builderGuideSections.querySelectorAll("[data-guide-section]")];
8802:   let visibleCount = 0;
8803:   sections.forEach((section) => {
8804:     const topicMatch = activeBuilderGuideTopic === "all" || (section.dataset.guideSection ||
8805:       "").split(/\s+/).includes(activeBuilderGuideTopic);
8806:     const queryMatch = !query || guideSectionText(section).includes(query);
8807:     const visible = topicMatch && queryMatch;
8808:     section.hidden = !visible;
8809:     if (visible) {
8810:       visibleCount += 1;
8811:       section.open = false;
8812:     }
8813:   });
8814:   if (builderGuideResults) {
8815:     const topicText = activeBuilderGuideTopic === "all" ? "all topics" : activeBuilderGuideTopic;
8816:     builderGuideResults.textContent = visibleCount
8817:       ? `Showing ${visibleCount} guide topic${visibleCount === 1 ? "" : "s"} for ${topicText}${query ? `
8818:         matching "${query}"` : ""}`
8819:       : `No guide topics match "${query}"`;

```

setBuilderGuideTopic - lines 8821-8827

Item	Explanation
Source location	Lines 8821-8827 (7 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function setBuilderGuideTopic(topic = "all") {
Touches	browser/Electron DOM
Calls detected	querySelectorAll, forEach, setAttribute, filterBuilderGuide
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

8821: function setBuilderGuideTopic(topic = "all") {
8822:   activeBuilderGuideTopic = topic || "all";
8823:   builderGuideDialog?.querySelectorAll("[data-guide-topic]").forEach((button) => {
8824:     button.setAttribute("aria-pressed", button.dataset.guideTopic === activeBuilderGuideTopic ? "true" :
"false");
8825:   });
8826:   filterBuilderGuide();
8827: }

```

closeGuideAndFocus - lines 8829-8835

Item	Explanation
Source location	Lines 8829-8835 (7 source lines).
Purpose	Controls a window, dialog, panel, menu, or visible state.
Declaration	function closeGuideAndFocus(target) {
Touches	Mostly local calculations or local UI state.
Calls detected	closeDialogElement, requestAnimationFrame, focus
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

8829: function closeGuideAndFocus(target) {
8830:   closeDialogElement(builderGuideDialog);
8831:   requestAnimationFrame(() => {
8832:     target?.scrollIntoView?({ behavior: "smooth", block: "center" });
8833:     if (target?.focus) target.focus({ preventScroll: true });
8834:   });
8835: }

```

handleBuilderGuideAction - lines 8837-8854

Item	Explanation
Source location	Lines 8837-8854 (18 source lines).
Purpose	Handles an event, request, command, or user action.
Declaration	function handleBuilderGuideAction(action = "") {
Touches	Mostly local calculations or local UI state.
Calls detected	closeGuideAndFocus, closeDialogElement, click
API endpoints	None detected inside this function body.

Item	Explanation
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

8837: function handleBuilderGuideAction(action = "") {
8838:   if (action === "profile") closeGuideAndFocus(profileDisplayNameInput);
8839:   if (action === "site-content") closeGuideAndFocus(siteHeroTitleInput);
8840:   if (action === "fun-facts") closeGuideAndFocus(funFactsInput);
8841:   if (action === "projects") closeGuideAndFocus(projectTree);
8842:   if (action === "preview") {
8843:     closeDialogElement(builderGuideDialog);
8844:     openPortfolioPreviewButton?.click();
8845:   }
8846:   if (action === "target") {
8847:     closeDialogElement(builderGuideDialog);
8848:     publishTargetOpen?.click();
8849:   }
8850:   if (action === "updates") {
8851:     closeDialogElement(builderGuideDialog);
8852:     appUpdateCheckButton?.click();
8853:   }
8854: }

```

openBuilderGuideTopic - lines 8856-8878

Item	Explanation
Source location	Lines 8856-8878 (23 source lines).
Purpose	Controls a window, dialog, panel, menu, or visible state.
Declaration	function openBuilderGuideTopic(section) {
Touches	browser/Electron DOM, authentication/security data
Calls detected	querySelector, trim, replaceChildren, cloneNode, add, append, createElement, showModal, requestAnimationFrame, focus, updateDialogWindowButtons
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

8856: function openBuilderGuideTopic(section) {
8857:   if (!section || !builderGuideTopicDialog || !builderGuideTopicTitle || !builderGuideTopicBody) return;
8858:   const summary = section.querySelector("summary");
8859:   const source = section.querySelector(":scope > div");
8860:   const title = summary?.textContent?.trim() || "Builder guide topic";
8861:   builderGuideTopicTitle.textContent = title;
8862:   builderGuideTopicBody.replaceChildren();
8863:   if (source) {
8864:     const copy = source.cloneNode(true);
8865:     copy.classList.add("builder-guide-topic-copy");
8866:     builderGuideTopicBody.append(copy);
8867:   } else {
8868:     const empty = document.createElement("p");
8869:     empty.textContent = "This guide topic does not have written instructions yet.";
8870:     builderGuideTopicBody.append(empty);
8871:   }
8872:   if (!builderGuideTopicDialog.open) builderGuideTopicDialog.showModal();
8873:   requestAnimationFrame(() => {
8874:     builderGuideTopicBody.scrollTop = 0;
8875:     builderGuideTopicBody.focus({ preventScroll: true });
8876:     updateDialogWindowButtons(builderGuideTopicDialog);
8877:   });
8878: }

```

handleRichEditorContextMenu - lines 9312-9372

Item	Explanation
Source location	Lines 9312-9372 (61 source lines).
Purpose	Handles an event, request, command, or user action.
Declaration	function handleRichEditorContextMenu(event) {
Touches	browser/Electron DOM
Calls detected	richEditorFromContextEvent, preventDefault, stopPropagation, closest, clearPersistentTextSelection, hideTextSelectionInspector, configureSummaryContextMenu, selectionRangeInsideEditor, cloneRange, contains, restoreRichSelection, captureRichSelection
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

9312: function handleRichEditorContextMenu(event) {
9313:     const editor = richEditorFromContextEvent(event);
9314:     const textBlock = event.target.closest?(".rich-text-block");
9315:
9316:     if (!editor && !textBlock) return;
9317:
9318:     event.preventDefault();
9319:     event.stopPropagation();
9320:     const nextEditor = editor || textBlock.closest("[data-rich-editor]");
9321:     if (activeSummaryEditor && nextEditor && activeSummaryEditor !== nextEditor) {
9322:         activeRichSelectionRange = null;
9323:         activeTextSelectionRange = null;
9324:         clearPersistentTextSelection(true);
9325:         summaryContextMenu.hidden = true;
9326:     }
9327:     hideTextSelectionInspector();
9328:
9329:     activePlainTextControl = null;
9330:     activePlainTextSelection = null;
9331:     activeSummaryEditor = nextEditor;
9332:     configureSummaryContextMenu("rich", { textOnly: activeSummaryEditor.dataset.richTextOnly === "true"
});
9333:     const liveRange = selectionRangeInsideEditor(activeSummaryEditor);
9334:     const savedRange = activeTextSelectionRange && !activeTextSelectionRange.collapsed
9335:         ? activeTextSelectionRange.cloneRange()
9336:         : null;
9337:     const savedContainer = savedRange?.commonAncestorContainer?.nodeType === Node.TEXT_NODE
9338:         ? savedRange.commonAncestorContainer.parentElement
9339:         : savedRange?.commonAncestorContainer;
9340:     if (liveRange && !liveRange.collapsed) {
9341:         activeRichSelectionRange = liveRange.cloneRange();
9342:         activeTextSelectionRange = liveRange.cloneRange();
9343:     } else if (savedRange && savedContainer && activeSummaryEditor.contains(savedContainer)) {
9344:         activeRichSelectionRange = savedRange.cloneRange();
9345:         restoreRichSelection(activeSummaryEditor);
9346:     } else {
9347:         captureRichSelection(activeSummaryEditor);
9348:     }
9349:     selectRichBlock(event.target.closest?(".rich-block") || currentRichBlock(activeSummaryEditor));
9350:     syncRichContextMenuControls(activeSummaryBlock);
9351:
9352:     const menuHost = activeSummaryEditor.closest("dialog") || document.body;
9353:     if (summaryContextMenu.parentElement !== menuHost) menuHost.append(summaryContextMenu);
9354:
9355:     summaryContextMenu.style.left = `${event.clientX}px`;
9356:     summaryContextMenu.style.top = `${event.clientY}px`;
9357:     summaryContextMenu.style.maxHeight = "";
9358:     summaryContextMenu.hidden = false;
9359:     const hostRect = menuHost.getBoundingClientRect();
9360:     const bounds = {
9361:         bottom: Math.min(window.innerHeight - 8, hostRect.bottom - 8),
9362:         left: Math.max(8, hostRect.left + 8),
9363:         right: Math.min(window.innerWidth - 8, hostRect.right - 8),
9364:         top: Math.max(8, hostRect.top + 8)
9365:     };
9366:     summaryContextMenu.style.maxHeight = `${Math.max(180, bounds.bottom - bounds.top)}px`;
9367:     const menuRect = summaryContextMenu.getBoundingClientRect();
9368:     const left = Math.max(bounds.left, Math.min(event.clientX, bounds.right - menuRect.width));
9369:     const top = Math.max(bounds.top, Math.min(event.clientY, bounds.bottom - menuRect.height));
9370:     summaryContextMenu.style.left = `${left}px`;
9371:     summaryContextMenu.style.top = `${top}px`;
9372: }

```

handlePlainTextContextMenu - lines 9398-9433

Item	Explanation
Source location	Lines 9398-9433 (36 source lines).
Purpose	Handles an event, request, command, or user action.
Declaration	function handlePlainTextContextMenu(event) {
Touches	browser/Electron DOM
Calls detected	plainTextControlFromContextEvent, preventDefault, hideTextSelectionInspector, configureSummaryContextMenu, syncPlainContextMenuControls, closest, append, getBoundingClientRect, min, max
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
9398: function handlePlainTextContextMenu(event) {
9399:   const control = plainTextControlFromContextEvent(event);
9400:   if (!control) return;
9401:
9402:   event.preventDefault();
9403:   hideTextSelectionInspector();
9404:   activeSummaryEditor = null;
9405:   activeSummaryBlock = null;
9406:   activePlainTextControl = control;
9407:   activePlainTextSelection = {
9408:     start: control.selectionStart ?? 0,
9409:     end: control.selectionEnd ?? control.value.length
9410:   };
9411:   configureSummaryContextMenu("plain");
9412:   syncPlainContextMenuControls(control);
9413:
9414:   const menuHost = control.closest("dialog") || document.body;
9415:   if (summaryContextMenu.parentElement !== menuHost) menuHost.append(summaryContextMenu);
9416:   summaryContextMenu.style.left = `${event.clientX}px`;
9417:   summaryContextMenu.style.top = `${event.clientY}px`;
9418:   summaryContextMenu.style.maxHeight = "";
9419:   summaryContextMenu.hidden = false;
9420:   const hostRect = menuHost.getBoundingClientRect();
9421:   const bounds = {
9422:     bottom: Math.min(window.innerHeight - 8, hostRect.bottom - 8),
9423:     left: Math.max(8, hostRect.left + 8),
9424:     right: Math.min(window.innerWidth - 8, hostRect.right - 8),
9425:     top: Math.max(8, hostRect.top + 8)
9426:   };
9427:   summaryContextMenu.style.maxHeight = `${Math.max(140, bounds.bottom - bounds.top)}px`;
9428:   const menuRect = summaryContextMenu.getBoundingClientRect();
9429:   const left = Math.max(bounds.left, Math.min(event.clientX, bounds.right - menuRect.width));
9430:   const top = Math.max(bounds.top, Math.min(event.clientY, bounds.bottom - menuRect.height));
9431:   summaryContextMenu.style.left = `${left}px`;
9432:   summaryContextMenu.style.top = `${top}px`;
9433: }
```

handleRichEditorPaste - lines 9437-9480

Item	Explanation
Source location	Lines 9437-9480 (44 source lines).
Purpose	Handles an event, request, command, or user action.
Declaration	async function handleRichEditorPaste(event) {
Touches	browser/Electron DOM
Calls detected	closest, find, startsWith, preventDefault, setStatus, uploadSummaryImageFile, getAsFile, insertRichBlockAtCursor, createRichImageBlock, saveRichEditorToProject, getData, insertHtmlIntoRichEditor
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	1 await expression(s), 5 return statement(s).

Item	Explanation
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

9437: async function handleRichEditorPaste(event) {
9438:   const editor = event.target.closest("[data-rich-editor]");
9439:   if (!editor) return;
9440:
9441:   const imageItem = [...(event.clipboardData?.items || [])].find((item) =>
item.type.startsWith("image/"));
9442:
9443:   if (imageItem) {
9444:     event.preventDefault();
9445:     if (editor.dataset.richTextOnly === "true") {
9446:       setStatus("Images can only be pasted into overview editors, not title, profile, or contact
fields.");
9447:       return;
9448:     }
9449:
9450:     const imageBlock = await uploadSummaryImageFile(imageItem.getAsFile(), {
9451:       align: "center",
9452:       display: "show",
9453:       folder: editor.dataset.richFolder || "summary",
9454:       projectId: editor.dataset.richProjectId || "",
9455:       title: ""
9456:     });
9457:
9458:     if (imageBlock) {
9459:       insertRichBlockAtCursor(editor, createRichImageBlock(imageBlock));
9460:       saveRichEditorToProject(editor);
9461:     }
9462:
9463:     return;
9464:   }
9465:
9466:   const pastedHtml = event.clipboardData?.getData("text/html") || "";
9467:   if (pastedHtml) {
9468:     event.preventDefault();
9469:     insertHtmlIntoRichEditor(editor, pastedHtml);
9470:     saveRichEditorToProject(editor);
9471:     return;
9472:   }
9473:
9474:   const pastedText = event.clipboardData?.getData("text/plain") || "";
9475:   if (!pastedText) return;
9476:
9477:   event.preventDefault();
9478:   insertPlainTextIntoRichEditor(editor, pastedText);
9479:   saveRichEditorToProject(editor);
9480: }

```

restoreRichInsertionPoint - lines 9494-9503

Item	Explanation
Source location	Lines 9494-9503 (10 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function restoreRichInsertionPoint(editor) {
Touches	browser/Electron DOM
Calls detected	restoreRichSelection, focusTextBlock, selectedRichBlock, querySelector, focus, selectionRangeInsideEditor, deleteContents
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

9494: function restoreRichInsertionPoint(editor) {
9495:   if (!restoreRichSelection(editor)) focusTextBlock(selectedRichBlock(editor) ||
editor.querySelector(".rich-text-block"));
9496:   editor.focus({ preventScroll: true });
9497:
9498:   const range = selectionRangeInsideEditor(editor);

```

```

9499:     if (!range) return null;
9500:
9501:     range.deleteContents();
9502:     return range;
9503: }

```

insertFragmentIntoRichEditor - lines 9505-9521

Item	Explanation
Source location	Lines 9505-9521 (17 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function insertFragmentIntoRichEditor(editor, fragment) {
Touches	browser/Electron DOM
Calls detected	restoreRichInsertionPoint, insertNode, createRange, setStartAfter, setStart, collapse, getSelection, removeAllRanges, addRange, captureRichSelection
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

9505: function insertFragmentIntoRichEditor(editor, fragment) {
9506:     const range = restoreRichInsertionPoint(editor);
9507:     if (!range || !fragment) return;
9508:
9509:     const lastInsertedNode = fragment.lastChild;
9510:     range.insertNode(fragment);
9511:
9512:     const nextRange = document.createRange();
9513:     if (lastInsertedNode) nextRange.setStartAfter(lastInsertedNode);
9514:     else nextRange.setStart(range.startContainer, range.startOffset);
9515:     nextRange.collapse(true);
9516:
9517:     const selection = window.getSelection();
9518:     selection.removeAllRanges();
9519:     selection.addRange(nextRange);
9520:     captureRichSelection(editor);
9521: }

```

insertPlainTextIntoRichEditor - lines 9523-9533

Item	Explanation
Source location	Lines 9523-9533 (11 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function insertPlainTextIntoRichEditor(editor, pastedText) {
Touches	browser/Electron DOM
Calls detected	replace, createDocumentFragment, split, forEach, append, createElement, createTextNode, insertFragmentIntoRichEditor
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

9523: function insertPlainTextIntoRichEditor(editor, pastedText) {
9524:     const textToInsert = String(pastedText || "").replace(/\r\n?/g, "\n");
9525:     if (!textToInsert) return;
9526:
9527:     const fragment = document.createDocumentFragment();
9528:     textToInsert.split("\n").forEach((line, index) => {
9529:         if (index > 0) fragment.append(document.createElement("br"));
9530:         if (line) fragment.append(document.createTextNode(line));

```

```

9531:     });
9532:     insertFragmentIntoRichEditor(editor, fragment);
9533: }

```

insertHtmlIntoRichEditor - lines 9535-9543

Item	Explanation
Source location	Lines 9535-9543 (9 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function insertHtmlIntoRichEditor(editor, pastedHtml) {
Touches	browser/Electron DOM, filesystem
Calls detected	replace, createElement, sanitizeRichInlineHtml, insertFragmentIntoRichEditor, cloneNode
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

9535: function insertHtmlIntoRichEditor(editor, pastedHtml) {
9536:     const normalizedHtml = String(pastedHtml || "")
9537:       .replace(/<\/(?<:p|div|li|h[1-6]|tr)>/gi, "<br>")
9538:       .replace(/</(?<:p|div|li|h[1-6]|tr|td|th)[^>]*>/gi, "")
9539:       .replace(/<br\s*\/*>\s*(<br\s*\/*>\s*){2,}/gi, "<br><br>");
9540:     const template = document.createElement("template");
9541:     template.innerHTML = sanitizeRichInlineHtml(normalizedHtml);
9542:     insertFragmentIntoRichEditor(editor, template.content.cloneNode(true));
9543: }

```

handleRichEditorKeydown - lines 9565-9610

Item	Explanation
Source location	Lines 9565-9610 (46 source lines).
Purpose	Handles an event, request, command, or user action.
Declaration	function handleRichEditorKeydown(event) {
Touches	browser/Electron DOM
Calls detected	closest, includes, toLowerCase, preventDefault, applyRichInlineCommand, execCommand, captureRichSelection, selectionRangeInsideEditor, contains, deleteContents, createRange, setStart
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 6 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

9565: function handleRichEditorKeydown(event) {
9566:     const editor = event.target.closest("[data-rich-editor]");
9567:     if (!editor) return;
9568:
9569:     if ((event.ctrlKey || event.metaKey) && ["b", "i", "u"].includes(event.key.toLowerCase())) {
9570:         event.preventDefault();
9571:         const command = event.key.toLowerCase() === "b"
9572:           ? "bold"
9573:           : event.key.toLowerCase() === "i"
9574:             ? "italic"
9575:             : "underline";
9576:         applyRichInlineCommand(editor, command);
9577:         return;
9578:     }
9579:
9580:     if (event.key !== "Enter") return;
9581:
9582:     const block = event.target.closest(".rich-text-block");
9583:     if (!block) return;

```

```

9584:
9585:     event.preventDefault();
9586:     if (event.shiftKey) {
9587:         document.execCommand("insertLineBreak", false);
9588:         captureRichSelection(editor);
9589:         return;
9590:     }
9591:
9592:     const range = selectionRangeInsideEditor(editor);
9593:     if (!range || !block.contains(range.startContainer)) return;
9594:     range.deleteContents();
9595:     const tail = document.createRange();
9596:     tail.setStart(range.startContainer, range.startOffset);
9597:     tail.setEnd(block, block.childNodes.length);
9598:     const trailingContent = tail.extractContents();
9599:     const nextBlock = matchingTextBlock(block, "");
9600:     nextBlock.append(trailingContent);
9601:     block.after(nextBlock);
9602:     const nextRange = document.createRange();
9603:     nextRange.selectNodeContents(nextBlock);
9604:     nextRange.collapse(true);
9605:     const selection = window.getSelection();
9606:     selection.removeAllRanges();
9607:     selection.addRange(nextRange);
9608:     captureRichSelection(editor);
9609:     setStatus("Unsaved local changes.");
9610: }

```

handleRichEditorFocus - lines 9632-9637

Item	Explanation
Source location	Lines 9632-9637 (6 source lines).
Purpose	Handles an event, request, command, or user action.
Declaration	function handleRichEditorFocus(event) {
Touches	browser/Electron DOM
Calls detected	closest, execCommand, captureRichSelection
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

9632: function handleRichEditorFocus(event) {
9633:     const editor = event.target.closest("[data-rich-editor]");
9634:     if (!editor) return;
9635:     document.execCommand("defaultParagraphSeparator", false, "p");
9636:     captureRichSelection(editor);
9637: }

```

handleRichEditorInput - lines 9639-9644

Item	Explanation
Source location	Lines 9639-9644 (6 source lines).
Purpose	Handles an event, request, command, or user action.
Declaration	function handleRichEditorInput(event) {
Touches	Mostly local calculations or local UI state.
Calls detected	closest, captureRichSelection, setStatus
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:


```

9639: function handleRichEditorInput(event) {
9640:   const editor = event.target.closest("[data-rich-editor]");
9641:   if (!editor) return;
9642:   captureRichSelection(editor);
9643:   setStatus("Unsaved local changes.");
9644: }

```

handleRichEditorBeforeInput - lines 9646-9653

Item	Explanation
Source location	Lines 9646-9653 (8 source lines).
Purpose	Handles an event, request, command, or user action.
Declaration	function handleRichEditorBeforeInput(event) {
Touches	Mostly local calculations or local UI state.
Calls detected	closest, clearPersistentTextSelection
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

9646: function handleRichEditorBeforeInput(event) {
9647:   const editor = event.target.closest("[data-rich-editor]");
9648:   if (!editor) return;
9649:   const inputType = event.inputType || "";
9650:   if (!/insertFromPaste|insertFromDrop/i.test(inputType) && /^(insert|delete|history)/i.test(inputType))
{
9651:     clearPersistentTextSelection(true);
9652:   }
9653: }

```

finishTextSelectionGesture - lines 9728-9753

Item	Explanation
Source location	Lines 9728-9753 (26 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function finishTextSelectionGesture(event) {
Touches	Mostly local calculations or local UI state.
Calls detected	getSelection, selectionRangeInsideEditor, cloneRange, showPersistentTextSelection, rangeBelongsToEditor, clearPersistentTextSelection, refreshTextSelectionInspector
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

9728: function finishTextSelectionGesture(event) {
9729:   if (event?.button === 2 || event?.target?.closest?("#text-selection-inspector, #summary-context-
menu")) return;
9730:   selectionGestureActive = false;
9731:   clearTimeout(selectionGestureFinishTimer);
9732:   selectionGestureFinishTimer = setTimeout(() => {
9733:     const selection = window.getSelection();
9734:     let node = selection?.anchorNode;
9735:     if (node?.nodeType === Node.TEXT_NODE) node = node.parentElement;
9736:     const editor = node?.closest?("[data-rich-editor]");
9737:     const range = editor ? selectionRangeInsideEditor(editor) : null;
9738:     if (editor && range && !range.collapsed) {
9739:       activeSummaryEditor = editor;
9740:       activeRichSelectionRange = range.cloneRange();
9741:       activeTextSelectionRange = range.cloneRange();
9742:       selectionGestureProducedRange = true;

```

```

9743:     showPersistentTextSelection(range);
9744:   } else if (!selectionGestureProducedRange) {
9745:     if (editor && rangeBelongsToEditor(activeTextSelectionRange, editor)) {
9746:       showPersistentTextSelection(activeTextSelectionRange);
9747:     } else {
9748:       clearPersistentTextSelection(true);
9749:     }
9750:   }
9751:   refreshTextSelectionInspector();
9752: }, 40);
9753: }

```

endSelectionInspectorDrag - lines 9814-9819

Item	Explanation
Source location	Lines 9814-9819 (6 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function endSelectionInspectorDrag() {
Touches	browser/Electron DOM
Calls detected	remove
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

9814: function endSelectionInspectorDrag() {
9815:   if (!activeSelectionInspectorDrag) return;
9816:   activeSelectionInspectorDrag.target?.releasePointerCapture?.(activeSelectionInspectorDrag.pointerId);
9817:   textSelectionInspector?.classList.remove("is-dragging");
9818:   activeSelectionInspectorDrag = null;
9819: }

```

handleSelectionInspectorTextField - lines 9843-9849

Item	Explanation
Source location	Lines 9843-9849 (7 source lines).
Purpose	Handles an event, request, command, or user action.
Declaration	function handleSelectionInspectorTextField(event) {
Touches	filesystem
Calls detected	preventDefault, applySelectionInspectorFormat
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

9843: function handleSelectionInspectorTextField(event) {
9844:   if (event.type === "keydown" && event.key !== "Enter") return;
9845:   if (event.type === "keydown") event.preventDefault();
9846:   if (event.target === selectionFontInput) applySelectionInspectorFormat("font", event.target.value);
9847:   if (event.target === selectionColorInput) applySelectionInspectorFormat("color", event.target.value);
9848:   if (event.target === selectionSizeInput) applySelectionInspectorFormat("size", event.target.value);
9849: }

```

scheduleSelectionInspectorTextField - lines 9851-9859

Item	Explanation
Source location	Lines 9851-9859 (9 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function scheduleSelectionInspectorTextField(event) {
Touches	filesystem
Calls detected	cleanFontFamily, applySelectionInspectorFormat, normalizeTextColor, normalizeFontPx
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

9851: function scheduleSelectionInspectorTextField(event) {
9852:   clearTimeout(selectionInspectorInputTimer);
9853:   const target = event.target;
9854:   selectionInspectorInputTimer = setTimeout(() => {
9855:     if (target === selectionFontInput && cleanFontFamily(target.value))
applySelectionInspectorFormat("font", target.value);
9856:     if (target === selectionColorInput && normalizeTextColor(target.value))
applySelectionInspectorFormat("color", target.value);
9857:     if (target === selectionSizeInput && normalizeFontPx(target.value))
applySelectionInspectorFormat("size", target.value);
9858:   }, 350);
9859: }

```

focusAdjacentImageText - lines 9871-9882

Item	Explanation
Source location	Lines 9871-9882 (12 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function focusAdjacentImageText(block, direction) {
Touches	Mostly local calculations or local UI state.
Calls detected	closest, matches, matchingTextBlock, before, after, focusTextBlock, saveRichEditorToProject
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

9871: function focusAdjacentImageText(block, direction) {
9872:   const editor = block?.closest("[data-rich-editor]");
9873:   if (!editor) return;
9874:   const sibling = direction === "before" ? block.previousElementSibling : block.nextElementSibling;
9875:   const textBlock = sibling?.matches(".rich-text-block") ? sibling : matchingTextBlock(block, "");
9876:   if (textBlock !== sibling) {
9877:     if (direction === "before") block.before(textBlock);
9878:     else block.after(textBlock);
9879:   }
9880:   focusTextBlock(textBlock);
9881:   saveRichEditorToProject(editor);
9882: }

```

handleRichCaretClick - lines 9884-9889

Item	Explanation
Source location	Lines 9884-9889 (6 source lines).
Purpose	Handles an event, request, command, or user action.

Item	Explanation
Declaration	function handleRichCaretClick(event) {
Touches	browser/Electron DOM
Calls detected	closest, focusAdjacentImageText
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

9884: function handleRichCaretClick(event) {
9885:   const caretButton = event.target.closest("[data-rich-caret]");
9886:   if (!caretButton) return false;
9887:   focusAdjacentImageText(caretButton.closest(".rich-image-block"), caretButton.dataset.richCaret);
9888:   return true;
9889: }

```

handleRichDragStart - lines 9891-9910

Item	Explanation
Source location	Lines 9891-9910 (20 source lines).
Purpose	Handles an event, request, command, or user action.
Declaration	function handleRichDragStart(event) {
Touches	browser/Electron DOM
Calls detected	closest, preventDefault, selectRichBlock, add, clearData, setData
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

9891: function handleRichDragStart(event) {
9892:   const handle = event.target.closest("[data-rich-drag-handle]");
9893:   const imageBlock = event.target.closest(".rich-image-block");
9894:   const isControl = event.target.closest("button, input, select, textarea, a, .rich-block-actions, [data-rich-caret]");
9895:   const block = handle?.closest(".rich-block") || (!isControl ? imageBlock : null);
9896:   if (!block) {
9897:     if (imageBlock) event.preventDefault();
9898:     return;
9899:   }
9900:   activeRichDragBlock = block;
9901:   selectRichBlock(activeRichDragBlock);
9902:   activeRichDragBlock.classList.add("is-dragging");
9903:   if (event.dataTransfer) {
9904:     event.dataTransfer.clearData();
9905:     event.dataTransfer.effectAllowed = "move";
9906:     event.dataTransfer.dropEffect = "move";
9907:     event.dataTransfer.setData("text/x-rich-block", "move");
9908:     event.dataTransfer.setData("text/plain", activeRichDragBlock.dataset.title ||
activeRichDragBlock.dataset.type || "image");
9909:   }
9910: }

```

handleRichDragOver - lines 9912-9919

Item	Explanation
Source location	Lines 9912-9919 (8 source lines).
Purpose	Handles an event, request, command, or user action.

Item	Explanation
Declaration	function handleRichDragOver(event) {
Touches	Mostly local calculations or local UI state.
Calls detected	closest, preventDefault, updateRichDropMarker
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

9912: function handleRichDragOver(event) {
9913:   if (!activeRichDragBlock) return;
9914:   const editor = event.target.closest("[data-rich-editor]");
9915:   if (!editor || editor !== activeRichDragBlock.closest("[data-rich-editor]")) return;
9916:   event.preventDefault();
9917:   if (event.dataTransfer) event.dataTransfer.dropEffect = "move";
9918:   updateRichDropMarker(editor, activeRichDragBlock, event.clientX, event.clientY, event.target);
9919: }

```

richRangeFromPoint - lines 9921-9933

Item	Explanation
Source location	Lines 9921-9933 (13 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function richRangeFromPoint(x, y) {
Touches	browser/Electron DOM
Calls detected	caretRangeFromPoint, caretPositionFromPoint, createRange, setStart, collapse
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 3 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

9921: function richRangeFromPoint(x, y) {
9922:   if (document.caretRangeFromPoint) return document.caretRangeFromPoint(x, y);
9923:   if (document.caretPositionFromPoint) {
9924:     const position = document.caretPositionFromPoint(x, y);
9925:     if (position) {
9926:       const range = document.createRange();
9927:       range.setStart(position.offsetNode, position.offset);
9928:       range.collapse(true);
9929:       return range;
9930:     }
9931:   }
9932:   return null;
9933: }

```

richDropLocation - lines 9935-9950

Item	Explanation
Source location	Lines 9935-9950 (16 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function richDropLocation(editor, movingBlock, x, y, eventTarget = null) {
Touches	browser/Electron DOM
Calls detected	validDropRangeForEditor, richRangeFromPoint, contains, elementFromPoint
API endpoints	None detected inside this function body.

Item	Explanation
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

9935: function richDropLocation(editor, movingBlock, x, y, eventTarget = null) {
9936:   const previousPointerEvents = movingBlock?.style.pointerEvents || "";
9937:   if (movingBlock) movingBlock.style.pointerEvents = "none";
9938:   try {
9939:     const dropRange = validDropRangeForEditor(editor, movingBlock, richRangeFromPoint(x, y));
9940:     const pointTarget = eventTarget && !movingBlock?.contains(eventTarget)
9941:       ? eventTarget
9942:       : document.elementFromPoint(x, y);
9943:     return {
9944:       dropRange,
9945:       target: pointTarget?.closest?(".rich-block") || null
9946:     };
9947:   } finally {
9948:     if (movingBlock) movingBlock.style.pointerEvents = previousPointerEvents;
9949:   }
9950: }

```

validDropRangeForEditor - lines 9952-9960

Item	Explanation
Source location	Lines 9952-9960 (9 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function validDropRangeForEditor(editor, movingBlock, range) {
Touches	Mostly local calculations or local UI state.
Calls detected	contains, closest
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 4 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

9952: function validDropRangeForEditor(editor, movingBlock, range) {
9953:   const rangeNode = range?.startContainer?.nodeType === Node.TEXT_NODE
9954:     ? range.startContainer.parentElement
9955:     : range?.startContainer;
9956:   if (!range || !rangeNode || !editor.contains(rangeNode)) return null;
9957:   if (movingBlock?.contains(rangeNode)) return null;
9958:   if (rangeNode.closest("[contenteditable='false']")) return null;
9959:   return range;
9960: }

```

ensureRichDropMarker - lines 9962-9971

Item	Explanation
Source location	Lines 9962-9971 (10 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function ensureRichDropMarker() {
Touches	browser/Electron DOM
Calls detected	createElement, setAttribute, append
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).

Item	Explanation
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

9962: function ensureRichDropMarker() {
9963:   if (!activeRichDropMarker) {
9964:     activeRichDropMarker = document.createElement("div");
9965:     activeRichDropMarker.className = "rich-drop-marker";
9966:     activeRichDropMarker.setAttribute("aria-hidden", "true");
9967:     document.body.append(activeRichDropMarker);
9968:   }
9969:   activeRichDropMarker.hidden = false;
9970:   return activeRichDropMarker;
9971: }

```

hideRichDropMarker - lines 9973-9975

Item	Explanation
Source location	Lines 9973-9975 (3 source lines).
Purpose	Controls a window, dialog, panel, menu, or visible state.
Declaration	function hideRichDropMarker() {
Touches	Mostly local calculations or local UI state.
Calls detected	No obvious named calls detected by the generator.
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

9973: function hideRichDropMarker() {
9974:   if (activeRichDropMarker) activeRichDropMarker.hidden = true;
9975: }

```

updateRichDropMarker - lines 9977-10006

Item	Explanation
Source location	Lines 9977-10006 (30 source lines).
Purpose	Updates UI, state, cache, or stored data.
Declaration	function updateRichDropMarker(editor, movingBlock, x, y, eventTarget = null) {
Touches	Mostly local calculations or local UI state.
Calls detected	ensureRichDropMarker, richDropLocation, getClientRects, getBoundingClientRect, max, min, contains
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

9977: function updateRichDropMarker(editor, movingBlock, x, y, eventTarget = null) {
9978:   if (!editor || !movingBlock) return;
9979:   const marker = ensureRichDropMarker();
9980:   const { dropRange, target } = richDropLocation(editor, movingBlock, x, y, eventTarget);
9981:   if (dropRange) {
9982:     const rangeRect = dropRange.getClientRects()[0];
9983:     const anchor = dropRange.startContainer.nodeType === Node.TEXT_NODE
9984:       ? dropRange.startContainer.parentElement
9985:       : dropRange.startContainer;
9986:     const anchorRect = anchor?.getBoundingClientRect?().();

```

```

9987:     const rect = rangeRect || anchorRect || editor.getBoundingClientRect();
9988:     marker.className = "rich-drop-marker caret";
9989:     marker.style.left = `${Math.max(rect.left, Math.min(x, rect.right || rect.left))}px`;
9990:     marker.style.top = `${rect.top}px`;
9991:     marker.style.width = "2px";
9992:     marker.style.height = `${Math.max(22, rect.height || 22)}px`;
9993:     return;
9994: }
9995:
9996: const targetRect = (target && editor.contains(target) && target !== movingBlock)
9997:   ? target.getBoundingClientRect()
9998:   : editor.getBoundingClientRect();
9999: const imageRect = movingBlock.getBoundingClientRect();
10000: const beforeTarget = !target || y < targetRect.top + targetRect.height / 2;
10001: marker.className = "rich-drop-marker block";
10002: marker.style.left = `${targetRect.left}px`;
10003: marker.style.top = `${beforeTarget ? targetRect.top : targetRect.bottom}px`;
10004: marker.style.width = `${Math.min(targetRect.width, Math.max(160, imageRect.width ||
targetRect.width))}px`;
10005: marker.style.height = "3px";
10006: }

```

moveRichBlockToPoint - lines 10008-10029

Item	Explanation
Source location	Lines 10008-10029 (22 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function moveRichBlockToPoint(editor, movingBlock, x, y, eventTarget = null, options = {}) {
Touches	browser/Electron DOM
Calls detected	richDropLocation, remove, insertRichBlockAtCursor, contains, append, getBoundingClientRect, before, after, cleanupEmptyRichTextBlocks, focusAdjacentImageText, saveRichEditorToProject, selectRichBlock
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

10008: function moveRichBlockToPoint(editor, movingBlock, x, y, eventTarget = null, options = {}) {
10009:   if (!editor || !movingBlock) return;
10010:   const { dropRange, target } = richDropLocation(editor, movingBlock, x, y, eventTarget);
10011:
10012:   movingBlock.classList.remove("is-dragging");
10013:   if (target === movingBlock) return;
10014:
10015:   if (dropRange) {
10016:     insertRichBlockAtCursor(editor, movingBlock, dropRange);
10017:   } else if (!target || !editor.contains(target)) {
10018:     editor.append(movingBlock);
10019:   } else if (y < target.getBoundingClientRect().top + target.getBoundingClientRect().height / 2) {
10020:     target.before(movingBlock);
10021:   } else {
10022:     target.after(movingBlock);
10023:   }
10024:
10025:   cleanupEmptyRichTextBlocks(editor);
10026:   if (options.focusAfter !== false) focusAdjacentImageText(movingBlock, "after");
10027:   if (options.commit !== false) saveRichEditorToProject(editor);
10028:   if (options.focusAfter === false) selectRichBlock(movingBlock);
10029: }

```

handleRichDrop - lines 10031-10039

Item	Explanation
Source location	Lines 10031-10039 (9 source lines).
Purpose	Handles an event, request, command, or user action.
Declaration	function handleRichDrop(event) {
Touches	Mostly local calculations or local UI state.
Calls detected	closest, preventDefault, moveRichBlockToPoint, hideRichDropMarker

Item	Explanation
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

10031: function handleRichDrop(event) {
10032:   if (!activeRichDragBlock) return;
10033:   const editor = event.target.closest("[data-rich-editor]");
10034:   if (!editor || editor !== activeRichDragBlock.closest("[data-rich-editor]")) return;
10035:   event.preventDefault();
10036:   moveRichBlockToPoint(editor, activeRichDragBlock, event.clientX, event.clientY, event.target);
10037:   hideRichDropMarker();
10038:   activeRichDragBlock = null;
10039: }

```

imageMoveDragControl - lines 10041-10043

Item	Explanation
Source location	Lines 10041-10043 (3 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function imageMoveDragControl(target) {
Touches	Mostly local calculations or local UI state.
Calls detected	closest
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

10041: function imageMoveDragControl(target) {
10042:   return target.closest("button, input, select, textarea, a, .rich-block-actions, [data-rich-caret]");
10043: }

```

beginRichImageMoveDrag - lines 10045-10061

Item	Explanation
Source location	Lines 10045-10061 (17 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function beginRichImageMoveDrag(event) {
Touches	Mostly local calculations or local UI state.
Calls detected	closest, imageMoveDragControl
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 4 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

10045: function beginRichImageMoveDrag(event) {
10046:   if (event.button !== 0) return;
10047:   const block = event.target.closest(".rich-image-block");
10048:   if (!block || imageMoveDragControl(event.target)) return;
10049:   if (event.target.closest(".rich-image-viewport.crop-active")) return;

```

```

10050:   const editor = block.closest("[data-rich-editor]");
10051:   if (!editor) return;
10052:   activeRichImageMoveDrag = {
10053:     block,
10054:     editor,
10055:     moved: false,
10056:     pointerId: event.pointerId,
10057:     startX: event.clientX,
10058:     startY: event.clientY
10059:   };
10060:   block.setPointerCapture?.(event.pointerId);
10061: }

```

moveRichImageMoveDrag - lines 10063-10077

Item	Explanation
Source location	Lines 10063-10077 (15 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function moveRichImageMoveDrag(event) {
Touches	browser/Electron DOM
Calls detected	hypot, selectRichBlock, add, updateRichDropMarker, preventDefault
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

10063: function moveRichImageMoveDrag(event) {
10064:   if (!activeRichImageMoveDrag) return;
10065:   const drag = activeRichImageMoveDrag;
10066:   const distance = Math.hypot(event.clientX - drag.startX, event.clientY - drag.startY);
10067:   if (!drag.moved && distance < 6) return;
10068:   if (!drag.moved) {
10069:     drag.moved = true;
10070:     activeRichDragBlock = drag.block;
10071:     selectRichBlock(drag.block);
10072:     drag.block.classList.add("is-dragging");
10073:     document.body.classList.add("rich-image-dragging");
10074:   }
10075:   updateRichDropMarker(drag.editor, drag.block, event.clientX, event.clientY, event.target);
10076:   event.preventDefault();
10077: }

```

endRichImageMoveDrag - lines 10079-10092

Item	Explanation
Source location	Lines 10079-10092 (14 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function endRichImageMoveDrag(event) {
Touches	browser/Electron DOM
Calls detected	moveRichBlockToPoint, hideRichDropMarker, preventDefault, remove
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

10079: function endRichImageMoveDrag(event) {
10080:   if (!activeRichImageMoveDrag) return;
10081:   const drag = activeRichImageMoveDrag;
10082:   drag.block.releasePointerCapture?.(drag.pointerId);
10083:   if (drag.moved) {

```

```

10084:     moveRichBlockToPoint(drag.editor, drag.block, event.clientX, event.clientY);
10085:     hideRichDropMarker();
10086:     event.preventDefault();
10087: }
10088: drag.block.classList.remove("is-dragging");
10089: document.body.classList.remove("rich-image-dragging");
10090: activeRichImageMoveDrag = null;
10091: activeRichDragBlock = null;
10092: }

```

beginImageCropDrag - lines 10094-10109

Item	Explanation
Source location	Lines 10094-10109 (16 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function beginImageCropDrag(event) {
Touches	browser/Electron DOM, filesystem
Calls detected	closest, normalizeCropPosition, max, getBoundingClientRect, preventDefault
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

10094: function beginImageCropDrag(event) {
10095:   const viewport = event.target.closest(".rich-image-viewport.crop-active");
10096:   if (!viewport || event.button !== 0) return;
10097:   const block = viewport.closest(".rich-image-block");
10098:   activeImageCropDrag = {
10099:     block,
10100:     editor: block.closest("[data-rich-editor]"),
10101:     startX: event.clientX,
10102:     startY: event.clientY,
10103:     cropX: normalizeCropPosition(block.dataset.cropX),
10104:     cropY: normalizeCropPosition(block.dataset.cropY),
10105:     width: Math.max(1, viewport.getBoundingClientRect().width),
10106:     height: Math.max(1, viewport.getBoundingClientRect().height)
10107:   };
10108:   event.preventDefault();
10109: }

```

moveImageCropDrag - lines 10111-10121

Item	Explanation
Source location	Lines 10111-10121 (11 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function moveImageCropDrag(event) {
Touches	browser/Electron DOM, filesystem
Calls detected	normalizeCropPosition, querySelector, setProperty
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

10111: function moveImageCropDrag(event) {
10112:   if (!activeImageCropDrag) return;
10113:   const { block, startX, startY, cropX, cropY, width, height } = activeImageCropDrag;
10114:   const nextX = normalizeCropPosition(cropX - ((event.clientX - startX) / width) * 100);
10115:   const nextY = normalizeCropPosition(cropY - ((event.clientY - startY) / height) * 100);
10116:   block.dataset.cropX = String(nextX);
10117:   block.dataset.cropY = String(nextY);

```

```

10118:   const viewport = block.querySelector(".rich-image-viewport");
10119:   viewport?.style.setProperty("--crop-x", `${nextX}%`);
10120:   viewport?.style.setProperty("--crop-y", `${nextY}%`);
10121: }

```

endImageCropDrag - lines 10123-10127

Item	Explanation
Source location	Lines 10123-10127 (5 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	function endImageCropDrag() {
Touches	Mostly local calculations or local UI state.
Calls detected	saveRichEditorToProject
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

10123: function endImageCropDrag() {
10124:   if (!activeImageCropDrag) return;
10125:   saveRichEditorToProject(activeImageCropDrag.editor);
10126:   activeImageCropDrag = null;
10127: }

```

handleStandaloneRichClick - lines 10211-10226

Item	Explanation
Source location	Lines 10211-10226 (16 source lines).
Purpose	Handles an event, request, command, or user action.
Declaration	async function handleStandaloneRichClick(event) {
Touches	browser/Electron DOM
Calls detected	handleRichCaretClick, closest, selectRichBlock, copyRichCodeBlock, editSelectedRichBlock, deleteSelectedRichBlock, saveRichEditorToProject
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	2 await expression(s), 3 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

10211: async function handleStandaloneRichClick(event) {
10212:   if (handleRichCaretClick(event)) return;
10213:   const blockAction = event.target.closest("[data-rich-block-action]");
10214:   const richBlock = event.target.closest(".rich-block");
10215:   if (richBlock) selectRichBlock(richBlock);
10216:   if (!blockAction) return;
10217:   const editor = blockAction.closest("[data-rich-editor]");
10218:   selectRichBlock(blockAction.closest(".rich-block"));
10219:   if (blockAction.dataset.richBlockAction === "copy-code") {
10220:     await copyRichCodeBlock(blockAction.closest(".rich-block"));
10221:     return;
10222:   }
10223:   if (blockAction.dataset.richBlockAction === "edit") await editSelectedRichBlock(editor);
10224:   if (blockAction.dataset.richBlockAction === "delete") deleteSelectedRichBlock(editor);
10225:   saveRichEditorToProject(editor);
10226: }

```

hasDataset - lines 10323-10327

Item	Explanation
Source location	Lines 10323-10327 (5 source lines).
Purpose	Part of the private builder frontend and editing experience.
Declaration	<code>const hasDataset = (key) => Object.prototype.hasOwnProperty.call(button.dataset, key);</code>
Touches	browser/Electron DOM
Calls detected	<code>call</code> , <code>addCustomSection</code>
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	1 <code>await</code> expression(s), 1 <code>return</code> statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```

10323:   const hasDataset = (key) => Object.prototype.hasOwnProperty.call(button.dataset, key);
10324:   if (button.dataset.addSection) {
10325:     await addCustomSection();
10326:     return;
10327:   }

```

Representative opening snippet

```

1: const builderGuideOpen = document.querySelector("#builder-guide-open");
2: const builderGuideDialog = document.querySelector("#builder-guide-dialog");
3: const builderGuideClose = document.querySelector("#builder-guide-close");
4: const builderGuideSearch = document.querySelector("#builder-guide-search");
5: const builderGuideResults = document.querySelector("#builder-guide-results");
6: const builderGuideSections = document.querySelector("#builder-guide-sections");
7: const builderGuideTopicDialog = document.querySelector("#builder-guide-topic-dialog");
8: const builderGuideTopicTitle = document.querySelector("#builder-guide-topic-title");
9: const builderGuideTopicBody = document.querySelector("#builder-guide-topic-body");
10: const builderGuideTopicClose = document.querySelector("#builder-guide-topic-close");
11: const templateSelect = document.querySelector("#template-select");
12: const newCategorySelect = document.querySelector("#new-category-select");
13: const placementSelect = document.querySelector("#placement-select");
14: const newProjectTitle = document.querySelector("#new-project-title");
15: const addProjectButton = document.querySelector("#add-project");
16: const addCategoryButton = document.querySelector("#add-category");
17: const addSiteSectionButton = document.querySelector("#add-site-section");
18: const funFactsInput = document.querySelector("#fun-facts-input");
19: const funFactsCount = document.querySelector("#fun-facts-count");
20: const saveFunFactsButton = document.querySelector("#save-fun-facts");
21: const clearFunFactsButton = document.querySelector("#clear-fun-facts");
22: const siteHeroEyebrowInput = document.querySelector("#site-hero-eyebrow");
23: const siteHeroTitleInput = document.querySelector("#site-hero-title");
24: const siteHeroCopyInput = document.querySelector("#site-hero-copy");
25: const saveSiteContentButton = document.querySelector("#save-site-content");
26: const profileDisplayNameInput = document.querySelector("#profile-display-name");
27: const profilePortfolioLabelInput = document.querySelector("#profile-portfolio-label");
28: const profileContactIntroInput = document.querySelector("#profile-contact-intro");
29: const profileEmailInput = document.querySelector("#profile-email");
30: const profilePhoneInput = document.querySelector("#profile-phone");
31: const profileGithubInput = document.querySelector("#profile-github");
32: const profileLinkedinInput = document.querySelector("#profile-linkedin");
33: const profileWebsiteInput = document.querySelector("#profile-website");
34: const saveProfileContentButton = document.querySelector("#save-profile-content");
35: const profileImageUploadButton = document.querySelector("#profile-image-upload");
36: const profileHeroUploadButton = document.querySelector("#profile-hero-upload");
37: const profileResumeUploadButton = document.querySelector("#profile-resume-upload");
38: const profileBrandUploadButton = document.querySelector("#profile-brand-upload");
39: const profileAssetStatus = document.querySelector("#profile-asset-status");
40: const categoryDropdown = document.querySelector("#category-dropdown");
41: const siteSectionList = document.querySelector("#site-section-list");
42: const projectCreateDialog = document.querySelector("#project-create-dialog");
43: const projectCreateForm = document.querySelector("#project-create-form");
44: const projectCreateCategory = document.querySelector("#project-create-category");
45: const projectCreateCancel = document.querySelector("#project-create-cancel");
46: const categoryDialog = document.querySelector("#category-dialog");
47: const categoryForm = document.querySelector("#category-form");
48: const categoryTitle = document.querySelector("#category-title");
49: const categoryDescription = document.querySelector("#category-description");
50: const categoryAccent = document.querySelector("#category-accent");
51: const categoryCancel = document.querySelector("#category-cancel");
52: const projectTree = document.querySelector("#project-tree");
53: const projectSearchInput = document.querySelector("#project-search");
54: const projectSearchClear = document.querySelector("#project-search-clear");
55: const projectSearchStatus = document.querySelector("#project-search-status");
56: const projectDialog = document.querySelector("#project-dialog");

```

```
57: const projectWindowTitle = document.querySelector("#project-window-title");
58: const projectWindowClose = document.querySelector("#project-window-close");
59: const projectWindowDelete = document.querySelector("#project-window-delete");
60: const viewProjectPreviewButton = document.querySelector("#view-project-preview");
61: const saveProjectButton = document.querySelector("#save-project");
62: const saveProjectCloseButton = document.querySelector("#save-project-close");
63: const projectTitleMenu = document.querySelector("#project-title-menu");
64: const titleRenameActions = document.querySelector("#title-rename-actions");
65: const titleRenameSave = document.querySelector("#title-rename-save");
66: const titleRenameCancel = document.querySelector("#title-rename-cancel");
67: const projectMetaDialog = document.querySelector("#project-meta-dialog");
68: const projectMetaTitle = document.querySelector("#project-meta-title");
69: const projectMetaGrid = document.querySelector("#project-meta-grid");
70: const projectMetaClose = document.querySelector("#project-meta-close");
71: const projectFields = document.querySelector("#project-fields");
72: const sectionTabs = document.querySelector("#section-tabs");
```

Safe change checklist

- Confirm which app, website, installer, workflow, or document consumes this file.
- Change it on a feature branch from development.
- Regenerate docs if the file purpose, API surface, or behavior changes.
- Run the smallest relevant smoke test before merging into development.